

The Backend Engineer's Library

Cloud, Containers, and Infrastructure

Volume 12

Contents

Containers Deep Dive: Images, Runtimes, and Isolation

Kubernetes Architecture: Control Plane and Data Plane

Kubernetes Workloads, Networking, and Storage

Infrastructure as Code

Cloud Primitives: Compute, Storage, and Network

Managed Data and Platform Services

Multi-Tenancy and Isolation

Cloud Cost and Capacity Engineering

Capacity Planning and Performance at Cloud Scale

Cloud Networking, IAM, and Security Foundations

Platform Engineering: Paved Roads and Internal Developer Platforms

Chapter 1 — Containers Deep Dive: Images, Runtimes, and Isolation

What this chapter covers. Containers are the deployment unit of modern backend systems — every microservice, batch job, and sidecar you ship to production runs inside one. Yet most engineers treat them as lightweight VMs, misunderstanding what they actually are, what they isolate, and what they do not. This chapter opens the container from bottom to top: the OCI image as a content-addressed stack of filesystem layers, the build pipeline that produces it, the runtime stack that executes it, and the Linux kernel primitives that isolate it. You will learn to write minimal, fast, cache-efficient Dockerfiles with multi-stage BuildKit builds, to read and reason about OCI manifests and layer history, to choose and configure a runtime (runc, crun, gVisor, Kata Containers), and to harden isolation with namespaces, cgroups v2, capabilities, seccomp, and mandatory access control. The chapter closes with the operational reality of running containers at scale — image distribution, signing (cosign), scanning, and debugging — grounding every abstraction in commands and manifests you run in production.

Learning goals — after this chapter you should be able to:

- Explain what a container is in terms of Linux namespaces, cgroups, and filesystem isolation — and why it is not a VM.
- Read an OCI image manifest and config, describe how content-addressed layers compose via a union filesystem, and use `crane`, `skopeo`, and `buildah` to inspect images without pulling them.
- Write production Dockerfiles that are minimal (`distroless` / `scratch`), cache-efficient (layer ordering, BuildKit cache mounts), reproducible, and rootless.
- Compare OCI runtimes — `runc`, `crun`, `gVisor` (`runsc`), `Kata Containers`, `Firecracker` / `microVMs` — on isolation strength, start latency, and overhead, and choose correctly for multi-tenant vs. trusted workloads.
- Configure isolation: namespaces (all 8 types), cgroups v2 (`cpu`, `memory`, `pids`, `io`), capability drops, seccomp profiles, AppArmor /

SELinux, read-only root filesystems, and user namespaces / rootless mode.

- Operate an image supply chain: registry distribution (OCI Distribution Spec), image signing with cosign and Sigstore, SBOM generation, and vulnerability scanning with Trivy / Grype.
- Diagnose container failures with `crictrl`, `nsenter`, `crun` / `runc` introspection, and eBPF-based tracing.

Boundary note. Volume 2, Chapter 9 — Namespaces, cgroups, and Container Internals — develops the *kernel mechanism* in detail: how each namespace is created with `unshare` / `clone`, how cgroups v2 controllers enforce limits, and the system-call surface that containers expose. This chapter is the *platform user's view*: how images are built and shipped, which runtime to pick, how to configure isolation for production, and what breaks when you get it wrong. Read Vol 2 Ch 9 for the kernel primitives; read here for the packaging and operations layer that sits on top.

What a container really is

A container is a set of kernel-enforced constraints around an ordinary Linux process. There is no hypervisor, no guest kernel, no emulated hardware. The container's process runs directly on the host kernel; the kernel merely lies to it about what it can see and how much it can consume.

```
host kernel (one kernel for all containers)
├─ cgroup /kubepods/pod-abc/container-app (cpu.max, memory.max, pid
s.max)
├─ mount namespace (its own / – overlayfs)
├─ pid namespace (PID 1 inside is PID 42319 outside)
├─ net namespace (its own veth + iptables)
├─ ipc, uts, user, cgroup, time namespaces
└─ seccomp + capabilities + LSM (AppArmor/SELinux)
    └─ process: /usr/bin/myapp (PID 1 in the container)
```

Three properties distinguish containers from VMs:

Property	Container	Virtual machine
Isolation boundary	Kernel namespaces + cgroups + LSM	Hypervisor + hardware virtualization (Intel VT-x / AMD-V, KVM)
Kernel	Shared with host	Own guest kernel
Start time	Milliseconds (fork + unshare)	Seconds (boot guest kernel + init)
Density	100s-1000s per host (limited by cgroups)	10s per host (limited by RAM for guest kernels)
Security boundary	Kernel is shared — a kernel exploit escapes all containers on the host	Hypervisor + guest kernel — escape requires hypervisor or hardware flaw
Image size	Megabytes (layers sharing a base)	Gigabytes (full disk image)

The shared-kernel design is both the strength and the weakness. It makes containers cheap and fast, and it makes the host kernel the most critical attack surface. Every container escape in recent history — CVE-2019-5736 (runc), CVE-2022-0492 (cgroups), CVE-2024-21626 (runc internal file descriptor leak) — was a bug in the shared kernel-facing code, not in the application.

The OCI standardization

The Open Container Initiative (OCI) standardizes three specs that let images and runtimes interoperate:

- **OCI Image Specification (v1.1)** — the format of an image (manifest, config, layers as tar+gz blobs, all content-addressed by SHA-256).
- **OCI Runtime Specification (v1.2)** — the JSON bundle (config.json) and lifecycle (create → start → kill → delete) that any runtime (runc, crun, gVisor, Kata) must honor.
- **OCI Distribution Specification (v1.1)** — the HTTP API that registries implement (/v2/<name>/manifests/<ref> , /v2/<name>/blobs/<digest>).

Any tool that speaks these specs interoperates: Docker can push to a Harbor registry, Podman can pull from Docker Hub, containerd can run an image built with Buildah, and Kubernetes can schedule it via any CRI runtime.

```
flowchart LR
    subgraph Build[Build]
        DF[Dockerfile] --> BK[BuildKit / Buildah]
        BK --> IMG[OCI Image]
    end
    layers + manifest + config
    end
    subgraph Distribute[Distribute]
        IMG --> REG["(OCI Registry  
Distribution Spec)"]
        REG --> SIGN["cosign sign  
Sigstore"]
        REG --> SCAN["Trivy / Gype  
scan"]
    end
    end
    subgraph Run[Run]
        REG --> CR["Container Runtime  
runc / crun / gVisor"]
        CR --> NS["Namespaces + cgroups  
+ seccomp + LSM"]
        NS --> PROC[Process]
    end
    end
    style IMG fill:#e3f2fd
    style REG fill:#fff3e0
    style PROC fill:#e8f5e9
```

Figure 1-1: The container lifecycle — build an OCI image, distribute through a registry with signing and scanning, run via an OCI runtime that configures kernel isolation.

OCI images: layers, manifests, and content addressing

The anatomy of an image

An OCI image is not a single file. It is a JSON manifest that points — by SHA-256 digest — to a config blob and an ordered list of layer blobs. Every blob is immutable and deduplicated by its digest.

```

myapp:v1.42
├─ Index (optional, for multi-arch) → manifest for amd64, manifest
for arm64
├─ Manifest (application/vnd.oci.image.manifest.v1+json)
│   └─ config: sha256:abc123... (JSON: env, entrypoint, exposed
ports, layer diff_ids)
│       └─ layer 0: sha256:111... (tar.gz – base filesystem, e.g. debian
bookworm-slim)
│           └─ layer 1: sha256:222... (tar.gz – apt-get install +
dependencies)
│               └─ layer 2: sha256:333... (tar.gz – COPY app binary)
│                   └─ layer 3: sha256:444... (tar.gz – config files)

```

When you `docker pull myapp:v1.42`, the registry serves the manifest; the runtime fetches only the layer blobs it does not already have (deduplicated by digest across all images on the host). The union filesystem (overlayfs) stacks them:

```

flowchart BT
    L0["Layer 0: base  
debian:bookworm-slim  
sha256:111..."]
    L1["Layer 1: dependencies  
RUN apt-get install libssl3  
sha256:222..."]
    L2["Layer 2: application  
COPY myapp /usr/local/bin/  
sha256:333..."]
    L3["Layer 3: config  
COPY config.yaml /etc/  
sha256:444..."]
    RW["Writable layer  
(container layer, ephemeral)"]
    Merged["Merged view (overlayfs)  
what the process sees as /"]

    L0 --> L1 --> L2 --> L3 --> RW --> Merged

    style L0 fill:#e3f2fd
    style L1 fill:#e8f5e9
    style L2 fill:#fff3e0
    style L3 fill:#fce4ec
    style RW fill:#fff9c4
    style Merged fill:#f3e5f5

```

Figure 1-2: OCI image layers — each Dockerfile instruction creates a content-addressed tar layer. OverlayFS presents the union as a single

filesystem; the top writable layer captures runtime mutations and is discarded when the container is removed.

Key consequences:

- **Immutability.** A layer's digest is `sha256(tar.gz contents)`. Changing one byte changes the digest; you cannot mutate a layer in place. Tags (`:v1.42`, `:latest`) are mutable pointers to a manifest digest — the digest is the true identity. Always pin deployments to `myapp@sha256:abc...`, not `:latest`.
- **Deduplication.** If ten images share `debian:bookworm-slim` as a base, that layer is stored once on the host. `docker system df` shows shared size vs. unique size.
- **Copy-on-write.** OverlayFS is CoW: reading a file traverses layers top-to-bottom until found; writing a file copies it up to the writable layer (copy-up). This is fast for reads, but write-heavy workloads (databases writing to the container filesystem) pay a copy-up penalty on first write — mount a volume instead.

Inspecting images without pulling

```
# Inspect a remote manifest without pulling layers (Go crane – no
daemon needed)
crane manifest gcr.io/distroless/static-debian12:nonroot | jq .
# {
#   "schemaVersion": 2,
#   "mediaType": "application/vnd.oci.image.manifest.v1+json",
#   "config": { "mediaType": "...", "digest": "sha256:abc...", "size":
1234 },
#   "layers": [
#     { "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
#       "digest": "sha256:111...", "size": 20971520 },
#     ...
#   ]
# }

# Show the config (entrypoint, env, history)
crane config gcr.io/distroless/static-debian12:nonroot | jq '{Env,
Entrypoint, WorkingDir}'

# Show layer history (which Dockerfile instruction created each layer)
crane history gcr.io/distroless/static-debian12:nonroot
# or with Docker:
docker history --no-trunc gcr.io/distroless/static-debian12:nonroot

# Alternative: skopeo (Red Hat ecosystem)
skopeo inspect docker://gcr.io/distroless/static-debian12:nonroot --
format '{{.Digest}}'
```

The config blob

The config blob (`sha256:abc...` referenced by the manifest) is JSON that describes *how* to run the image:

```
{
  "architecture": "amd64",
  "os": "linux",
  "config": {
    "Env": ["PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"],
    "Entrypoint": ["/usr/local/bin/myapp"],
    "Cmd": ["--config", "/etc/myapp/config.yaml"],
    "ExposedPorts": { "8080/tcp": {} },
    "WorkingDir": "/app",
    "User": "65532:65532"
  },
  "rootfs": {
    "type": "layers",
    "diff_ids": [
      "sha256:111...",
      "sha256:222..."
    ]
  },
  "history": [
    { "created": "2026-01-15T10:00:00Z", "created_by": "FROM gcr.io/distroless/static-debian12:nonroot" }
  ]
}
```

Note `diff_ids` are the *uncompressed* layer digests (`sha256(uncompressed tar)`), while the manifest references *compressed* digests (`sha256(gzip(tar))`). The runtime decompresses each layer and verifies both digests — a mismatch fails the pull.

Building images: Dockerfiles, BuildKit, and caching

Dockerfile best practices

A naive Dockerfile works but produces large, slow, insecure images. A production Dockerfile is minimal, cache-efficient, and reproducible.

Anti-pattern — single-stage, runs as root, no layer caching:

```
# BAD – do not do this
FROM ubuntu:22.04
COPY . /app
RUN apt-get update && apt-get install -y python3 python3-pip
RUN pip install -r /app/requirements.txt
CMD ["python3", "/app/main.py"]
# Problems: 400 MB base, runs as root, COPY . invalidates cache on any
# file change,
# apt cache left in layer, no .dockerignore discipline
```

Production — multi-stage, distroless, cache-efficient, non-root:

```

# syntax=docker/dockerfile:1.15
# Multi-stage Go build – stage 1 compiles, stage 2 ships only the
binary.

# — Stage 1: build —————
FROM golang:1.22-bookworm AS builder
WORKDIR /src

# Leverage layer caching: copy dependency manifests first, download,
then copy source.
# Changing app code does NOT invalidate the dependency layer.
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod \
    --mount=type=cache,target=/root/.cache/go-build \
    go mod download

COPY . .
RUN --mount=type=cache,target=/root/.cache/go-build \
    CGO_ENABLED=0 GOOS=linux go build \
        -trimpath \
        -ldflags="-s -w -extldflags '-static'" \
        -o /out/myapp ./cmd/myapp

# Optional: run tests inside the build (fail the build if tests fail)
# RUN go test ./...

# — Stage 2: runtime —————
FROM gcr.io/distroless/static-debian12:nonroot

# Copy ONLY the binary – no shell, no package manager, no OS utilities.
COPY --from=builder /out/myapp /usr/local/bin/myapp

# Non-root is already set in the distroless nonroot variant (uid
65532).
# If using `static` (root), add:
# USER 65532:65532

EXPOSE 8080
ENTRYPOINT ["/usr/local/bin/myapp"]
CMD ["--config", "/etc/myapp/config.yaml"]

```

For a Python service, the pattern is similar but uses a virtual environment to avoid system-wide installs:

```
# syntax=docker/dockerfile:1.15
FROM python:3.12-slim-bookworm AS builder
WORKDIR /src
COPY requirements.txt ./
# Build wheels in an isolated step (cache pip downloads)
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install --prefix=/out --no-warn-script-location -r requirements
    .txt

FROM gcr.io/distroless/python3-debian12:nonroot
COPY --from=builder /out /usr/local
COPY app/ /app/
WORKDIR /app
# Distroless python variant includes the interpreter but no shell.
ENTRYPOINT ["/usr/bin/python3", "-m", "app.main"]
```

Key techniques:

Technique	Why it matters
Multi-stage	Final image contains only runtime artifacts. Build tools, caches, and intermediate files never ship. A Go binary image drops from ~1 GB to ~15 MB.
Layer ordering	Copy dependency manifests first (<code>go.mod</code> , <code>requirements.txt</code>), install, <i>then</i> copy source. Source changes do not invalidate the dependency layer — 10× faster rebuilds.
BuildKit cache mounts (<code>--mount=type=cache</code>)	<code>go mod</code> downloads and <code>pip</code> caches persist across builds without being baked into the layer. Without this, every <code>go mod</code> download re-downloads the world.
Distroless / <code>scratch</code>	No shell, no <code>apt</code> , no <code>curl</code> — the attack surface is the app and its linked libraries only. <code>gcr.io/distroless/static</code> is ~2 MB; <code>scratch</code> is zero bytes.
<code>-trimpath</code> + <code>-ldflags="-s -w"</code>	Removes filesystem paths and debug symbols from Go binaries — smaller, deterministic builds.
<code>.dockerignore</code>	Exclude <code>.git</code> , <code>*.md</code> , <code>node_modules</code> , test fixtures — they bloat the build context and invalidate <code>COPY</code> . caching.

BuildKit and buildx

Docker's legacy builder (`docker build`) is deprecated. BuildKit (`DOCKER_BUILDKIT=1`, or `docker buildx build`) is the production builder:

```
# Enable BuildKit (default in Docker 23+)
export DOCKER_BUILDKIT=1

# Multi-arch build – produce amd64 + arm64 in one invocation
docker buildx create --name multi --use
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  --tag ghcr.io/myorg/myapp:v1.42 \
  --push \
  .

# Inspect build cache usage
docker buildx du

# Build with provenance and SBOM attestations (SLSA / supply-chain)
docker buildx build \
  --provenance mode=max \
  --sbom true \
  --tag ghcr.io/myorg/myapp:v1.42 \
  --push .
```

BuildKit features you should use in production:

- **Parallel stage execution** — independent stages build concurrently.
- **Cache backends** — `--cache-from type=registry,ref=ghcr.io/myorg/myapp:cache` `--cache-to type=registry,ref=ghcr.io/myorg/myapp:cache,mode=max` shares cache across CI runners.
- **Secrets** — `--mount=type=secret,id=npnrc,target=/root/.npnrc` injects credentials without baking them into a layer.
- **Provenance attestations** — generates in-toto SLSA provenance (who built it, from which commit, with which base image) — consumed by admission controllers (see Book 6, Ch 5 — Image Signing in Kubernetes).

Runtimes: runc, crun, gVisor, Kata, and beyond

The runtime stack

When Kubernetes runs a container, the request traverses a stack of runtimes. Understanding the layering clarifies where performance and isolation are decided.

```
flowchart TB
    subgraph K8s[Kubernetes]
        Kubelet[kubelet]
        Kubelet -->|CRI gRPC| CR[Container Runtime
        containerd / CRI-O]
    end
    subgraph Runtime[OCI Runtime Layer]
        CR -->|OCI Runtime Spec| LowR[Low-level Runtime
        runc / crun / runsc / kata-runtime]
        LowR -->|unshare + cgroups| Kernel[(Linux Kernel)]
    end
    subgraph Isolation[Isolation Strength]
        direction LR
        Runc["runc / crun
        namespaces + cgroups
        fast, shared kernel"]
        GVisor["gVisor (runsc)
        user-space kernel
        syscall interception"]
        Kata["Kata Containers
        lightweight VM
        QEMU / Firecracker"]
        Firecracker["Firecracker
        microVM
        minimal VMM"]
        Runc --- GVisor --- Kata --- Firecracker
    end
    style Kubelet fill:#e3f2fd
    style CR fill:#fff3e0
    style LowR fill:#e8f5e9
    style Kernel fill:#fce4ec
```

Figure 1-3: The Kubernetes runtime stack — kubelet speaks CRI to a high-level runtime (containerd/CRI-O), which delegates to a low-level OCI runtime that configures kernel isolation.

Component	Role	Examples
High-level runtime	Image pull, layer unpack, snapshot management, CRI gRPC server	<code>containerd</code> , <code>CRI-O</code>
Low-level (OCI) runtime	Takes an OCI bundle (<code>config.json</code> + <code>rootfs</code>), calls <code>unshare / clone</code> , sets cgroups, execs the process	<code>runc</code> (Go, reference), <code>crun</code> (C, faster), <code>runsc</code> (gVisor), <code>kata-runtime</code>
Shim	Keeps container alive if the high-level runtime restarts	<code>containerd-shim-runc-v2</code>

Choosing a runtime

Runtime	Isolation	Start latency	Memory overhead	Use case
runc	Namespaces + cgroups (shared kernel)	~50-100 ms	~0 (process)	Default for trusted workloads; fastest, lowest overhead
crun	Same as runc, written in C	~30-60 ms (faster than runc)	~0	Drop-in runc replacement; preferred on Fedora / Podman
gVisor (runsc)	User-space kernel (Sentry) intercepts syscalls; ~200 of 300+ Linux syscalls reimplemented	~100-200 ms	~20-50 MB per sandbox (Go runtime)	Multi-tenant isolation stronger than runc, without VM cost; GKE Autopilot / GKE Sandbox

Runtime	Isolation	Start latency	Memory overhead	Use case
Kata Containers	Lightweight VM (QEMU or Firecracker) per pod; own guest kernel	~300-800 ms (QEMU) / ~100-150 ms (Firecracker)	~100-150 MB per VM (guest kernel + memory)	Strongest isolation for untrusted / multi-tenant workloads; IBM Cloud, Kata on bare metal
Firecracker	MicroVM — minimal VMM (AWS)	~100-150 ms	~5 MB VMM + guest memory	AWS Lambda / Fargate; not a standalone OCI runtime but used under Kata or directly

Guidance for backend engineers:

- **Default to `runc/crun`** for first-party microservices you control. The isolation is sufficient when you trust the image.
- **Use `gVisor`** when running untrusted or semi-trusted code (user-submitted functions, CI jobs, multi-tenant SaaS) and you cannot afford Kata's memory cost.
- **Use `Kata` / `Firecracker`** when you need VM-grade isolation — regulatory requirements, running untrusted kernels, or when a kernel exploit must not escape to other tenants.

On a Kubernetes node you can offer multiple runtimes simultaneously via `RuntimeClass`:

```

# runtimeclass-gvisor.yaml – register gVisor as a scheduling option
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: gvisor
handler: runsc          # must match containerd config:
[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runsc]
---
# runtimeclass-kata.yaml
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: kata
handler: kata          # containerd runtime handler for kata-runtime
---
# Pod that opts into gVisor – scheduler places it only on nodes
supporting runsc
apiVersion: v1
kind: Pod
metadata:
  name: untrusted-job
spec:
  runtimeClassName: gvisor
  containers:
  - name: worker
    image: ghcr.io/myorg/untrusted-processor:v1.42@sha256:abc...
    resources:
      requests: { cpu: "500m", memory: "256Mi" }
      limits:   { cpu: "1",    memory: "512Mi" }

```

Isolation primitives

Volume 2, Chapter 9 builds each primitive from the syscall level. Here we configure them for production.

Namespaces

Eight namespace types exist (as of Linux 6.x); containers typically use seven (time namespace is opt-in):

Namespace	Flag	What it isolates	Visible effect inside container
<code>mnt</code>	<code>CLONE_NEWNS</code>	Filesystem mount table	Container sees its own <code>/</code> (overlayfs rootfs)

Namespace	Flag	What it isolates	Visible effect inside container
<code>pid</code>	<code>CLONE_NEWPID</code>	Process IDs	<code>ps</code> shows PID 1 as the app; host PIDs invisible
<code>net</code>	<code>CLONE_NEWNET</code>	Network stack (interfaces, routes, iptables, ports)	Container has its own <code>eth0</code> (veth pair) and port 80 does not conflict with host
<code>ipc</code>	<code>CLONE_NEWIPC</code>	SysV IPC, POSIX message queues	<code>ipcs</code> inside is independent of host
<code>uts</code>	<code>CLONE_NEWUTS</code>	Hostname and domain name	<code>hostname</code> inside is the pod name, not the node name
<code>user</code>	<code>CLONE_NEWUSER</code>	UID/GID mapping	Root (0) inside maps to unprivileged UID (e.g. 100000) outside
<code>cgroup</code>	<code>CLONE_NEWCGROUP</code>	cgroup mount view	Container sees only its own cgroup path
<code>time</code>	<code>CLONE_NEWTIME</code>	System clock offsets (rarely used)	Per-container clock skew for testing

Inspect namespaces of a running container:

```
# Find container PID (containerd / Docker)
PID=$(crictl inspect $(crictl ps -q | head -1) | jq .info.pid)
# or: PID=$(docker inspect -f '{{.State.Pid}}' mycontainer)

# List namespaces
ls -l /proc/$PID/ns
# lrwxrwxrwx cgroup -> 'cgroup:[4026532394]'
# lrwxrwxrwx ipc -> 'ipc:[4026532395]'
# lrwxrwxrwx mnt -> 'mnt:[4026532392]'
# lrwxrwxrwx net -> 'net:[4026532397]'
# lrwxrwxrwx pid -> 'pid:[4026532396]'
# lrwxrwxrwx user -> 'user:[4026531837]'
# lrwxrwxrwx uts -> 'uts:[4026532393]'

# Enter the container's network namespace to debug
nsenter -t $PID -n ip addr show
```

cgroups v2

cgroups enforce resource limits. cgroups v2 (unified hierarchy, default since Kubernetes 1.25 on most distros) exposes a single tree at `/sys/fs/cgroup`:

```
# On the host – find the cgroup for a pod
cat /proc/$PID/cgroup
# 0::/kubepods.slice/kubepods-burstable.slice/kubepods-burstable-
podabc.slice/cri-containerd-xyz.scope

# Inspect limits (cgroups v2)
cat /sys/fs/cgroup/kubepods.slice/.../memory.max      # e.g.
536870912 (512 MiB)
cat /sys/fs/cgroup/kubepods.slice/.../cpu.max          # e.g. "50000
100000" (0.5 CPU)
cat /sys/fs/cgroup/kubepods.slice/.../pids.max         # e.g. 1024
cat /sys/fs/cgroup/kubepods.slice/.../memory.oom.group # 1 = kill
whole cgroup on OOM
```

In Kubernetes you never write cgroup files directly — you set `resources.requests` / `resources.limits` and the kubelet + container runtime program cgroups for you (see Ch 3 for QoS classes and OOM behavior).

Capabilities, seccomp, and LSM

By default, Docker and Kubernetes containers retain 14 of the 38 Linux capabilities. The principle of least privilege says drop all and add back only what the process needs.

```
# Pod security – drop all capabilities, add back only what is required
apiVersion: v1
kind: Pod
metadata:
  name: hardened-app
spec:
  containers:
  - name: app
    image: ghcr.io/myorg/myapp:v1.42@sha256:abc...
    securityContext:
      runAsUser: 65532
      runAsGroup: 65532
      runAsNonRoot: true
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
        # Add back only if the app binds to a privileged port or needs
        # raw sockets
        # add: ["NET_BIND_SERVICE"]
    seccompProfile:
      type: RuntimeDefault          # use containerd's default seccomp
  filter
    # or: type: Localhost
    #   localhostProfile: profiles/myapp-seccomp.json
  resources:
    requests: { cpu: "200m", memory: "128Mi" }
    limits:   { cpu: "500m", memory: "256Mi" }
```

seccomp filters which syscalls the process may invoke. `RuntimeDefault` (the containerd / Docker default) blocks ~40 dangerous syscalls (`mount` , `clock_settime` , `ptrace` , `bpf` , `reboot` , etc.) while allowing the rest. Custom profiles can tighten further:

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "architectures": ["SCMP_ARCH_X86_64"],
  "syscalls": [
    { "names": ["read", "write", "open", "close", "fstat", "mmap", "munmap", "brk", "exit_group", "futex", "nanosleep", "getpid", "socket", "connect", "sendto", "recvfrom", "epoll_wait", "epoll_ctl"], "action": "SCMP_ACT_ALLOW" }
  ]
}
```

Generate a least-privilege seccomp profile by tracing actual syscalls with `strace` or `oci-seccomp-bpf-hook`.

AppArmor / **SELinux** add mandatory access control on files and capabilities beyond DAC:

```
# AppArmor profile (on Ubuntu/Debian nodes)
securityContext:
  appArmorProfile:
    type: Localhost
    localhostProfile: myapp-deny-write # /etc/apparmor.d/myapp-deny-write on the node
```

```
flowchart TB
    Proc[Process inside container] --> Caps{Capabilities?}
    Caps -->|required cap dropped| Deny1[EPERM]
    Caps -->|cap present| Sec{Seccomp filter?}
    Sec -->|syscall blocked| Deny2[EPERM / EPERM]
    Sec -->|syscall allowed| LSM{LSM}
    LSM -->|profile denies| Deny3[Permission denied]
    LSM -->|allowed| Kernel[Kernel executes syscall]
    style Deny1 fill:#ffcdd2
    style Deny2 fill:#ffcdd2
    style Deny3 fill:#ffcdd2
    style Kernel fill:#c8e6c9
```

Figure 1-4: Layered syscall filtering — capabilities gate privileged operations, seccomp filters the syscall table, and LSM (AppArmor/SELinux) enforces file and capability policy. All three must allow the call for it to reach the kernel.

Rootless and user namespaces

Running the container runtime itself as an unprivileged user (rootless Docker / Podman / containerd) eliminates the daemon-as-root attack surface. Inside the container, user namespaces map container UIDs to unprivileged host UIDs:

```
# /etc/subuid on the host – allocate 65536 UIDs starting at 100000 for
user 'ubuntu'
ubuntu:100000:65536

# Podman rootless – no daemon, no root
podman run --rm -it --user 1000:1000 alpine id
# uid=1000 gid=1000

# On the host, that process is actually uid 101000 (100000 + 1000)
ps -o pid,uid,cmd -p $PID
```

Kubernetes 1.25+ supports user namespaces for pods (`hostUsers: false`), mapping pod UIDs to host subordinate ranges — an important hardening step for multi-tenant clusters (see Ch 3 and Vol 12 Ch 7 — Multi-Tenancy).

Distribution, signing, and scanning

Registry and the OCI Distribution Spec

Registries implement a simple HTTP API. A push is: `POST /v2/<name>/blobs/uploads/` → `PUT ...?digest=sha256:...` for each layer, then `PUT /v2/<name>/manifests/<tag>` with the manifest JSON. A pull reverses it. Content negotiation supports multi-arch indexes (`application/vnd.oci.image.index.v1+json`) — the client sends `Accept: ...index...` and receives the manifest for its `GOARCH`.

Mirroring and pull-through caches are essential at scale to avoid Docker Hub / GHCR rate limits and to keep pulls fast inside the VPC:

```
# containerd - /etc/containerd/config.toml - registry mirrors
[plugins."io.containerd.grpc.v1.cri".registry.mirrors]
  [plugins."io.containerd.grpc.v1.cri".registry.mirrors."docker.io"]
    endpoint = ["https://mirror.internal.example.com", "https://registry-1.docker.io"]
  [plugins."io.containerd.grpc.v1.cri".registry.mirrors."ghcr.io"]
    endpoint = ["https://mirror.internal.example.com/ghcr"]
```

Signing with cosign and Sigstore

An unsigned image is an unauthenticated artifact — anyone who can push to the registry can substitute a malicious layer. Cosign signs the manifest digest with a key (or keyless via Fulcio/OIDC) and stores the signature as an OCI artifact alongside the image. Admission controllers (Kyverno, OPA Gatekeeper, Sigstore policy-controller) verify before scheduling.

```
# Keyless signing (OIDC via GitHub Actions, Fulcio issues a short-lived cert, Rekor logs)
cosign sign ghcr.io/myorg/myapp:v1.42@sha256:abc...

# Verify in CI / admission
cosign verify ghcr.io/myorg/myapp:v1.42@sha256:abc... \
  --certificate-identity-regexp "https://github.com/myorg/myapp/.github/workflows/release.yml@refs/tags/v.*" \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com

# Generate and attach an SBOM
syft ghcr.io/myorg/myapp:v1.42 -o spdx-json > sbom.spdx.json
cosign attest --predicate sbom.spdx.json --type spdx ghcr.io/myorg/myapp:v1.42@sha256:abc...
```

Scanning

Scan at build time (fail the build on **CRITICAL** with a fix), at push time (registry webhook), and continuously (new CVEs after push):


```
# Trivy – scan an image before push, fail on CRITICAL with fix
available
trivy image --severity CRITICAL --ignore-unfixed ghcr.io/myorg/
myapp:v1.42

# Gype – alternative (Anchore), better for distroless where no package
DB exists in the image
gype ghcr.io/myorg/myapp:v1.42 -o json | jq '.matches[] |
{vuln: .vulnerability.id, severity: .vulnerability.severity}'

# Admission-time scanning – Trivy Operator / Kyverno policy blocks pods
with CRITICAL CVEs
```

Prefer scanners that read SBOMs rather than re-extracting packages from the filesystem — they are faster and less prone to false positives on distroless images that lack `dpkg` / `rpm` databases.

Debugging containers in production

```
# crictl – the CRI-native equivalent of docker CLI (works with
containerd and CRI-O)
crictl ps -a                                # list all containers (including
exited)
crictl logs <container-id>                  # fetch logs without kubectl
crictl inspect <container-id> | jq .        # full CRI inspect (pid, mounts,
labels)
crictl exec -it <container-id> sh           # exec (if image has a shell –
distroless does not)

# Distroless debugging – ephemeral debug container shares the target's
namespaces
kubectl debug -it pod/myapp-xyz --image=busybox:1.36 --target=myapp --
sh
# --target shares pid + network + filesystem namespaces; busybox
provides the shell

# Enter namespaces directly on the node (for node-level debugging)
nsenter -t $PID -m -u -n -i -p -- sh
# -m mount, -u uts, -n net, -i ipc, -p pid – drops you into the
container's view

# Trace syscalls without a shell in the image (eBPF)
kubectl debug -it pod/myapp-xyz --image=nicolaka/netshoot --profile=sys
admin -- crictl inspect ...
# or on the node with bpftrace / kubectl-trace
bpftrace -e 'tracepoint:syscalls:sys_enter_execve { printf("%d %s
%s\n", pid, comm, str(args->filename)); }'
```

Key takeaways

- A container is a host kernel process with kernel-enforced lies: namespaces hide what it can see, cgroups limit what it can consume, capabilities/seccomp/LSM limit what it can do. There is no guest kernel — the host kernel is the trust boundary.
- OCI images are content-addressed stacks of tar layers referenced by SHA-256 digests. Tags are mutable pointers; digests are identities. Pin deployments to digests, share base layers for density, and prefer overlayfs CoW over writing to the container filesystem.
- Production Dockerfiles use multi-stage builds, BuildKit cache mounts, layer ordering (dependencies before source), distroless final

stages, and non-root users. BuildKit (`docker buildx`) adds multi-arch, remote cache, secret mounts, and SLSA provenance.

- The runtime stack is kubelet → CRI (containerd/CRI-O) → OCI runtime (runc/crun/gVisor/Kata) → kernel. Choose runc/crun for trusted workloads, gVisor for stronger syscall isolation at moderate cost, Kata/Firecracker for VM-grade isolation of untrusted code. Offer multiple runtimes via `RuntimeClass`.
- Isolation is layered: capabilities (drop ALL), seccomp (`RuntimeDefault` or custom), AppArmor/SELinux, read-only root filesystems, and user namespaces / rootless mode. No single layer is sufficient; defense in depth is mandatory.
- Distribution is an OCI HTTP API; sign images with cosign/Sigstore, generate SBOMs with Syft, scan with Trivy/Grype, and verify at admission. Treat unsigned images as untrusted input.
- Debug without assuming a shell in the image: `crictl`, `kubectl debug --target` (ephemeral debug containers), `nsenter`, and eBPF tracing work regardless of what the image contains.

Further reading

- OCI Specifications — Image Spec v1.1, Runtime Spec v1.2, Distribution Spec v1.1. <https://opencontainers.org/> and <https://github.com/opencontainers/image-spec>, <https://github.com/opencontainers/runtime-spec>, <https://github.com/opencontainers/distribution-spec>
- Docker BuildKit documentation — Dockerfile frontend, cache mounts, multi-platform builds. <https://docs.docker.com/build/buildkit/>
- Google Distroless images. <https://github.com/GoogleContainerTools/distroless>
- gVisor documentation — architecture, Sentry, Gofer, runsc. <https://gvisor.dev/docs/>
- Kata Containers architecture. <https://github.com/kata-containers/kata-containers/blob/main/docs/how-to/how-it-works.md>
- Firecracker design document (AWS). <https://github.com/firecracker-microvm/firecracker/blob/main/docs/design.md>
- NIST SP 800-190 — Application Container Security Guide. <https://csrc.nist.gov/publications/detail/sp/800-190/final>

- Liz Rice — *Container Security* (O'Reilly, 2020) — practical treatment of namespaces, capabilities, and seccomp.
- Sigstore / cosign documentation — keyless signing and verification. <https://docs.sigstore.dev/cosign/overview/>
- Trivy and Grype — container image vulnerability scanning. <https://aquasecurity.github.io/trivy/> , <https://github.com/anchore/grype>

Chapter 2 — Kubernetes

Architecture: Control Plane and Data Plane

What this chapter covers. Kubernetes is a declarative, level-triggered control system that turns a desired state (YAML in etcd) into an observed state (containers running on nodes). Every `kubectl apply` writes an object to etcd through the API server; a constellation of controllers, schedulers, and node agents then reconciles the world until it matches. This chapter dissects that system end to end: the control plane that stores and decides (kube-apiserver, etcd, kube-scheduler, kube-controller-manager, cloud-controller-manager), the data plane that acts (kubelet, kube-proxy, the CRI runtime, CNI, CSI), and the contracts between them (the Kubernetes API conventions, the watch/informer cache, CRI, CNI, and CSI). You will learn how the API server authenticates, authorizes, admits, validates, and persists every request; how etcd provides the strongly consistent store that makes leader election and object storage correct; how the scheduler scores and binds pods; how controllers implement reconciliation loops with informers and work queues; and how the kubelet drives pod lifecycle on a single node. The chapter closes with high-availability topologies, upgrades, and failure modes — the operational reality of keeping the control plane alive while the data plane does the work.

Learning goals — after this chapter you should be able to:

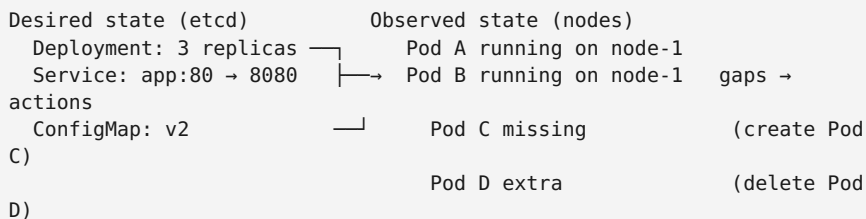
- Describe the control-plane / data-plane split, name every component, and explain the contract each one fulfills.

- Trace a `kubectl apply` from TLS termination through authentication, authorization, admission, validation, and etcd persistence — and explain what `kubectl get --raw` actually hits.
- Explain etcd's role (Raft consensus, MVCC store, watch stream, compaction), why etcd health determines cluster health, and how to operate and back up an etcd cluster.
- Describe the scheduler's pipeline (queue → filter → score → bind → reserve/permit) and the Scheduling Framework extension points, and write a `KubeSchedulerConfiguration` that tunes it.
- Implement the controller pattern: informers, listers, work queues, level-triggered reconciliation, leader election, and status vs. spec — and explain why `observedGeneration` and `resourceVersion` exist.
- Explain the kubelet's responsibilities (pod sync loop, probes, static pods, CRI interaction, cgroup management) and how kube-proxy (iptables, IPVS, eBPF) implements Services — or why Cilium/eBPF replaces it.
- Reason about HA topologies (stacked vs. external etcd), API server scaling, controller-manager and scheduler leader election, and the upgrade sequence that keeps workloads running while the control plane rolls.
- Diagnose control-plane and node failures with API server audit logs, etcd metrics, scheduler events, kubelet logs, and `crictl`.

Boundary note. Volume 6 — Distributed Systems — develops the consensus and coordination theory that the control plane depends on: Raft (Ch 6), linearizability and watches, and leases for leader election (Ch 8). Volume 2, Chapters 9–10 develop the Linux isolation and networking primitives that the data plane configures. This chapter is the *orchestration layer*: how Kubernetes composes etcd, the API server, scheduling, and node agents into a declarative control system. For Raft internals, read Vol 6 Ch 6; for how cgroups enforce the limits the kubelet sets, read Vol 2 Ch 9; for how a load balancer distributes traffic to the pods that Services select, read Vol 7 Ch 4. Here we focus on the Kubernetes-specific architecture that sits on top.

The cluster as a control system

Kubernetes is not a script that creates containers. It is a continuously running control loop — many loops, in fact — that observe the current state of the world, compare it to the desired state stored in etcd, and act to close the gap. This is *level-triggered* reconciliation: if a controller crashes and restarts, it re-reads the desired state and retries the same actions; if a node disappears, controllers reschedule its pods; if a user edits a Deployment, the Deployment controller creates a new ReplicaSet and the old one scales down.



Two planes implement this:

- **Control plane** — the brain. Runs on dedicated nodes (or managed by a cloud provider) and never runs user workloads in production. Components: kube-apiserver, etcd, kube-scheduler, kube-controller-manager, cloud-controller-manager.
- **Data plane** — the muscle. The fleet of worker nodes that actually run pods. On each node: kubelet, kube-proxy (or eBPF datapath), a container runtime (containerd / CRI-O via CRI), a CNI plugin, and a CSI driver.

The API server is the only component that talks to etcd, and every other component talks to the API server. No controller, scheduler, or kubelet accesses etcd directly — the API server is the sole stateful gateway, which lets it enforce authentication, authorization, validation, admission, and audit on every read and write.

```

flowchart TB
    User([User / CI / Controller])
    API[kube-apiserver]
    authn --> authz --> admission --> validation --> etcd
    ETCD[(etcd)]
    Raft[MVCC store]
    Sched[kube-scheduler]
    filter --> score --> bind
    CCM[kube-controller-manager]
    Deployment, ReplicaSet,
    Node, Endpoint controllers
    CloudCM[cloud-controller-manager]
    LoadBalancer, Node, Route
    Kubelet[kubelet]
    pod sync loop + CRI
    Proxy[kube-proxy / eBPF]
    Service --> iptables / IPVS
    Runtime[Container Runtime]
    containerd / CRI-O
    CNI[CNI Plugin]
    Cilium / Calico / Flannel
    CSI[CSI Driver]
    EBS / GCE PD / Ceph

    User <--> API
    API <--> ETCD
    Sched -->|watch pods + nodes| API
    bind decisions| API
    CCM -->|watch + reconcile| API
    CloudCM -->|cloud API| API
    Kubelet -->|watch pods bound| API
    to this node| API
    Kubelet --> Runtime
    Kubelet --> CNI
    Kubelet --> CSI
    Proxy -->|watch Services| API
    + Endpoints| API
    Proxy -->|programs| Runtime

    style API fill:#e3f2fd
    style ETCD fill:#fff3e0
    style Kubelet fill:#e8f5e9
    style Proxy fill:#fce4ec

```

Figure 2-1: Kubernetes control plane and data plane — every arrow passes through the API server. No component bypasses it to reach etcd.

The API server (kube-apiserver)

The API server is the front door and the only writer to etcd. It is a stateless, horizontally scalable Go HTTP server (typically 2-5 replicas behind a load balancer) that exposes a RESTful API over TLS. Understanding its request pipeline is the key to debugging every Kubernetes operation.

Request pipeline

```
TLS termination
→ Authentication (who are you?)
→ Authorization (are you allowed?)
→ Admission – mutating webhooks and controllers (modify the object)
→ Validation – OpenAPI schema + CEL + validating webhooks (reject if
invalid)
→ etcd write (persist, assign resourceVersion, emit watch event)
→ Response (201 Created / 200 OK with the stored object)
```

Every step is pluggable:

Authentication — the API server supports multiple authenticators simultaneously (x509 client certs, bearer tokens / ServiceAccount JWTs, OIDC, webhook token review). They run in order; the first to succeed sets `user + groups`.


```
# kube-apiserver flags (managed – shown for understanding)
# /etc/kubernetes/manifests/kube-apiserver.yaml (static pod)
spec:
  containers:
    - command:
      - kube-apiserver
      - --etcd-servers=https://10.0.0.5:2379,https://
10.0.0.6:2379,https://10.0.0.7:2379
      - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
      - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.crt
      - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key
      - --authentication-mode=Node,ServiceAccount
      - --service-account-issuer=https://kubernetes.default.svc
      - --service-account-key-file=/etc/kubernetes/pki/sa.pub
      - --authorization-mode=Node,RBAC
      - --enable-admission-
plugins=NodeRestriction,MutatingAdmissionWebhook,ValidatingAdmissionWeb
hook,ResourceQuota
      - --audit-log-path=/var/log/kubernetes/audit.log
      - --audit-policy-file=/etc/kubernetes/audit-policy.yaml
      - --request-timeout=60s
      - --max-requests-inflight=400
      - --max-mutating-requests-inflight=200
```

Authorization — after authentication, every request is authorized. The built-in modes are **Node** (kubelets may only touch their own Node and pods bound to it) and **RBAC** (Role / ClusterRole bindings). Authorization is evaluated per verb (**get**, **list**, **watch**, **create**, **update**, **patch**, **delete**) on a resource in a namespace.

```
# RBAC – least-privilege role for a CI deployer that may only rollout
one Deployment
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: production
  name: deployer
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "watch", "update", "patch"]
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: production
  name: deployer-binding
subjects:
- kind: ServiceAccount
  name: ci-deployer
  namespace: production
roleRef:
  kind: Role
  name: deployer
  apiGroup: rbac.authorization.k8s.io
```

Admission control — runs *after* authz but *before* persistence, in two phases:

1. **Mutating admission** — may modify the object. Built-in controllers (e.g., `DefaultStorageClass` sets `storageClassName` if absent) and `MutatingAdmissionWebhook` (e.g., Istio sidecar injection, `vault-agent` injection) add defaults, inject sidecars, or set labels.
2. **Validating admission** — may reject the object. Built-in validation (OpenAPI schema, CEL `x-kubernetes-validations`), `ValidatingAdmissionWebhook`, and the newer `ValidatingAdmissionPolicy` (CEL-based, no webhook server needed since 1.28) enforce policy.

```
# ValidatingAdmissionPolicy – deny privileged pods without a webhook
server
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata:
  name: deny-privileged
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - apiGroups: [""]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["pods"]
  validations:
    - expression: "!has(object.spec.containers.exists(c, has(c.securityContext) && has(c.securityContext.privileged) && c.securityContext.privileged))"
      message: "Privileged containers are not allowed."
    - expression: "object.spec.containers.all(c, !has(c.securityContext) || !has(c.securityContext.allowPrivilegeEscalation) || c.securityContext.allowPrivilegeEscalation == false)"
      message: "allowPrivilegeEscalation must be false."
  ---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata:
  name: deny-privileged-binding
spec:
  policyName: deny-privileged
  validationActions: [Deny]
  matchResources:
    namespaceSelector:
    matchLabels:
      enforce-privileged-policy: "true"
```

The API server rejects the request before it ever reaches etcd if any validating step fails — the audit log records the denial, and the client receives `400` or `403` with the policy message.

API conventions — every Kubernetes object shares a shape:

```

apiVersion: apps/v1          # group/version – "" is the legacy core
group
kind: Deployment
metadata:
  name: myapp
  namespace: production
  uid: 550e8400-e29b-41d4-a716-446655440000 # assigned by apiserver,
immutable
  resourceVersion: "123456"          # MVCC version – changes
on every write
  generation: 4                     # increments only when
spec changes
  creationTimestamp: "2026-01-15T10:00:00Z"
  labels: { app: myapp }
spec: {}      # desired state – what the user wants
status: {}    # observed state – what controllers report (subresource,
              separate RBAC)

```

`spec` is what you declare; `status` is what controllers observe and report. `resourceVersion` is the etcd MVCC revision — clients use it for optimistic concurrency (`If-Match`) and for `watch` resumption. `generation` vs. `observedGeneration` lets controllers know when they have reconciled the latest spec.

Scaling and HA

The API server is stateless, so you scale it horizontally — 3 or 5 replicas behind a TCP load balancer (or cloud managed control plane). Each replica connects to the same etcd cluster. Requests are not sticky; any replica can serve any read or write. etcd provides the linearizability.

Flow-control (APF — API Priority and Fairness, GA since 1.26) prevents a single misbehaving client from starving the API server:

```
# FlowSchema – boring workloads get lower priority
apiVersion: flowcontrol.apiserver.k8s.io/v1beta3
kind: FlowSchema
metadata:
  name: low-priority-batch
spec:
  priorityLevelConfiguration:
    name: low-priority
  distinguisherMethod:
    type: ByUser
  matchingPrecedence: 1000
  rules:
  - resourceRules:
    - apiGroups: ["batch"]
      resources: ["jobs"]
      verbs: ["create"]
    subjects:
    - kind: Group
      group:
        name: system:serviceaccounts:batch
  ---
apiVersion: flowcontrol.apiserver.k8s.io/v1beta3
kind: PriorityLevelConfiguration
metadata:
  name: low-priority
spec:
  type: Limited
  limited:
    nominalConcurrencyShares: 10
    limitResponse:
      type: Queue
    queuing:
      queues: 4
      queueLengthLimit: 20
      handSize: 3
```

etcd

etcd is the single source of truth. Every object you create lives as a key in etcd's MVCC key-value store under a prefix derived from its group/version/resource and namespace:

```
/registry/pods/production/myapp-xyz
/registry/deployments.apps/production/myapp
/registry/secrets/production/myapp-tls
```

Raft and the MVCC store

etcd is a Raft cluster (3 or 5 members; even numbers do not improve fault tolerance). One member is the leader that sequences writes; followers replicate the log. A write is committed when a majority (quorum) acknowledges it — 2 of 3, 3 of 5 — so a 3-member cluster tolerates 1 failure, a 5-member cluster tolerates 2. The MVCC store retains multiple revisions of each key, enabling `watch` from a past `resourceVersion` and optimistic concurrency.

Critical operational details:

- **Quorum, not count.** Adding a 4th member to a 3-member cluster does *not* increase fault tolerance — it still tolerates 1 failure but now needs 3 of 4 to form quorum (worse for latency). Scale from 3 → 5 when you need to tolerate 2.
- **Disk is the bottleneck.** etcd is fsync-bound — every committed write fsyncs the WAL. Slow disks (network-attached without provisioned IOPS, noisy neighbors) cause leader elections and API server timeouts. Run etcd on local SSDs or provisioned-IOPS volumes; monitor `etcd_disk_wal_fsync_duration_seconds` (p99 should be < 10ms).
- **Size limits.** The default `--quota-backend-bytes` is 8 GiB. Large objects (oversized ConfigMaps, Helm release secrets that store full release history, verbose CRDs) fill etcd and cause `etcdserver: mvcc: database space exceeded` — the cluster becomes read-only. Set `--auto-compaction-mode=periodic --auto-compaction-retention=5m`, monitor `etcd_mvcc_db_total_size_in_bytes`, and defragment periodically. Keep objects under 1 MiB; never store large blobs in etcd.
- **Defragmentation** reclaims space after compaction. It requires extra disk space (needs to copy the DB) and briefly holds a lock — run it during maintenance or rolling across members.

```
# etcd health and member list (from a control-plane node)
ETCDCTL_API=3 etcdctl --endpoints=https://10.0.0.5:2379,https://
10.0.0.6:2379,https://10.0.0.7:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  endpoint health --cluster

# Inspect raw keys (prefix scan)
ETCDCTL_API=3 etcdctl get /registry/pods/production/ --prefix --keys-
only

# Defragment one member at a time (never all at once – quorum)
ETCDCTL_API=3 etcdctl defrag --endpoints=https://10.0.0.5:2379

# Snapshot backup (the only supported backup method)
ETCDCTL_API=3 etcdctl snapshot save /var/backups/etcd-$(date +%F).db
ETCDCTL_API=3 etcdctl snapshot status /var/backups/etcd-2026-01-15.db -
w table
```

Watches

Watches are the nervous system. Instead of polling, every controller opens a long-lived `watch` on the resources it cares about (`GET /api/v1/pods?watch=1&resourceVersion=123`). `etcd` streams events (`ADDED` , `MODIFIED` , `DELETED`) as they occur. If the connection drops, the client resumes from the last `resourceVersion` it saw — because the MVCC store retains history until compaction, a short disconnection loses nothing. If `resourceVersion` is too old (compacted), the server returns `410 Gone` and the client must re-list.

This is why the informer cache exists — controllers list once, then watch forever, maintaining a local in-memory cache (the `Store`) without hammering the API server.

The scheduler (kube-scheduler)

The scheduler's job is narrow: assign each unscheduled pod (`spec.nodeName == ""`) to a node that can run it. It does not create or delete pods — it only writes `spec.nodeName` via the `pods/binding` subresource; the kubelet on that node then notices and starts the pod.

Scheduling pipeline

```
sequenceDiagram
    participant Pod as Pod (pending)
    participant Q as Scheduling Queue
    participant Cache as Scheduler Cache
    (nodes + pods)
    participant Filter as Filter plugins
    participant Score as Score plugins
    participant Bind as Bind

    Pod->>Q: watch: pod.spec.nodeName == ""
    Q->>Q: priority sort + backoff
    Q->>Cache: snapshot nodes + pods
    Cache->>Filter: predicates per node
    Note over Filter: NodeResourcesFit,
    TaintToleration,
    NodeAffinity,
    PodTopologySpread,
    VolumeBinding
    Filter-->>Score: feasible nodes
    Score->>Score: rank feasible nodes
    Note over Score: NodeResourcesFit score,
    ImageLocality,
    InterPodAffinity,
    NodeAffinity score
    Score-->>Bind: highest scoring node
    Bind->>Bind: Reserve → Permit → PreBind → Bind API call
    Bind->>Pod: POST /api/v1/namespaces/.../pods/.../binding
    {target: {name: node-3}}
```

Filtering is hard constraints (a pod that needs 4 GiB cannot land on a node with 2 GiB free). Scoring is soft preferences (prefer nodes that already have the image cached). The binding is a single optimistic write — if two schedulers race to bind the same pod, one fails with `409 Conflict` and retries.

The Scheduling Framework

Since 1.19, scheduling behavior is composed from framework plugins at extension points. You can tune, disable, or add plugins without forking the scheduler:

Extension point	When it runs	Example plugins
<code>QueueSort</code>	Orders pods in the queue	<code>PrioritySort</code> (default)

Extension point	When it runs	Example plugins
Filter	Hard predicate — can this pod run on this node?	NodeResourcesFit , NodePorts , TaintToleration , VolumeBinding , InterPodAffinity
Score	Soft ranking of feasible nodes (0-100)	NodeResourcesFit , ImageLocality , InterPodAffinity , PodTopologySpread
Reserve	After scoring, before bind — reserve resources	VolumeBinding reserves PV
Permit	Approve, deny, or delay the binding	CoScheduling (gang scheduling via PodGroup)
PreBind / PostBind	Immediately before/ after the bind API call	VolumeBinding provisions volume

Custom schedulers (batch, gang, bin-packing) either configure the default scheduler via `KubeSchedulerConfiguration` or run as a second scheduler (`spec.schedulerName: my-scheduler`) that watches only pods naming it.

```
# KubeSchedulerConfiguration – tune the default scheduler
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: default-scheduler
  plugins:
    filter:
      disabled:
        - name: InterPodAffinity # disable if you never use pod
          affinity (saves latency)
        score:
          disabled:
            - name: ImageLocality # disable on nodes with fast image
              pulls
          pluginConfig:
            - name: NodeResourcesFit
              args:
                scoringStrategy:
                  type: MostAllocated # bin-pack: prefer nodes with higher
                    utilization
                  # type: LeastAllocated # spread: prefer emptier nodes
                    (default)
                resources:
                  - name: cpu
                    weight: 1
                  - name: memory
                    weight: 2 # weight memory more heavily for
                    memory-bound workloads
```

Controller manager

The controller manager (`kube-controller-manager`) is not one controller but a process that hosts ~30 controllers, each a reconciliation loop. The cloud-controller-manager splits out the 3 controllers that need cloud API credentials (node, route, service/load-balancer) so the core controller manager needs no cloud IAM.

The reconciliation loop

Every controller follows the same pattern, built on `client-go`'s informer + work queue:

```

flowchart LR
    API[kube-apiserver  
watch stream]
    Informer[Informer  
List + Watch  
+ local cache Store]
    Queue[Work Queue  
rate-limited,  
deduplicated by key]
    Worker[Worker goroutines  
Reconcile loop]
    Act[Act via API server  
create / update / delete]

    API -->|watch events| Informer
    Informer -->|enqueue key  
ns/name| Queue
    Queue --> Worker
    Worker -->|get from cache  
compare spec vs status| Worker
    Worker -->|requeue on error  
with backoff| Queue
    Worker --> Act
    Act --> API

    style Informer fill:#e3f2fd
    style Queue fill:#fff3e0
    style Worker fill:#e8f5e9

```

Key design choices:

- **Level-triggered, not edge-triggered.** The work queue stores *keys* (namespace/name), not events. Multiple events for the same object collapse to one key. The worker always re-reads the latest object from the informer cache — if it missed an event, the next reconciliation corrects it. This is why controllers are resilient to missed watches.
- **Rate-limited requeue.** On error, the key is requeued with exponential backoff (5ms → 10ms → 20ms → ... → 1000s). Transient failures (API server temporarily unavailable) retry; persistent failures do not hot-loop.
- **Status subresource.** Controllers update `status` (observed state) separately from `spec` (desired state). RBAC grants controllers `update` on `status` but not on `spec` — users own `spec`, controllers

own `status.observedGeneration` in status records which `generation` was last reconciled.

- **Owner references and garbage collection.** When a controller creates an object, it sets `metadata.ownerReferences` pointing to itself. The garbage collector watches for owner deletion and cascades deletes (foreground, background, or orphan — controlled by `propagationPolicy`).

Key controllers and what they reconcile:

Controller	Watches	Action
Deployment	Deployments	Creates/updates ReplicaSets; orchestrates rolling updates
ReplicaSet	ReplicaSets + Pods	Ensures <code>replicas</code> pods exist matching the selector
StatefulSet	StatefulSets + Pods + PVCs	Ordered, sticky-identity pod creation with stable network/storage
Job / CronJob	Jobs / CronJobs	Creates pods to completion, handles parallelism and deadlines
Node	Nodes + Pods	Marks nodes <code>NotReady</code> after missed heartbeats; evicts pods via taints
Endpoints / EndpointSlice	Services + Pods	Populates the set of pod IPs backing each Service
ServiceAccount	ServiceAccounts	Creates mountable tokens / projected volumes

A minimal controller in Go (the pattern every controller follows):

```

// Reconcile is called with the key of a changed object.
// It must be idempotent – calling it twice with the same key produces
the same result.
func (c *MyController) Reconcile(ctx context.Context, key string)
error {
    ns, name, _ := cache.SplitMetaNamespaceKey(key)
    obj, exists, err := c.informer.GetIndexer().GetByKey(key)
    if err != nil { return err }
    if !exists {
        // Object was deleted – clean up external resources.
        return c.handleDelete(ns, name)
    }
    desired := obj.(*MyApp).Spec
    observed := obj.(*MyApp).Status

    if observed.ObservedGeneration == obj.(*MyApp).Generation {
        return nil // already reconciled
    }
    // ... create/update child resources, set ownerReferences ...
    // ... update status with new observedGeneration ...
    return c.client.Status().Update(ctx, obj)
}

```

Leader election

Controller-manager and scheduler are active-passive — only one replica acts at a time; the others stand by. They use a Lease object in `kube-system` (`kube-controller-manager`, `kube-scheduler`) with periodic renewals. If the leader's lease expires (it crashed or was partitioned), another replica acquires it within seconds. etcd is the linearizable store that makes this safe (see Vol 6 Ch 8 for the lease mechanics).

```

kubectl -n kube-system get lease kube-controller-manager -o yaml
# spec:
#   holderIdentity: kube-controller-manager-xyz_abc
#   leaseDurationSeconds: 15
#   renewTime: "2026-01-15T10:00:05Z"

```

Data plane

kubelet

The kubelet is the node agent — one per node, the only Kubernetes component that actually starts containers. Its responsibilities:

1. **Watch pods bound to this node** — the API server notifies it via watch; it also watches a local directory (`/etc/kubernetes/manifests`) for static pods (used to run the control plane itself).
2. **Sync loop (PLEG — Pod Lifecycle Event Generator)** — every second, compare desired pods (from API server + static pods) to running containers (from CRI). If a desired pod is missing, create it; if a running container should not exist, kill it; if a liveness probe fails, restart it. This is reconciliation at the node level, mirroring the control-plane controllers.
3. **Probes** — `livenessProbe` (restart the container if it fails), `readinessProbe` (remove the pod from Service endpoints if it fails), `startupProbe` (gate liveness/readiness until the app has started). All probes are executed by the kubelet, not by the pod.
4. **Resource management** — creates cgroups for pods/containers (via the cgroup driver — `systemd` is the only supported driver since 1.22), enforces `resources.limits`, reports `resources.requests` to the scheduler, and evicts pods when the node is under memory/disk pressure (`--eviction-hard=memory.available<500Mi,imagefs.available<10%`).
5. **Volume and secret mounting** — calls CSI drivers to mount volumes, fetches ConfigMaps/Secrets from the API server and projects them into the pod filesystem.
6. **Status reporting** — posts pod status, node status (conditions, capacity, allocatable), and events back to the API server.

```
# KubeletConfiguration (on each node - /var/lib/kubelet/config.yaml)
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
cgroupDriver: systemd
clusterDNS: ["10.96.0.10"]
clusterDomain: cluster.local
maxPods: 110
evictionHard:
  memory.available: "500Mi"
  imagefs.available: "10%"
  nodefs.available: "10%"
evictionSoft:
  memory.available: "1Gi"
evictionSoftGracePeriod:
  memory.available: "2m"
featureGates:
  UserNamespacesSupport: true
```

Static pods deserve a note — the kubelet can run pods without the API server. Any YAML file placed in `--pod-manifest-path` (default `/etc/kubernetes/manifests`) is run as a pod. `kubeadm` uses this to bootstrap the control plane: the API server, scheduler, controller-manager, and etcd each run as a static pod, so the cluster can start without an API server already running.

Container Runtime Interface (CRI)

The kubelet does not speak Docker directly (dockershim was removed in 1.24). It speaks CRI — a gRPC API (`RunPodSandbox`, `CreateContainer`, `StartContainer`, `StopContainer`, `RemoveContainer`, `ImageService.PullImage`) — to a high-level runtime:

- **containerd** (`containerd.io`) — the CNCF default; used by GKE, EKS, AKS, and kind.
- **CRI-O** — Red Hat's lightweight CRI-only runtime; used by OpenShift.

Both delegate to an OCI low-level runtime (`runc` / `crun`) for the actual `unshare` / `cgroups` / `exec` (see Ch 1 for runtime selection).

Debug CRI directly with `crictl` (not `docker`):

```
crictl ps -a                # list containers via CRI (not Docker)
crictl pods --name myapp    # list pod sandboxes
crictl inspect <container-id> # CRI inspect (pid, mounts, labels)
crictl logs <container-id>   # fetch logs via CRI log driver
crictl images               # images known to the runtime
```

kube-proxy and Service networking

Services are stable virtual IPs that load-balance across a dynamic set of pod IPs. kube-proxy is the component that programs that load-balancing on each node.

Three modes, in historical order:

Mode	How it programs the datapath	Performance	Status
userspace (legacy)	kube-proxy process proxies every connection in user space	Slow — extra copy per packet	Removed (1.25)
iptables	kube-proxy writes iptables NAT rules; kernel does DNAT	Good for < 1k Services; iptables-restore is O(n) on rule count	Default on most clusters
IPVS	kube-proxy writes IPVS virtual servers; kernel does load-balancing	Better for large clusters; O(1) per Service, supports more LB algorithms	Opt-in (--proxy-mode=ipvs)
eBPF (no kube-proxy)	Cilium / Cilium-eBPF replaces kube-proxy entirely with eBPF programs	Best — no iptables traversal, socket-level LB, XDP fast path	Preferred on new clusters with Cilium

In iptables mode, a ClusterIP Service 10.96.42.10:80 → pods 10.0.1.3:8080, 10.0.1.4:8080 is implemented as:

```
# On the node – what kube-proxy wrote (simplified)
iptables -t nat -A KUBE-SERVICES -d 10.96.42.10/32 -p tcp --dport 80
-j KUBE-SVC-XXXX
iptables -t nat -A KUBE-SVC-XXXX -m statistic --mode random --
probability 0.5 -j KUBE-SEP-YYYY # 50% → pod 1
iptables -t nat -A KUBE-SVC-XXXX -j KUBE-SEP-
ZZZZ # 50% → pod 2
iptables -t nat -A KUBE-SEP-YYYY -j DNAT --to-destination 10.0.1.3:8080
iptables -t nat -A KUBE-SEP-ZZZZ -j DNAT --to-destination 10.0.1.4:8080
```


Cilium in eBPF mode replaces all of this with a single eBPF map lookup at the socket layer — no iptables chain traversal, and it handles NetworkPolicy enforcement in the same program. New clusters should prefer Cilium and set `kubeProxyReplacement: true`.

Ch 3 covers Services, Ingress, Gateway API, DNS, and NetworkPolicies at the workload level — here the focus is the datapath mechanism.

CNI and CSI

CNI (Container Network Interface) is the plugin that wires a pod's network namespace. The kubelet calls `ADD` on pod creation (allocate an IP, create a veth pair, attach to a bridge or eBPF datapath) and `DEL` on deletion. Popular plugins: Cilium (eBPF), Calico (BGP / eBPF), Flannel (VXLAN), Weave. Only one CNI runs per cluster.

CSI (Container Storage Interface) is the analogous plugin for storage. A CSI driver (EBS, GCE PD, Ceph, Longhorn) implements `CreateVolume`, `NodeStageVolume`, `NodePublishVolume` — the kubelet and the external CSI controller call these to provision and mount persistent volumes (see Ch 3).

High availability, upgrades, and failure modes

HA topologies

Two standard topologies (from `kubeadm`):

Stacked etcd — etcd members colocated with control-plane nodes (3 nodes, each runs API server + scheduler + controller-manager + etcd). Simpler, fewer machines, but etcd shares CPU/disk with the API server and a node loss takes both a control-plane replica and an etcd member.

External etcd — 3 dedicated etcd hosts + 3 control-plane nodes (6 machines). etcd gets dedicated disks and is isolated from API server load. Preferred for large or latency-sensitive clusters.

Both tolerate 1 failure with 3 members and 2 with 5. Cloud managed control planes (GKE, EKS, AKS) hide this entirely — they run a regional, multi-AZ control plane with external etcd and a managed load balancer.

```

flowchart TB
    subgraph Stacked[Stacked etcd - 3 nodes]
        S1[Node 1  
API + sched + c-m + etcd-1]
        S2[Node 2  
API + sched + c-m + etcd-2]
        S3[Node 3  
API + sched + c-m + etcd-3]
        S1 --- S2 --- S3
    end
    subgraph External[External etcd - 6 nodes]
        direction TB
        E1[etcd-1] --- E2[etcd-2] --- E3[etcd-3]
        C1[API + sched + c-m]
        C2[API + sched + c-m]
        C3[API + sched + c-m]
        E1 --> C1
        E2 --> C2
        E3 --> C3
    end
    LB[Load Balancer  
apiserver.example.com:443]
    S1 --> LB
    S2 --> LB
    S3 --> LB
    C1 --> LB
    C2 --> LB
    C3 --> LB
    style LB fill:#e3f2fd

```

Figure 2-2: Stacked vs. external etcd — stacked is simpler but couples etcd to control-plane load; external isolates etcd on dedicated hosts with dedicated disks.

Upgrades

Kubernetes upgrades one minor version at a time (1.29 → 1.30, never 1.29 → 1.31). The sequence:

1. Upgrade the control plane — API server first (it must understand both old and new object versions), then controller-manager and scheduler. etcd is upgraded separately and rarely (it is backward-compatible across many Kubernetes versions).
2. Drain and upgrade worker nodes one at a time — `kubectl drain --ignore-daemonsets` evicts pods, upgrade kubelet + container runtime, `kubectl uncordon`.

3. Workloads stay running throughout — Deployments reschedule evicted pods onto remaining nodes (assuming `PodDisruptionBudget` and sufficient capacity).

Managed clusters automate this; self-hosted clusters use `kubeadm upgrade plan` / `kubeadm upgrade apply v1.30.x`.

Failure modes

Failure	Symptom	Mitigation
etcd quorum loss (2 of 3 down)	API server returns 500; no writes succeed; existing pods keep running but no new scheduling	Restore from snapshot; never run etcd on ephemeral disks
API server overload (too many watches or large lists)	429 / timeouts; controllers fall behind	APF flow control; paginate lists (<code>limit + continue</code>); avoid <code>list</code> without label selectors
Scheduler down	New pods stay <code>Pending</code> indefinitely; running pods unaffected	Multiple scheduler replicas with leader election (or a second scheduler for critical pods)
kubelet down on a node	Node goes <code>NotReady</code> after 40s (default <code>node-monitor-grace-period</code>); pods stay bound until the Node controller evicts them (5 min default <code>pod-eviction-timeout</code> or immediately if tainted <code>NoExecute</code>)	Node auto-repair (GKE) or Cluster API machine health checks
CNI failure	Pods stuck in <code>ContainerCreating</code> (<code>FailedCreatePodSandBox</code>); network policy not enforced	CNI DaemonSet health checks; Cilium/Calico operator with self-healing

Failure	Symptom	Mitigation
etcd disk full	API server writes fail with <code>database space exceeded</code> ; cluster read-only	Compaction + defrag; quota alerts; never store large blobs in etcd

Key takeaways

- Kubernetes is a set of level-triggered reconciliation loops. The desired state lives in etcd; controllers, the scheduler, and kubelets continuously drive observed state toward it. Every loop is built on informer + work queue + compare spec vs. status.
- The API server is the sole gateway to etcd and the only place where authentication, authorization, admission, validation, and audit are enforced. It is stateless and horizontally scalable; etcd is the only stateful control-plane component.
- etcd is a Raft MVCC store. Its health determines cluster health. Run it on dedicated SSDs, keep the DB under quota (compact + defrag), never store large objects in it, back up with `etcdctl snapshot save`, and prefer 3 members (or 5 for 2-failure tolerance — never 4).
- The scheduler filters (hard predicates) then scores (soft preferences) nodes, then binds via a single optimistic `Pods/Binding` write. The Scheduling Framework makes filtering and scoring pluggable; tune it with `KubeSchedulerConfiguration` rather than forking.
- Controllers watch, enqueue keys, and reconcile. They collapse events, retry with backoff, and separate `spec` (user-owned) from `status` (controller-owned). Owner references drive garbage collection.
- The data plane is kubelet (pod sync loop + probes + cgroups + CRI) + container runtime (containerd/CRI-O → runc/crun) + kube-proxy or eBPF (Service load-balancing) + CNI (pod networking) + CSI (volumes). `crictl` is the CRI-native debug tool; `kubectl debug` works even for distroless images.
- HA is stacked or external etcd (3 or 5 members) with 2-5 API server replicas behind a load balancer and leader-elected scheduler/

controller-manager. Upgrades are control plane first, then nodes one at a time, with `PodDisruptionBudget` protecting availability.

Further reading

- Kubernetes Concepts — Cluster Architecture. <https://kubernetes.io/docs/concepts/architecture/>
- Kubernetes Reference — kube-apiserver, kube-scheduler, kube-controller-manager, kubelet. <https://kubernetes.io/docs/reference/command-line-tools-reference/>
- etcd documentation — operations, tuning, disaster recovery. <https://etcd.io/docs/>
- Kubernetes Enhancement Proposals — APF (KEP-1040), Scheduling Framework (KEP-624), ValidatingAdmissionPolicy (KEP-3488). <https://github.com/kubernetes/enhancements>
- Stefan Schimanski and Michael Hausenblas — *Programming Kubernetes* (O'Reilly, 2019) — informers, work queues, and controller patterns.
- Brendan Burns et al. — *Kubernetes: Up and Running*, 3rd ed. (O'Reilly, 2022) — control-plane and data-plane walkthroughs.
- Cilium documentation — eBPF datapath and kube-proxy replacement. <https://docs.cilium.io/>
- kubeadm — Creating Highly Available Clusters. <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/high-availability/>

Chapter 3 — Kubernetes Workloads, Networking, and Storage

What this chapter covers. Pods, Services, and PersistentVolumes are the three primitives that turn Kubernetes from a container runner into a platform that can host stateful, networked, auto-scaled backend systems. This chapter builds the workload layer that backend engineers use every day: how a Pod spec actually maps to containers, probes, lifecycle hooks, and cgroups on a node; how workload controllers (Deployment, StatefulSet, DaemonSet, Job, CronJob) provide rollout, identity, and completion semantics on top of raw pods; how autoscalers (HPA, VPA,

KEDA, Karpenter/Cluster Autoscaler) close the loop between load and capacity; how Services, EndpointSlices, Ingress, Gateway API, CoreDNS, and NetworkPolicy compose the networking model; and how volumes, PersistentVolumeClaims, StorageClasses, and CSI drivers give pods durable storage with dynamic provisioning, snapshots, and topology awareness. Every abstraction is grounded in manifests you apply to a real cluster, with the failure modes and operational trade-offs that determine whether a system survives load, upgrades, and zone failures.

Learning goals — after this chapter you should be able to:

- Write correct Pod specs with multi-container patterns (sidecar, adapter, ambassador), init containers, native sidecar containers (1.29+ `restartPolicy: Always`), probes, lifecycle hooks, and resource requests/limits that produce the right QoS class.
- Choose and configure workload controllers — Deployment (stateless rollouts), StatefulSet (stable identity + ordered storage), DaemonSet (per-node agents), Job/CronJob (run-to-completion) — and explain their reconciliation guarantees.
- Configure autoscaling at three levels: pod count (HPA on CPU/custom metrics), pod resources (VPA), event-driven scaling (KEDA), and node count (Cluster Autoscaler / Karpenter), and reason about their interactions and lag.
- Describe the Service abstraction (ClusterIP, headless, NodePort, LoadBalancer, ExternalName), how EndpointSlices are populated, and how kube-proxy or Cilium/eBPF implements Service load-balancing — and when to use Ingress vs. Gateway API for L7 routing.
- Explain cluster DNS (CoreDNS), CNI networking (pod IP allocation, overlay vs. native routing), and NetworkPolicy (default-deny, namespace isolation, Cilium ClusterwideNetworkPolicy), and write correct policies that enforce tenant isolation.
- Provision storage with PVCs, StorageClasses, and CSI drivers — dynamic provisioning, `volumeBindingMode: WaitForFirstConsumer`, topology constraints, snapshots, expansion, and the lifecycle of PV/PVC/Pod binding — and explain why StatefulSet + `volumeClaimTemplates` is the correct pattern for databases.

Boundary note. Chapter 2 dissected the control plane and data plane machinery (API server, etcd, scheduler, kubelet, CRI, kube-proxy/CNI/CSI as components). This chapter is the *workload author's view* — the objects

you declare and the semantics they give you. Volume 6 — Distributed Systems — provides the replication and consistency theory behind StatefulSet identity and storage; Volume 3 — Networking — develops the L3/L4/L7 concepts that Services and Ingress implement; Volume 5 — Databases — covers the storage engines that run inside the pods you schedule here. Read Ch 2 for *how* the kubelet runs a pod; read here for *which* pod controller and *which* Service and *which* volume to use, and what breaks when you choose wrong.

Pods: the atomic unit

A Pod is not a container — it is a *group* of containers that share fate, network, and storage. The Pod is the smallest schedulable unit; the scheduler binds an entire pod to a node, and the kubelet creates a single network namespace (the "pause" or "sandbox" container) that all containers in the pod join. They share an IP, a port space, IPC, and optionally volumes and process namespace.

Anatomy of a Pod spec


```

apiVersion: v1
kind: Pod
metadata:
  name: api-7f9c4
  namespace: production
  labels: { app: api, version: v2.1 }
spec:
  # Scheduling
  nodeSelector: { workload: api }
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector: { matchLabels: { app: api } }
            topologyKey: kubernetes.io/hostname # spread across nodes
  tolerations:
    - key: workload
      operator: Equal
      value: api
      effect: NoSchedule
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
      labelSelector: { matchLabels: { app: api } }

  # Security – pod-level defaults (container can override)
  securityContext:
    runAsUser: 65532
    runAsNonRoot: true
    seccompProfile: { type: RuntimeDefault }
    fsGroup: 65532

  # Volumes shared across containers in this pod
  volumes:
    - name: config
      configMap: { name: api-config }
    - name: cache
      emptyDir: {}
    - name: tls
      secret: { secretName: api-tls }

  # Init containers – run sequentially to completion before app
  containers start
  initContainers:
    - name: wait-for-migrations
      image: ghcr.io/myorg/migrate:v1.42@sha256:abc...
      command: ["/wait-for-db", "--timeout=120s"]
      resources: { requests: { cpu: "50m", memory: "64Mi" } }

```

```

containers:
# Main application container
- name: api
  image: ghcr.io/myorg/api:v2.1@sha256:def...
  ports:
  - containerPort: 8080
    name: http
    protocol: TCP
  env:
  - name: POD_IP
    valueFrom: { fieldRef: { fieldPath: status.podIP } }
  - name: CONFIG_PATH
    value: /etc/config/config.yaml
  volumeMounts:
  - { name: config, mountPath: /etc/config }
  - { name: tls, mountPath: /etc/tls, readOnly: true }

  resources:
    requests: { cpu: "500m", memory: "512Mi" } # scheduler uses
this; also initial cgroup share
    limits: { cpu: "1000m", memory: "768Mi" } # cgroup hard cap;
OOMKill if exceeded

    # Probes – the kubelet's health checks (not the pod's)
    startupProbe: # gate liveness/
readiness until app has started
      httpGet: { path: /healthz/startup, port: http }
      periodSeconds: 5
      failureThreshold: 30 # 30 × 5s = 150s
    startup budget
      livenessProbe:
# restart the container if this fails
      httpGet: { path: /healthz/live, port: http }
      periodSeconds: 10
      failureThreshold: 3
      readinessProbe: # remove from Service
endpoints if this fails
      httpGet: { path: /healthz/ready, port: http }
      periodSeconds: 5
      failureThreshold: 2
      successThreshold: 1

  lifecycle:
    preStop: # called before SIGTERM
      exec:
        command: ["/bin/sh", "-c", "sleep 10"]
# give the LB time to drain

    # Native sidecar container (1.29+ – restartPolicy: Always keeps it
running)

```

```

- name: envoy-proxy
  image: ghcr.io/envoyproxy/envoy:v1.30@sha256:789...
  restartPolicy: Always
  ports:
  - { containerPort: 15001, name: proxy }
  resources: { requests: { cpu: "100m", memory: "128Mi" } }

  terminationGracePeriodSeconds: 30
  restartPolicy: Always # Always (Deployment)
vs. OnFailure (Job) vs. Never

```

Key decisions in this spec and why they matter:

Field	Why it matters for backend systems
<code>resources.requests</code> vs. <code>limits</code>	Requests drive scheduling (bin-packing) and initial cgroup shares; limits are the hard cap. Setting <code>requests == limits</code> gives Guaranteed QoS (never evicted for resource pressure). <code>requests < limits</code> is Burstable (can burst, but may be throttled/killed). Omitting both is BestEffort (first to be evicted).
<code>startupProbe</code>	Without it, a slow-starting app (JVM warmup, large cache load) has its liveness probe fail during startup and gets killed in a crash loop. <code>startupProbe</code> disables liveness/readiness until it succeeds.
<code>livenessProbe</code> vs. <code>readinessProbe</code>	Liveness = "is the process stuck?" → restart. Readiness = "can this pod serve traffic?" → remove from endpoints. Conflating them causes restart storms when a downstream dependency is slow — readiness should fail, liveness should not.

Field	Why it matters for backend systems
<code>preStop</code> + <code>terminationGracePeriodSeconds</code>	On deletion, kubelet sends <code>preStop</code> , then <code>SIGTERM</code> , waits <code>terminationGracePeriodSeconds</code> (default 30s), then <code>SIGKILL</code> . <code>preStop: sleep 10</code> gives kube-proxy/Cilium time to remove the pod from Service backends before the process stops accepting connections.
<code>topologySpreadConstraints</code>	Without it, the scheduler may pack all replicas onto one zone or node. <code>maxSkew: 1</code> across <code>topology.kubernetes.io/zone</code> ensures even distribution — critical for zone failure tolerance.
<code>securityContext</code>	<code>runAsNonRoot</code> + <code>RuntimeDefault</code> seccomp + dropping capabilities (Ch 1) should be the default for every pod.

```

flowchart LR
    subgraph Pod[Pod – one IP, one cgroup, shared volumes]
        Pause[Pause container]
    end
    holds net + ipc + pid ns
    Init[Init containers]
    sequential, to completion
    App[api container]
    8080]
    Sidecar[envoy-proxy]
    sidecar Always]
    Vol1[(config
    ConfigMap)]
    Vol2[(cache
    emptyDir)]
    end
    Pause --- App
    Pause --- Sidecar
    Init -->|all succeed then| App
    App --- Vol1
    Sidecar --- Vol1
    App --- Vol2

    style Pause fill:#e3f2fd
    style App fill:#e8f5e9
    style Sidecar fill:#fff3e0
    style Init fill:#fce4ec

```

Figure 3-1: Pod anatomy — init containers run to completion in order, then app and native sidecar containers start sharing the pause container's namespaces and volumes.

QoS classes

The kubelet and the eviction manager treat pods differently based on their resource spec:

QoS	Condition	Eviction order	Use for
Guaranteed	Every container has <code>requests == limits</code> for cpu and memory (and both set)	Last to be evicted	Latency-sensitive, stateful workloads (databases, API servers)

QoS	Condition	Eviction order	Use for
Burstable	At least one container has <code>requests != limits</code> or only requests set	Middle — evicted after BestEffort, before Guaranteed	Most microservices — request what you need, allow burst headroom
BestEffort	No container has requests or limits	First to be evicted under pressure	Batch jobs, best-effort workers — never use for serving traffic

Set `requests` to what the pod needs under steady state, `limits` to the maximum you will tolerate before throttling (cpu) or OOMKill (memory). For memory, `limits` should have headroom above `requests` for GC spikes; for CPU, overcommitting `limits` is common (CPU is compressible — throttling slows the pod but does not kill it).

Workload controllers

Pods are ephemeral — they are not resurrected if they die. Controllers provide the semantics that make pods useful.

```

flowchart TB
    Deploy[Deployment
declarative rollout
ReplicaSet + strategy]
    RS[ReplicaSet
ensure N pods
with selector]
    Pod1[Pod]
    Pod2[Pod]
    Pod3[Pod]

    Deploy -->|creates / updates| RS
    RS -->|ensures replicas| Pod1
    RS -->|ensures replicas| Pod2
    RS -->|ensures replicas| Pod3

    STS[StatefulSet
stable identity
ordered, sticky storage]
    STSPod0[Pod-0
pvc-0]
    STSPod1[Pod-1
pvc-1]
    STS --> STSPod0
    STS --> STSPod1

    DS[DaemonSet
one per node
node agent]
    DSPodA[Pod on node-1]
    DSPodB[Pod on node-2]
    DS --> DSPodA
    DS --> DSPodB

    Job[Job
run to completion
completions / parallelism]
    JobPod[Pod]
    restartPolicy OnFailure]
    Job --> JobPod

    style Deploy fill:#e3f2fd
    style RS fill:#fff3e0
    style STS fill:#e8f5e9
    style DS fill:#fce4ec
    style Job fill:#f3e5f5

```

Figure 3-2: Workload controller taxonomy — Deployments manage ReplicaSets for stateless rollouts; StatefulSets provide ordered identity; DaemonSets run per-node agents; Jobs run to completion.

Deployment — stateless rollouts

The workhorse for stateless services. A Deployment owns a ReplicaSet, which owns pods. Updating `spec.template` (new image tag) creates a new ReplicaSet and progressively shifts replicas from old to new according to `strategy`.


```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: production
spec:
  replicas: 6
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
# at most 1 pod unavailable during rollout (availability)
      maxSurge: 2 # at most 2 extra pods above replicas
(capacity headroom)
  selector:
    matchLabels: { app: api }
  template:
    metadata:
      labels: { app: api, version: v2.1 }
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
          labelSelector: { matchLabels: { app: api } }
      containers:
        - name: api
          image: ghcr.io/myorg/api:v2.1@sha256:def...
          ports: [{ containerPort: 8080, name: http }]
          readinessProbe:
            httpGet: { path: /healthz/ready, port: 8080 }
            periodSeconds: 5
          resources:
            requests: { cpu: "500m", memory: "512Mi" }
            limits: { cpu: "1000m", memory: "768Mi" }
          lifecycle:
            preStop: { exec: { command: ["/bin/sh", "-c", "sleep 10"] } }
          terminationGracePeriodSeconds: 30

# Rollout controls
revisionHistoryLimit: 5
progressDeadlineSeconds: 600 # mark rollout as failed if not
complete in 10 min

```

Rollout mechanics:

```
kubectl rollout status deployment/api -n production
kubectl rollout history deployment/api -n production
kubectl rollout undo deployment/api -n production --to-revision=3
kubectl rollout pause deployment/api # pause before next ReplicaSet
scale-up (manual gate)
kubectl rollout resume deployment/api
```

During a rolling update with `maxUnavailable: 1`, `maxSurge: 2` and `replicas: 6`:

1. New ReplicaSet created with 0 pods; old has 6.
2. New ReplicaSet scaled to 2 (surge); 8 total, 6 available (old) + 0 ready (new, not yet passing readiness).
3. As new pods become ready (readinessProbe passes), old ReplicaSet scales down one at a time, new scales up, always keeping ≥ 5 available.
4. At completion: new ReplicaSet has 6, old has 0 (kept for rollback, scaled to 0 — not deleted).

For zero-downtime, readiness probes are essential — the Deployment waits for each new pod to become ready before continuing. Without readiness probes, the rollout proceeds pod-by-pod regardless of whether the new pods can serve traffic.

`Recreate` strategy (delete all old before creating new) is used only when pods cannot coexist (e.g., singleton writers, migration jobs that hold an exclusive lock).

StatefulSet — stable identity and storage

When pods need identity (each replica is distinguishable) and sticky storage (each replica reattaches to the same volume after rescheduling), use StatefulSet. It is the correct controller for databases, queues, and any system where `pod-0` is not interchangeable with `pod-2`.

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: data
spec:
  serviceName: postgres-headless # headless Service that gives each
pod a DNS record
  replicas: 3
  podManagementPolicy: OrderedReady
# create 0 → 1 → 2, delete 2 → 1 → 0 (default)
# podManagementPolicy: Parallel # create all at once (faster, but
no ordering)
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      partition: 0 # update all; set to N to canary
# (only pods >= N update)
  selector:
    matchLabels: { app: postgres }
  template:
    metadata:
      labels: { app: postgres }
    spec:
      terminationGracePeriodSeconds: 30
      containers:
        - name: postgres
          image: postgres:16-bookworm@sha256:abc...
          ports: [{ containerPort: 5432, name: postgres }]
          env:
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
          readinessProbe:
            exec: { command: ["pg_isready", "-U", "postgres"] }
            periodSeconds: 5
          resources:
            requests: { cpu: "1000m", memory: "2Gi" }
            limits: { cpu: "2000m", memory: "4Gi" }
      volumeClaimTemplates: # one PVC per pod, named data-
postgres-0, data-postgres-1, ...
        - metadata: { name: data }
          spec:
            accessModes: ["ReadWriteOnce"]
            storageClassName: gp3-encrypted
            resources: { requests: { storage: 100Gi } }
      ---
# Headless Service – no ClusterIP, DNS returns pod IPs directly
apiVersion: v1

```

```

kind: Service
metadata:
  name: postgres-headless
  namespace: data
spec:
  clusterIP: None
  selector: { app: postgres }
  ports:
    - { port: 5432, name: postgres }

```

Guarantees:

- **Stable network identity** — pods are named `postgres-0`, `postgres-1`, `postgres-2`; DNS records `postgres-0.postgres-headless.data.svc.cluster.local` are stable across rescheduling.
- **Stable storage** — `volumeClaimTemplates` creates one PVC per ordinal; when `postgres-1` is rescheduled to a new node, it reattaches to `data-postgres-1` (the same EBS/GPD volume, subject to `volumeBindingMode` and zone topology).
- **Ordered creation and deletion** — `OrderedReady` ensures `postgres-0` is Running and Ready before `postgres-1` is created — essential for consensus systems (Raft, primary election) that bootstrap from ordinal 0.
- **At-most-one semantics** — `StatefulSet` never runs two pods with the same ordinal simultaneously. Scaling down deletes the highest ordinal first.

DaemonSet — per-node agents

Runs exactly one pod per node (or per node matching a selector). Used for node-level concerns that must run everywhere:

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
  namespace: observability
spec:
  selector: { matchLabels: { app: fluent-bit } }
  updateStrategy:
    type: RollingUpdate
    rollingUpdate: { maxUnavailable: 1 }
  template:
    metadata: { labels: { app: fluent-bit } }
    spec:
      tolerations:
        - operator: Exists          # run on every node, including
tainted control-plane nodes
      containers:
        - name: fluent-bit
          image: fluent/fluent-bit:3.0@sha256:abc...
          volumeMounts:
            - { name: varlog, mountPath: /var/log }
            - { name: varlibdockercontainers, mountPath: /var/lib/docker/
containers, readOnly: true }
          resources: { requests: { cpu: "100m", memory: "128Mi" } }
          volumes:
            - name: varlog
              hostPath: { path: /var/log }
            - name: varlibdockercontainers
              hostPath: { path: /var/lib/docker/containers }

```

Other DaemonSet use cases: CNI plugins (Cilium, Calico), CSI node drivers, `node-exporter`, security agents (Falco), kube-proxy itself.

Job and CronJob — run to completion

Jobs create pods that run until they succeed (`completions` successes). Unlike Deployments, `restartPolicy` is `OnFailure` or `Never` — the pod is not restarted indefinitely.

```

# One-off migration job – runs exactly once to completion
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate-v42
  namespace: production
spec:
  completions: 1
  parallelism: 1
  backoffLimit: 3                # retry at most 3 times on failure
  activeDeadlineSeconds: 600     # kill if not complete in 10 min
  ttlSecondsAfterFinished: 3600  # auto-delete 1 hour after
completion
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: migrate
        image: ghcr.io/myorg/migrate:v42@sha256:abc...
        command: ["/migrate", "--to=v42"]
        resources: { requests: { cpu: "500m", memory: "256Mi" } }
---
# CronJob – run a Job on a schedule (cron expression, UTC)
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-report
  namespace: production
spec:
  schedule: "0 2 * * *"          # 02:00 UTC daily
  concurrencyPolicy: Forbid       # Forbid concurrent runs (vs.
Allow / Replace)
  startingDeadlineSeconds: 300    # skip if not started within 5 min
of schedule (controller was down)
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 5
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: report
            image: ghcr.io/myorg/reporter:v1.42@sha256:def...
            command: ["/generate-report"]

```

For workloads that need ordered, indexed completion (sharded batch processing), use `completionMode: Indexed` — each pod gets `JOB_COMPLETION_INDEX` (0 ... completions-1) and can claim a shard.

Scaling

Horizontal Pod Autoscaler (HPA)

HPA scales `replicas` based on observed metrics (CPU, memory, custom, external). The controller polls metrics every 15s and computes $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times \text{currentMetric} / \text{targetMetric})$.

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 6
  maxReplicas: 50
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 } # 70% of
requests
    - type: Pods
      pods:
        metric: { name: http_requests_per_second }
        target: { type: AverageValue, averageValue: "1000" } #
requires custom metrics API (Prometheus Adapter)
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300 # wait 5 min before scaling
down (avoid flapping)
      policies:
        - type: Percent
          value: 25
          periodSeconds: 60 # at most 25% down per minute
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
        - type: Percent
          value: 100
          periodSeconds: 30 # double at most every 30s
        - type: Pods
          value: 4
          periodSeconds: 30
      selectPolicy: Max # take the more aggressive scale-
up

```

HPA requires a metrics source — `metrics-server` for CPU/memory, or `prometheus-adapter` / `kube-metrics-adapter` for custom metrics (request rate, queue depth, p99 latency). Without metrics, HPA cannot act and the Deployment stays at its current replica count.

Vertical Pod Autoscaler (VPA)

VPA adjusts `requests/limits` rather than replica count — useful for workloads that cannot scale horizontally (single-writer databases, memory-bound batch jobs). It has three modes: `Off` (recommend only), `Initial` (set on creation), `Auto` (evict and recreate pods with new resources).

VPA and HPA on the same metric (CPU/memory) conflict — do not enable both on the same resource for the same Deployment. Use VPA for right-sizing during staging, then disable it and use HPA in production — or use VPA in `Off` mode as a recommendation engine.

KEDA — event-driven autoscaling

KEDA (Kubernetes Event-Driven Autoscaling) extends HPA to queue depth, stream lag, cron schedules, and 60+ scalers (Kafka, SQS, Redis, PostgreSQL, Prometheus). It can scale to zero (HPA cannot — `minReplicas` is at least 1 without KEDA).

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: order-processor
  namespace: production
spec:
  scaleTargetRef: { name: order-processor } # Deployment or
StatefulSet
  minReplicaCount: 0                       # scale to zero when
queue is empty
  maxReplicaCount: 30
  triggers:
    - type: kafka
      metadata:
        bootstrapServers: kafka.production.svc:9092
        consumerGroup: order-processor
        topic: orders
        lagThreshold: "100"
  # scale up when lag > 100 messages per partition
  activationLagThreshold: "10"             # keep at zero until lag
  > 10
```

Node scaling — Cluster Autoscaler and Karpenter

Pod autoscaling is useless if there are no nodes to schedule new pods onto.

- **Cluster Autoscaler (CA)** — watches for unschedulable pods and scales managed node groups (ASGs) up; scales down underutilized nodes. Respects `PodDisruptionBudget` when draining. Slow (minutes — ASG launch time) and tied to pre-defined node groups / instance types.

```
# PodDisruptionBudget — protect availability during voluntary
disruptions (drain, upgrade)
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api-pdb
  namespace: production
spec:
  minAvailable: 4                # always keep 4 pods available (or:
  maxUnavailable: 2)             # always keep 2 pods unavailable)
  selector: { matchLabels: { app: api } }
```

- **Karpenter** — provisions nodes just-in-time with the right instance type for the pending pod (no pre-defined node groups). Consolidates (replaces fragmented nodes with fewer, better-fitting ones) and is significantly faster than CA. Preferred for new clusters.

```

# Karpenter NodePool – provision right-sized nodes on demand
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: api-pool
spec:
  template:
    spec:
      requirements:
        - key: kubernetes.io/arch
          operator: In
          values: ["amd64"]
        - key: karpenter.k8s.aws/instance-category
          operator: In
          values: ["c", "m", "r"]
        - key: topology.kubernetes.io/zone
          operator: In
          values: ["us-east-1a", "us-east-1b", "us-east-1c"]
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: default
      expireAfter: 720h
    limits:
      cpu: 1000
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 30s

```

```

flowchart TB
    Load[Load spike  
queue depth / RPS]  
    HPA[HPA / KEDA  
increase replicas  
15s loop]  
    Pending[Pod Pending  
no node capacity]  
    CA[Karpenter / CA  
provision node  
30s - 3m]  
    Node[New Node  
kubelet ready]  
    Sched[scheduler  
bind pod → node]  
    Pod[Pod Running]

    Load --> HPA --> Pending --> CA --> Node --> Sched --> Pod
    Pod --> Load

    PDB[PDB  
minAvailable]  
    PDB -.->|protects during| CA

    style HPA fill:#e3f2fd
    style CA fill:#fff3e0
    style Pod fill:#e8f5e9
    style PDB fill:#fce4ec

```

Figure 3-3: Autoscaling chain — HPA/KEDA scale pods on metrics, Karpenter/CA scale nodes for pending pods, PDB protects availability during disruption. End-to-end lag is 30s (HPA poll) + node launch (30s Karpenter, 2-3m CA).

Networking

The Kubernetes networking model

Four invariants (from the original design doc, still true):

1. Every pod gets its own IP (no NAT between pods — flat network).
2. Pods can communicate with all other pods without NAT (CNI implements this).
3. Nodes can communicate with all pods without NAT.

4. The IP a pod sees as its own is the same IP others see for it.

This is implemented by the CNI plugin, which allocates a pod CIDR per node and programs routes (BGP, VXLAN overlay, or eBPF) so pod IPs are routable within the cluster.

Services and EndpointSlices

A Service is a stable virtual IP + DNS name that selects a dynamic set of pods via label selector. The mapping from Service to pod IPs is stored in EndpointSlice objects (which replaced Endpoints in 1.21 for scalability — Endpoints was a single object per Service that hit the 1 MiB etcd limit at ~5k endpoints).

```

# ClusterIP – the default; stable IP reachable only inside the cluster
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: production
spec:
  selector: { app: api }
  ports:
    - name: http
      port: 80
      targetPort: 8080          # port on the pod (named port resolves via
pod spec)
      protocol: TCP
      type: ClusterIP          # also: NodePort, LoadBalancer,
ExternalName

# clusterIP: None            # headless – no virtual IP, DNS returns pod
IPs (for StatefulSet)
---
# EndpointSlice – auto-populated by the EndpointSlice controller
# (shown for understanding; you never write these by hand)
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  name: api-xyz
  namespace: production
  labels: { kubernetes.io/service-name: api }
addressType: IPv4
ports:
- { name: http, port: 8080, protocol: TCP }
endpoints:
- addresses: ["10.0.1.3"]
  conditions: { ready: true, serving: true, terminating: false }
  targetRef: { kind: Pod, name: api-7f9c4-abcde, namespace:
production }
- addresses: ["10.0.1.4"]
  conditions: { ready: true, serving: true, terminating: false }

```

Service types:

Type	What it does	When to use
ClusterIP	Stable IP + DNS (<code>api.production.svc.cluster.local</code> → <code>10.96.42.10</code>); kube-proxy/Cilium load-balances to endpoints	Internal service- to-service communication (the default)

Type	What it does	When to use
ClusterIP: None (headless)	No virtual IP; DNS returns the pod IPs directly (A records per pod)	StatefulSet (each pod needs direct address), or client-side load-balancing
NodePort	Allocates a port on every node's IP (30000-32767) that forwards to the ClusterIP	Rarely directly — LoadBalancer and Ingress build on it
LoadBalancer	Provisions a cloud load balancer (ELB/NLB/CLB) that forwards to NodePorts; <code>status.loadBalancer.ingress</code> gets the LB hostname/IP	Exposing a Service to the internet when you need TCP/UDP LB (not HTTP routing)
ExternalName	DNS CNAME (<code>my-db</code> → <code>db.example.com</code>) — no proxying, just DNS	Pointing a cluster-local name at an external service

Readiness gates interaction: a pod is included in EndpointSlices only when `readinessProbe` passes *and* any `readinessGates` (custom conditions like cloud LB attachment) are true. `preStop` + `terminationGracePeriodSeconds` ensure the pod is removed from endpoints *before* `SIGTERM` stops the process — otherwise in-flight requests get `connection refused`.

Ingress and Gateway API

Services are L4 (TCP/UDP). For L7 (HTTP routing, TLS termination, path/header-based routing), use Ingress (the legacy API, still widely deployed) or Gateway API (the successor, designed to fix Ingress's limitations).

```
# Ingress – single manifest, one controller (nginx, ALB, Cilium, etc.)
implements it
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api
  namespace: production
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
spec:
  ingressClassName: nginx
  tls:
  - hosts: [api.example.com]
    secretName: api-tls
  rules:
  - host: api.example.com
    http:
      paths:
      - path: /v1
        pathType: Prefix
        backend: { service: { name: api, port: { number: 80 } } }
      - path: /admin
        pathType: Prefix
        backend: { service: { name: admin, port: { number: 80 } } }
```

Ingress limitations that motivate Gateway API: single resource for all routing (no RBAC separation), annotation-driven configuration (controller-specific, not portable), no first-class support for header/method/mirror/weight-based routing, and no role separation between infra operator and app developer.


```

# Gateway API – role-separated: infra owns Gateway, teams own
HTTPEndpoints
---
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: external
  namespace: infra
spec:
  gatewayClassName: cilium          # or: istio, nginx, gke-l7-gxlb
  listeners:
  - name: https
    port: 443
    protocol: HTTPS
    hostname: "*.example.com"
    tls: { mode: Terminate, certificateRefs: [{ name: wildcard-tls }] }
    allowedRoutes:
      namespaces: { from: All }    # which namespaces may attach
HTTPEndpoints
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: api-route
  namespace: production
spec:
  parentRefs:
  - { name: external, namespace: infra }
  hostnames: ["api.example.com"]
  rules:
  - matches:
    - path: { type: PathPrefix, value: /v1 }
    backendRefs:
    - name: api
      port: 80
      weight: 90
    - name: api-canary
      port: 80
      weight: 10          # 10% canary – no annotation needed
    filters:
    - type: RequestHeaderModifier
      requestHeaderModifier:
        add: [{ name: X-Canary, value: "true" }]
  - matches:
    - path: { type: PathPrefix, value: /admin }
      headers: [{ name: X-Admin-Token, type: Exact, value: secret }]
    backendRefs:
    - { name: admin, port: 80 }

```

Gateway API also defines `GRPCRoute`, `TCPRoute`, `TLSRoute`, and `ReferenceGrant` (explicit cross-namespace backend references) — all as standard, portable APIs rather than controller-specific annotations.

Cluster DNS (CoreDNS)

Every Service gets a DNS record. CoreDNS (a Deployment with 2+ replicas, fronted by a Service at `10.96.0.10` / `kube-dns`) serves the `cluster.local` zone.

Query	Returns
<code>api.production.svc.cluster.local</code>	ClusterIP (<code>10.96.42.10</code>)
<code>api.production.svc.cluster.local</code> (headless)	Pod IPs (<code>10.0.1.3</code> , <code>10.0.1.4</code>)
<code>postgres-0.postgres-headless.data.svc.cluster.local</code>	Pod IP of <code>postgres-0</code> (StatefulSet stable DNS)
<code>api</code> (short name, same namespace)	ClusterIP (via <code>search production.svc.cluster.local</code>)

Each pod's `/etc/resolv.conf` is configured by the kubelet:

```
nameserver 10.96.0.10
search production.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

`ndots:5` means any name with fewer than 5 dots is tried with each search suffix before being treated as absolute — this causes extra DNS queries for external names (`google.com` → `google.com.production.svc.cluster.local` first). For pods that make many external DNS queries, set `dnsConfig: { options: [{ name: ndots, value: "2" }] }` or use fully-qualified names (`google.com.` with trailing dot).

CoreDNS tuning for large clusters — increase replicas, enable `autopath` (reduces search-suffix queries), and watch `coredns_dns_request_duration_seconds` and `coredns_cache_hits_total`:

```
# CoreDNS Corefile (ConfigMap coredns in kube-system) – key plugins
.:53 {
  errors
  health
  kubernetes cluster.local in-addr.arpa ip6.arpa {
    pods insecure
    fallthrough in-addr.arpa ip6.arpa
  }
  prometheus :9153
  forward . /etc/resolv.conf
  cache 30
  loop
  reload
  loadbalance
  autopath @kubernetes    # optimize ndots search
}
```

CNI — pod networking

The CNI plugin is called by the kubelet (via the container runtime) on every pod create/delete. Two architectural families:

Family	How pod IPs are routed	Example	Trade-off
Overlay (VXLAN / Geneve)	Encapsulates pod traffic in VXLAN between nodes; no cloud route changes needed	Flannel, Calico VXLAN, Cilium VXLAN	Works on any cloud/VPC without route configuration; encapsulation overhead (~50 bytes, slight MTU reduction)
Native routing (BGP / cloud routes)	Each node's pod CIDR is advertised via BGP or cloud route table; no encapsulation	Calico BGP, Cilium native routing, AWS VPC CNI (ENI)	No encapsulation overhead; requires BGP peering or cloud route table integration
eBPF	eBPF programs on each node handle forwarding, load-balancing, and policy	Cilium eBPF	Replaces both CNI routing and kube-proxy; best performance, most features (L7 policy, Hubble observability)

AWS's VPC CNI is a special case: it allocates real VPC IPs (ENIs) to pods, so pods are directly addressable within the VPC — no overlay, no BGP — but ENI limits per instance cap pod density.

Choose Cilium for new clusters — its eBPF datapath replaces kube-proxy, implements NetworkPolicy, and provides Hubble (network flow observability) in one component.

NetworkPolicy — microsegmentation

By default, every pod can talk to every other pod — flat, open network. NetworkPolicy is a firewall rule that selects pods and restricts ingress/egress. Without a CNI that enforces policy (Cilium, Calico, Antrea), NetworkPolicy objects exist but do nothing.

```

# Default-deny – once applied, any pod not matched by a policy is
isolated
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: production
spec:
  podSelector: {} # selects ALL pods in this namespace
  policyTypes: [Ingress, Egress]
  # no ingress/egress rules → deny all
---
# Allow api pods to receive from ingress and egress to postgres + DNS +
egress
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-allow
  namespace: production
spec:
  podSelector: { matchLabels: { app: api } }
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
      - namespaceSelector: { matchLabels: { name: infra } } # ingress
        controller
      - podSelector: { matchLabels: { app: api } } # same-app
        traffic
      ports: [{ port: 8080 }]
  egress:
    - to:
      - namespaceSelector: { matchLabels: { name: data } }
        podSelector: { matchLabels: { app: postgres } }
      ports: [{ port: 5432 }]
    - to:
      - namespaceSelector: { matchLabels: { name: kube-system } }
        podSelector: { matchLabels: { k8s-app: kube-dns } }
      ports: [{ port: 53, protocol: UDP }, { port: 53, protocol: TCP }]
    - to:
      - podSelector: { matchLabels: { app: cache } } # Redis in
        same namespace
      ports: [{ port: 6379 }]

```

Patterns for backend systems:

- **Default-deny per namespace** — apply a `podSelector: {}` deny-all in every namespace, then explicitly allow required flows. Without default-deny, a forgotten Service is reachable from any compromised pod.

- **DNS egress** — every pod that needs DNS must have an egress rule to `kube-dns` on port 53, or name resolution fails silently.
- **Cilium ClusterwideNetworkPolicy / CiliumNetworkPolicy** — adds L7 rules (HTTP method/path, Kafka topic, DNS-aware egress to `api.stripe.com`) and `toFQDNs` that standard NetworkPolicy cannot express.

```

flowchart TB
    Internet([Internet])
    GW[Gateway / Ingress]
    TLS_termination[TLS termination]
    L7_routing[L7 routing]
    SVC_API[Service api]
    ClusterIP_10_96_42_10[ClusterIP 10.96.42.10]
    EPS[EndpointSlice  
10.0.1.3, 10.0.1.4]
    Pod1[Pod api-xyz  
10.0.1.3:8080]
    Pod2[Pod api-abc  
10.0.1.4:8080]
    SVC_DB[Service postgres  
headless]
    DB0[Pod postgres-0  
10.0.2.1]
    DB1[Pod postgres-1  
10.0.2.2]
    DNS[(CoreDNS  
10.96.0.10)]

    Internet --> GW --> SVC_API --> EPS --> Pod1
    EPS --> Pod2
    Pod1 -->|NetworkPolicy allow| SVC_DB --> DB0
    Pod2 --> SVC_DB --> DB1
    Pod1 -->|DNS query| DNS
    Pod2 -->|DNS query| DNS

    style GW fill:#e3f2fd
    style SVC_API fill:#fff3e0
    style Pod1 fill:#e8f5e9
    style Pod2 fill:#e8f5e9
    style DNS fill:#fce4ec

```

Figure 3-4: Request path — Gateway/Ingress routes externally, Service load-balances to EndpointSlices, NetworkPolicy gates pod-to-pod traffic, CoreDNS resolves Service names.

Storage

Containers are ephemeral — when a pod is deleted, its writable layer and `emptyDir` volumes disappear. Persistent storage decouples data lifetime from pod lifetime.

Volume types

Type	Lifetime	Scope	Use case
<code>emptyDir</code>	Pod — deleted when pod is deleted	Node-local, shared across containers in the pod	Scratch space, sidecar communication via shared filesystem
<code>configMap</code> / <code>secret</code> / <code>projected</code> / <code>downwardAPI</code>	Pod — projected from API server objects	Node-local (kubelet fetches and mounts)	Configuration, credentials, pod metadata
<code>hostPath</code>	Node — outlives pods, tied to one node	Node-local	DaemonSet agents that read host files (avoid for app data — not portable, not replicated)
<code>persistentVolumeClaim</code>	Independent — outlives pods, managed by CSI	Cluster-wide, bound to a PV (EBS, GCE PD, Ceph, etc.)	Databases, queues, any durable state
<code>ephemeral</code> (generic ephemeral volume)	Pod — like PVC but defined inline in the pod spec	Cluster-wide, provisioned per pod	Per-pod scratch that needs <code>StorageClass</code> features (snapshots, topology) but not persistence beyond the pod

PersistentVolumes, Claims, and StorageClasses

Three objects collaborate:

- **StorageClass** — the template (provisioner, parameters, binding mode, reclaim policy). Written once by the cluster operator.
- **PersistentVolumeClaim (PVC)** — a request for storage ("I need 100 Gi, ReadWriteOnce, from class gp3-encrypted"). Written by the workload author in the pod/StatefulSet spec.
- **PersistentVolume (PV)** — the actual volume (an EBS volume `vol-0abc...`, a GCE PD). Created dynamically by the CSI external-provisioner when a PVC is created, or pre-provisioned by an admin.

```
flowchart LR
    SC[StorageClass  
gp3-encrypted  
provisioner: ebs.csi.aws.com]
    PVC[PVC  
data-postgres-0  
100Gi RW0]
    PV[PV  
pvc-xxxx  
vol-0abc...  
100Gi gp3]
    Pod[Pod postgres-0  
mounts PVC]

    SC -->|provisions| PV
    PVC -->|binds 1:1| PV
    Pod -->|mounts| PVC

    style SC fill:#e3f2fd
    style PVC fill:#fff3e0
    style PV fill:#e8f5e9
    style Pod fill:#fce4ec
```

Figure 3-5: Storage binding — StorageClass provisions a PV for each PVC; the PVC binds 1:1 to a PV; the pod mounts the PVC. Deleting the pod does not delete the PVC or PV.


```

# StorageClass – cluster operator defines once
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3-encrypted
provisioner: ebs.csi.aws.com          # CSI driver name
parameters:
  type: gp3
  encrypted: "true"
  kmsKeyId: arn:aws:kms:us-east-1:123456789:key/abc...
reclaimPolicy: Delete                 # Delete PV when PVC is deleted
(vs. Retain)
volumeBindingMode: WaitForFirstConsumer
# provision only when a pod is scheduled (topology-aware)
allowVolumeExpansion: true           # allow PVC resize without
recreation
---
# PVC – workload author requests storage (or StatefulSet's
volumeClaimTemplates does)
apiVersion: v1
kind: PVC
metadata:
  name: data-postgres-0
  namespace: data
spec:
  accessModes: [ReadWriteOnce]
# RW0 (one node), R0X (many nodes read), RWX (many nodes read-write)
  storageClassName: gp3-encrypted
  resources: { requests: { storage: 100Gi } }
---
# Pod mounts the PVC
apiVersion: v1
kind: Pod
metadata: { name: postgres-0, namespace: data }
spec:
  containers:
    - name: postgres
      image: postgres:16-bookworm@sha256:abc...
      volumeMounts: [{ name: data, mountPath: /var/lib/postgresql/data }]
  volumes:
    - name: data
      persistentVolumeClaim: { claimName: data-postgres-0 }

```

Critical details:

- **volumeBindingMode: WaitForFirstConsumer** — without it, the PV is provisioned immediately in a random zone; the pod may then be scheduled to a different zone and fail to attach (EBS/GPD volumes are zone-local). **WaitForFirstConsumer** delays provisioning until the

scheduler has chosen a node, then provisions in that node's zone. Always use it for zone-local storage.

- **Access modes** — `ReadWriteOnce` (RWO) is the only mode most block storage (EBS, GCE PD) supports. `ReadWriteMany` (RWX) requires a shared filesystem (EFS, GCE Filestore, CephFS, Longhorn) — do not request RWX from an RWO-only `StorageClass`.
- **allowVolumeExpansion** — lets you `kubectl patch pvc data-postgres-0 --patch '{"spec":{"resources":{"requests":{"storage":"200Gi"}}}}'` without recreating the PVC. The CSI driver expands the underlying volume online (for most drivers, without pod restart — the kubelet resizes the filesystem).
- **Reclaim policy** — `Delete` removes the cloud volume when the PVC is deleted (convenient, dangerous — accidental `kubectl delete pvc` destroys data). `Retain` keeps the volume even after PVC deletion (safer for production databases — requires manual cleanup via cloud API).
- **Snapshots** — `VolumeSnapshot` (via the CSI external-snapshotter) creates a crash-consistent snapshot of a PV for backup or cloning:

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata: { name: postgres-snap-2026-01-15, namespace: data }
spec:
  volumeSnapshotClassName: gp3-snap
  source: { persistentVolumeClaimName: data-postgres-0 }
---
# Restore — create a new PVC from the snapshot
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data-postgres-restore, namespace: data }
spec:
  storageClassName: gp3-encrypted
  dataSource:
    name: postgres-snap-2026-01-15
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 100Gi } }
```

CSI architecture

CSI decouples Kubernetes from storage backends. A CSI driver runs as two components:

- **Controller** (Deployment, 2+ replicas, leader-elected) — handles `CreateVolume`, `DeleteVolume`, `CreateSnapshot` — calls the cloud storage API.
- **Node** (DaemonSet, one per node) — handles `NodeStageVolume`, `NodePublishVolume`, `NodeUnpublishVolume` — mounts the volume on the node so the kubelet can bind-mount it into the pod.

```
flowchart TB
    User([User[kubectl apply PVC]])
    API([API[kube-apiserver]])
    Prov([Prov[external-provisioner]])
    CSI_C[CSI_C[CSI Controller  
Deployment]]
    Cloud[(Cloud[(Cloud Storage API  
EBS / GCE PD)])]
    Sched[sched[scheduler]]
    Kubelet[kubelet]
    CSI_N[CSI_N[CSI Node  
DaemonSet]]
    Pod([Pod])

    User --> API
    API --> Prov --> CSI_C --> Cloud
    API --> Sched
    Sched --> Kubelet --> CSI_N --> Pod
    CSI_N -. "NodeStage + NodePublish" .-> Cloud

    style Prov fill:#e3f2fd
    style CSI_C fill:#fff3e0
    style CSI_N fill:#e8f5e9
    style Cloud fill:#fce4ec
```

Figure 3-6: CSI volume lifecycle — external-provisioner calls the controller to create the cloud volume; after scheduling, the node driver mounts it for the pod.

StatefulSet storage revisited

The reason `StatefulSet` + `volumeClaimTemplates` exists is that Deployments cannot safely own PVCs — if a Deployment scales from 3 to 4, which PVC does the new pod get? `StatefulSet` solves this with ordinal-indexed PVCs (`data-postgres-0`, `data-postgres-1`, ...) that are created alongside their pod and never reassigned. Deleting a `StatefulSet` does *not* delete its PVCs (you must delete them explicitly), so data survives even if the controller is removed — a safety property for databases.

For local storage (NVMe on the node, for high-throughput caches or scratch), use the local CSI driver or `hostPath` with `volumeBindingMode: WaitForFirstConsumer` and `nodeAffinity` — but understand that local volumes die with the node and are not replicated.

Putting it together: a production workload

```

# Complete production Deployment – stateless API with HPA, PDB,
NetworkPolicy, and Gateway API
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: production
  labels: { app: api, version: v2.1 }
spec:
  replicas: 6
  strategy: { type: RollingUpdate, rollingUpdate: { maxUnavailable: 1,
maxSurge: 2 } }
  selector: { matchLabels: { app: api } }
  template:
    metadata: { labels: { app: api, version: v2.1 } }
    spec:
      securityContext: { runAsUser: 65532, runAsNonRoot: true,
seccompProfile: { type: RuntimeDefault } }
      topologySpreadConstraints:
        - { maxSkew: 1, topologyKey: topology.kubernetes.io/zone,
whenUnsatisfiable: DoNotSchedule, labelSelector: { matchLabels: { app:
api } } }
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - { weight: 100, podAffinityTerm: { labelSelector: {
matchLabels: { app: api } }, topologyKey: kubernetes.io/hostname } }
      containers:
        - name: api
          image: ghcr.io/myorg/api:v2.1@sha256:def...
          ports: [{ containerPort: 8080, name: http } ]
          env:
            - { name: POD_NAME, valueFrom: { fieldRef: { fieldPath: metadat
a.name } } }
          resources: { requests: { cpu: "500m", memory: "512Mi" },
limits: { cpu: "1000m", memory: "768Mi" } }
          startupProbe: { httpGet: { path: /healthz/startup, port:
http }, periodSeconds: 5, failureThreshold: 30 }
          livenessProbe: { httpGet: { path: /healthz/live, port:
http }, periodSeconds: 10, failureThreshold: 3 }
          readinessProbe: { httpGet: { path: /healthz/ready, port:
http }, periodSeconds: 5, failureThreshold: 2 }
          lifecycle: { preStop: { exec: { command: ["/bin/sh", "-c", "sle
ep 10"] } } }
          securityContext: { allowPrivilegeEscalation: false,
readOnlyRootFilesystem: true, capabilities: { drop: [ALL] } }
          terminationGracePeriodSeconds: 30
---
apiVersion: v1

```

```

kind: Service
metadata: { name: api, namespace: production }
spec:
  selector: { app: api }
  ports: [{ name: http, port: 80, targetPort: 8080 }]
  type: ClusterIP
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: { name: api-route, namespace: production }
spec:
  parentRefs: [{ name: external, namespace: infra }]
  hostnames: [api.example.com]
  rules:
    - matches: [{ path: { type: PathPrefix, value: / } }]
      backendRefs: [{ name: api, port: 80 }]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: api, namespace: production }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: api }
  minReplicas: 6
  maxReplicas: 50
  metrics:
    - { type: Resource, resource: { name: cpu, target: { type: Utilization, averageUtilization: 70 } } }
  behavior:
    scaleDown: { stabilizationWindowSeconds: 300, policies: [{ type: Percent, value: 25, periodSeconds: 60 } ] }
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: api-pdb, namespace: production }
spec: { minAvailable: 4, selector: { matchLabels: { app: api } } }
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: api-allow, namespace: production }
spec:
  podSelector: { matchLabels: { app: api } }
  policyTypes: [Ingress, Egress]
  ingress:
    - from: [{ namespaceSelector: { matchLabels: { name: infra } } }]
      ports: [{ port: 8080 }]
  egress:
    - to: [{ namespaceSelector: { matchLabels: { name: data } } },
podSelector: { matchLabels: { app: postgres } } }]
      ports: [{ port: 5432 }]
    - to: [{ namespaceSelector: { matchLabels: { name: kube-system } } },

```

```
podSelector: { matchLabels: { k8s-app: kube-dns } } }]  
ports: [{ port: 53, protocol: UDP }]
```

Key takeaways

- A Pod is the schedulable unit — containers in a pod share a network namespace (one IP), IPC, and volumes via the pause container. Init containers run sequentially to completion before app containers start; native sidecar containers (`restartPolicy: Always` , 1.29+) stay running alongside the main container.
- Resource requests drive scheduling and QoS class (Guaranteed when `requests == limits` , Burstable otherwise, BestEffort when unset); limits are cgroup hard caps. Always set requests; set limits with headroom for memory spikes. Separate liveness (restart) from readiness (remove from endpoints) — conflating them causes restart storms.
- Workload controllers encode intent: Deployment for stateless rollouts (ReplicaSet + rolling update with `maxUnavailable / maxSurge`), StatefulSet for stable identity and sticky storage (ordinal pods + `volumeClaimTemplates` + headless Service), DaemonSet for per-node agents, Job/CronJob for run-to-completion. Choose by whether replicas are interchangeable.
- Autoscaling is three levels: HPA (pod count on metrics, needs metrics-server or Prometheus Adapter), KEDA (event-driven, scales to zero, 60+ scalers), and node scaling (Karpenter preferred over Cluster Autoscaler — faster, no pre-defined node groups). PDBs (`minAvailable / maxUnavailable`) protect availability during voluntary disruptions.
- Services are stable virtual IPs + DNS names backed by EndpointSlices (not the legacy Endpoints object). Types are ClusterIP (internal), headless (direct pod IPs), NodePort (node port), LoadBalancer (cloud LB), ExternalName (CNAME). Gateway API (Gateway + HTTPRoute/GRPCRoute) replaces Ingress for L7 routing with RBAC-separated, portable, annotation-free configuration.
- Cluster DNS is CoreDNS at `10.96.0.10` — watch `ndots:5` search-suffix overhead for external names. CNI (Cilium eBPF preferred) implements the flat pod network (overlay vs. native routing).

NetworkPolicy is deny-by-default per namespace — apply a default-deny policy and explicitly allow required flows, including DNS egress.

- Storage is StorageClass (template) → PVC (request) → PV (volume) → pod mount, implemented by CSI (controller + node DaemonSet). Always use `volumeBindingMode: WaitForFirstConsumer` for zone-local block storage to avoid cross-zone attach failures. StatefulSet + `volumeClaimTemplates` is the correct pattern for databases — ordinal-indexed PVCs survive pod rescheduling and StatefulSet deletion.

Further reading

- Kubernetes Concepts — Workloads (Pods, Deployments, StatefulSets, DaemonSets, Jobs, CronJobs). <https://kubernetes.io/docs/concepts/workloads/>
- Kubernetes Concepts — Services, Networking (Service, Ingress, Gateway API, Network Policies, DNS). <https://kubernetes.io/docs/concepts/services-networking/>
- Kubernetes Concepts — Storage (Volumes, Persistent Volumes, Storage Classes, CSI). <https://kubernetes.io/docs/concepts/storage/>
- Kubernetes Reference — Gateway API (v1.1). <https://gateway-api.sigs.k8s.io/>
- CoreDNS documentation — plugins, Corefile, tuning. <https://coredns.io/manual/toc/>
- Cilium documentation — CNI, eBPF datapath, NetworkPolicy, Hubble. <https://docs.cilium.io/>
- Karpenter documentation — NodePools, consolidation, drift. <https://karpenter.sh/docs/>
- KEDA documentation — scalers, ScaledObjects, scaling to zero. <https://keda.sh/docs/>
- Ian Lewis et al. — *Kubernetes Best Practices* (O'Reilly, 2019) — workload patterns and operational guidance.

- Brendan Burns et al. — *Kubernetes: Up and Running*, 3rd ed. (O'Reilly, 2022) — workloads, networking, and storage walkthroughs.

Chapter 4 — Infrastructure as Code

What this chapter covers. ClickOps — provisioning infrastructure by hand in a console — does not survive contact with a team of twenty engineers deploying fifty times a day. Infrastructure as Code (IaC) replaces imperative clicks with declarative, version-controlled definitions that can be reviewed, tested, and reproducibly applied. This chapter builds the mental model and practical toolkit for managing cloud infrastructure as software: the declarative contract and convergence loop that underpins every IaC tool; Terraform and OpenTofu in depth — HCL, providers, resources, state, modules, and backends — with production-grade configurations you can apply to a real AWS account; the CI/CD pipeline that turns `terraform plan` into a safe, auditable change workflow (Atlantis, Terraform Cloud, Spacelift, GitHub Actions); drift detection, policy-as-code, and testing; secrets handling; state refactoring and import; and how IaC fits alongside configuration management (Ansible), cloud CDKs (Pulumi, AWS CDK), and Kubernetes-native provisioning (Crossplane). Every abstraction is grounded in configs that `plan` cleanly and failure modes that have taken down production when ignored.

Learning goals — after this chapter you should be able to:

- Articulate why declarative IaC dominates imperative scripting for fleet-scale infrastructure, and explain idempotency, convergence, and drift in precise terms.
- Write, structure, and operate Terraform/OpenTofu configurations — HCL, providers, resources, data sources, variables, outputs, locals, `count / for_each`, dynamic blocks, and modules — that pass `validate` and produce minimal, predictable plans.
- Design remote state with locking (S3 + DynamoDB, GCS, Terraform Cloud), explain the state file's role as a mapping between desired and real, and operate state safely (backups, `moved` blocks, `import`, `state mv/rm`).
- Build an IaC delivery pipeline — branch → plan → policy check → approval → apply — with drift detection, cost estimation, and

automated remediation, and choose between directory-per-environment vs. workspace isolation.

- Enforce policy-as-code (OPA/Conftest, Sentinel, Checkov) and test IaC (terraform validate, tfint, Terratest, ephemeral test workspaces) before it reaches production.
- Handle secrets, sensitive outputs, and provider credentials without leaking them into state or logs, and refactor live infrastructure without downtime.

Boundary note. This chapter is the *provisioning* layer — how you declare and converge the infrastructure that workloads run on. Volume 2 — Operating Systems — covers the kernel mechanisms those machines expose; Chapter 1 of this volume covers container images and runtimes that run atop them; Chapters 2-3 cover Kubernetes as the scheduler for those containers. Volume 6 — Distributed Systems — provides the consistency theory behind state backends; Volume 8 — APIs — covers the provider APIs that Terraform calls. For supply-chain concerns specific to IaC modules and providers (typosquatting, compromised registry artifacts), see the Companion Series, Book 2, Chapter 3 and Book 6, Chapter 8.

Why infrastructure as code

From ClickOps to GitOps

Every infrastructure change is a state transition on a distributed system (the cloud control plane). Doing it by hand in a console has four fatal properties at scale:

ClickOps	IaC
Undocumented — the only record is CloudTrail, if enabled	Every change is a commit with author, diff, and review
Unrepeatable — the next environment is rebuilt from memory	<code>terraform apply</code> reproduces the same graph from the same commit
Unreviewable — no diff before the change	<code>plan</code> shows the exact diff; policy gates block violations

ClickOps	IaC
Undetectable drift — a manual tweak diverges silently	Drift detection re-plans on schedule and alerts or re-converges

The fix is to treat infrastructure definitions as software: versioned in Git, reviewed in pull requests, tested in CI, and applied through an automated pipeline that enforces the same guarantees as application delivery.

Declarative vs. imperative vs. idempotent

- **Imperative:** "Run these steps in order" (`aws ec2 run-instances ...; aws ec2 create-tags ...`). If a step fails halfway, re-running may duplicate or error. Order matters.
- **Declarative:** "The world should look like this" (`resource "aws_instance" "api" { instance_type = "m5.large" }`). The tool computes the diff between desired and actual and issues the minimal API calls to converge. Re-applying the same declaration is a no-op.
- **Idempotent:** Applying the operation N times has the same effect as applying it once. Declarative IaC is idempotent by construction; imperative scripts must be written carefully to be.

Terraform, OpenTofu, Pulumi (in declarative mode), CloudFormation, and Crossplane are declarative. Ansible is mostly imperative (ordered tasks) with idempotent modules. Shell scripts and AWS CLI invocations are imperative.

The convergence loop

Every declarative IaC tool implements the same loop, whether it calls it reconciliation, convergence, or apply:

```

flowchart LR
    A["Desired State  
(Git: .tf / .yaml)"] --> B["Plan  
diff desired vs actual"]
    B --> C{"Changes?"}
    C -->|No| D["No-op  
already converged"]
    C -->|Yes| E["Approval / Policy Gate"]
    E --> F["Apply  
call provider APIs"]
    F --> G["Actual State  
(cloud + state file)"]
    G -->|Drift detection  
(scheduled re-plan)| B
    G -->|Persist| H["State Backend  
S3 / GCS / TFC"]

    style A fill:#e3f2fd
    style G fill:#fff3e0
    style H fill:#fce4ec

```

Figure 4-1: The declarative convergence loop — desired state in Git is diffed against actual state (cloud + state file); the plan is gated and applied; drift detection re-enters the loop on a schedule.

Drift is the inevitable divergence between desired (Git) and actual (cloud) caused by manual changes, provider-side defaults, or eventual consistency. Without scheduled re-planning, drift accumulates silently until the next apply produces a surprising, large diff.

Terraform and OpenTofu: architecture

Core, providers, and state

Terraform (HashiCorp, BSL) and OpenTofu (Linux Foundation fork, MPL-2.0, drop-in replacement as of 2024) share the same architecture. The binary is split into:

- **Core** — parses HCL, builds a dependency graph, evaluates expressions, and orchestrates plan/apply. Core has no cloud knowledge.
- **Providers** — plugins (separate binaries, e.g., registry.opentofu.org/hashicorp/aws) that implement CRUD for a specific API. Each

resource type maps to one or more API calls. Providers are versioned independently and pinned via `required_providers`.

- **State** — a JSON file (`terraform.tfstate`) that maps each resource address to the real cloud object ID and its last-known attributes. State is the *only* record linking `aws_instance.api` in config to `i-0a1b2c3d4e5f` in AWS. Lose it, and Terraform forgets what it manages.

```
flowchart TB
    subgraph Config["Configuration (HCL)"]
        R["resource / data / module / variable"]
    end
    end
    subgraph Core["Terraform / OpenTofu Core"]
        Parse["Parse HCL"] --> Graph["Build DAG"]
        Graph --> Eval["Evaluate + Plan"]
        Eval --> Apply["Apply (walk DAG)"]
    end
    end
    subgraph Providers["Providers (gRPC plugins)"]
        AWS["hashicorp/aws"]
        GCP["hashicorp/google"]
        K8s["hashicorp/kubernetes"]
    end
    end
    subgraph State["State Backend"]
        S3["S3 + DynamoDB"]
    end
    or GCS / TFC / local
    end
    subgraph Cloud["Cloud APIs"]
        EC2["EC2 / VPC / RDS / ..."]
    end
    end

    R --> Parse
    Apply <--> |"CRUD"| Providers
    Providers <--> |"API calls"| Cloud
    Eval <--> |"read/write"| S3
    Apply <--> |"persist"| S3

    style Core fill:#e3f2fd
    style Providers fill:#fff3e0
    style State fill:#fce4ec
    style Cloud fill:#e8f5e9
```

Figure 4-2: Terraform/OpenTofu architecture — core builds a DAG from HCL, providers translate resource operations to cloud API calls, and the state backend persists the desired↔actual mapping.

HCL essentials

HCL (HashiCorp Configuration Language) is JSON-superset with expressions, functions, and meta-arguments. Every `.tf` file in a directory is merged into one configuration.

```
# versions.tf – pin everything; unpinned versions are a supply-chain
risk
terraform {
  required_version = ">= 1.8"
  required_providers {
    aws = {
      source = "registry.opentofu.org/hashicorp/aws"
      version = "~> 5.60"
    }
    random = {
      source = "registry.opentofu.org/hashicorp/random"
      version = "~> 3.6"
    }
  }
}

# Remote state – S3 with DynamoDB locking (see State section)
backend "s3" {
  bucket      = "myorg-tofu-state"
  key         = "prod/network/terraform.tfstate"
  region      = "us-east-1"
  dynamodb_table = "tofu-state-lock"
  encrypt     = true
}

provider "aws" {
  region = var.region
  default_tags {
    tags = {
      Project      = "platform"
      Environment = var.environment
      ManagedBy    = "opentofu"
    }
  }
}
```

```

# variables.tf – typed inputs with validation
variable "region" {
  type      = string
  default    = "us-east-1"
  description = "AWS region for all resources"
}

variable "environment" {
  type      = string
  description = "Deployment environment"
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

variable "vpc_cidr" {
  type      = string
  default    = "10.0.0.0/16"
  validation {
    condition     = can(cidrhost(var.vpc_cidr, 0))
    error_message = "Must be a valid CIDR block."
  }
}

variable "az_count" {
  type      = number
  default    = 3
  validation {
    condition     = var.az_count >= 2 && var.az_count <= 4
    error_message = "Use 2-4 AZs."
  }
}

# locals.tf – derived values, not inputs
locals {
  azs = slice(data.aws_availability_zones.available.names, 0, var.az_count)
  common_tags = {
    Environment = var.environment
    ManagedBy   = "opentofu"
  }
}

data "aws_availability_zones" "available" {
  state = "available"
}

output "vpc_id" {

```



```
    description = "ID of the created VPC"
    value       = aws_vpc.main.id
  }

  output "private_subnet_ids" {
    value = aws_subnet.private[*].id
  }
```

```

# network.tf – the actual infrastructure
resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true
  tags = merge(local.common_tags, { Name = "${var.environment}-main" })
}

resource "aws_subnet" "private" {
  count          = var.az_count
  vpc_id         = aws_vpc.main.id
  cidr_block     = cidrsubnet(var.vpc_cidr, 8, count.index)
  availability_zone = local.azs[count.index]
  tags = merge(local.common_tags, {
    Name = "${var.environment}-private-${local.azs[count.index]}"
    Type = "private"
  })
}

resource "aws_subnet" "public" {
  count          = var.az_count
  vpc_id         = aws_vpc.main.id
  cidr_block     = cidrsubnet(var.vpc_cidr, 8, count.index + v
ar.az_count)
  availability_zone = local.azs[count.index]
  map_public_ip_on_launch = false
  tags = merge(local.common_tags, {
    Name = "${var.environment}-public-${local.azs[count.index]}"
    Type = "public"
  })
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags = merge(local.common_tags, { Name = "${var.environment}-igw" })
}

resource "aws_eip" "nat" {
  count = var.az_count
  domain = "vpc"
  tags = merge(local.common_tags, { Name = "${var.environment}-nat-${count.index}" })
}

resource "aws_nat_gateway" "main" {
  count          = var.az_count
  allocation_id = aws_eip.nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id
  tags          = merge(local.common_tags, { Name = "${

```

```

{var.environment}-nat-${count.index}" })
  depends_on    = [aws_internet_gateway.main]
}

resource "aws_route_table" "private" {
  count = var.az_count
  vpc_id = aws_vpc.main.id
  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main[count.index].id
  }
  tags = merge(local.common_tags, { Name = "${var.environment}-private-
rt-${count.index}" })
}

resource "aws_route_table_association" "private" {
  count          = var.az_count
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private[count.index].id
}

```

Meta-arguments: count, for_each, lifecycle, depends_on

```

# for_each – preferred over count when managing named instances
variable "services" {
  type = map(object({
    port      = number
    health_path = string
    cpu       = number
    memory    = number
  }))
  default = {
    api    = { port = 8080, health_path = "/healthz", cpu = 512,
memory = 1024 }
    worker = { port = 8081, health_path = "/healthz", cpu = 256,
memory = 512 }
  }
}

resource "aws_security_group" "service" {
  for_each = var.services
  name     = "${var.environment}-${each.key}-sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    from_port = each.value.port
    to_port   = each.value.port
    protocol  = "tcp"
    cidr_blocks = [var.vpc_cidr]
    description = "Allow ${each.key} on ${each.value.port}"
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = local.common_tags
}

# dynamic blocks – generate repeated nested blocks
variable "ingress_rules" {
  type = list(object({
    port      = number
    protocol   = string
    cidr_blocks = list(string)
    description = string
  }))
  default = []
}

resource "aws_security_group" "dynamic_example" {
  name = "${var.environment}-dynamic-sg"

```

```

vpc_id = aws_vpc.main.id

dynamic "ingress" {
  for_each = var.ingress_rules
  content {
    from_port    = ingress.value.port
    to_port      = ingress.value.port
    protocol     = ingress.value.protocol
    cidr_blocks  = ingress.value.cidr_blocks
    description  = ingress.value.description
  }
}

# lifecycle – control replacement and drift behavior
resource "aws_instance" "api" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "m7i.large"
  subnet_id    = aws_subnet.private[0].id

  lifecycle {
    create_before_destroy = true          # for ASG/launch template
    prevent_destroy       = true          # guard prod DB/infra from
accidental destroy
    ignore_changes        = [ami]         # AMI updated out-of-band by
image pipeline
    replace_triggered_by  = [aws_launch_template.api.id] # force
replacement when LT changes
  }
}

```

Rules of thumb:

- Prefer `for_each` over `count` when instances have identity (named services, per-AZ resources). `count` re-indexes on removal, causing spurious replacements; `for_each` keys are stable.
- Use `lifecycle.ignore_changes` sparingly — it hides drift. Prefer fixing the source of drift.
- `depends_on` is rarely needed — Terraform infers dependencies from interpolations (`aws_vpc.main.id`). Use it only for hidden dependencies (NAT gateway needs IGW attachment to complete first).

State: the source of truth you must not lose

Why state exists

Cloud APIs are eventually consistent and have no declarative "desired state" — they only know the current state. State bridges the gap: it records what Terraform *last* created so the next plan can diff desired vs. actual. Without it, Terraform would have to list every cloud object and guess which ones it owns.

State is JSON, roughly:

```
{
  "version": 4,
  "resources": [{
    "mode": "managed", "type": "aws_vpc", "name": "main",
    "provider": "registry.opentofu.org/hashicorp/aws",
    "instances": [{ "attributes": { "id": "vpc-0a1b2c", "cidr_block": "
10.0.0.0/16" }}]
  }]
}
```

Remote backends and locking

Local state (`terraform.tfstate` on disk) is unsuitable for teams — no sharing, no locking, secrets in plaintext on a laptop. Production always uses a remote backend with locking:

```

# S3 + DynamoDB (AWS) – the most common pattern
terraform {
  backend "s3" {
    bucket      = "myorg-tofu-state"
    key         = "prod/network/terraform.tfstate"
    region     = "us-east-1"
    dynamodb_table = "tofu-state-lock" # conditional writes for
locking
    encrypt      = true                # SSE-S3 or SSE-KMS
  }
}

# GCS (GCP) – locking is native via object preconditions
terraform {
  backend "gcs" {
    bucket = "myorg-tofu-state"
    prefix = "prod/network"
  }
}

# Terraform Cloud / Spacelift – managed state, run history, policy
gates
terraform {
  cloud {
    organization = "myorg"
    workspaces { name = "network-prod" }
  }
}

```

Backend	Locking	Versioning	Encryption	Notes
S3 + DynamoDB	DynamoDB conditional write	S3 versioning	SSE-S3 / KMS	Most common on AWS; set <code>encrypt = true</code> + bucket versioning
GCS	Native (preconditions)	GCS versioning	Google- managed / CMEK	No extra lock table needed
Terraform Cloud	Managed	Managed	Managed	Adds run history, variables, policy sets, drift detection

Backend	Locking	Versioning	Encryption	Notes
Local	None	None	None	Dev only; never for shared stacks

State locking prevents two `apply` runs from concurrently mutating the same state and corrupting it. If a run crashes holding the lock, `terraform force-unlock <lock-id>` releases it — but only after confirming no other run is active.

State operations

```
# Inspect state
tofu state list
# aws_vpc.main
# aws_subnet.private[0]
# aws_subnet.private[1]

tofu state show aws_vpc.main

# Import an existing resource created outside Terraform (e.g., ClickOps VPC)
tofu import aws_vpc.main vpc-0a1b2c3d4e5f

# Modern import block (declarative, reviewable in PR) – preferred over CLI import
# imports.tf
import {
  to = aws_vpc.main
  id = "vpc-0a1b2c3d4e5f"
}

# Move/rename without destroying – e.g., refactoring into a module
tofu state mv aws_subnet.private module.network.aws_subnet.private

# Declarative move (Terraform 1.1+ / OpenTofu) – reviewable, no CLI needed
moved {
  from = aws_subnet.private
  to   = module.network.aws_subnet.private
}

# Remove from state without destroying the real object (hand off to another stack)
tofu state rm aws_eip.nat[2]

# Pull/push for disaster recovery (rare – prefer backend replication)
tofu state pull > backup.tfstate
tofu state push backup.tfstate
```

Treat state as critical data: enable versioning on the bucket, replicate cross-region, restrict IAM to the CI role only, and never commit it to Git. State contains secrets in plaintext (see Secrets section).

Workspaces vs. directory-per-environment

Two patterns for managing dev / staging / prod :

- **Directory-per-environment** (recommended) — `envs/dev/` , `envs/prod/` each with their own backend key and `terraform.tfvars` . Clear isolation, different IAM roles per env, no risk of `terraform workspace select` mistakes. Each directory is an independent root module.
- **Workspaces** (`terraform workspace new prod`) — same config, different state file keyed by workspace name. Convenient but error-prone: a forgotten `workspace select` applies prod config to dev state. Suitable only for ephemeral, identical environments (per-PR preview).

```
repo/
├── modules/
│   ├── network/      # reusable VPC module
│   ├── compute/     # ASG / ECS module
│   └── database/     # RDS module
├── envs/
│   ├── dev/
│   │   ├── main.tf   # calls modules/network with dev vars
│   │   ├── variables.tf
│   │   └── terraform.tfvars # dev-specific values
│   ├── staging/
│   │   └── ...
│   └── prod/
│       ├── main.tf
│       └── terraform.tfvars
└── imports.tf        # one-off import blocks, removed after apply
```

Modules: packaging reusable infrastructure

A module is a directory of `.tf` files with inputs (`variable`), outputs (`output`), and resources. Every root configuration is itself a module.

```

# modules/network/variables.tf
variable "environment" { type = string }
variable "vpc_cidr"    { type = string }
variable "az_count"    { type = number default = 3 }
variable "enable_nat"  { type = bool   default = true }

# modules/network/main.tf (the VPC + subnets from above,
# ... (aws_vpc, aws_subnet, aws_nat_gateway, etc.)

# modules/network/outputs.tf
output "vpc_id" { value = aws_vpc.main.id }
output "private_subnet_ids" { value = aws_subnet.private[*].id }
output "public_subnet_ids" { value = aws_subnet.public[*].id }
output "nat_gateway_ids" { value = aws_nat_gateway.main[*].id }

# envs/prod/main.tf – consuming the module
module "network" {
  source      = "../../modules/network"
  environment = "prod"
  vpc_cidr    = "10.0.0.0/16"
  az_count    = 3
  enable_nat  = true
}

module "database" {
  source      = "../../modules/database"
  vpc_id      = module.network.vpc_id
  subnet_ids  = module.network.private_subnet_ids
  # ...
}

```

Module sources:

```

module "vpc" {
  source = "terraform-aws-modules/vpc/aws" # Terraform Registry
  version = "~> 5.0"
}

module "network" {
  source = "git:https://github.com/myorg/tf-modules.git//network?
ref=v2.1.0" # Git
}

module "network" {
  source = "../../modules/network" # local path
}

```

Best practices:

- Pin every external module to a version or commit SHA — unpinned `source = "github.com/.../network"` fetches `main` and breaks reproducibly.
 - Keep modules small and composable (network, compute, database) rather than one mega-module.
 - Expose only necessary variables; validate them. Use `nullable = false` and `validation` blocks.
 - Publish internal modules to a private registry (Terraform Cloud private registry, or Git tags) so consumers get versioned, discoverable artifacts.
-

The IaC delivery pipeline

Plan → policy → approve → apply

The production pipeline mirrors application CI/CD but with an extra safety property: the plan is the change, and it must be reviewed before apply. No `apply` without a prior `plan` from the same commit.

```

flowchart LR
    A["Git Push / PR"] --> B["tofu fmt -check  
tofu validate  
tflint / checkov"]
    B --> C["tofu plan  
(save plan file)"]
    C --> D["Policy Gate  
OPA / Sentinel  
cost estimate"]
    D -->|"pass"| E["Human Approval  
(PR review / env gate)"]
    D -->|"fail"| X["Block + Comment"]
    E --> F["tofu apply  
(plan file)"]
    F --> G["State Backend  
persist"]
    G --> H["Drift Detection  
(scheduled re-plan)"]
    H -->|"drift found"| I["Alert / Auto-remediate"]
    X --> A

    style C fill:#e3f2fd
    style D fill:#fff3e0
    style F fill:#e8f5e9
    style H fill:#fce4ec

```

Figure 4-3: The IaC delivery pipeline — every commit is formatted, validated, planned, policy-checked, approved, and applied from the saved plan file; drift detection re-plans on a schedule.

```

# .github/workflows/tofu-plan.yaml – plan on PR, apply on merge to main
name: tofu-plan-apply
on:
  pull_request:
    paths: ["envs/**", "modules/**"]
  push:
    branches: [main]
    paths: ["envs/**", "modules/**"]

jobs:
  plan:
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
      id-token: write          # OIDC to AWS – no static credentials
    strategy:
      matrix:
        env: [dev, staging, prod]
    steps:
      - uses: actions/checkout@v4
      - uses: opentofu/setup-opentofu@v1
        with: { tofu_version: "1.8.5" }
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/tofu-plan-${{ matrix.env }}
          aws-region: us-east-1

      - run: tofu fmt -check -recursive
      - run: tofu init -backend-config="key=${{ matrix.env }}"
      terraform.tfstate"
      - run: tofu validate
        working-directory: envs/${{ matrix.env }}
      - run: tflint --init && tflint
        working-directory: envs/${{ matrix.env }}
      - run: tofu plan -out=tfplan -input=false
        working-directory: envs/${{ matrix.env }}
      # Post plan as PR comment for review
      - uses: borcherio/terraform-plan-comment@v2
        with:
          working-directory: envs/${{ matrix.env }}
          planfile: tfplan

  apply:
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    needs: [] # or gate on plan success via workflow_run

```

```

environment: prod    # GitHub environment protection – requires
approval
permissions: { contents: read, id-token: write }
steps:
  - uses: actions/checkout@v4
  - uses: opentofu/setup-opentofu@v1
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123456789012:role/tofu-apply-
prod
      aws-region: us-east-1
  - run: tofu init -backend-config="key=prod/terraform.tfstate"
    working-directory: envs/prod
  - run: tofu apply -input=false -auto-approve tfplan
    working-directory: envs/prod

```

Atlantis, Terraform Cloud, Spacelift — managed runners

Running `plan` / `apply` on a developer laptop is unsafe (stale state, leaked credentials, no audit). Managed runners enforce the pipeline:

Runner	Model	Drift detection	Policy	Notes
Atlantis	Self-hosted, runs as a GitHub/ GitLab webhook; <code>plan</code> on PR, <code>apply</code> via comment	No (add cron)	Conftest/ Checkov via custom workflow	Open-source, you operate it; needs locking
Terraform Cloud (TFC)	SaaS; VCS-driven runs, remote execution, private registry	Native (health checks)	Sentinel (paid) + OPA	Remote state + run history built-in
Spacelift	SaaS; stack-per-directory, dependencies between stacks	Native	OPA natively, drift remediation	Strong multi-stack orchestration
Digger / Terrateam	GitHub Actions-native; <code>plan</code> / <code>apply</code> in Actions	Via Actions cron	OPA/Checkov	Lightweight, no extra service

Atlantis workflow in `atlantis.yaml`:


```
version: 3
projects:
  - name: network-prod
    dir: envs/prod
    workspace: default
    workflow: prod
    autoplan:
      when_modified: ["*.tf", "*.tfvars", "../..modules/network/**/
*.tf"]
workflows:
  prod:
    plan:
      steps:
        - run: tofu fmt -check
        - init
        - plan
        - run: conftest test --policy policy/ plan.json
    apply:
      steps: [apply]
```

Drift detection and reconciliation

Drift happens. Someone changes a security group in the console during an incident and forgets to revert; a provider default changes; an AWS-managed update modifies a resource out-of-band. Without detection, the next `plan` is a surprise.

```

# Scheduled drift detection – run plan on cron, alert if non-empty
# GitHub Actions cron (daily 06:00 UTC)
# .github/workflows/drift-detection.yaml
name: drift-detection
on:
  schedule: [{ cron: "0 6 * * *" }]
  workflow_dispatch:
jobs:
  drift:
    runs-on: ubuntu-latest
    strategy: { matrix: { env: [dev, staging, prod] } }
    steps:
      - uses: actions/checkout@v4
      - uses: opentofu/setup-opentofu@v1
      - uses: aws-actions/configure-aws-credentials@v4
        with: { role-to-assume: arn:aws:iam::123456789012:role/tofu-
plan-${{ matrix.env }}, aws-region: us-east-1 }
      - run: tofu init -backend-config="key=${{ matrix.env }}/"
terraform.tfstate"
      - working-directory: envs/${{ matrix.env }}
      - run: tofu plan -detailed-exitcode -input=false
        id: plan
      - working-directory: envs/${{ matrix.env }}
      # exit 0 = no changes, 1 = error, 2 = drift detected
      - if: steps.plan.outputs.exitcode == '2'
        run: |
          echo "::warning::Drift detected in ${{ matrix.env }}"
          # Post to Slack / PagerDuty / create an issue
          gh issue create --title "Drift detected: ${{ matrix.env }}" -
-body "Plan shows unexpected changes"

```

Terraform Cloud and Spacelift have native drift detection (health assessments) that run `plan` on a schedule and surface drift in the UI. For self-hosted setups, the cron workflow above plus `driftctl` (now part of Snyk) provides deeper detection by scanning the cloud account directly, catching resources Terraform does not manage at all.

Remediation choices:

- **Alert-only** — notify and let a human reconcile Git to match reality (safe, auditable).
- **Auto-remediate** — automatically `apply` to re-converge (fast, but risks reverting an intentional hotfix during an incident). Prefer alert-only for production; auto-remediate only for tightly controlled, non-critical stacks.

Policy as code and testing

Policy gates

Policy-as-code blocks non-compliant plans before they apply. Common controls: no public S3 buckets, no 0.0.0.0/0 ingress, mandatory tags, approved instance families, encrypted volumes.

```
# policy/deny_public_s3.rego - OPA/Conftest
package main

deny contains msg if {
    rc := input.resource_changes[_]
    rc.type == "aws_s3_bucket_public_access_block"
    # Every S3 bucket must have a public_access_block
    not rc.change.after.block_public_acls
    msg := sprintf("S3 bucket %q must block public ACLs", [rc.address])
}

deny contains msg if {
    rc := input.resource_changes[_]
    rc.type == "aws_security_group"
    ingress := rc.change.after.ingress[_]
    ingress.cidr_blocks[_] == "0.0.0.0/0"
    ingress.from_port == 22
    msg := sprintf("Security group %q allows SSH from 0.0.0.0/0", [rc.address])
}

deny contains msg if {
    rc := input.resource_changes[_]
    rc.type == "aws_db_instance"
    not rc.change.after.storage_encrypted
    msg := sprintf("RDS instance %q must have storage_encrypted = true",
[rc.address])
}
```

```
# Run in CI - convert plan to JSON, evaluate against policy
tofu show -json tfplan > plan.json
conftest test --policy policy/ plan.json
# FAIL - plan.json - main - S3 bucket "aws_s3_bucket.data" must block
public ACLs
```

Alternatives: **Sentinel** (TFC-native, similar to Rego), **Checkov** / **tfsec** (static analysis on HCL, no plan needed — `checkov -d envs/prod`), **OPA via Spacelift** (native integration).

Testing IaC

Layer	Tool	What it checks	When
Format/lint	<code>tofu fmt</code> <code>-check</code> , <code>tflint</code>	Syntax, naming, provider rules	Every PR
Validate	<code>tofu validate</code>	HCL validity, variable types	Every PR
Static analysis	Checkov, tfsec, KICS	Security/compliance on HCL	Every PR
Policy gate	Conftest/OPA, Sentinel	Plan JSON against org policy	Every plan
Unit test	<code>tofu test</code> (1.6+) / Terratest	Resource attributes after apply in ephemeral env	PR or nightly
Integration	Terratest (Go), Kitchen-Terraform	Real infra in a sandbox account, then destroy	Nightly / on module release

```
# tests/network.tfctest.hcl – native tofu test (1.6+, also in OpenTofu)
run "vpc_has_correct_cidr" {
  command = plan
  variables { environment = "test", vpc_cidr = "10.1.0.0/16", az_count
= 2 }
  assert {
    condition      = aws_vpc.main.cidr_block == "10.1.0.0/16"
    error_message = "VPC CIDR mismatch"
  }
  assert {
    condition      = length(aws_subnet.private) == 2
    error_message = "Expected 2 private subnets"
  }
}

run "vpc_tags_present" {
  command = plan
  assert {
    condition      = aws_vpc.main.tags["ManagedBy"] == "opentofu"
    error_message = "Missing ManagedBy tag"
  }
}
```

```
// terratest – Go integration test that applies, validates, and
// destroys
func TestNetworkModule(t *testing.T) {
    opts := &terraform.Options{
        TerraformDir: "../envs/dev",
        Vars: map[string]interface{}{
            "environment": "test",
            "vpc_cidr":    "10.1.0.0/16",
        },
    }
    defer terraform.Destroy(t, opts)
    terraform.InitAndApply(t, opts)
    vpcID := terraform.Output(t, opts, "vpc_id")
    assert.NotEmpty(t, vpcID)
    // Verify via AWS API
    subnets := aws.GetSubnetsForVpc(t, vpcID, "us-east-1")
    assert.Equal(t, 3, len(subnets))
}
```

Run `tofu test` is fast (no real apply) and catches logic errors; Terratest is slow (real apply/destroy, ~2–5 min) but catches provider and IAM errors. Use both: `tofu test` on every PR, Terratest nightly or on module version bumps.

Secrets and sensitive data

State is the Achilles' heel: every resource attribute — including passwords, keys, and tokens — is stored in plaintext in the state file. Anyone with state read access sees all secrets.

```

# BAD – password in plaintext in config and state
resource "aws_db_instance" "main" {
  password = "supersecret123" # committed to Git, stored in state,
  shown in plan
}

# GOOD – generate or fetch, mark sensitive, use a secrets manager
resource "random_password" "db" {
  length = 32
  special = true
}

resource "aws_secretsmanager_secret" "db" {
  name = "${var.environment}/db/password"
  tags = local.common_tags
}

resource "aws_secretsmanager_secret_version" "db" {
  secret_id      = aws_secretsmanager_secret.db.id
  secret_string = random_password.db.result
}

resource "aws_db_instance" "main" {
  # Reference the secret version – the password still appears in state
  via
  # aws_db_instance.password, but at least it was never in Git.
  # For stronger isolation, create the DB outside Terraform and inject
  via secrets manager.
  password = random_password.db.result # still in state – see
  mitigations below
  lifecycle { ignore_changes = [password] } # avoid perpetual diff if
  rotated externally
}

# Mark outputs sensitive so they are redacted in CLI output
output "db_password" {
  value      = random_password.db.result
  sensitive = true
}

# Variables that carry secrets
variable "github_token" {
  type      = string
  sensitive = true # redacted in plan/apply output
}

```

Mitigations, in order of strength:

1. **Do not put secrets in Terraform at all** — create the secret out-of-band (manual, or a separate secrets-management pipeline) and

reference it via `data "aws_secretsmanager_secret_version"` (read-only). The secret value still briefly appears in state if any resource consumes it, but the source of truth is outside IaC.

2. **Encrypt state at rest** — `encrypt = true` on the S3 backend (KMS), GCS CMEK, or TFC encryption. This protects against bucket exfiltration but not against anyone with state read IAM.
3. **Restrict state access** — IAM policy allowing `s3:GetObject` on the state bucket only to the CI runner role; no human `GetObject`. Use `terraform_cloud` remote execution so humans never download state.
4. **Use ephemeral values** (Terraform 1.10+ / OpenTofu) — `ephemeral` resources and `write-only` arguments that never persist to state. Emerging pattern for secrets — track provider support.
5. **Rotate after exposure** — if state was ever stored unencrypted or committed to Git, rotate every secret that appeared in it.

Provider credentials themselves should never be in config — use OIDC federation (GitHub Actions → AWS IAM role via `id-token: write`), environment variables, or a credentials helper. Never provider `"aws"` `{ access_key = "..."` }.

Refactoring, import, and lifecycle management

Infrastructure lives for years; the IaC that describes it must be refactored without destroying what it manages.

```

# Rename a resource address without destroying – declarative (1.1+)
moved {
  from = aws_instance.api
  to   = aws_instance.api_v2
}

# Move into a module
moved {
  from = aws_vpc.main
  to   = module.network.aws_vpc.main
}

# Change a resource type (rare, e.g., aws_s3_bucket → module)
moved {
  from = aws_s3_bucket.data
  to   = module.storage.aws_s3_bucket.this[0]
}

# Import existing infrastructure declaratively
import {
  to = aws_vpc.existing
  id = "vpc-0alb2c3d4e5f"
}
# Then run plan – Terraform shows the diff between the imported actual
and desired,
# so you can adjust config until plan is clean before applying.

# Remove from management without destroying (hand off or decommission
outside IaC)
removed {
  from = aws_eip.legacy
  lifecycle { destroy = false } # keep the real EIP, just stop
managing it
}

```

Operational tips:

- Always `plan` after a `moved` or `import` block before `apply` — verify the diff is empty or minimal.
- For large refactors, move one resource type at a time and apply incrementally.
- `terraform state mv` / `import` CLI commands still work but are not reviewable — prefer the declarative blocks in Git.
- When renaming a `for_each` key, Terraform sees it as destroy+create (different key) — use `moved` with the old and new key:


```
moved { from = aws_security_group.service["old"] to =  
aws_security_group.service["new"] }.
```

Alternatives and complements

Tool	Language	State	Strength	Weakness
Terraform / OpenTofu	HCL	Remote (S3/GCS/TFC)	Largest provider ecosystem, mature, declarative	HCL is not a general-purpose language; complex logic is awkward
Pulumi	TypeScript, Python, Go, etc.	Pulumi Cloud / S3 / local	Real language, loops, tests, IDE support	Smaller provider coverage; state model less transparent
AWS CDK / CDKTF	TypeScript/Python (CDK), HCL synth (CDKTF)	CloudFormation / Terraform state	Best for AWS-native stacks; constructs are high-level	AWS-centric; abstraction leaks when you need raw control
Crossplane	YAML (K8s CRDs)	etcd (K8s state)	K8s-native, GitOps with ArgoCD/Flux, drift via controllers	K8s required; provider maturity varies
Ansible	YAML (playbooks)	None (push-based)	Great for config management inside VMs; agentless	Not declarative for infra; no state/drift tracking

Tool	Language	State	Strength	Weakness
CloudFormation / SAM	YAML/ JSON	AWS-managed	No state to operate; rollback native	AWS-only, verbose, slower iteration

Choosing:

- **Default to Terraform/OpenTofu** for cloud infrastructure — the provider ecosystem and module registry are unmatched, and every cloud hire knows it.
- **Use Pulumi** when your infra logic is complex (dynamic graph construction, heavy abstraction) and your team prefers a real language.
- **Use Crossplane** when you already run Kubernetes and want infra as K8s objects reconciled by controllers (platform teams exposing `XRD / Composition` to tenants).
- **Use Ansible** for *configuration inside* the machines Terraform provisions (install packages, write configs, start services) — not for provisioning the machines themselves. The classic split: Terraform provisions, Ansible configures.

```

flowchart LR
    subgraph Provision["Provision (Terraform / OpenTofu)"]
        VPC[VPC + Subnets]
        ASG[ASG / Instances]
        RDS[RDS / ElastiCache]
    end
    end
    subgraph Configure["Configure (Ansible / cloud-init)"]
        PKG[Install packages]
        CFG[Write configs]
        SVC[Start services]
    end
    end
    subgraph Deploy["Deploy (K8s / ECS / systemd)"]
        APP[Application]
    end
    end
    Provision --> Configure --> Deploy
    CF["Crossplane  
(K8s-native alternative  
to Provision)"] -.->|"replaces"| Provision
    Pulumi["Pulumi / CDK  
(alternative IaC)"] -.->|"replaces"| Provision

    style Provision fill:#e3f2fd
    style Configure fill:#fff3e0
    style Deploy fill:#e8f5e9

```

Figure 4-4: The provisioning → configuration → deployment pipeline — Terraform/OpenTofu provisions the infrastructure, Ansible or cloud-init configures the machines, and the scheduler deploys the application; Crossplane and Pulumi/CDK are alternative provisioning paths.

Anti-patterns and operational lessons

Unpinned providers and modules. `source = "terraform-aws-modules/vpc/aws"` without `version` fetches the latest on every `init` — a breaking change can silently alter your next plan. Always pin: `version = "~> 5.0"` or `?ref=v2.1.0`.

Monolithic root module. One directory with 200 resources, one state file, one blast radius. A bad apply touches everything; plan takes minutes; locking blocks all teams. Split by blast radius and lifecycle: `network` (rarely changes), `compute` (frequently), `data` (stateful, guarded).

State in Git. Committing `terraform.tfstate` to Git leaks secrets and creates merge conflicts. Use a remote backend from day one.

Manual console fixes without back-porting. An on-call engineer opens a security group in the console to restore traffic, then forgets to commit the fix. Drift detection catches it — but only if you run it.

count with unstable ordering. `count = length(var.subnets)` where `var.subnets` is a list that can reorder — removing the first element shifts every index, causing Terraform to destroy and recreate all subsequent resources. Use `for_each` with stable keys.

Ignoring plan output. Approving a plan without reading it. A plan that says `forces replacement` on a database means downtime and data loss. Require plan review and policy gates; block `forces replacement` on stateful resources via policy.

No `prevent_destroy` on stateful resources. One `terraform destroy` or a bad `for_each` key change deletes the production database. Set `lifecycle { prevent_destroy = true }` on every stateful resource and require an explicit config change to remove it.

Key takeaways

- IaC replaces imperative ClickOps with declarative, versioned, reviewable definitions that converge actual state toward desired state via a plan/apply loop with drift detection.
- Terraform/OpenTofu's architecture — core + providers + remote state — is the de facto standard; pin providers and modules, use HCL's type system and validation, and prefer `for_each` over `count` for named resources.
- State is critical data — encrypt it, version it, lock it, and restrict access; never commit it to Git and assume every secret that touches a resource appears in state.
- The delivery pipeline is `fmt → validate → plan → policy gate → approval → apply (from saved plan)`; managed runners (Atlantis, TFC, Spacelift) enforce it, and scheduled re-planning catches drift.
- Policy-as-code (OPA/Conftest, Sentinel, Checkov) and testing (`tofu test` , Terratest) shift compliance and correctness left — block violations before they reach the cloud.

- Refactor safely with declarative `moved / import / removed` blocks and `lifecycle` guards; split stacks by blast radius and lifecycle, not into one monolithic root.
- Choose the right tool for the layer: Terraform/OpenTofu for provisioning, Ansible/cloud-init for host configuration, and Crossplane or Pulumi when K8s-native or general-purpose-language ergonomics justify the trade-off.

Further reading

- OpenTofu documentation — <https://opentofu.org/docs/> (language, CLI, state, modules, backends)
- Terraform documentation — <https://developer.hashicorp.com/terraform/docs> (providers, registry, CDKTF)
- Terraform AWS Provider — <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
- Atlantis — <https://www.runatlantis.io/docs>
- OPA / Rego — <https://www.openpolicyagent.org/docs/latest/policy-language/>
- Checkov — <https://www.checkov.io/> and tfsec — <https://aquasecurity.github.io/tfsec/>
- Terratest — <https://terratest.gruntwork.io/docs/>
- Pulumi vs. Terraform — <https://www.pulumi.com/docs/concepts/vs/terraform/>
- Crossplane — <https://docs.crossplane.io/latest/>
- Infracost (cost estimation in plan) — <https://www.infracost.io/docs/>

Chapter 5 — Cloud Primitives: Compute, Storage, and Network

What this chapter covers. Every cloud workload — whether it runs on Kubernetes, on VMs, or on managed services — composes the same three primitives: compute that runs code, storage that persists bytes, and network that moves packets between them. The primitives differ across providers in naming and API but converge on architecture: virtualized

compute with varying isolation (VMs, containers, functions), storage with distinct durability/latency/consistency trade-offs (object, block, file, ephemeral), and a software-defined network (VPC, subnets, routing, load balancing, DNS, CDN) that gives you a private data center inside a shared physical fabric. This chapter builds the mental model for each primitive, shows how cloud providers actually implement them (AWS as primary, with GCP/Azure mappings), and grounds every abstraction in production-grade Terraform/OpenTofu and cloud-native configs you can apply — including the failure modes that only appear at scale.

Learning goals — after this chapter you should be able to:

- Model the compute spectrum — bare metal, VMs, containers, functions — and choose the right primitive for a workload based on isolation, start latency, cost, and operational overhead.
- Provision and operate EC2 (and its GCP/Azure equivalents) with launch templates, auto scaling, placement, tenancy, and lifecycle hooks, and explain what actually happens when a VM boots inside a VPC.
- Distinguish object (S3/GCS), block (EBS/PD), file (EFS/Filestore), and ephemeral storage by durability, latency, consistency, and access pattern, and select correctly for databases, assets, shared state, and scratch.
- Design a VPC — CIDR planning, subnets, AZs, route tables, NAT, Internet and VPC endpoints, peering vs. Transit Gateway, PrivateLink — and reason about packet flow from public internet to private subnet.
- Configure L4 and L7 load balancing (NLB/ALB, GLB/ALB equivalents), DNS, and CDN (CloudFront/Cloud CDN) with health checks, TLS termination, and failover, and explain how they compose with the VPC data path.
- Apply cost and resilience trade-offs across primitives — Spot/Preemptible, Savings Plans, storage tiers, NAT Gateway vs. NAT instance, single-AZ vs. multi-AZ — and avoid the common misconfigurations that cause outages or bill shock.

Boundary note. Chapter 1 covered container images and runtimes as the packaging and isolation layer; Chapters 2–3 covered Kubernetes as the scheduler that orchestrates those containers. This chapter is the *cloud substrate* those schedulers run on — the VMs, disks, and networks

that Kubernetes nodes, databases, and load balancers are built from. Volume 3 — Networking — develops packet-level mechanics (TCP, TLS, DNS, HTTP); this chapter shows how cloud providers virtualize those mechanics into managed primitives. Volume 7 — System Design — uses these primitives as building blocks for multi-region architectures; Chapter 8 (Cloud Cost) optimizes their economics.

The cloud as a virtual data center

A cloud region is a set of isolated data centers (Availability Zones) connected by high-bandwidth, low-latency fiber, presented as a single logical data center with an API. The provider virtualizes physical resources — servers, disks, switches — into primitives you provision declaratively. Understanding the mapping from physical to virtual clarifies every performance and failure characteristic.

```

flowchart TB
    subgraph Physical["Physical Infrastructure (provider-operated)"]
        Hosts["Hosts / Racks / AZs"]
        Disks["Disks / Arrays"]
        Fabric["Network Fabric / Spine-Leaf"]
    end
    end
    subgraph Virtual["Virtual Primitives (you provision)"]
        Compute["Compute  
EC2 / GCE / Azure VM  
Lambda / Cloud Run"]
        Storage["Storage  
S3 / EBS / EFS  
GCS / PD / Filestore"]
        Network["Network  
VPC / Subnet / Route Table  
IGW / NAT / LB / DNS"]
    end
    end
    subgraph Control["Control Plane (API)"]
        API["Cloud APIs  
(eventually consistent)"]
        IAM["IAM / Policies"]
        State["IaC State  
(Terraform / Tofu)"]
    end
    end

    Hosts -->|"virtualize"| Compute
    Disks -->|"virtualize"| Storage
    Fabric -->|"virtualize"| Network
    API -->|"CRUD"| Compute & Storage & Network
    IAM -->|"authorize"| API
    State -->|"desired ↔ actual"| API

    style Physical fill:#e3f2fd
    style Virtual fill:#fff3e0
    style Control fill:#fce4ec

```

Figure 5-1: The cloud as a virtual data center — physical hosts, disks, and fabric are virtualized into compute, storage, and network primitives exposed through eventually consistent control-plane APIs governed by IAM and driven by IaC state.

Cross-provider mapping (names differ, architecture converges):

Primitive	AWS	GCP	Azure	Abstraction
VM	EC2	Compute Engine (GCE)	Virtual Machines	Virtualized host with vCPU/RAM/disk

Primitive	AWS	GCP	Azure	Abstraction
Container host	ECS / EKS	Cloud Run / GKE	Container Instances / AKS	Managed container scheduling
Function	Lambda	Cloud Functions / Cloud Run Jobs	Functions	Event-driven, scale-to-zero
Object storage	S3	Cloud Storage (GCS)	Blob Storage	Durable, eventually-consistent blob store
Block storage	EBS	Persistent Disk (PD)	Managed Disks	Network-attached low-latency volume
File storage	EFS	Filestore	Files	NFS/SMB shared filesystem
Network	VPC	VPC	VNet	Isolated L3 domain
L7 LB	ALB	External ALB / GLB	Application Gateway	HTTP-aware load balancing
L4 LB	NLB	Network LB / TCP Proxy	Load Balancer	TCP/UDP load balancing
CDN	CloudFront	Cloud CDN	Front Door / CDN	Edge caching and acceleration

We use AWS as the primary example (largest API surface, most documentation) and note GCP/Azure equivalents where they diverge.

Compute

The isolation spectrum

Compute primitives trade isolation strength for density and start latency — the same spectrum introduced in Chapter 1 for container runtimes, now at the cloud level:

```
flowchart LR
    BM["Bare Metal  
i3.metal / n2d-metal  
no hypervisor"] --> VM["VM  
EC2 / GCE  
KVM / Nitro"]
    VM --> Kata["MicroVM  
Firecracker / Kata  
per-pod VM"]
    Kata --> CTR["Container  
ECS / GKE pod  
namespaces + cgroups"]
    CTR --> FN["Function  
Lambda / Cloud Run  
scale-to-zero"]
    style BM fill:#e3f2fd
    style VM fill:#e8f5e9
    style Kata fill:#fff3e0
    style CTR fill:#fce4ec
    style FN fill:#f3e5f5
```

Primitive	Isolation	Start latency	Density	When to use
Bare metal	Physical (no sharing)	Minutes (provision)	1 tenant / host	DPDK, HPC, licensing that forbids virtualization
VM (EC2)	Hypervisor (KVM/Nitro)	~30-90 s (boot + init)	~10-100 VMs/host	General-purpose, databases, stateful workloads
MicroVM	Lightweight VM per sandbox	~100-150 ms	~100s/host	Multi-tenant, untrusted code (Fargate, GKE Sandbox)

Primitive	Isolation	Start latency	Density	When to use
Container	Kernel namespaces + cgroups	~1-5 s (image pull + start)	~100s/host	Microservices on ECS/EKS/GKE
Function	MicroVM or container, managed	~10-500 ms (cold start)	1000s/host (provider managed)	Event-driven, spiky, short-lived (<15 min)

Most backend systems compose two or three: VMs or containers for the steady-state API, functions for async/event-driven work, and bare metal only when virtualization overhead is measurable (high-throughput packet processing, NVMe-local databases).

EC2 in depth

An EC2 instance is a VM with vCPU, RAM, network, and storage defined by an instance type, launched from an AMI into a subnet, governed by security groups and IAM.

Instance families — the first letter defines the optimization:

Family	Optimizes	Examples	Use case
m (general)	Balanced	<code>m7i.large</code> (2 vCPU, 8 GB)	Application servers, default choice
c (compute)	vCPU per dollar	<code>c7i.xlarge</code>	CPU-bound API, batch
r (memory)	RAM per vCPU	<code>r7i.2xlarge</code> (8 vCPU, 64 GB)	Caches, in-memory DBs
i / d (storage)	NVMe-local disk	<code>i4i.large</code> (NVMe SSD)	Kafka, Cassandra, local NVMe cache
g / p (accelerated)	GPU	<code>g5.xlarge</code> (A10G), <code>p4d.24xlarge</code> (A100)	ML training/inference

Suffix letters denote CPU generation: `m7i` = 7th gen, Intel; `m7g` = 7th gen, Graviton (ARM); `m7a` = 7th gen, AMD. Graviton (`g`) instances are

~20-40% cheaper per unit of performance for compiled languages — prefer them when your toolchain supports ARM.

Launch template — the declarative definition of how to launch an instance (replaces the older launch configuration):

```

# ec2.tf – launch template + auto scaling group
data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]
  filter { name = "name"      values = ["al2023-ami-*-x86_64"] }
  filter { name = "virtualization-type" values = ["hvm"] }
}

resource "aws_launch_template" "api" {
  name_prefix   = "${var.environment}-api-"
  image_id      = data.aws_ami.al2023.id
  instance_type = "m7i.large"           # or m7g.large for Graviton
  key_name      = var.key_name          # prefer SSM Session Manager
  over SSH keys

  iam_instance_profile { name = aws_iam_instance_profile.api.name }

  vpc_security_group_ids = [aws_security_group.api.id]

  block_device_mappings {
    device_name = "/dev/xvda"
    ebs {
      volume_size      = 30
      volume_type       = "gp3"      # gp3 is cheaper and more
configurable than gp2
      iops              = 3000
      throughput        =
125      # MB/s – gp3 allows independent tuning
      encrypted         = true
      delete_on_termination = true
    }
  }

  metadata_options {
    http_endpoint      = "enabled"
    http_tokens        = "required" # IMDSv2 – mitigates
SSRF token exfiltration
    http_put_response_hop_limit = 1
  }

  tag_specifications {
    resource_type = "instance"
    tags = merge(local.common_tags, { Role = "api" })
  }

  user_data = base64encode(templatefile("${path.module}/user-data.sh",
{
  environment = var.environment
}))

```

```

    lifecycle { create_before_destroy = true }
}

resource "aws_autoscaling_group" "api" {
    name                = "${var.environment}-api"
    vpc_zone_identifier = module.network.private_subnet_ids
    min_size            = 3
    max_size            = 20
    desired_capacity    = 6
    health_check_type   = "ELB"
    health_check_grace_period = 90

    launch_template {
        id      = aws_launch_template.api.id
        version = "$Latest"
    }

    instance_refresh {
        strategy = "Rolling"
        preferences {
            checkpoint_delay      = 60
            checkpoint_percentages = [50, 100]
            min_healthy_percentage = 90
        }
    }
}

tag {
    key      = "Name"
    value    = "${var.environment}-api"
    propagate_at_launch = true
}

    lifecycle { create_before_destroy = true }
}

# Target-tracking autoscaling – scale on CPU or request count
resource "aws_autoscaling_policy" "cpu_tracking" {
    name                = "${var.environment}-api-cpu-tracking"
    autoscaling_group_name = aws_autoscaling_group.api.name
    policy_type          = "TargetTrackingScaling"
    target_tracking_configuration {
        predefined_metric_specification { predefined_metric_type = "ASGAverageCPUUtilization" }
        target_value = 55.0
    }
}

# Scheduled scaling – handle known traffic patterns (e.g., 9am spike)
resource "aws_autoscaling_schedule" "morning_ramp" {
    scheduled_action_name = "morning-ramp"
    autoscaling_group_name = aws_autoscaling_group.api.name

```

```

recurrence          = "0 8 * * MON-FRI"
desired_capacity     = 10
min_size            = 6
}

```

```

# user-data.sh – runs once on first boot (cloud-init style)
#!/bin/bash
set -eou pipefail
dnf update -y
dnf install -y amazon-cloudwatch-agent aws-cli

# Join ECS cluster or bootstrap k8s / systemd service
cat >/etc/myapp/config.yaml <<EOF
environment: ${environment}
log_level: info
EOF

systemctl enable --now myapp
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config -m ec2 -s -c ssm:AmazonCloudWatch-linux

```

Key details:

- **gp3 over gp2** — gp3 lets you provision IOPS and throughput independently of volume size; gp2 ties them to size (3 IOPS/GB). For most workloads gp3 is cheaper and avoids the "small volume, low IOPS" trap.
- **IMDSv2 (http_tokens = required)** — mitigates SSRF where an attacker tricks the app into fetching `http://169.254.169.254/latest/meta-data/iam/security-credentials/` — IMDSv2 requires a `PUT` to fetch a token first, which SSRF typically cannot do.
- **instance_refresh** — rolling replacement on launch template change; without it, updating the AMI or instance type requires manual instance replacement.
- **Lifecycle hooks** can pause termination to drain connections or flush state: `aws_autoscaling_lifecycle_hook` with a heartbeat timeout and an SQS/Lambda handler.

Placement and tenancy:

- **Spread** (default for ASG) — instances distributed across AZs for availability.

- **Cluster placement group** — instances in the same AZ, same network, for low-latency HPC (10 Gbps+, jumbo frames). Single-AZ, single failure domain.
- **Partition placement group** — spread across logical partitions for large distributed systems (HDFS, Cassandra) — limits blast radius within an AZ.
- **Dedicated hosts / tenancy** — `tenancy = "dedicated"` pins to a physical host (licensing, compliance); otherwise `default` shares the host.

Purchasing options:

Option	Discount	Commitment	Interruption	Use case
On-Demand	—	None	Never	Baseline, auto scaling
Spot	~60-70%	None	2-min warning	Batch, stateless, fault-tolerant
Reserved (1/3 yr)	~30-60%	1 or 3 years	Never	Steady-state baseline
Savings Plan	~20-40%	\$/hr commitment	Never	Flexible across families/regions
Dedicated Host	—	Host reservation	Never	BYOL licensing

Mix Spot + On-Demand in an ASG for cost without availability risk:


```

resource "aws_autoscaling_group" "mixed" {
  mixed_instances_policy {
    launch_template { launch_template_specification { launch_template_id = aws_launch_template.api.id } }
    instances_distribution {
      on_demand_base_capacity = 2      # always 2 on-demand
      on_demand_percentage_above_base_capacity = 30      # 30% on-demand above base, rest Spot
      spot_allocation_strategy = "price-capacity-optimized"
    }
    # Allow multiple instance types – ASG picks cheapest available
    launch_template {
      launch_template_specification { launch_template_id = aws_launch_template.api.id }
      override { instance_type = "m7i.large" }
      override { instance_type = "m7g.large" }
      override { instance_type = "m6i.large" }
    }
  }
}

```

Containers on cloud

Managed container services remove the need to operate the control plane or the host fleet:

Service	Abstraction	You manage	Provider manages
ECS + EC2	Containers on your VMs	Cluster, scaling, AMI	Scheduling, service discovery
ECS + Fargate	Containers, no VMs	Task definition, scaling	Hosts, patching, bin-packing
EKS + managed node group	K8s on your VMs	Node group, add-ons	Control plane (API server, etcd)
EKS + Fargate	K8s pods, no nodes	Pod spec	Nodes, patching, scaling
Lambda	Function	Code + event binding	Everything (invoke, scale, isolate)

```

# ecs.tf – ECS service on Fargate (no EC2 to manage)
resource "aws_ecs_cluster" "main" {
  name = "${var.environment}-main"
  setting { name = "containerInsights" value = "enabled" }
}

resource "aws_ecs_task_definition" "api" {
  family           = "${var.environment}-api"
  network_mode     = "awsvpc"
  requires_compatibilities = ["FARGATE"]
  cpu              = 512
  memory          = 1024
  execution_role_arn = aws_iam_role.ecs_execution.arn
  task_role_arn     = aws_iam_role.ecs_task.arn
  container_definitions = jsonencode([
    {
      name       = "api"
      image      = "${aws_ecr_repository.api.repository_url}:v1.42"
      essential = true
      portMappings = [
        { containerPort = 8080, protocol = "tcp" }
      ]
      logConfiguration = {
        logDriver = "awslogs"
        options = {
          awslogs-group      = aws_cloudwatch_log_group.api.name
          awslogs-region     = var.region
          awslogs-stream-prefix = "api"
        }
      }
    }
  ])
  environment = [
    { name = "ENVIRONMENT", value = var.environment }
  ]
  secrets = [
    {
      name      = "DATABASE_URL"
      valueFrom = aws_secretsmanager_secret.db.arn
    }
  ]
  healthCheck = {
    command = ["CMD-SHELL", "curl -f http://localhost:8080/healthz || exit 1"]
    interval = 30, timeout = 5, retries = 3, startPeriod = 30
  }
})

resource "aws_ecs_service" "api" {
  name           = "${var.environment}-api"
  cluster        = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.api.arn
  desired_count  = 6
  launch_type    = "FARGATE"
  network_configuration {
    subnets      = module.network.private_subnet_ids
    security_groups = [aws_security_group.api.id]
    assign_public_ip = false
  }
}

```

```
}
load_balancer {
    target_group_arn = aws_lb_target_group.api.arn
    container_name    = "api"
    container_port    = 8080
}
# ECS deployment circuit breaker – auto-rollback on failed deployment
deployment_circuit_breaker { enable = true, rollback = true }
}
```

Functions

Lambda (and equivalents) trades control for operational simplicity — no capacity planning, scale-to-zero, pay-per-invocation. Constraints: 15 min max duration, 10 GB RAM, 512 MB `/tmp`, cold start latency, and concurrency limits that can throttle under burst.

```

# lambda.tf – event-driven function
resource "aws_lambda_function" "ingest" {
  function_name = "${var.environment}-ingest"
  role          = aws_iam_role.lambda.arn
  package_type  = "Image"
  image_uri     = "${aws_ecr_repository.ingest.repository_url}:v1.42"
  timeout       = 30
  memory_size   = 512
  architectures = ["arm64"] # Graviton – cheaper, same perf for most
                             # workloads

  environment { variables = { ENVIRONMENT = var.environment } }

  # VPC-attached – needs NAT for outbound; adds ENI cold-start latency
  (~ls)
  vpc_config {
    subnet_ids          = module.network.private_subnet_ids
    security_group_ids = [aws_security_group.lambda.id]
  }

  # Provisioned concurrency – eliminates cold start for latency-
  # sensitive paths
  # (costs like always-warm; use only for p99-critical functions)
}

resource "aws_lambda_event_source_mapping" "sqs" {
  event_source_arn = aws_sqs_queue.ingest.arn
  function_name     = aws_lambda_function.ingest.arn
  batch_size        = 10
  maximum_batching_window_in_seconds = 5
  function_response_types = ["ReportBatchItemFailures"] # partial
  # failure – only retry failed messages
}

# Concurrency guard – prevents runaway scale from poisoning downstream
resource "aws_lambda_function" "ingest_concurrency" {
  # reserved_concurrent_executions = 100 # cap at 100 concurrent
  # invocations
  # provisioned_concurrent_executions = 10 # keep 10 warm
}

```

When to use functions vs. containers:

- **Functions win** when invocations are infrequent or spiky (webhooks, S3 events, cron), duration is short, and you want zero idle cost.
- **Containers/VMs win** when you need long-lived connections (WebSocket, gRPC streaming), large memory/disk, or predictable latency without cold starts.

Storage

The four storage shapes

Cloud storage is not one thing — four shapes with different trade-offs, often confused because all are "durable":

```
flowchart TB
    subgraph Shapes["Storage Shapes"]
        Object["Object Storage  
S3 / GCS / Blob  
11 9s, HTTP, eventual  
ms latency, TB-EB scale"]
        Block["Block Storage  
EBS / PD / Managed Disks  
replicated, low ms latency  
attached to one VM"]
        File["File Storage  
EFS / Filestore / Files  
NFS/SMB, shared  
many readers/writers"]
        Eph["Ephemeral / Local  
instance store / local SSD  
fastest, not durable  
lost on stop/terminate"]
    end
    Object -->|"lifecycle →"| Archive["Archive  
Glacier / Archive  
hours retrieval, cheapest"]
    Block -->|"snapshot →"| Object

    style Object fill:#e3f2fd
    style Block fill:#fff3e0
    style File fill:#e8f5e9
    style Eph fill:#fce4ec
    style Archive fill:#f3e5f5
```

Figure 5-2: The four storage shapes — object (durable, HTTP, EB-scale), block (low-latency, VM-attached), file (shared, POSIX), and ephemeral (fastest, non-durable) — with lifecycle and snapshot relationships.

Dimension	Object (S3)	Block (EBS gp3)	File (EFS)	Ephemeral (instance store)
Durability	11 9s (cross-AZ replication)	Replicated within AZ (snapshot to S3 for cross-AZ)	11 9s (regional)	None (lost on stop)
Latency	~10-100 ms (first byte)	~1-2 ms (gp3), sub-ms (io2)	~1-5 ms	~0.1-1 ms (local NVMe)
Throughput	5.5 GB/s per prefix (scales with prefix)	125-1000 MB/s per volume	Burst or provisioned	GB/s (local)
Access	HTTP (GET/PUT), any host	Block device, one VM (or multi-attach io1/io2)	NFS v4.1, many VMs	Block device, one VM
Consistency	Strong read-after-write (since 2020)	POSIX block	POSIX file	POSIX block
Scale	Unlimited objects, 5 TB per object	1 GB - 64 TB per volume	PB-scale, elastic	Fixed per instance type
Cost (us-east-1)	\$0.023/GB-mo (Standard)	\$0.08/GB-mo (gp3) + IOPS	\$0.30/GB-mo (Standard)	Included in instance cost

Object storage (S3) in depth

S3 is the default for every durable artifact that is not a database: build artifacts, logs, backups, data lake, static assets, and increasingly the primary store for data systems (lakehouse).

```

# s3.tf – production bucket with encryption, versioning, lifecycle, and
access logging
resource "aws_s3_bucket" "assets" {
    bucket = "${var.environment}-assets-$
{data.aws_caller_identity.current.account_id}"
    tags   = local.common_tags
}

resource "aws_s3_bucket_versioning" "assets" {
    bucket = aws_s3_bucket.assets.id
    versioning_configuration { status = "Enabled" } # required for
replication, recovery from overwrite
}

resource "aws_s3_bucket_server_side_encryption_configuration" "assets"
{
    bucket = aws_s3_bucket.assets.id
    rule {
        apply_server_side_encryption_by_default {
            sse_algorithm     = "aws:kms"
            kms_master_key_id = aws_kms_key.s3.arn
        }
        bucket_key_enabled = true # reduces KMS calls by ~99% – always
enable with KMS
    }
}

resource "aws_s3_bucket_public_access_block" "assets" {
    bucket           = aws_s3_bucket.assets.id
    block_public_acls       = true
    block_public_policy    = true
    ignore_public_acls     = true
    restrict_public_buckets = true
}

resource "aws_s3_bucket_lifecycle_configuration" "assets" {
    bucket = aws_s3_bucket.assets.id
    rule {
        id      = "transition-and-expire"
        status = "Enabled"
        transition { days = 30, storage_class = "STANDARD_IA" }
        transition { days = 90, storage_class = "GLACIER" }
        expiration { days = 365 }
        noncurrent_version_expiration { noncurrent_days = 90 }
        abort_incomplete_multipart_upload { days_after_initiation = 7 }
    }
}

# Replication to another region for DR (requires versioning on both
buckets)

```

```

resource "aws_s3_bucket_replication_configuration" "assets" {
  bucket = aws_s3_bucket.assets.id
  role   = aws_iam_role.s3_replication.arn
  rule {
    id      = "replicate-all"
    status  = "Enabled"
    destination {
      bucket          = aws_s3_bucket.assets_replica.arn
      storage_class   = "STANDARD"
      encryption_configuration { replica_kms_key_id = aws_kms_key.s3_re
plica.arn }
    }
  }
}

# Access via CloudFront (OAC), not public bucket policy
resource "aws_cloudfront_origin_access_control" "assets" {
  name = "${var.environment}-assets-oac"
  origin_access_control_origin_type = "s3"
  signing_behavior = "always"
  signing_protocol = "sigv4"
}

```

S3 consistency and performance details that matter:

- **Strong consistency** — since December 2020, S3 is strongly consistent for all operations (read-after-write, list-after-write). The old "eventual consistency for overwrite/delete" caveat is gone, but many blog posts still claim it.
- **Request rate** — S3 scales to 5,500 GET and 3,500 PUT per second *per prefix* (the part of the key before the last `/`). Randomizing key prefixes (e.g., `assets/{uuid}/file`) avoids hot partitions; sequential keys (`assets/0000001`, `assets/0000002`) concentrate load on one partition. For most workloads the default partitioning is sufficient, but high-throughput data pipelines must design key layout.
- **Multipart upload** — required for objects >5 GB, recommended for >100 MB (parallel upload, retry per part). Always set `abort_incomplete_multipart_upload` lifecycle to avoid paying for abandoned parts.
- **Pre-signed URLs** — grant temporary access without bucket policy changes: `aws s3 presign s3://bucket/key --expires-in 3600`. The backend generates the URL; the client uploads/downloads directly to S3, offloading bandwidth.

Block storage (EBS)

EBS volumes are network-attached block devices — they behave like a local disk but are replicated within a single AZ and can be snapshot-restored to another AZ.

```
# ebs.tf - gp3 volume with snapshot lifecycle
resource "aws_ebs_volume" "data" {
  availability_zone = local.azs[0]
  size             = 100
  type             = "gp3"
  iops              = 3000
  throughput        = 125
  encrypted         = true
  kms_key_id        = aws_kms_key.ebs.arn
  tags              = merge(local.common_tags, { Name = "${
{var.environment}-data" })
}

# DLM - automated snapshot lifecycle (replaces cron + aws ec2 create-
snapshot)
resource "aws_dlm_lifecycle_policy" "ebs_snapshots" {
  description = "Daily EBS snapshots, 30-day retention"
  state       = "ENABLED"
  execution_role_arn = aws_iam_role.dlm.arn
  policy_details {
    resource_types = ["VOLUME"]
    target_tags    = { Snapshot = "true" }
    schedule {
      name = "daily"
      create_rule { interval = 24, interval_unit = "HOURS", times =
["03:00"] }
      retain_rule { count = 30 }
      copy_tags = true
    }
  }
}
```

EBS type	Latency	Max IOPS	Max throughput	Durability	Use case
gp3	~1-2 ms	16,000	1,000 MB/s	AZ-replicated	Default for boot and data

EBS type	Latency	Max IOPS	Max throughput	Durability	Use case
io2 / io2 Block Express	sub-ms	64,000 / 256,000	1,000 / 4,000 MB/s	Higher (99.999%)	Databases needing provisioned IOPS
st1 (throughput)	~5 ms	500	500 MB/s	AZ-replicated	Big data, sequential
sc1 (cold HDD)	~10 ms	250	250 MB/s	AZ-replicated	Infrequent access

For databases, `io2` with multi-attach (attach one volume to multiple instances in the same AZ) enables active-active clustering without shared storage replication — but only `io1` / `io2` support it, and the filesystem must handle concurrent access (OCFS2, GFS2, or application-level).

File storage (EFS) and ephemeral

```
# efs.tf – elastic NFS for shared state (e.g., WordPress uploads, ML checkpoints)
resource "aws_efs_file_system" "shared" {
  creation_token = "${var.environment}-shared"
  encrypted      = true
  kms_key_id     = aws_kms_key.efs.arn
  throughput_mode = "bursting" # or "elastic" / "provisioned" for predictable perf
  tags           = local.common_tags
}

resource "aws_efs_mount_target" "shared" {
  for_each      = toset(module.network.private_subnet_ids)
  file_system_id = aws_efs_file_system.shared.id
  subnet_id     = each.value
  security_groups = [aws_security_group.efs.id]
}

# Mount on EC2: mount -t efs -o tls fs-abc123.efs.us-east-1.amazonaws.com:/mnt/shared
# Or via ECS/EKS: volume { efs_volume_configuration { file_system_id = aws_efs_file_system.shared.id } }
```

Ephemeral (instance store) — NVMe SSDs physically attached to the host, exposed as `/dev/nvme*n1`. Fastest and cheapest per IOPS but *data is lost on stop, terminate, or host failure*. Use for scratch, shuffle, page

cache, or Kafka log segments that are replicated elsewhere — never as the sole copy.

Network

VPC: your private data center

A VPC is an isolated L3 domain with its own CIDR, route tables, and gateways. Every resource (EC2, RDS, ECS task) lives in a subnet inside a VPC; nothing is reachable without an explicit route and security group rule.

```
flowchart TB
    Internet["Internet"] --> IGW["Internet Gateway (IGW)"]
    IGW --> PubRT["Public Route Table"]
    0.0.0.0/0 --> IGW
    PubRT --> PubSub1["Public Subnet AZ-a  
10.0.1.0/24"]
    PubRT --> PubSub2["Public Subnet AZ-b  
10.0.2.0/24"]
    PubSub1 --> NAT1["NAT Gateway AZ-a"]
    PubSub2 --> NAT2["NAT Gateway AZ-b"]
    NAT1 --> PrivRT1["Private Route Table AZ-a  
0.0.0.0/0 → NAT AZ-a"]
    NAT2 --> PrivRT2["Private Route Table AZ-b  
0.0.0.0/0 → NAT AZ-b"]
    PrivRT1 --> PrivSub1["Private Subnet AZ-a  
10.0.10.0/24  
EC2 / ECS / RDS"]
    PrivRT2 --> PrivSub2["Private Subnet AZ-b  
10.0.11.0/24  
EC2 / ECS / RDS"]
    PrivSub1 & PrivSub2 --> VPC["VPC Endpoints  
(Gateway: S3/Dynamo  
Interface: ECR/Secrets/KMS) | AWS[AWS Services  
(no NAT needed)"]
    PubSub1 & PubSub2 --> ALB["ALB (public subnets)"]
    ALB --> PrivSub1 & PrivSub2

    style Internet fill:#e3f2fd
    style IGW fill:#fff3e0
    style NAT1 fill:#fce4ec
    style NAT2 fill:#fce4ec
    style PrivSub1 fill:#e8f5e9
    style PrivSub2 fill:#e8f5e9
    style ALB fill:#f3e5f5
```

Figure 5-3: Canonical VPC layout — public subnets with IGW, private subnets with per-AZ NAT, ALB in public subnets forwarding to private workloads, and VPC endpoints for AWS services that bypass NAT.

CIDR planning — the most common irreversible mistake. Choose a range that will not collide when you peer VPCs or connect on-prem:

- Use RFC 1918 private space: `10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16` . Prefer `10.x.0.0/16` per VPC with room to peer.
- Size for growth: `/16` (65k IPs) per VPC is safe; `/20` (4k) is tight once you count ENIs per pod (EKS assigns one IP per pod by default). Account for secondary CIDRs if you outgrow the primary.
- Avoid `10.0.0.0/16` everywhere — every VPC and on-prem network that uses the same CIDR cannot peer without NAT. Allocate distinct `/16` s per VPC (e.g., `10.1.0.0/16` prod, `10.2.0.0/16` staging, `10.100.0.0/16` shared services).

```

# vpc.tf – complete VPC with per-AZ NAT and VPC endpoints
# (extends the network.tf from Chapter 4)

# Gateway endpoint – S3 and DynamoDB via route table (no ENI, no cost)
resource "aws_vpc_endpoint" "s3" {
  vpc_id      = aws_vpc.main.id
  service_name = "com.amazonaws.${var.region}.s3"
  route_table_ids = aws_route_table.private[*].id # adds route to S3
  prefix_list
  tags = local.common_tags
}

# Interface endpoints – ECR, Secrets Manager, KMS via PrivateLink (ENI
per AZ, ~$7/mo each)
resource "aws_vpc_endpoint" "ecr_api" {
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.${var.region}.ecr.api"
  vpc_endpoint_type = "Interface"
  private_dns_enabled = true
  subnet_ids        = aws_subnet.private[*].id
  security_group_ids = [aws_security_group.vpc_endpoints.id]
}

resource "aws_vpc_endpoint" "secretsmanager" {
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.${var.region}.secretsmanager"
  vpc_endpoint_type = "Interface"
  private_dns_enabled = true
  subnet_ids        = aws_subnet.private[*].id
  security_group_ids = [aws_security_group.vpc_endpoints.id]
}

# NAT cost note: one NAT Gateway per AZ ≈ $32/mo + $0.045/GB processed.
# For dev/low-traffic VPCs, a NAT instance (t4g.nano) or sharing one
NAT across AZs
# saves cost but sacrifices AZ isolation – a NAT AZ failure takes all
private subnets offline.

```

Security groups and NACLs

Two firewall layers, often confused:

Layer	Scope	State	Rules	Evaluation
Security group	Per-ENI (instance/task)	Stateful (return traffic auto-allowed)	Allow only	All rules evaluated (union)

Layer	Scope	State	Rules	Evaluation
NACL	Per-subnet	Stateless (explicit egress for return)	Allow + deny, numbered	First matching rule wins

Security groups are the primary control — stateful, tied to the workload, and composable (reference another SG as source: `security_groups = [aws_security_group.api.id]`). NACLs are a coarse subnet-level guardrail (e.g., block an abusive CIDR at the subnet boundary). In practice, most teams use security groups exclusively and leave NACLs at default-allow; NACLs become relevant for compliance requiring subnet-level deny or for blocking at the subnet before traffic reaches the instance.

```

# sg.tf – least-privilege security groups with SG-to-SG references
resource "aws_security_group" "alb" {
  name     = "${var.environment}-alb"
  vpc_id   = aws_vpc.main.id
  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "HTTPS from internet"
  }
  egress {
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    security_groups = [aws_security_group.api.id] # only to API – not
0.0.0.0/0
    description   = "Forward to API"
  }
}

resource "aws_security_group" "api" {
  name     = "${var.environment}-api"
  vpc_id   = aws_vpc.main.id
  ingress {
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    security_groups = [aws_security_group.alb.id]
    description   = "From ALB only"
  }
  egress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "Outbound HTTPS (ECR, Secrets Manager via NAT or
endpoint)"
  }
  # No SSH ingress – use SSM Session Manager (no open port, IAM-
audited)
}

resource "aws_security_group" "db" {
  name     = "${var.environment}-db"
  vpc_id   = aws_vpc.main.id
  ingress {
    from_port   = 5432
    to_port     = 5432
    protocol    = "tcp"

```

```

security_groups = [aws_security_group.api.id]
description     = "Postgres from API only"
}
# No egress needed – DB does not initiate connections
}

```

Connectivity beyond one VPC

Mechanism	Scope	L3 behavior	Use case
VPC Peering	2 VPCs, same or cross-region	Non-transitive (A↔B, B↔C does not imply A↔C)	Simple two-VPC connect
Transit Gateway	Hub for many VPCs + on-prem (VPN/Direct Connect)	Transitive via route tables (like a router)	Multi-VPC, multi-account, hybrid
PrivateLink	Service-to-service, cross-VPC/account	No peering, no CIDR overlap concern	Expose a service (ALB/NLB) privately to another VPC
VPN / Direct Connect	On-prem ↔ VPC	IPsec or dedicated fiber	Hybrid cloud, migration

Peering is simple but does not scale — N VPCs need $O(N^2)$ peerings. Transit Gateway is the hub that scales: each VPC attaches once, and route tables control which VPCs can reach which.

Load balancing

Two tiers, often both present:


```

flowchart LR
    User["Client"] --> DNS["Route 53 / Cloud DNS  
health-checked DNS"]
    DNS --> CF["CloudFront / Cloud CDN  
(edge, TLS, cache)"]
    CF --> ALB["ALB (L7)  
HTTP routing, TLS termination  
host/path/header rules"]
    ALB --> TGA["Target Group A  
api.prod:8080  
health: /healthz"]
    ALB --> TGB["Target Group B  
web.prod:3000  
health: /"]
    TGA --> ECS1["ECS1 [ECS / EC2 Targets  
AZ-a]"]
    TGA --> ECS2["ECS2 [ECS / EC2 Targets  
AZ-b]"]
    DNS -. -> |"failover  
active-passive"| ALB2["ALB (secondary region)"]

    style DNS fill:#e3f2fd
    style CF fill:#fff3e0
    style ALB fill:#e8f5e9
    style TGA fill:#fce4ec
    style TGB fill:#fce4ec

```

Figure 5-4: Load balancing composition — DNS failover across regions, CDN at the edge, ALB for L7 routing to target groups, with health-checked targets across AZs.

LB type	Layer	Protocols	TLS	Routing	Use case
ALB	L7 (HTTP)	HTTP/1.1, HTTP/2, gRPC	Terminate	Host, path, header, query, weighted	Web APIs, microservices
NLB	L4 (TCP/ UDP)	TCP, TLS, UDP	Pass- through or terminate	TCP, weighted	Low-latency, non-HTTP, static IP
GLB (GCP) / GWLB (AWS)	L3	GENEVE	—	—	Transparent firewall insertion

```

# alb.tf – ALB with HTTPS, target group, and health checks
resource "aws_lb" "main" {
  name           = "${var.environment}-main"
  load_balancer_type = "application"
  subnets       = module.network.public_subnet_ids
  security_groups = [aws_security_group.alb.id]
  # Access logs to S3 – essential for debugging 502/504 and WAF
  analysis {
    access_logs {
      bucket = aws_s3_bucket.alb_logs.id
      prefix = "alb"
      enabled = true
    }
  }
  tags = local.common_tags
}

resource "aws_lb_target_group" "api" {
  name       = "${var.environment}-api"
  port       = 8080
  protocol   = "HTTP"
  vpc_id     = module.network.vpc_id
  target_type = "ip" # for Fargate/ECS awsvpc; use "instance" for EC2

  health_check {
    path           = "/healthz"
    healthy_threshold = 2
    unhealthy_threshold = 3
    timeout        = 5
    interval       = 15
    matcher        = "200"
  }
}

# Deregistration delay – time to drain in-flight requests on deploy/
scale-in
deregistration_delay = 30

# Stickiness – only if you need it (stateful sessions); prefer
stateless
# stickiness { type = "lb_cookie" enabled = true }
}

resource "aws_lb_listener" "https" {
  load_balancer_arn = aws_lb.main.arn
  port              = 443
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-TLS13-1-2-2021-06" # TLS
1.2+1.3 only
  certificate_arn   = aws_acm_certificate.main.arn

  default_action {

```

```

        type                = "forward"
        target_group_arn = aws_lb_target_group.api.arn
    }
}

# Path-based routing - /api/* to API, /* to web
resource "aws_lb_listener_rule" "api" {
    listener_arn = aws_lb_listener.https.arn
    priority     = 10
    condition { path_pattern { values = ["/api/*"] } }
    action { type = "forward" target_group_arn = aws_lb_target_group.api.
arn }
}

# HTTP → HTTPS redirect
resource "aws_lb_listener" "http" {
    load_balancer_arn = aws_lb.main.arn
    port              = 80
    protocol           = "HTTP"
    default_action {
        type = "redirect"
        redirect { port = "443" protocol = "HTTPS" status_code =
"HTTP_301" }
    }
}

```

Health check design:

- Check a dedicated `/healthz` that verifies the app can serve (not just that the process is alive). A `200` that returns while the DB pool is exhausted is a lying health check — include dependency checks or at least a DB ping with a short timeout.
- `healthy_threshold = 2` / `unhealthy_threshold = 3` balances detection speed vs. flapping. Lower thresholds detect faster but flap on transient errors.
- `deregistration_delay` must exceed your longest request duration — otherwise in-flight requests are killed on deploy. For gRPC streaming, set it to minutes or use connection draining at the app layer.

DNS and CDN

Route 53 (AWS), Cloud DNS (GCP), and Azure DNS are authoritative DNS with health-checked routing:

- **Alias / ANAME** — point an apex domain (`example.com`, not just `www.example.com`) to an ALB/CloudFront without a CNAME (which

the DNS spec forbids at apex). Route 53 alias to ALB is free and health-aware.

- **Health-checked failover** — Route 53 health checks probe an endpoint; on failure, DNS fails over to a secondary (another region, a static S3 error page, or a standby ALB). TTL controls failover speed — lower TTL (60s) fails faster but increases DNS query cost.
- **Latency-based and geoproximity routing** — route users to the nearest healthy region.

CloudFront / Cloud CDN — edge caching and acceleration in front of S3 or ALB:

```

# cloudfront.tf – CDN in front of ALB + S3
resource "aws_cloudfront_distribution" "main" {
  enabled = true
  aliases = ["api.example.com"]

  origin {
    domain_name = aws_lb.main.dns_name
    origin_id   = "alb-origin"
    custom_origin_config {
      http_port      = 80
      https_port     = 443
      origin_protocol_policy = "https-only"
      origin_ssl_protocols  = ["TLSv1.2"]
    }
  }
  origin {
    domain_name      = aws_s3_bucket.assets.bucket_regional_domain_name
    origin_id        = "s3-assets"
    origin_access_control_id = aws_cloudfront_origin_access_control.assets.id
  }

  default_cache_behavior {
    target_origin_id      = "alb-origin"
    viewer_protocol_policy = "redirect-to-https"
    allowed_methods       = ["GET", "HEAD", "OPTIONS", "PUT", "POST", "PATCH", "DELETE"]
    cached_methods       = ["GET", "HEAD"]
    cache_policy_id       = data.aws_cloudfront_cache_policy.caching_disabled.id # API – no cache
  }
  ordered_cache_behavior {
    path_pattern      = "/assets/*"
    target_origin_id  = "s3-assets"
    viewer_protocol_policy = "redirect-to-https"
    allowed_methods   = ["GET", "HEAD"]
    cached_methods   = ["GET", "HEAD"]
    cache_policy_id   = data.aws_cloudfront_cache_policy.caching_optimized.id
    compress           = true
  }

  viewer_certificate {
    acm_certificate_arn      = aws_acm_certificate.main.arn
    ssl_support_method       = "sni-only"
    minimum_protocol_version = "TLSv1.2_2021"
  }
}

```

Cache invalidation (`aws cloudfront create-invalidation --distribution-id EDFDVBD6EXAMPLE --paths "/assets/*"`) is eventually consistent (~1-2 min) and costs after the free tier — prefer versioned asset URLs (`/assets/app.v1.42.js`) so invalidation is rarely needed.

Putting it together: a minimal production stack

```
flowchart TB
    User["Internet"] --> R53["Route 53  
api.example.com → CloudFront / ALB"]
    R53 --> CF["CloudFront  
(TLS, cache /assets/*)"]
    CF --> ALB["ALB (public subnets)  
443 → target groups"]
    ALB --> ECS["ECS Fargate / EC2 ASG  
(private subnets, SG: from ALB only)"]
    ECS --> RDS["RDS Postgres  
(private subnets, SG: from ECS only)"]
    ECS --> ElastiCache["ElastiCache Redis  
(private subnets)"]
    ECS --> S3["S3  
(via VPC endpoint)"]
    ECS --> CW["logs/metrics" | CloudWatch]
```

```
style R53 fill:#e3f2fd
style CF fill:#fff3e0
style ALB fill:#fce4ec
style ECS fill:#e8f5e9
style RDS fill:#f3e5f5
```

Figure 5-5: Minimal production stack on AWS — Route 53 → CloudFront → ALB → ECS/EC2 in private subnets → RDS + ElastiCache + S3, with VPC endpoints and security-group isolation.

This is the stack that Chapters 4 (IaC) provisions, Chapter 6 (Managed Services) extends with data services, and Chapter 10 (Cloud Networking & IAM) secures.

Anti-patterns and operational lessons

Single-AZ everything. One AZ is one data center — a power or network failure takes the whole workload offline. Run at least two AZs for every

tier (ALB, ECS/EC2, RDS Multi-AZ, ElastiCache with replica). The extra NAT Gateway cost is the price of availability.

Public subnets for workloads. Running EC2/RDS in a public subnet (route to IGW) exposes them to direct internet scanning even if the security group blocks it — a single SG misconfiguration is then an open port. Private subnets with no IGW route are a second layer of defense.

Oversized VPC CIDR or overlapping CIDRs. A `/20` that seemed generous becomes exhausted when EKS allocates one IP per pod; overlapping `10.0.0.0/16` across VPCs blocks peering. Plan CIDRs as carefully as you plan schema.

Ignoring ENI/IP exhaustion. EKS (default VPC CNI) allocates one ENI IP per pod; a `t3.medium` has `~17` IPs. At scale you hit `InsufficientFreeAddressesInSubnet` — monitor `IPAddressUtilization` and use prefix delegation or secondary CIDRs.

ALB health check that always returns 200. A health check that does not verify dependencies keeps unhealthy targets in rotation, causing 5xx for users. Make `/healthz` fail when the app cannot serve.

Unencrypted EBS / S3 without versioning. Unencrypted volumes leak data on snapshot sharing or host decommission; S3 without versioning cannot recover from accidental overwrite. Enable both by default via SCP/organization policy.

Key takeaways

- Cloud primitives are virtualized physical resources — compute, storage, network — exposed through eventually consistent APIs governed by IAM and driven by IaC.
- Compute spans VM → MicroVM → container → function; choose by isolation, latency, and operational overhead, and use mixed Spot/On-Demand for cost without sacrificing availability.
- Storage has four shapes — object (S3, 11 9s, HTTP), block (EBS, low-latency, AZ-bound), file (EFS, shared POSIX), ephemeral (local, non-durable) — each with distinct durability, latency, and access trade-offs; S3's strong consistency and prefix-based scaling govern data pipeline design.

- A production VPC uses private subnets for workloads, per-AZ NAT, VPC endpoints for AWS services, SG-to-SG references for least privilege, and Transit Gateway for multi-VPC scale; CIDR planning is irreversible and must account for pod IP allocation.
- ALB (L7) and NLB (L4) compose with Route 53 health-checked DNS and CloudFront/CDN for resilient, edge-accelerated delivery; health checks must verify serving readiness and deregistration delay must cover in-flight requests.
- The minimal stack — Route 53 → CloudFront → ALB → ECS/EC2 (private) → RDS/ElastiCache/S3 — is the foundation that IaC provisions and managed services extend.

Further reading

- AWS Well-Architected Framework — Reliability and Performance pillars — <https://docs.aws.amazon.com/wellarchitected/latest/framework/>
- EC2 instance types and Nitro — <https://aws.amazon.com/ec2/instance-types/> and <https://aws.amazon.com/ec2/nitro/>
- EBS volume types — <https://docs.aws.amazon.com/ebs/latest/userguide/ebs-volume-types.html>
- S3 performance and consistency — <https://docs.aws.amazon.com/AmazonS3/latest/userguide/optimizing-performance.html>
- VPC and Transit Gateway — <https://docs.aws.amazon.com/vpc/latest/userguide/> and <https://docs.aws.amazon.com/vpc/latest/tgw/>
- ALB/NLB — <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/> and <https://docs.aws.amazon.com/elasticloadbalancing/latest/network/>
- CloudFront — <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html>
- GCP networking and storage — <https://cloud.google.com/vpc/docs/overview> and <https://cloud.google.com/storage/docs>

- Azure networking — <https://learn.microsoft.com/en-us/azure/virtual-network/>

Chapter 6 — Managed Data and Platform Services

What this chapter covers. Running a production database, cache, or search cluster on raw EC2 is a second full-time job — patching, backups, replication, failover, monitoring, and 3 AM pages. Managed services trade a degree of control for a decisive reduction in operational burden: the provider automates the undifferentiated heavy lifting while you retain control over schema, queries, and application semantics. This chapter covers the managed data and platform services every backend engineer provisions — relational databases (RDS/Aurora/Cloud SQL), caches (ElastiCache/Memorystore), search and analytics (OpenSearch/BigQuery/Redshift/Athena), and the platform glue (Secrets Manager, Parameter Store, ECR, queue and stream services) — with production-grade Terraform/OpenTofu, the failure modes that survive the managed abstraction, and a decision framework for when managed is the right call and when self-hosting still wins.

Learning goals — after this chapter you should be able to:

- Decide build-vs-managed for databases, caches, and search by weighing operational cost, scale, compliance, and escape velocity, and articulate what managed does *not* absolve you from.
- Provision RDS Postgres (and Aurora) with Multi-AZ, read replicas, parameter and option groups, automated backups, point-in-time recovery, and performance insights, and explain the replication and failover mechanics underneath.
- Right-size and operate ElastiCache for Redis/Valkey (and its GCP/Azure equivalents) — cluster vs. replication group, sharding, persistence, and eviction — and explain when a cache should be managed vs. embedded.
- Use managed search (OpenSearch Service) and analytics (Athena/Redshift/BigQuery) for log search and warehousing without operating the underlying cluster, and reason about indexing, partitioning, and cost.

- Consume platform services — Secrets Manager, Systems Manager Parameter Store, AppConfig, ECR, SQS/SNS/Kinesis — as the glue that replaces self-operated infrastructure, with IaC that wires them to the VPC from Chapter 5.
- Detect and handle the failure modes managed services do not hide: replica lag, failover DNS propagation, cache avalanche, backup retention gaps, maintenance windows, and the provider's own control-plane outages.

Boundary note. Chapter 4 provisioned infrastructure with IaC; Chapter 5 built the compute/storage/network substrate. This chapter adds the *stateful managed services* that run on that substrate — the databases and caches your application connects to, and the platform services it depends on. Volume 5 — Databases — develops storage engines, indexing, transactions, and replication theory; this chapter is the *operational* view — how those engines are offered as a service and what you must still get right. Volume 10 — Messaging & Streaming — is the deep treatment of queues, logs, and delivery semantics; this chapter covers the managed *provisioning* of those primitives. Volume 11 — Reliability & SRE — builds the observability and incident-response practices that keep managed services healthy in production.

Why managed services

The undifferentiated heavy lifting

Every stateful system requires the same operational work regardless of whether it stores orders, sessions, or search indexes:

- OS and engine patching (Postgres minor versions, Redis CVEs)
- Backups, point-in-time recovery, and restore drills
- Replication, failover, and split-brain handling
- Monitoring, alerting, and performance tuning
- Capacity planning and vertical/horizontal scaling
- Encryption at rest and in transit, audit logging

Self-hosting means owning all of it. Managed means the provider owns the *infrastructure* layer — host, disk, OS, engine binary, replication plumbing, backup automation — while you own the *data* layer — schema,

queries, indexes, access control, and application-level resilience to the failure modes that remain.

```
flowchart TB
    subgraph You["You own (always)"]
        Schema["Schema / Queries / Indexes"]
        AppLogic["Application logic"]
        retries, timeouts, idempotency
        Access["Access control / IAM / grants"]
        Capacity["Capacity intent"]
        (instance size, replica count)
    end
    subgraph Provider["Provider owns (managed)"]
        Host["Hosts / Disks / OS patching"]
        Engine["Engine binary / minor version"]
        Replication["Replication plumbing"]
        Multi-AZ sync, failover
        Backup["Automated backups"]
        snapshots, PITR, retention
        Monitoring["Metrics / logs"]
        Performance Insights, slow query
    end
    subgraph Shared["Shared / leaky abstraction"]
        Failover["Failover behavior"]
        (DNS flip, replica promotion)
        Lag["Replica lag"]
        (read-after-write hazards)
        Maintenance["Maintenance windows"]
        (restarts, upgrades)
        Limits["Quotas / throttling"]
        (connections, IOPS, throughput)
    end
    You -.-> Shared
    Provider -.-> Shared

    style You fill:#e3f2fd
    style Provider fill:#e8f5e9
    style Shared fill:#fff3e0
```

Figure 6-1: The managed-service responsibility split — the provider owns host, engine, replication plumbing, and backup automation; you own schema, queries, and application resilience; the boundary leaks around failover, lag, maintenance, and limits.

What managed does **not** do:

- **Not zero-downtime** — failover takes 30-120 seconds (RDS Multi-AZ) or longer (Aurora Global); the application must handle DNS re-resolution, connection retries, and transaction replay.
- **Not infinitely scalable** — every managed service has quotas (max connections, IOPS, storage, shard count) that must be planned for, just like self-hosted.
- **Not free of data modeling** — a bad schema or missing index is slow on RDS exactly as it is on bare EC2.
- **Not a backup strategy alone** — automated backups are necessary but not sufficient; you still need cross-region copies, restore drills, and retention that matches compliance.

Decision framework: managed vs. self-hosted

Factor	Prefer managed	Prefer self-hosted
Team size	Small platform team, few DBAs	Dedicated storage team, deep engine expertise
Scale	Up to ~10 TB, ~100k QPS per cluster	100 TB+, custom sharding, bespoke engine tuning
Compliance	Provider handles patching, encryption, audit	Air-gapped, custom hardening, exotic OS
Feature velocity	Standard Postgres/Redis/Search features	Engine forks, extensions, custom builds
Cost at scale	Cheaper below ~\$10k/mo; premium at high throughput	Cheaper at sustained high I/O when you can bin-pack hosts
Escape velocity	Accept provider-specific extensions carefully	Need portability across clouds/on-prem

Default to managed. Self-host only when you have a concrete, measured reason that managed cannot satisfy — and budget for the operational team that self-hosting requires.

Managed relational databases

RDS for PostgreSQL / MySQL

RDS is a managed VM running the database engine with automated provisioning, patching, backup, and Multi-AZ replication. The instance class, storage type, and parameter group define the performance envelope; everything else is API-driven.

```

# rds.tf – production RDS Postgres with Multi-AZ, encrypted storage,
and secrets integration

resource "aws_db_subnet_group" "main" {
  name      = "${var.environment}-main"
  subnet_ids = 
module.network.private_subnet_ids # private subnets, no public access
  tags      = local.common_tags
}

resource "aws_db_parameter_group" "postgres16" {
  name      = "${var.environment}-postgres16"
  family    = "postgres16"

  # Production-tuned – override defaults that are wrong for backend
workloads
  parameter {
    name      = "shared_preload_libraries"
    value     = "pg_stat_statements,auto_explain"
  }
  parameter {
    name      = "log_min_duration_statement"
    value     = "1000" # log queries >1s
  }
  parameter {
    name      = "rds.force_ssl"
    value     = "1"    # require TLS – no plaintext connections
  }
  parameter {
    name      = "max_connections"
    value     = "200"  # explicit, not the engine default that
scales with RAM
    apply_method = "pending-reboot"
  }
}

resource "aws_secretsmanager_secret" "db_master" {
  name      = "${var.environment}/rds/master"
  tags      = local.common_tags
}

resource "random_password" "db_master" {
  length    = 32
  special    = false # avoid characters that break connection strings
}

resource "aws_secretsmanager_secret_version" "db_master" {
  secret_id      = aws_secretsmanager_secret.db_master.id
  secret_string  = jsonencode({ username = "platform_admin", password = 
random_password.db_master.result })
}

```

```

}

# KMS key for storage encryption (customer-managed, not aws/rds default
- so you control rotation and grants)
resource "aws_kms_key" "rds" {
  description      = "${var.environment} RDS encryption"
  deletion_window_in_days = 10
  enable_key_rotation = true
  tags             = local.common_tags
}

resource "aws_db_instance" "main" {
  identifier      = "${var.environment}-main"
  engine          = "postgres"
  engine_version = "16.4"
  instance_class = "db.r7g.large" # Graviton – ~20% cheaper than r7i
  for Postgres
  # For MySQL: engine = "mysql", engine_version = "8.0.36",
  instance_class = "db.r7g.large"

  db_name = "appdb"
  username = jsondecode(aws_secretsmanager_secret_version.db_master.secret_string)["username"]
  password = jsondecode(aws_secretsmanager_secret_version.db_master.secret_string)["password"]

  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.db.id]
  publicly_accessible   = false
  port                  = 5432

  # Storage – gp3 with independent IOPS/throughput, encrypted
  allocated_storage      = 100
  max_allocated_storage = 500 # autoscaling – grows
  automatically to 500 GB
  storage_type           = "gp3"
  storage_encrypted      = true
  kms_key_id             = aws_kms_key.rds.arn
  iops                   = 6000
  storage_throughput     = 250 # MB/s – gp3 allows tuning
  without IOPS coupling

  # Availability – Multi-AZ synchronous standby in another AZ
  multi_az      = true
  availability_zone = null # let RDS choose; pin only for
  debugging
  backup_retention_period = 14 # days – PITR window
  backup_window          = "03:00-04:00"
  maintenance_window     = "sun:04:00-sun:05:00"
  copy_tags_to_snapshot  = true
  delete_automated_backups =

```

```

false      # retain backups after instance deletion (safety)
deletion_protection    = true
skip_final_snapshot    = false
final_snapshot_identifier = "${var.environment}-main-final-${
formatdate("20060102-150405", timestamp())}"

parameter_group_name    = aws_db_parameter_group.postgres16.name
enabled_cloudwatch_logs_exports = ["postgresql", "upgrade"]

# Performance Insights – instrumented query-level latency, free for
7-day retention
performance_insights_enabled      = true
performance_insights_retention_period = 7
performance_insights_kms_key_id    = aws_kms_key.rds.arn

# Enhanced Monitoring – OS-level metrics at 30s granularity (requires
IAM role)
monitoring_interval = 30
monitoring_role_arn = aws_iam_role.rds_enhanced_monitoring.arn

auto_minor_version_upgrade = false # pin minor version – upgrade
explicitly via IaC, not silently

tags = local.common_tags

lifecycle {
  prevent_destroy = true
  ignore_changes = [password] # rotated via Secrets Manager,
not Terraform
}
}

# Read replica – async, cross-AZ or cross-region
resource "aws_db_instance" "read_replica" {
  identifier      = "${var.environment}-main-ro"
  replicate_source_db = aws_db_instance.main.identifier
  instance_class   = "db.r7g.large"
  publicly_accessible = false
  skip_final_snapshot = true
  auto_minor_version_upgrade = false
  tags              = merge(local.common_tags, { Role = "read-
replica" })
}

```



```

flowchart TB
    App["Application  
(private subnets)"] -->|"read/write  
5432 + TLS"| Primary["RDS Primary  
AZ-a  
read/write"]
    Primary -->|"sync replication  
(block-level for Multi-AZ)"| Standby["RDS Standby  
AZ-b  
not readable  
auto-promoted on failure"]
    Primary -.->|"async replication  
(WAL streaming)"| Replica["Read Replica  
AZ-c / cross-region  
readable, lag ~10-100ms"]
    App -.->|"read-only  
replica endpoint"| Replica
    Primary --> Snap["Automated Snapshots  
S3, 14-day PITR  
+ cross-region copy"]
    Standby -.->|"DNS flip  
~60-120s"| App

    style Primary fill:#e3f2fd
    style Standby fill:#fff3e0
    style Replica fill:#e8f5e9
    style Snap fill:#fce4ec

```

Figure 6-2: RDS Multi-AZ and read replica topology — synchronous standby for HA (automatic DNS failover), asynchronous read replica for scale-out, and automated snapshots for PITR.

Replication and failover — what actually happens:

- **Multi-AZ** is block-level synchronous replication to a standby in another AZ — not Postgres streaming replication. The standby is not readable and shares no data with read replicas. On primary failure (host, AZ, or engine crash), RDS promotes the standby, flips the DNS record (`<id>.<hash>.<region>.rds.amazonaws.com`), and the application must re-resolve DNS and reconnect. Failover takes ~60-120 seconds; during that window writes fail.
- **Read replicas** use Postgres WAL streaming (async). Lag is typically 10-100 ms but can spike to seconds under write pressure or large transactions. Never read-your-writes from a replica without handling lag — either use the primary for read-after-write, or gate on `pg_last_wal_replay_lag`.

- **Connection handling** — RDS has a hard `max_connections` (tunable via parameter group, but memory-bound). Use a pooler (RDS Proxy, PgBouncer, or application-level pooling) rather than opening a connection per request. RDS Proxy also handles failover more gracefully by preserving pooled connections across DNS flips.

RDS Proxy — managed PgBouncer for RDS/Aurora, essential when using Lambda (which can open thousands of concurrent connections):

```
resource "aws_db_proxy" "main" {
  name                = "${var.environment}-main-proxy"
  engine_family       = "POSTGRESQL"
  role_arn            = aws_iam_role.rds_proxy.arn
  vpc_security_group_ids = [aws_security_group.db_proxy.id]
  vpc_subnet_ids      = module.network.private_subnet_ids
  auth {
    iam_auth          = "REQUIRED" # IAM database auth – no password in
  }
  app_config {
    secret_arn        = aws_secretsmanager_secret.db_master.arn
  }
  connection_pool_config {
    max_connections_percent      = 80
    max_idle_connections_percent = 50
    connection_borrow_timeout   = 10
  }
}

resource "aws_db_proxy_default_target_group" "main" {
  db_proxy_name = aws_db_proxy.main.name
  connection_pool_config {
    max_connections_percent = 80
  }
}

resource "aws_db_proxy_target" "main" {
  db_proxy_name           = aws_db_proxy.main.name
  target_group_name       = aws_db_proxy_default_target_group.main.name
  db_instance_identifiers = [aws_db_instance.main.identifier]
}
```

Aurora: when RDS is not enough

Aurora is a re-implementation of the MySQL/Postgres wire protocol on a distributed, multi-tenant storage layer (6 copies across 3 AZs, quorum writes). It separates compute (the instance) from storage (the Aurora storage fleet), enabling instant failover, storage autoscaling to 128 TB, and Global Database (cross-region with <1 s replication lag).

Dimension	RDS Postgres	Aurora Postgres
Storage	EBS gp3/io2, per-instance, manual scaling	Distributed, 6-way, auto-scales to 128 TB
Failover	60-120 s (DNS flip, promote standby)	~30 s (storage already has the data, just promote)
Read scale	5 read replicas (async WAL)	15 Aurora Replicas, shared storage (no WAL replay lag)
Cross-region	Snapshot copy or async replica	Global Database (<1 s lag, 1 s RTO)
Cost	Lower for small/medium	Higher baseline, cheaper at scale (storage-only replicas)
Compatibility	Native Postgres	Postgres-compatible (~95% — check extension support)

```

# aurora.tf – Aurora Postgres cluster with two instances and Global
Database readiness
resource "aws_rds_cluster" "aurora" {
  cluster_identifier = "${var.environment}-aurora"
  engine             = "aurora-postgresql"
  engine_version     = "15.4"
  database_name      = "appdb"
  master_username    = "platform_admin"
  master_password    = random_password.db_master.result
  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.db.id]
  storage_encrypted   = true
  kms_key_id         = aws_kms_key.rds.arn
  backup_retention_period = 14
  preferred_backup_window = "03:00-04:00"
  deletion_protection = true
  copy_tags_to_snapshot = true
  enabled_cloudwatch_logs_exports = ["postgresql"]

  # Serverless v2 – scales ACUs (Aurora Capacity Units) automatically
  within bounds
  # Use for variable workloads; use provisioned for steady-state
  # serverlessv2_scaling_configuration { min_capacity = 0.5
  max_capacity = 16 }

  lifecycle { prevent_destroy = true }
}

resource "aws_rds_cluster_instance" "aurora" {
  count          = 2
  identifier     = "${var.environment}-aurora-${count.index}"
  cluster_identifier = aws_rds_cluster.aurora.id
  instance_class = "db.r7g.large" # or db.serverless for
  Serverless v2
  engine         = aws_rds_cluster.aurora.engine
  engine_version = aws_rds_cluster.aurora.engine_version
  performance_insights_enabled = true
  monitoring_interval = 30
  monitoring_role_arn = aws_iam_role.rds_enhanced_monitoring.arn
}

# Cluster endpoints – writer and reader (reader load-balances across
replicas)
# Writer: <cluster>.cluster-abc.us-east-1.rds.amazonaws.com → current
primary
# Reader: <cluster>.cluster-ro-abc.us-east-1.rds.amazonaws.com → load-
balanced replicas

```

When to choose Aurora over RDS:

- You need sub-minute failover or cross-region Global Database.
 - Storage exceeds ~10 TB or grows unpredictably (Aurora autoscales; RDS requires `modify` with downtime for large grows).
 - You need many read replicas without per-replica storage cost (Aurora replicas share the storage fleet).
 - You can tolerate the ~5% Postgres incompatibility (check `aurora_postgresql` extension coverage) and higher baseline cost.
-

Managed caches

ElastiCache for Redis / Valkey

ElastiCache offers Redis (and its open-source fork Valkey, adopted after Redis's 2024 license change) as a managed in-memory store with replication, persistence, and cluster-mode sharding. Memorystore (GCP) and Azure Cache for Redis are the equivalents.

Two deployment modes:

Mode	Topology	Scaling	Use case
Replication group (non-cluster)	1 primary + up to 5 replicas, single shard	Vertical (instance size)	Dataset fits one host, simple
Cluster mode (sharded)	Up to 500 shards × (1 primary + 5 replicas)	Horizontal (add shards, reshard online)	Dataset exceeds one host, high throughput

```

# elasticache.tf – Redis/Valkey replication group with cluster mode,
# TLS, and persistence

resource "aws_elasticache_subnet_group" "main" {
  name      = "${var.environment}-cache"
  subnet_ids = module.network.private_subnet_ids
}

resource "aws_elasticache_parameter_group" "valkey8" {
  name      = "${var.environment}-valkey8"
  family    = "valkey8"
  parameter {
    name     = "maxmemory-policy"
    value    = "noeviction" # for cache-aside with explicit TTLs – never
                           # Alternatives: allkeys-lru (pure cache), volatile-lru (only keys
                           # with TTL)
  }
  parameter {
    name     = "timeout"
    value    = "0"          # no idle timeout – let the app manage
                           # connection lifecycle
  }
}

resource "aws_elasticache_replication_group" "main" {
  replication_group_id = "${var.environment}-cache"
  description          = "${var.environment} Valkey cluster"
  engine               = "valkey"
  engine_version       = "8.0"
  node_type            = "cache.r7g.large"
  num_cache_clusters   = 3          # 1 primary + 2 replicas (non-
  cluster_mode         = true       # cluster mode)
  # For cluster mode (sharded):
  # num_node_groups      = 3          # 3 shards
  # replicas_per_node_group = 2       # 2 replicas per shard
  # automatic_failover_enabled = true

  subnet_group_name = aws_elasticache_subnet_group.main.name
  security_group_ids = [aws_security_group.cache.id]
  parameter_group_name = aws_elasticache_parameter_group.valkey8.name
  port                = 6379

  # Security – encryption in transit and at rest, auth token via
  # Secrets Manager
  at_rest_encryption_enabled = true
  transit_encryption_enabled = true
  auth_token                 = random_password.cache_auth.result #
  stored_in_secrets_manager  = true
  kms_key_id                 = aws_kms_key.cache.arn
}

```

```

    # Persistence – snapshot to S3 for warm restart (not a backup – Redis
    is a cache)
    snapshot_retention_limit = 7
    snapshot_window          = "02:00-03:00"
    maintenance_window       = "sun:03:00-sun:04:00"
    auto_minor_version_upgrade = false

    # Automatic failover – promotes replica on primary failure (~1-2 min
    without cluster mode, seconds with)
    automatic_failover_enabled = true
    multi_az_enabled          = true

    apply_immediately = false # changes wait for maintenance window –
    set true only for urgent fixes

    tags = local.common_tags
}

resource "aws_secretsmanager_secret" "cache_auth" {
    name = "${var.environment}/cache/auth-token"
}

resource "random_password" "cache_auth" {
    length    = 32
    special   = false
}

```

```

flowchart TB
    App["Application  
(private subnets)"] -->|"TLS 6379  
auth token"| Primary["Primary  
AZ-a  
read/write"]
    Primary -->|"async replication"| Replica1["Replica AZ-b  
readable"]
    Primary -->|"async replication"| Replica2["Replica AZ-c  
readable"]
    App -->|"read scale  
(replica endpoint)"| Replica1 & Replica2

    subgraph ClusterMode["Cluster Mode (sharded) – alternative"]
        Shard1["Shard 1  
slots 0-5460"]
        Shard2["Shard 2  
slots 5461-10922"]
        Shard3["Shard 3  
slots 10923-16383"]
        Shard1 & Shard2 & Shard3 -->|"each: 1 primary + 2 replicas"|
    end
    Rep["Replicas"]
    end

    style Primary fill:#e3f2fd
    style Replica1 fill:#e8f5e9
    style Replica2 fill:#e8f5e9
    style ClusterMode fill:#fff3e0

```

Figure 6-3: ElastiCache Valkey/Redis — replication group (primary + replicas for HA and read scale) and cluster mode (sharded across slots, each shard independently replicated).

Operational details that survive the managed abstraction:

- **Eviction policy** — `noeviction` returns errors on OOM (safe for session stores that must not lose data); `allkeys-lru` silently evicts (safe for pure caches). Choosing wrong causes either silent data loss or unexpected write errors.
- **Persistence is not durability** — ElastiCache snapshots are for warm restarts, not for recovery of critical data. If the data must survive cache loss, the source of truth is the database; the cache is reconstructible.
- **Failover still blips** — replica promotion takes seconds to minutes; the application must retry with backoff and handle `READONLY` errors during the transition.

- **Client sharding vs. cluster mode** — with cluster mode, the client must support `MOVED` / `ASK` redirections (most modern clients do); with replication group, the client connects to a single endpoint.

Serverless cache (ElastiCache Serverless / Memystore Serverless) — pay-per-request, scales to zero, no node sizing. Ideal for spiky or unpredictable workloads; more expensive per operation at sustained high throughput than provisioned nodes. Use when you cannot predict capacity or want zero idle cost.

Choosing the cache layer

Pattern	Where	Managed service	When
Look-aside (lazy)	App ↔ cache ↔ DB	ElastiCache / Memystore	Default for read-heavy workloads; app loads on miss
Write-through	App → cache → DB (sync)	Same	Strong consistency between cache and DB (rare — adds latency)
Write-behind	App → cache → DB (async)	Same + queue	High write throughput, eventual durability is OK
Embedded (in-process)	App heap (Caffeine, groupcache)	None	Sub-ms, no network hop, but per-instance, not shared

Most backend systems use look-aside with a managed cache for shared state (sessions, rate-limit counters, feature flags) and an embedded cache (Caffeine in Java, `singleflight` in Go) for per-instance hot keys — two layers, complementary.

Managed search and analytics

OpenSearch Service

Managed OpenSearch (fork of Elasticsearch 7.10) for log search, product search, and observability. The service manages cluster provisioning,

patching, snapshots to S3, and AZ awareness — you manage index templates, mappings, and shard strategy.

```

# opensearch.tf – domain with fine-grained access control and VPC
isolation
resource "aws_opensearch_domain" "logs" {
  domain_name      = "${var.environment}-logs"
  engine_version   = "OpenSearch_2.15"

  cluster_config {
    instance_type      = "r7g.large.search"
    instance_count     = 3
    zone_awareness_enabled = true
    zone_awareness_config { availability_zone_count = 3 }
    dedicated_master_enabled = true
    dedicated_master_type   = "r7g.medium.search"
    dedicated_master_count  = 3
    warm_enabled            = true
    warm_count              = 2
    warm_type               =
    "ultrawarm1.medium.search" # for older, infrequently queried indexes
  }

  ebs_options {
    ebs_enabled = true
    volume_type = "gp3"
    volume_size = 100
    iops        = 3000
    throughput  = 125
  }

  vpc_options {
    subnet_ids      = module.network.private_subnet_ids
    security_group_ids = [aws_security_group.opensearch.id]
  }

  encrypt_at_rest { enabled = true, kms_key_id = aws_kms_key.opensearch
.arn }
  node_to_node_encryption { enabled = true }
  domain_endpoint_options { enforce_https = true, tls_security_policy
= "Policy-Min-TLS-1-2-2019-07" }

  advanced_security_options {
    enabled                = true
    internal_user_database_enabled = false
    master_user_arn        = aws_iam_role.opensearch_admin.arn
  }

  log_publishing_options {
    cloudwatch_log_group_arn = aws_cloudwatch_log_group.opensearch_sear
ch.arn
    log_type                = "SEARCH_SLOW_LOGS"
  }
}

```

```

    snapshot_options { automated_snapshot_start_hour = 3 }

    tags = local.common_tags
}

# Index template – control sharding and mapping before data arrives
# (applied via API after domain creation, not Terraform – shown as curl
# for clarity)
# PUT _index_template/logs
# {
#   "index_patterns": ["logs-*"],
#   "template": {
#     "settings": {
#       "number_of_shards": 3,          # size for ~30 GB per shard –
# oversharding kills heap
#       "number_of_replicas": 1,
#       "index.lifecycle.name": "logs-30d",
#       "index.codec": "best_compression"
#     },
#     "mappings": {
#       "properties": {
#         "timestamp": { "type": "date" },
#         "level":      { "type": "keyword" },
#         "message":    { "type": "text" }
#       }
#     }
#   }
# }

```

Pitfalls that survive the managed label:

- **Oversharding** — each shard is a Lucene index with heap overhead. 1000 small shards on a 3-node cluster will OOM the heap. Rule of thumb: ~20-50 GB per shard, shards $\approx$ data size / 30 GB.
- **Mapping explosion** — dynamic mapping creates a field per unique JSON key; high-cardinality fields (UUIDs as field names) can create thousands of fields and crash the cluster. Use `dynamic: strict` or `index.mapping.total_fields.limit`.
- **No cross-AZ awareness without `zone_awareness_enabled`** — without it, all replicas may land in one AZ, losing HA.

Analytics: Athena, Redshift, BigQuery

Service	Model	Query engine	Storage	When
Athena	Serverless, pay-per-TB-scanned	Presto/Trino on S3	S3 (Parquet/ORC)	Ad-hoc, infrequent, data already in S3
Redshift	Provisioned or Serverless, columnar	Custom (Postgres-derived)	Managed (or S3 via Spectrum)	Frequent, complex, warehouse workloads
BigQuery (GCP)	Serverless, pay-per-TB or slots	Dremel	Colossus (managed)	GCP-native, large-scale analytics
Snowflake (cross-cloud)	Multi-cluster, pay-per-second	Custom	S3/GCS/Azure Blob	Cross-cloud, strong isolation

```

-- Athena – query S3 access logs without loading into a warehouse
-- Partition by date so each query scans only one day's data (10x cost
reduction)
CREATE EXTERNAL TABLE alb_logs (
  type          string,
  time          string,
  elb           string,
  client_ip     string,
  target_ip     string,
  request_processing_time double,
  target_processing_time double,
  response_processing_time double,
  elb_status_code int,
  target_status_code int
)
PARTITIONED BY (dt string)
STORED AS INPUTFORMAT 'org.apache.hadoop.mapred.TextInputFormat'
OUTPUTFORMAT 'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat'
LOCATION 's3://myorg-alb-logs/prod/'
TBLPROPERTIES ('projection.enabled' = 'true',
               'projection.dt.type' = 'date',
               'projection.dt.range' = '2024-01-01,NOW',
               'projection.dt.format' = 'yyyy-MM-dd',
               'storage.location.template' = 's3://myorg-alb-logs/prod/
dt=${dt}/');

-- Then: SELECT elb_status_code, count(*) FROM alb_logs WHERE dt =
'2026-08-19' GROUP BY 1;

```

Cost control for analytics is storage layout: partition by date, use Parquet/ORC (columnar, compressed), and compress with `gzip` or `zstd`. An unpartitioned JSON scan over a year of ALB logs can cost 100× more than a partitioned Parquet scan over one day.

Platform services: the glue

Beyond data stores, managed platform services replace self-operated infrastructure for the cross-cutting concerns that every service needs.

Secrets, parameters, and configuration

Service	AWS	GCP	Azure	Use case
Secrets	Secrets Manager	Secret Manager	Key Vault	DB passwords, API keys — rotation, audit
Parameters	SSM Parameter Store	— (Secret Manager)	App Configuration	Non-secret config, feature flags
App config	AppConfig	—	App Configuration	Validated, staged rollouts of config

```

# platform.tf – secrets, parameters, and ECR

# Secrets Manager – rotation via Lambda (managed rotation for RDS is
one-click)
resource "aws_secretsmanager_secret" "api_key" {
  name           = "${var.environment}/api/external-key"
  recovery_window_in_days = 30      # soft-delete so recovery is
possible
  kms_key_id      = aws_kms_key.secrets.arn
  tags            = local.common_tags
}

# SSM Parameter Store – hierarchical, versioned, free for standard tier
resource "aws_ssm_parameter" "feature_flags" {
  name = "/${var.environment}/api/feature_flags"
  type = "String"
  value = jsonencode({ new_checkout = true, dark_mode = false })
  tags = local.common_tags
}

# AppConfig – validated, staged config rollout with automatic rollback
on CloudWatch alarm
resource "aws_appconfig_application" "main" {
  name           = "${var.environment}-main"
  description    = "Application configuration"
}
resource "aws_appconfig_environment" "prod" {
  application_id = aws_appconfig_application.main.id
  name          = "prod"
  monitor {
    alarm_arn      = aws_cloudwatch_metric_alarm.error_rate.arn
    alarm_role_arn = aws_iam_role.appconfig.arn
  }
}
resource "aws_appconfig_configuration_profile" "flags" {
  application_id = aws_appconfig_application.main.id
  name          = "feature-flags"
  location_uri  = "hosted"
  type         = "AWS.Freeform"
  validator { type = "JSON_SCHEMA" content = file("${path.module}/
flags.schema.json") }
}
# Deploy with validators – invalid JSON is rejected before rollout;
alarm triggers auto-rollback

# ECR – private container registry (replaces self-hosted Harbor/
registry)
resource "aws_ecr_repository" "api" {
  name           = "${var.environment}/api"
  image_tag_mutability = "IMMUTABLE"      # prevent tag overwrite
}

```



```

- deploy by digest
  image_scanning_configuration { scan_on_push = true }
  encryption_configuration { encryption_type = "KMS" kms_key = aws_kms_
key.ecr.arn }
  tags = local.common_tags
}

resource "aws_ecr_lifecycle_policy" "api" {
  repository = aws_ecr_repository.api.name
  policy = jsonencode({
    rules = [{
      rulePriority = 1
      description = "Keep last 50 images"
      selection = { tagStatus = "any", countType =
"imageCountMoreThan", countNumber = 50 }
      action = { type = "expire" }
    }]
  })
}

```

Application consumption — IAM-authenticated, no static credentials:

```

# Python – fetch config at startup, refresh on schedule (not per-
request)
import boto3, json, os

ssm = boto3.client("ssm")
appconfig = boto3.client("appconfigdata")

# Simple parameter
flags = json.loads(ssm.get_parameter(Name=f"/
{os.environ['ENVIRONMENT']}/api/feature_flags")["Parameter"]["Value"])

# AppConfig – session-based polling with token (efficient, long-poll)
session = appconfig.start_configuration_session(
    ApplicationIdentifier="prod-main",
    EnvironmentIdentifier="prod",
    ConfigurationProfileIdentifier="feature-flags",
)
# Poll with NextPollInterval from response; update is pushed when
config changes
resp = appconfig.get_latest_configuration(ConfigurationToken=session["I
nitialConfigurationToken"])

# Secrets – cached with rotation awareness (use aws-secretsmanager-
caching if available)
secrets = boto3.client("secretsmanager")
creds = json.loads(secrets.get_secret_value(SecretId=f"{os.environ['ENV
IRONMENT']}/rds/master")["SecretString"])

```

Queues, notifications, and streams

Primitive	AWS	GCP	Azure	Semantics
Queue	SQS (Standard/ FIFO)	Pub/Sub (pull)	Queue Storage / Service Bus	At-least-once (Standard) / exactly-once per group (FIFO)
Pub/Sub	SNS	Pub/Sub (push)	Event Grid / Service Bus Topics	Fan-out, push to SQS/Lambda/ HTTP
Stream	Kinesis Data Streams	Pub/Sub (streaming)	Event Hubs	Ordered, replayable, sharded log
Managed Kafka	MSK / MSK Serverless	—	HDInsight Kafka	Kafka API, managed brokers

These are the managed provisioning of the primitives from Volume 10. The delivery semantics are identical — managed does not change at-least-once vs. exactly-once — but the operational burden (broker patching, partition rebalancing, consumer lag monitoring) is absorbed.

```

# messaging.tf – SQS + SNS + Kinesis as the managed messaging fabric

resource "aws_sqs_queue" "orders" {
  name = "${var.environment}-orders.fifo" # FIFO for ordering + dedup
  fifo_queue = true
  content_based_deduplication = true
  visibility_timeout_seconds = 60
  message_retention_seconds = 1209600 # 14 days
  redrive_policy = jsonencode({
    deadLetterTargetArn = aws_sqs_queue.orders_dlq.arn
    maxReceiveCount = 5
  })
  tags = local.common_tags
}

resource "aws_sqs_queue" "orders_dlq" {
  name = "${var.environment}-orders-dlq.fifo"
  fifo_queue = true
  tags = local.common_tags
}

resource "aws_sns_topic" "order_events" {
  name = "${var.environment}-order-events.fifo"
  fifo_topic = true
  content_based_deduplication = true
  tags = local.common_tags
}

# Fan-out: SNS → SQS (each consumer gets its own queue, no polling contention)
resource "aws_sns_topic_subscription" "orders_to_queue" {
  topic_arn = aws_sns_topic.order_events.arn
  protocol = "sqs"
  endpoint = aws_sqs_queue.orders.arn
}

resource "aws_kinesis_stream" "events" {
  name = "${var.environment}-events"
  shard_count = 4
  retention_period = 48 # hours – replay window
  shard_level_metrics = ["IncomingRecords", "OutgoingRecords",
"ReadProvisionedThroughputExceeded"]
  stream_mode_details { stream_mode = "PROVISIONED" }
  # For variable throughput: stream_mode = "ON_DEMAND" (scales
  automatically, higher per-record cost)
  encryption_type = "KMS"
  kms_key_id = aws_kms_key.kinesis.arn
  tags = local.common_tags
}

```

```
# Lambda consumer for the stream (alternative to Kinesis Data
Analytics / Flink)
resource "aws_lambda_event_source_mapping" "kinesis_to_lambda" {
  event_source_arn = aws_kinesis_stream.events.arn
  function_name    = aws_lambda_function.stream_processor.arn
  starting_position = "TRIM_HORIZON"
  batch_size       = 100
  bisect_batch_on_function_error = true
  maximum_record_age_in_seconds = 3600
}
```

For Kafka workloads, MSK (Managed Streaming for Apache Kafka) or MSK Serverless provides the Kafka API without operating brokers, ZooKeeper/KRaft, or partition reassignment — but Volume 10, Chapter 3 is the deep treatment of Kafka architecture; here you provision it.

Cross-cutting concerns

Backups, retention, and restore drills

Automated backups are necessary but not sufficient. A production backup strategy answers:

- **RTO/RPO** — how long to restore, how much data can be lost. RDS PITR gives ~5 min RPO (continuous WAL archiving) and ~10–30 min RTO (restore to new instance). S3 versioning gives immediate RPO for object overwrites.
- **Cross-region** — a region failure must not lose backups. Enable cross-region snapshot copy (RDS `copy_automated_backups_to_region`), S3 cross-region replication, and ElastiCache snapshot copy.
- **Retention vs. compliance** — `backup_retention_period = 14` may not satisfy a 7-year retention requirement. Export snapshots to S3 and lifecycle to Glacier for long-term retention.
- **Restore drills** — a backup that has never been restored is not a backup. Automate a monthly drill: restore to a temporary instance, run `pg_checksums` or application-level consistency checks, tear down.

```
# backup.tf – AWS Backup for centralized, cross-service backup with
vault lock
resource "aws_backup_vault" "main" {
  name          = "${var.environment}-main"
  kms_key_arn = aws_kms_key.backup.arn
}

resource "aws_backup_vault_lock_configuration" "main" {
  backup_vault_name = aws_backup_vault.main.name
  min_retention_days = 30
  max_retention_days = 365
  changeable_for_days =
3 # after 3 days, retention is immutable (WORM – compliance)
}

resource "aws_backup_plan" "main" {
  name = "${var.environment}-main"
  rule {
    rule_name          = "daily-cross-region"
    target_vault_name = aws_backup_vault.main.name
    schedule            = "cron(0 3 * * ? *)"
    lifecycle { delete_after = 30 }
    copy_action {
      destination_vault_arn = aws_backup_vault.replica.arn # cross-
region vault
      lifecycle { delete_after = 90 }
    }
  }
  advanced_backup_setting {
    backup_options = { WindowsVSS = "enabled" } # for Windows EC2, if
applicable
    resource_type = "EC2"
  }
}
```

Scaling and maintenance

Every managed service has a maintenance window and scaling mechanics that affect availability:

Service	Scaling	Maintenance impact	Mitigation
RDS	modify instance class (brief failover) or storage autoscaling (online for gp3)	Minor version upgrades restart the instance (~1-2 min)	Multi-AZ for HA, <code>auto_minor_version_upgrade = false</code> , schedule upgrades explicitly
Aurora	Add/remove instances, Serverless v2 auto-scales ACUs	Same as RDS, but storage never needs scaling	Global Database for cross-region, Serverless v2 for variable load
ElastiCache	Vertical (modify node type, brief failover) or sharded (online reshard)	Engine upgrades restart nodes	Multi-AZ + cluster mode, test client <code>MOVED</code> handling
OpenSearch	Add nodes (blue/green deploy, ~10-30 min)	Upgrades are blue/green with no downtime but double capacity during	Provision with headroom, use warm/cold tiers
SQS/Kinesis	SQS scales automatically; Kinesis provisioned shards need manual reshard	None (serverless)	Use <code>ON_DEMAND</code> for Kinesis if throughput is unpredictable

Never set `apply_immediately = true` on production stateful resources — changes should wait for the maintenance window and be applied during a planned deployment, not mid-day.

Monitoring the managed abstraction

Managed does not mean unmonitored. The provider exposes metrics, but you must alert on them:

```

# monitoring.tf – CloudWatch alarms for the managed data layer
resource "aws_cloudwatch_metric_alarm" "rds_cpu" {
  alarm_name      = "${var.environment}-rds-cpu-high"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  threshold       = 80
  evaluation_periods = 3
  metric_name      = "CPUUtilization"
  namespace        = "AWS/RDS"
  dimensions       = { DBInstanceIdentifier = aws_db_instance.main.i
  identifier }
  statistic        = "Average"
  period           = 300
  alarm_actions     = [aws_sns_topic.alerts.arn]
}

resource "aws_cloudwatch_metric_alarm" "rds_replica_lag" {
  alarm_name      = "${var.environment}-rds-replica-lag"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  threshold       = 5000 # ms – alert when replica falls behind
  evaluation_periods = 2
  metric_name      = "ReplicaLag"
  namespace        = "AWS/RDS"
  dimensions       = { DBInstanceIdentifier = aws_db_instance.read_r
  replica.identifier }
  statistic        = "Maximum"
  period           = 60
  alarm_actions     = [aws_sns_topic.alerts.arn]
}

resource "aws_cloudwatch_metric_alarm" "cache_evictions" {
  alarm_name      = "${var.environment}-cache-evictions"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  threshold       = 100 # evictions/min – indicates undersized
  cache or bad policy
  evaluation_periods = 2
  metric_name      = "Evictions"
  namespace        = "AWS/ElastiCache"
  dimensions       = { CacheClusterId = aws_elasticache_replication_
  group.main.id }
  statistic        = "Sum"
  period           = 300
  alarm_actions     = [aws_sns_topic.alerts.arn]
}

resource "aws_cloudwatch_metric_alarm" "opensearch_cpu" {
  alarm_name      = "${var.environment}-opensearch-cpu"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  threshold       = 80
  evaluation_periods = 3
  metric_name      = "CPUUtilization"

```



```
namespace          = "AWS/ES"
dimensions          = { DomainName = aws_opensearch_domain.logs.domain_name, ClientId = data.aws_caller_identity.current.account_id }
statistic           = "Maximum"
period              = 300
alarm_actions       = [aws_sns_topic.alerts.arn]
}
```

Beyond CloudWatch, enable **Performance Insights** (RDS/Aurora) for query-level latency attribution, **ElastiCache slowlog** for hot-key detection, and **OpenSearch slow logs** for expensive queries — the managed console surfaces these, but they must be actively monitored, not just enabled.

Putting it together: service dependencies on managed infrastructure

```

flowchart TB
    App["Application (ECS Fargate)  
private subnets"] --> RDS["RDS Postgres  
Multi-AZ + read replica  
via RDS Proxy"]
    App --> Cache["ElastiCache Valkey  
replication group  
TLS + auth token"]
    App --> Search["OpenSearch  
3+3+warm  
VPC isolated"]
    App --> Queue["SQS FIFO  
orders.fifo → DLQ"]
    App --> Stream["Kinesis  
events (4 shards)  
→ Lambda processor"]
    App --> Secrets["config/secrets" | Secrets Manager  
+ SSM + AppConfig]
    App --> ECR["images" | ECR  
immutable tags]
    App --> CW["logs/metrics" | CloudWatch  
+ Performance Insights]

    RDS --> Snap["Snapshots → S3  
cross-region copy  
Backup Vault (WORM)"]
    Cache --> S3Cache["snapshot" | S3Cache  
cache warm-restart]
    Search --> S3Snap["S3  
OpenSearch snapshots"]

    style App fill:#e3f2fd
    style RDS fill:#fff3e0
    style Cache fill:#fce4ec
    style Search fill:#f3e5f5
    style Queue fill:#e8f5e9

```

Figure 6-4: A backend service's managed dependencies — RDS + Proxy for primary data, ElastiCache for shared cache, OpenSearch for search, SQS/Kinesis for messaging, Secrets/SSM/AppConfig for configuration, ECR for images, and centralized backup — all in private subnets with VPC endpoints.

This is the data and platform layer that Chapters 4–5 provision and that application code connects to. Chapter 7 (Multi-Tenancy) isolates these dependencies per tenant; Chapter 8 (Cost) optimizes their spend; Chapter 10 (Networking & IAM) secures their access.

Anti-patterns and operational lessons

Treating managed as magic. "RDS handles HA so the app doesn't need retries." It doesn't — failover still drops connections, and the app must reconnect with backoff. Every managed service has a failure mode that reaches the application; handle it.

No read-replica lag handling. Reading from a replica immediately after writing to the primary and assuming the read reflects the write. Under load, lag spikes and users see stale data or duplicate submissions. Use the primary for read-after-write or gate on lag.

Cache as source of truth. Storing the only copy of critical data in ElastiCache with `snapshot_retention_limit = 1`. A cluster replacement or AZ failure loses it. Caches are reconstructible; the database is the source of truth.

Oversized or undersized cache eviction policy. `allkeys-lru` on a session store silently evicts active sessions; `noeviction` on a pure cache returns errors when full. Match policy to data class.

Unencrypted or publicly accessible data stores. `publicly_accessible = true` on RDS or `block_public_acls = false` on S3 — both have been the root cause of breaches. Default to private subnets, `publicly_accessible = false`, and `block_public_acls = true`; require explicit justification to relax.

No backup restore drills. Automated snapshots exist but have never been restored. The first restore attempt during an actual incident discovers that the snapshot is corrupt, the KMS grant is missing, or the subnet group was deleted.

Pinning to provider-specific extensions without escape hatch. Using Aurora-only or BigQuery-only features pervasively makes migration prohibitively expensive. Isolate provider-specific code behind an interface; keep the core SQL portable.

Key takeaways

- Managed services absorb host, patching, replication plumbing, and backup automation — you retain ownership of schema, queries,

access control, and application-level resilience to the failure modes that leak through.

- RDS Postgres with Multi-AZ (synchronous block replication, 60–120 s failover) and async read replicas, plus RDS Proxy for connection pooling and IAM auth, is the default for relational workloads; Aurora trades higher baseline cost for distributed storage, 30 s failover, and Global Database.
- ElastiCache for Valkey/Redis offers replication groups (single shard) and cluster mode (500 shards); choose eviction policy by data class (`noeviction` for session stores, `allkeys-lru` for pure caches) and never treat the cache as the sole copy.
- OpenSearch Service requires disciplined index design (shard sizing ~30 GB, strict mappings, zone awareness); analytics on S3 via Athena/Redshift/BigQuery is cost-controlled by partitioning and columnar formats (Parquet/ORC).
- Platform services — Secrets Manager, SSM Parameter Store, AppConfig (with staged rollout and alarm-driven rollback), ECR (immutable tags, scan on push), SQS/SNS/Kinesis — replace self-operated glue and are consumed via IAM, not static credentials.
- Every managed service still requires backup strategy (cross-region, WORM vault, restore drills), maintenance window planning, scaling mechanics, and alerting on the metrics that matter (replica lag, evictions, CPU, shard pressure).

Further reading

- RDS User Guide — <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html>
- Aurora documentation — https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_AuroraOverview.html
- RDS Proxy — <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy.html>
- ElastiCache for Valkey/Redis — <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/WhatIs.html>
- OpenSearch Service — <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/what-is.html>

- Athena — <https://docs.aws.amazon.com/athena/latest/ug/what-is.html> and Redshift — https://docs.aws.amazon.com/redshift/latest/dg/c_intro.html
- AWS Backup — <https://docs.aws.amazon.com/aws-backup/latest/devguide/whatisbackup.html>
- GCP managed data — Cloud SQL, Memorystore, BigQuery — <https://cloud.google.com/products/databases> and <https://cloud.google.com/bigquery/docs>
- Azure managed data — <https://learn.microsoft.com/en-us/azure/product-categories/databases>

Chapter 7 — Multi-Tenancy and Isolation

What this chapter covers. Every SaaS product is multi-tenant — many customers share the same fleet, the same control plane, and often the same data stores. How thoroughly you isolate tenants determines your blast radius, your compliance posture, your cost structure, and your operational complexity. Isolate too little and one tenant's burst, bug, or breach spills onto everyone; isolate too much and you reimplement single-tenancy at SaaS prices. This chapter covers the tenancy models that span that spectrum — silo, pool, bridge, and cell — the isolation mechanisms at each layer (compute, data, network, identity), the Kubernetes and cloud primitives that enforce them, and the hard trade-offs — noisy neighbors, data residency, per-tenant rate limiting, and migration between models — with production-grade configs for namespaces, quotas, network policies, row-level security, and tenant-aware routing.

Learning goals — after this chapter you should be able to:

- Compare silo, pool, bridge (hybrid), and cell-based tenancy on isolation strength, cost efficiency, operational overhead, and blast radius — and choose the right model per tier (free, pro, enterprise) and per compliance requirement.
- Design data isolation — discriminator column, schema-per-tenant, database-per-tenant, and physical sharding — with row-level security,

per-tenant encryption, and backup/restore that respects tenancy boundaries.

- Implement Kubernetes multi-tenancy with namespaces, RBAC, `ResourceQuota`, `LimitRange`, `NetworkPolicy`, `PodSecurity`, hierarchical namespaces (Capsule/vCluster), and node-level isolation (taints, `RuntimeClass` with `gVisor/Kata`).
- Enforce tenant context propagation — JWT claims, header-based routing, and service-mesh tenancy — and build per-tenant rate limiting, bulkheads, and concurrency limits that prevent noisy neighbors.
- Reason about control-plane vs. data-plane isolation, cell-based architecture for blast-radius containment, and the noisy-neighbor problem at every layer.
- Plan tenancy migrations — pool → bridge → silo — with dual-write, backfill, and per-tenant cutover strategies.

Boundary note. Volume 2, Chapter 9 covered the kernel mechanisms — namespaces and cgroups — as isolation primitives. Volume 12, Chapter 1 showed how containers use them at fleet scale. This chapter is the *SaaS architecture* layer — how those primitives compose into tenancy models, how data and identity are isolated above the kernel, and how platform teams enforce tenancy at the API, mesh, and storage layers. Volume 5 — Databases — covers storage engines and replication; here we focus on tenancy-aware schema design and row-level enforcement. Volume 11, Chapter 10 covered bulkheads and concurrency limiting as resilience patterns; here we apply them per tenant.

Why multi-tenancy

A tenant is a security and resource boundary — typically a customer organization, but also a team, environment, or workload class that must be isolated from others. Multi-tenancy is the discipline of sharing infrastructure among tenants while preserving isolation where it matters and sharing where it is safe.

The tension is fundamental:

- **Sharing** improves utilization, reduces cost, and simplifies operations — one deployment, one pipeline, one upgrade.

- **Isolation** contains failures, protects data, satisfies compliance, and prevents noisy neighbors — at the cost of duplication and operational overhead.

Every tenancy decision is a placement on this spectrum, and different tenants often deserve different placements. A free tier with 10,000 small tenants is efficiently served by a shared pool; a single enterprise tenant paying six figures and requiring SOC 2 + data residency may justify a dedicated silo. Most mature SaaS products end up with a *tiered* tenancy model — pool for the many, bridge or silo for the few.

```

flowchart LR
    subgraph Pool[Pool – Shared Everything]
        P_API[Shared API] --> P_SVC[Shared Services]
        P_SVC --> P_DB[(Shared DB
tenant_id column)]
        P_SVC --> P_CACHE[(Shared Cache
key prefix)]
    end
    subgraph Bridge[Bridge – Hybrid]
        B_API[Shared API +
tenant router] --> B_STD[Pool – standard tenants]
        B_API --> B_ISO[Isolated – enterprise]
        B_STD --> B_DB1[(Shared DB)]
        B_ISO --> B_DB2[(Dedicated DB
per enterprise tenant)]
    end
    subgraph Silo[Silo – Dedicated Everything]
        S_API1[Dedicated Stack
tenant A] --> S_DB1[(DB A)]
        S_API2[Dedicated Stack
tenant B] --> S_DB2[(DB B)]
        S_API3[Dedicated Stack
tenant C] --> S_DB3[(DB C)]
    end
    Pool -.->|evolve as
enterprise needs grow| Bridge
    Bridge -.->|compliance or
blast radius demands| Silo

    style Pool fill:#e3f2fd
    style Bridge fill:#fff3e0
    style Silo fill:#fce4ec

```

Figure 7-1: Tenancy spectrum — pool shares everything with logical isolation, bridge shares the control plane but isolates data/compute for

enterprise tenants, silo dedicates a full stack per tenant. Most SaaS evolves left to right as requirements grow.

Isolation dimensions

Isolation must be considered at every layer — a gap at any layer breaks the boundary:

Dimension	Shared (pool)	Isolated (silo/ cell)	Mechanism
Compute	Shared pods/ nodes, cgroup limits	Dedicated nodes or clusters	ResourceQuota , taints, RuntimeClass
Data	Row-level (tenant_id) + RLS	Dedicated DB/ schema per tenant	Postgres RLS, schema-per-tenant, DB-per-tenant
Network	Shared VPC, logical separation	Dedicated VPC or subnet per tenant	NetworkPolicy , VPC per tenant, PrivateLink
Identity	Shared IdP, tenant claim in JWT	Per-tenant IdP or OIDC issuer	JWT tenant_id claim, per-tenant ClusterRole
Noisy neighbor	Per-tenant rate limits + bulkheads	Physical isolation	Token bucket per tenant, concurrency limits
Blast radius	One bad deploy affects all tenants	Cell contains failure to subset	Cell-based architecture
Compliance	Logical controls, shared audit log	Physical data residency, per-tenant encryption keys	KMS per tenant, region pinning

A common failure is strong isolation at one layer with none at another — e.g., per-tenant databases but a shared Redis without key prefixing, so one tenant's cache flush evicts everyone's entries.

Tenancy models

Pool (shared everything, logically isolated)

Every tenant shares the same deployment, the same database, the same cache. Tenancy is a *column* — `tenant_id` on every row — and a *claim* in the JWT. This is the most cost-efficient model and the correct default for tenants with similar requirements and no data-residency constraints.

Data access must be tenant-scoped at the query layer — every query includes `WHERE tenant_id = ?`, and row-level security (RLS) enforces it even if application code forgets:

```
-- Postgres row-level security – tenant isolation at the database layer
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON orders
  USING (tenant_id = current_setting('app.current_tenant')::uuid);

-- Application sets the tenant before every transaction – no query can
-- escape
BEGIN;
SET LOCAL app.current_tenant = 'a1b2c3d4-...'; -- from JWT claim
SELECT * FROM orders WHERE status = 'pending'; -- RLS appends
-- tenant_id filter automatically
COMMIT;

-- Per-tenant encryption – each tenant's sensitive columns encrypted
-- with its own KMS key
CREATE TABLE tenant_keys (
  tenant_id uuid PRIMARY KEY,
  kms_key_arn text NOT NULL,
  created_at timestamptz DEFAULT now()
);
-- Application encrypts PII with tenant's key before writing; decrypts
-- on read
```

```
// Tenant context propagation – Spring middleware sets tenant from JWT,
// RLS enforces at DB
@Component
public class TenantFilter implements Filter {
    @Override public void doFilter(ServletRequest req, ServletResponse
    res, FilterChain chain)
        throws IOException, ServletException {
        HttpServletRequest http = (HttpServletRequest) req;
        String token = http.getHeader("Authorization").replace("Bearer
        ", "");
        Claims claims = jwtParser.parseClaimsJws(token).getBody();
        String tenantId = claims.get("tenant_id", String.class);
        if (tenantId == null) {
            ((HttpServletResponse) res).sendError(403, "missing
            tenant_id claim");
            return;
        }
        TenantContext.set(tenantId);
        MDC.put("tenant_id", tenantId); // structured logging per
        tenant
        try (Connection c = dataSource.getConnection()) {
            c.createStatement().execute("SET LOCAL app.current_tenant = '" + tenant
            Id + "'");
            chain.doFilter(req, res);
        } finally {
            TenantContext.clear();
            MDC.remove("tenant_id");
        }
    }
}
```

Pool trade-offs: minimal operational overhead, best bin-packing, simplest deployments — but noisy neighbors share the same failure domain, and a query without `tenant_id` (or an RLS bypass via superuser) leaks data across tenants. Mitigate with mandatory RLS, per-tenant rate limiting, and query auditing.

Silo (dedicated everything)

Each tenant gets its own deployment, database, and often its own VPC or cluster. Tenancy is physical — there is no cross-tenant query to get wrong because there is no shared store. This is the strongest isolation and the simplest to reason about for compliance — but cost scales linearly with tenant count and operations multiply.

Silo is appropriate when: the tenant count is small (tens, not thousands), per-tenant revenue justifies dedicated infrastructure, compliance requires physical data residency or dedicated encryption domains, or blast-radius containment demands that no single failure affects more than one tenant.

Infrastructure as code for a per-tenant silo (Terraform/OpenTofu pattern):

```
# Per-tenant silo – one module instantiation per enterprise tenant
module "tenant_silo" {
  for_each = var.enterprise_tenants # map of tenant_id -> config
  source   = "../modules/tenant-silo"

  tenant_id   = each.key
  tenant_name = each.value.name
  region      = each.value.region          # data residency
  cidr_block  = each.value.cidr            # dedicated VPC

  db_instance_class = each.value.db_class
  kms_key_arn       = aws_kms_key.tenant[each.key].arn # per-tenant
  encryption        = true
  domain            = "${each.value.slug}.app.example.com"
}

# Tenant silo module – VPC + EKS node group + RDS per tenant
resource "aws_vpc" "tenant" {
  cidr_block           = var.cidr_block
  enable_dns_hostnames = true
  tags = { Tenant = var.tenant_id, Type = "silo" }
}

resource "aws_db_instance" "tenant" {
  identifier           = "db-${var.tenant_id}"
  engine              = "postgres"
  instance_class       = var.db_instance_class
  allocated_storage    = 100
  storage_encrypted    = true
  kms_key_id           = var.kms_key_arn
  vpc_security_group_ids = [aws_security_group.db.id]
  db_subnet_group_name = aws_db_subnet_group.tenant.name
  backup_retention_period = 7
  tags = { Tenant = var.tenant_id }
}
```

Silo trade-offs: strongest isolation, simplest compliance narrative, contained blast radius — but N tenants means N databases to patch, N deployments to roll out, N dashboards to watch. Automation is non-

negotiable; without it, silo becomes an operational nightmare beyond ~20 tenants.

Bridge (hybrid — shared control plane, isolated data)

Bridge is the pragmatic middle: the control plane (API gateway, routing, auth, billing) is shared, but the data plane isolates per tenant where it matters. Standard tenants share a pool database; enterprise tenants get a dedicated database; routing decides at request time.

```
flowchart TB
    Client --> GW[API Gateway  
auth + tenant resolution]
    GW --> Router{Tenant Router  
tenant_id → backend}
    Router -->|standard| Pool[Pool Fleet  
shared DB + cache]
    Router -->|enterprise_a| SiloA[Dedicated DB + Fleet  
tenant enterprise_a]
    Router -->|enterprise_b| SiloB[Dedicated DB + Fleet  
tenant enterprise_b]
    Router -->|residency: eu| EUPool[EU Pool  
region eu-west-1]

    Pool --> SharedCP[Shared Control Plane  
billing, metering, admin]
    SiloA --> SharedCP
    SiloB --> SharedCP
    EUPool --> SharedCP

    style GW fill:#e3f2fd
    style Router fill:#fff3e0
    style Pool fill:#e8f5e9
    style SiloA fill:#fce4ec
    style SiloB fill:#fce4ec
    style EUPool fill:#f3e5f5
```

Figure 7-2: Bridge model — shared gateway and control plane, tenant-aware routing to pool or dedicated backends per tenant tier and region. Combines pool efficiency for the many with silo isolation for the few.

Tenant-aware routing at the gateway (Envoy / Istio):

```

# Istio VirtualService – route by tenant header (set by gateway after
JWT verification)
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata: { name: tenant-router }
spec:
  hosts: [api.example.com]
  gateways: [prod-gateway]
  http:
    - match: [{ headers: { x-tenant-id: { exact: enterprise-a } } }]
      route:
        - destination: { host: api-enterprise-
a.prod.svc.cluster.local }
          headers:
            request:
              set: { x-tenant-tier: silo }
    - match: [{ headers: { x-tenant-id: { exact: enterprise-b } } }]
      route:
        - destination: { host: api-enterprise-
b.prod.svc.cluster.local }
        - route: # default – pool
          - destination: { host: api-pool.prod.svc.cluster.local }
  ---
# Envoy rate limiting per tenant – token bucket at the gateway
apiVersion: networking.istio.io/v1alpha1
kind: EnvoyFilter
metadata: { name: per-tenant-ratelimit }
spec:
  workloadSelector: { labels: { istio: ingressgateway } }
  configPatches:
    - applyTo: HTTP_FILTER
      match: { context: GATEWAY }
      patch:
        operation: INSERT_BEFORE
        value:
          name: envoy.filters.http.ratelimit
          typed_config:
            "@type": type.googleapis.com/
envoy.extensions.filters.http.ratelimit.v3.RateLimit
            domain: tenant_ratelimit
            request_type: external
            rate_limit_service:
              grpc_service: { envoy_grpc: { cluster_name: ratelimit } }
              transport_api_version: V3

```

Cell-based architecture

Cells extend the bridge model by sharding *all* tenants — including pool tenants — into isolated failure domains called cells. Each cell is a full

copy of the stack (compute + data) serving a subset of tenants. A failure in one cell affects only its tenants, not the entire fleet. AWS, Stripe, and Slack use cells to bound blast radius at scale.

```
flowchart TB
    Router[Cell Router]
    tenant_id --> cell --> CellA[Cell A]
    tenants 0-999
    Router --> CellB[Cell B]
    tenants 1000-1999
    Router --> CellC[Cell C]
    tenants 2000-2999
    Router --> CellD[Cell D]
    new tenants]

    CellA --> DBA[(DB A)]
    CellB --> DBB[(DB B)]
    CellC --> DBC[(DB C)]
    CellD --> DBD[(DB D)]

    CP[Global Control Plane]
    tenant-cell map
    billing, auth] --> Router

    Note[Benefits:
    • Blast radius = 1/N cells
    • Independent deploys per cell
    • Horizontal scale by adding cells
    • Per-cell chaos testing] --- CellA

    style Router fill:#fff3e0
    style CP fill:#e3f2fd
    style CellA fill:#e8f5e9
    style CellB fill:#e8f5e9
    style CellC fill:#e8f5e9
    style CellD fill:#f3e5f5
```

Figure 7-3: Cell-based architecture — tenants are sharded across independent cells, each a full stack. The cell router maps tenant to cell; the global control plane manages the mapping. Blast radius is one cell, not the fleet.

Cell assignment is typically consistent hashing on `tenant_id` so that adding a cell reassigns minimal tenants. Cell routing metadata is stored in a global, low-latency lookup (DynamoDB Global Tables, Spanner, or etcd) that the gateway reads on every request.

Kubernetes multi-tenancy

Kubernetes has no native hard multi-tenancy — namespaces are a soft boundary. True isolation requires layering several primitives:

Namespace isolation — the baseline


```

# Namespace per tenant (or per tenant tier)
apiVersion: v1
kind: Namespace
metadata:
  name: tenant-enterprise-a
  labels:
    tenant: enterprise-a
    tier: silo
    pod-security.kubernetes.io/enforce: restricted # no privileged
pods
---
# ResourceQuota – cap aggregate consumption per tenant
apiVersion: v1
kind: ResourceQuota
metadata: { name: quota, namespace: tenant-enterprise-a }
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    limits.cpu: "16"
    limits.memory: 32Gi
    count/pods: "20"
    count/services: "10"
    requests.storage: 100Gi
---
# LimitRange – default and max per pod/container (prevents one pod from
grabbing the whole quota)
apiVersion: v1
kind: LimitRange
metadata: { name: limits, namespace: tenant-enterprise-a }
spec:
  limits:
    - type: Container
      default: { cpu: 500m, memory: 512Mi }
      defaultRequest: { cpu: 200m, memory: 256Mi }
      max: { cpu: "2", memory: 4Gi }
      min: { cpu: 100m, memory: 128Mi }
    - type: Pod
      max: { cpu: "4", memory: 8Gi }
---
# NetworkPolicy – deny cross-tenant traffic by default, allow only
within namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: deny-cross-tenant, namespace: tenant-enterprise-a }
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
  ingress:
    - from:

```

```

    - podSelector: {}          # only same-namespace pods
    - namespaceSelector:
        matchLabels: { name: ingress-nginx } # plus ingress
    ports: [{ port: 8080 }]
    egress:
    - to: [{ podSelector: {} }] # only same-namespace
    - to: [{ namespaceSelector: { matchLabels: { name: kube-
system } } }]
        ports: [{ port: 53, protocol: UDP }] # DNS
    - to: [{ namespaceSelector: { matchLabels: { name: tenant-
enterprise-a } } }]
    ---
# RBAC — per-tenant access, no cross-namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: tenant-admin, namespace: tenant-enterprise-a }
rules:
  - apiGroups: [ "", apps, networking.k8s.io ]
    resources: [ pods, deployments, services, ingresses, configmaps, sec
rets ]
    verbs: [ get, list, watch, create, update, patch, delete ]
  ---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: tenant-admin, namespace: tenant-enterprise-a }
subjects: [{ kind: Group, name: enterprise-a:developers, apiGroup: rbac
.authorization.k8s.io }]
roleRef: { kind: Role, name: tenant-admin, apiGroup: rbac.authorization
.k8s.io }

```

For stronger isolation, two extensions are common:

- **Capsule / Hierarchical Namespace Controller (HNC)** — enforces quota and policy inheritance across namespace hierarchies, prevents tenants from escaping their subtree, and manages per-tenant `ResourceQuota` centrally.
- **vCluster** — gives each tenant a virtual Kubernetes control plane (virtual apiserver + etcd) inside a host cluster namespace. Tenants get full cluster-admin within their vCluster without host-cluster privileges. Appropriate when tenants need CRDs or operators.

Node-level isolation

When cgroup and namespace isolation is insufficient (noisy neighbors at the kernel or hardware level, or compliance requiring dedicated hardware), isolate at the node:

```

# Dedicated node pool per enterprise tenant – taint + toleration +
nodeSelector
apiVersion: v1
kind: Node
metadata:
  labels: { tenant: enterprise-a, node-pool: enterprise-a }
spec:
  taints: [{ key: tenant, value: enterprise-a, effect: NoSchedule }]
---
apiVersion: apps/v1
kind: Deployment
metadata: { name: api, namespace: tenant-enterprise-a }
spec:
  template:
    spec:
      tolerations: [{ key: tenant, value: enterprise-a, effect: NoSchedule }]
      nodeSelector: { tenant: enterprise-a }
      runtimeClassName: gvisor # extra kernel isolation via gVisor
      containers:
        - name: api
          image: api:1.42.0
          resources:
            requests: { cpu: 500m, memory: 512Mi }
            limits: { cpu: 1000m, memory: 1Gi }
---
# RuntimeClass – gVisor for tenant workloads needing stronger syscall
filtering
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata: { name: gvisor }
handler: runsc
---
# Kata Containers alternative – VM-level isolation per pod
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata: { name: kata }
handler: kata

```

Isolation level	Mechanism	Overhead	Use when
Namespace + quota	ResourceQuota , LimitRange , NetworkPolicy	Negligible	Default for all tenants

Isolation level	Mechanism	Overhead	Use when
Node pool	Taints, <code>nodeSelector</code> , dedicated ASG	Low (bin-packing loss)	Enterprise, noisy-neighbor sensitive
Sandbox runtime	gVisor (<code>runsc</code>)	~5-10% syscall overhead	Untrusted tenant code execution
VM isolation	Kata Containers / Firecracker	~50-100 ms cold start, higher memory	Strongest isolation, compliance

Data isolation in depth

Choosing the model

Model	Isolation	Cost	Operations	Migration
Discriminator column (<code>tenant_id</code>)	Logical — RLS enforced	1 DB for N tenants	Simplest	Trivial
Schema per tenant	Logical — schema <code>search_path</code>	1 DB, N schemas	Moderate — per-schema migrations	Moderate
Database per tenant	Physical — separate DB	N DBs	Heavy — per-DB patching/backups	Heavy
Shard per tenant group	Physical — tenant → shard map	Shards scale with tenants	Heavy — rebalancing	Complex

Decision factors: tenant count, per-tenant data size, compliance (dedicated encryption domain), and query patterns (cross-tenant analytics is trivial with discriminator, impossible with DB-per-tenant without ETL).

Schema-per-tenant (Postgres)

```
-- One schema per tenant – search_path isolates
CREATE SCHEMA tenant_enterprise_a;
CREATE SCHEMA tenant_enterprise_b;

-- Per-tenant tables – identical DDL, isolated data
CREATE TABLE tenant_enterprise_a.orders (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    amount_cents int NOT NULL,
    status text NOT NULL,
    created_at timestamptz DEFAULT now()
);

-- Application sets search_path per request
SET search_path TO tenant_enterprise_a, public;
SELECT * FROM orders; -- resolves to tenant_enterprise_a.orders

-- Migration runner – applies DDL to every tenant schema
DO $$
DECLARE s text;
BEGIN
    FOR s IN SELECT schema_name FROM information_schema.schemata WHERE sc
hema_name LIKE 'tenant_%'
    LOOP
        EXECUTE format('CREATE INDEX IF NOT EXISTS idx_orders_status ON
%i.orders(status)', s);
    END LOOP;
END $$;
```

Schema-per-tenant simplifies per-tenant backup/restore (`pg_dump --schema=tenant_enterprise_a`) and per-tenant PITR, but schema count in Postgres has practical limits (~hundreds before catalog bloat and `pg_dump` slowness) and every migration must fan out to N schemas.

Per-tenant encryption

Whether using pool or silo, encrypt tenant-sensitive data with a per-tenant KMS key so that key revocation immediately renders that tenant's data unreadable — and a compromised key affects only one tenant:

```
# Envelope encryption per tenant – DEK per record, KEK per tenant in KMS
import boto3
kms = boto3.client("kms")

def encrypt_for_tenant(tenant_id: str, plaintext: bytes) -> dict:
    key_arn = tenant_key_arn(tenant_id) # lookup tenant -> KMS key
    resp = kms.generate_data_key(KeyId=key_arn, KeySpec="AES_256")
    dek_plaintext, dek_encrypted = resp["Plaintext"], resp["CiphertextBlob"]
    # Encrypt record with DEK, store DEK_encrypted alongside
    ciphertext = aes_gcm_encrypt(dek_plaintext, plaintext)
    return {"ciphertext": ciphertext, "dek_encrypted": dek_encrypted, "key_arn": key_arn}

def decrypt_for_tenant(record: dict) -> bytes:
    dek_plaintext = kms.decrypt(CiphertextBlob=record["dek_encrypted"])["Plaintext"]
    return aes_gcm_decrypt(dek_plaintext, record["ciphertext"])
```

Noisy neighbors — per-tenant fairness

Sharing without fairness is just contention. Every shared resource — CPU, downstream concurrency, cache, database connections — needs per-tenant limits.

Per-tenant rate limiting and concurrency

Apply at the gateway (Envoy RLS) and at the service (application-level token bucket per tenant). Gateway limits protect the fleet; service limits protect the downstream.

```

// Per-tenant token bucket – Caffeine-backed, tenant_id -> bucket
public class PerTenantRateLimiter {
    private final LoadingCache<String, Bucket> buckets = Caffeine.newBuilder()
        .expireAfterAccess(Duration.ofMinutes(10))
        .build(tenantId -> {
            TenantTier tier = tenantService.tierOf(tenantId);
            // Tier-aware limits – free: 100 rps, pro: 1000 rps,
enterprise: 5000 rps
            Bandwidth limit = Bandwidth.classic(tier.rps(), Refill.greedy(tier.rps(), Duration.ofSeconds(1)));
            return Bucket.builder().addLimit(limit).build();
        });

    public boolean tryConsume(String tenantId) {
        Bucket bucket = buckets.get(tenantId);
        ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);
;
        if (!probe.isConsumed()) {
            meterRegistry.counter("ratelimit.exceeded", "tenant", tenantId).increment();
        }
        return probe.isConsumed();
    }
}

// Per-tenant bulkhead – prevents one tenant's slow requests from starving others
public class PerTenantBulkhead {
    private final ConcurrentHashMap<String, Bulkhead> bulkheads = new ConcurrentHashMap<>();
    public <T> T execute(String tenantId, Supplier<T> call) {
        Bulkhead bh = bulkheads.computeIfAbsent(tenantId,
            id -> Bulkhead.of("tenant-" + id,
                BulkheadConfig.custom().maxConcurrentCalls(20).maxWaitDuration(Duration.ofMillis(50)).build()));
        return Bulkhead.decorateSupplier(bh, call).get();
    }
}

```

noisy-neighbor detection

Alert on tenant-level p95 and error rate, not just global — a global p95 of 80 ms can hide one tenant at 2 s:

```

groups:
  - name: tenancy
    rules:
      - alert: TenantP95High
        expr: histogram_quantile(0.95,
rate(http_request_duration_seconds_bucket{tenant!=""}[5m])) > 0.5
        for: 5m
        labels: { severity: warning }
        annotations: { summary: "Tenant {{ $labels.tenant }} p95 >500ms"
}

      - alert: TenantErrorRateHigh
        expr: rate(http_requests_total{status=~"5..", tenant!=""}
[5m]) / rate(http_requests_total{tenant!=""}[5m]) > 0.05
        for: 3m
        labels: { severity: warning }
        annotations: { summary: "Tenant {{ $labels.tenant }} 5xx rate >
5%" }

      - alert: TenantQuotaExhaustion
        expr: kube_resourcequota{type="used",
resource="requests.cpu"} / kube_resourcequota{type="hard",
resource="requests.cpu"} > 0.9
        labels: { severity: info }
        annotations: { summary: "Namespace {{ $labels.namespace }} quot
a >90%" }

```

Control plane vs. data plane

The control plane (tenant lifecycle, billing, auth, routing map, admin APIs) is global and shared — it must be strongly consistent and highly available. The data plane (request serving, storage, caching) is per-tenant or per-cell and can be eventually consistent within the cell.

Isolate them physically:

- **Separate clusters or namespaces** — control plane in `control-plane` cluster, data-plane cells in `cell-*` clusters. A data-plane outage does not take down tenant provisioning or billing.
- **Separate data stores** — control-plane state (tenant → cell mapping, entitlements) in a global strongly-consistent store (DynamoDB Global Tables, Spanner); cell state in the cell's own database.
- **Separate deploy pipelines** — control-plane deploys do not roll data-plane cells simultaneously; canary one cell at a time.

This separation also enables *reconciliation* — the control plane can detect that a cell is unhealthy and re-route tenants or trigger failover without coupling to the cell's own health.

Migrating between models

Tenancy migrations are among the highest-risk operations in SaaS — they move live customer data. Three patterns cover most cases:

Migration	Strategy	Downtime
Pool → bridge (one tenant to dedicated DB)	Dual-write + backfill + cutover per tenant	Zero with careful cutover
Bridge → silo (dedicated DB to dedicated VPC)	Snapshot + restore to new VPC, DNS cutover	Seconds (DNS TTL)
Pool → cells (shard tenants)	Consistent-hash reassignment, per-cell backfill	Zero — tenants moved one by one

Pool → bridge cutover (per tenant, zero-downtime):

```
Phase 1: Provision dedicated DB for tenant, enable CDC from pool (Debezium).
Phase 2: Dual-write – application writes to both pool and dedicated DB for this tenant.
Phase 3: Backfill – copy historical rows for this tenant from pool to dedicated DB.
Phase 4: Verify – row counts, checksums, canary reads against dedicated DB.
Phase 5: Flip router – tenant_router now points tenant_id → dedicated DB.
Phase 6: Drain – after TTL (e.g., 7 days), remove tenant rows from pool, disable dual-write.
```

Each phase must be reversible. Keep the pool rows until the dedicated path has served production traffic for at least one full business cycle.

Key takeaways

- Tenancy is a spectrum — pool (shared everything, `tenant_id` column + RLS) for cost efficiency at scale, silo (dedicated stack per tenant) for strongest isolation and compliance, bridge (shared control plane, isolated data plane per tier) for the pragmatic middle, and cells (sharded full stacks) for blast-radius containment at large scale. Most SaaS uses a tiered model: pool for the many, dedicated for the few.
- Isolation must be enforced at every layer — compute (`ResourceQuota` , node pools), data (RLS or schema/DB per tenant, per-tenant KMS keys), network (`NetworkPolicy` , VPC per tenant), and identity (tenant claim in JWT, per-tenant RBAC). A gap at any layer breaks the boundary.
- Kubernetes namespaces are soft boundaries — layer `ResourceQuota` , `LimitRange` , `NetworkPolicy` , `PodSecurity` , and RBAC as the baseline; add Capsule/HNC for hierarchy enforcement, vCluster for virtual control planes, and `RuntimeClass` (gVisor/Kata) or dedicated node pools when kernel or hardware isolation is required.
- Noisy neighbors are prevented with per-tenant rate limiting (token bucket at gateway + service), per-tenant bulkheads and concurrency limits, and tier-aware quotas. Alert on per-tenant p95 and error rate, not just global — global metrics hide per-tenant degradation.
- Cell-based architecture bounds blast radius to one cell (1/N of tenants) and enables independent deploys and chaos testing per cell. Control plane (tenant mapping, billing) must be isolated from data-plane cells in separate clusters and stores.
- Migrating tenancy models is high-risk — use dual-write + backfill + per-tenant cutover with verification and a drain period; never move all tenants at once. Keep the old path until the new path has served production traffic for a full business cycle.

Further reading

- AWS SaaS Boost and SaaS Factory — *SaaS Tenant Isolation Strategies* — <https://docs.aws.amazon.com/whitepapers/latest/saas-tenant-isolation-strategies/saas-tenant-isolation-strategies.html> — authoritative survey of pool/silo/bridge with AWS primitives.

- *Multi-Tenant Architecture for SaaS* — Microsoft Learn — <https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/overview> — tenancy models, data partitioning, and noisy-neighbor guidance for Azure.
- Kubernetes — *Multi-tenancy* documentation — <https://kubernetes.io/docs/concepts/security/multi-tenancy/> — namespace isolation, quotas, and policy enforcement.
- Capsule — <https://capsule.clastix.io> — policy-based multi-tenancy operator for Kubernetes.
- vCluster — <https://www.vcluster.com/docs> — virtual clusters for hard multi-tenancy.
- Postgres — *Row Security Policies* — <https://www.postgresql.org/docs/current/ddl-rowsecurity.html> — RLS syntax and performance considerations.
- Oracle — *Cell-Based Architecture* (AWS re:Invent talk, Adrian Hornsby) — <https://www.youtube.com/watch?v=8pQ9I8J8J8Q> — practical cell design at AWS scale.

Chapter 8 — Cloud Cost and Capacity Engineering

What this chapter covers. Cloud bills are the second-order effect of every architectural decision — instance family, replica count, retention window, cross-AZ chatter, uncompressed images, and forgotten EBS volumes all compound into a monthly invoice that can dwarf headcount if left unmanaged. Cost engineering is not finance — it is backend engineering applied to the cloud's economic model: understanding pricing primitives, instrumenting cost visibility, right-sizing compute and storage, exploiting commitment and spot markets, taming data-transfer costs, and building the capacity-planning discipline that keeps performance and spend jointly optimized. This chapter covers the full FinOps lifecycle — inform, optimize, operate — with real billing queries, Terraform tagging and policy, autoscaling and spot configs, and the capacity mathematics that connects Little's Law to purchase commitments.

Learning goals — after this chapter you should be able to:

- Explain cloud pricing models — on-demand, Reserved Instances, Savings Plans, spot/preemptible, and Graviton/ARM — and choose the right model per workload shape (steady-state, spiky, batch, fault-tolerant).
- Instrument cost visibility — tagging strategy, AWS CUR + Athena, Cost Explorer, GCP Billing Export + BigQuery, and allocation to team/service/tenant — so every dollar is attributable.
- Right-size compute — vertical (VPA, Compute Optimizer), horizontal (HPA, KEDA, Karpenter), and commitment (RI/SP coverage, spot diversification) — with production configs and the metrics that prove a saving is safe.
- Attack the three biggest cost drivers after compute: storage (lifecycle, tiering, orphaned volumes), data transfer (cross-AZ, egress, NAT Gateway), and managed services — with concrete reductions and guardrails.
- Apply capacity engineering — Little's Law, the Universal Scalability Law, and queueing models — to forecast demand, size fleets, and decide when to buy commitments vs. stay flexible.
- Operate FinOps — showback/chargeback, budgets and alerts, policy-as-code guardrails (OPA/Sentinel), and the organizational cadence that keeps cost optimization continuous rather than quarterly.

Boundary note. Volume 11, Chapter 7 introduced load testing and capacity planning as reliability practices — generating load, measuring breaking points, and relating utilization to latency. This chapter is the *economic* treatment — how capacity decisions translate to spend, how pricing models reward different commitments, and how to keep that spend efficient at scale. Volume 12, Chapter 5 covered cloud compute/storage/network primitives; pricing is the economic view of those same primitives. Volume 7, Chapter 2 covered system-design estimation; here we apply estimation to fleet sizing and purchase planning.

Why cost engineering is an engineering discipline

The cloud cost equation

Every cloud bill decomposes into a small number of drivers:

Total = Compute + Storage + Data Transfer + Managed Services + Support
~60-70% ~15-20% ~5-15% ~5-15%
~3-10% of above

For a typical backend fleet serving user-facing traffic, compute dominates — EC2/GKE node cost is usually 50-70% of the bill. Within compute, the distribution is highly skewed: a handful of large services or data pipelines often account for half the spend. The Pareto applies aggressively — optimizing the top five cost centers yields most of the saving.

The cost engineering loop is simple to state, hard to sustain:

1. **See** — attribute every dollar to a team, service, environment, and tenant via tags and billing exports.
2. **Decide** — compare the current shape (instance family, replica count, pricing model) to the efficient shape (right-sized, committed, spot where fault-tolerant).
3. **Act** — apply the change via IaC, autoscaling, or purchase — with guardrails so optimization does not break performance or availability.
4. **Verify** — confirm the saving landed, performance held, and the change did not create new waste elsewhere.

Without step 1, steps 2-4 are guesswork. Most teams' first win is not a clever optimization but simply making the bill legible.

```

flowchart LR
    subgraph Inform[Inform – See]
        Tag[Tagging]
    end
    + CUR/Billing Export]
    Tag --> Alloc[Allocation]
    team / service / tenant]
    Alloc --> Dash[Dashboards]
    Cost Explorer / Looker]
    end
    subgraph Optimize[Optimize – Decide + Act]
        Dash --> RS[Right-size]
        VPA / HPA / Karpenter]
        Dash --> Commit[Commit]
        RI / Savings Plans]
        Dash --> Spot[Spot / Preemptible]
        batch + fault-tolerant]
        Dash --> Storage[Storage + Transfer]
        lifecycle + tiering]
    end
    end
    subgraph Operate[Operate – Verify + Govern]
        RS --> Guard[Guardrails]
        OPA / budgets / alerts]
        Commit --> Guard
        Spot --> Guard
        Storage --> Guard
        Guard --> Tag
    end
    end
    style Inform fill:#e3f2fd
    style Optimize fill:#e8f5e9
    style Operate fill:#fff3e0

```

Figure 8-1: FinOps lifecycle — inform makes spend visible and attributable, optimize acts on the biggest drivers, operate governs continuously via budgets, alerts, and policy. The loop never terminates.

Cost visibility — making the bill legible

Tagging strategy

Tags are the primary key for cost allocation. Without consistent tags, the bill is an unindexed table — you know the total but cannot slice by owner or service.

Minimum viable tagging policy (enforced at provision time, not retroactively):

Tag	Example	Purpose	Enforced
team	checkout , platform	Chargeback to owning team	Required
service	catalog , api-gateway	Per-service cost and unit economics	Required
env	prod , staging , dev	Separate prod from non-prod	Required
tenant / tenant_tier	enterprise-a , pool	Per-tenant cost (see Ch. 7)	For SaaS
cost_center	CC-4721	Finance mapping	Required
managed_by	terraform , karpenter	Identify waste source	Recommended

Enforce via Terraform and policy — not documentation:

```
# Terraform – default tags on every resource via provider block
provider "aws" {
  default_tags {
    tags = {
      team      = var.team
      service   = var.service
      env       = var.env
      cost_center = var.cost_center
      managed_by = "terraform"
      repo      = "github.com/org/infra"
    }
  }
}

# Tag policy – OPA/Conftest gate in CI: every resource must carry
# required tags
# policy/tagging.rego
package main
required_tags := {"team", "service", "env", "cost_center"}
deny[msg] {
  resource := input.resource_changes[_]
  resource.type == "aws_instance"
  not has_required_tags(resource.change.after.tags)
  msg := sprintf("Resource %v missing required tags %v", [resource.address, missing_tags(resource.change.after.tags)])
}
has_required_tags(tags) { count({t | t := required_tags[_]; tags[t] != null}) == count(required_tags) }
missing_tags(tags) := {t | t := required_tags[_]; tags[t] == null}
```

```
# AWS Tag Policy (Organizations) – reject untagged EC2/RDS at creation
time
# organizations tag policy – attach to OU
tags:
  team:
    tag_key: { "@@assign": team }
    enforced_for: { "@@assign": [ec2:instance, rds:db, s3:bucket] }
  env:
    tag_key: { "@@assign": env }
    enforced_for: { "@@assign": [ec2:instance, rds:db, s3:bucket] }
```

Billing exports and allocation

Raw billing data is the source of truth — Cost Explorer is a summary; the Cost and Usage Report (CUR) / Billing Export is the ledger.


```

-- Athena on AWS CUR (Parquet, partitioned by year/month) – top 10
services by cost, last 30 days
SELECT
    line_item_product_code          AS service,
    resource_tags_user_service      AS service_tag,
    resource_tags_user_team         AS team,
    SUM(line_item_unblended_cost)   AS cost_usd,
    SUM(line_item_usage_amount)     AS usage
FROM cur_db.cur_table
WHERE line_item_usage_start_date >= date_add('day', -30, current_date)
    AND line_item_line_item_type = 'Usage'
    -- exclude credits, refunds, taxes for clean allocation
    AND line_item_unblended_cost > 0
GROUP BY 1, 2, 3
ORDER BY cost_usd DESC
LIMIT 20;

-- Orphaned EBS volumes – paying for storage with no attachment
SELECT
    line_item_resource_id,
    product_volume_type,
    line_item_usage_amount AS gb_months,
    line_item_unblended_cost AS cost
FROM cur_db.cur_table
WHERE line_item_product_code = 'AmazonEC2'
    AND line_item_usage_type LIKE '%EBS:VolumeUsage%'
    AND line_item_resource_id NOT IN (
        SELECT volume_id FROM ebs_attachments_snapshot -- join with Config
        snapshot
    )
ORDER BY cost DESC;

-- Cross-AZ transfer – often the largest hidden cost after compute
SELECT
    line_item_usage_type,
    SUM(line_item_unblended_cost) AS cost,
    SUM(line_item_usage_amount)   AS gb
FROM cur_db.cur_table
WHERE line_item_product_code = 'AWSDataTransfer'
    AND line_item_usage_type LIKE '%Regional%'
GROUP BY 1
ORDER BY cost DESC;

```

GCP equivalent — Billing Export to BigQuery:

```
-- BigQuery on GCP billing export – per-service, per-team (labels)
SELECT
  service.description,
  labels.value AS team,
  SUM(cost) AS cost_usd
FROM `project.billing.gcp_billing_export_v1_XXXXXX`
WHERE labels.key = 'team'
      AND usage_start_time >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 3
0 DAY)
GROUP BY 1, 2
ORDER BY cost_usd DESC
LIMIT 20;
```

Build a weekly cost review around these queries — top movers, new untagged spend, and cross-AZ transfer trends. A single untagged NAT Gateway or cross-AZ Kafka replication can quietly add thousands per month.

Cloud pricing models

```

flowchart TB
    OD[On-Demand] --> RI[Reserved Instances]
    RI --> RI1[1-3 yr commitment]
    RI1 --> RI2[~30-40% discount]
    RI2 --> RI3[capacity reservation]
    RI3 --> SP[Savings Plans]
    SP --> SP1[1-3 yr commit to $/hr]
    SP1 --> SP2[~20-30% discount]
    SP2 --> SP3[flexible family/region]
    SP3 --> Spot[Spot / Preemptible]
    Spot --> Spot1[~60-90% discount]
    Spot1 --> Spot2[interruptible]
    Spot2 --> Spot3[no commitment]
    Spot3 --> Graviton[Graviton / ARM]
    Graviton --> Graviton1[~20% better price/perf]
    Graviton1 --> Graviton2[no commitment]
    Graviton2 --> Graviton3[requires rebuild]

    RI & SP & Spot & Graviton --> Choice{Workload shape?}
    Choice -->|steady-state| CommitChoice[Commit: RI or SP]
    CommitChoice -->|spiky / elastic| ElasticChoice[On-Demand + Autoscaling]
    ElasticChoice -->|batch / fault-tolerant| SpotChoice[Spot with diversification]
    SpotChoice -->|portable compute| GravitonChoice[Graviton migration]

    style OD fill:#e3f2fd
    style RI fill:#fff3e0
    style SP fill:#fff3e0
    style Spot fill:#fce4ec
    style Graviton fill:#e8f5e9

```

Figure 8-2: Pricing model comparison. On-demand is the baseline; commitments (RI/SP) discount steady-state, spot discounts fault-tolerant batch, Graviton discounts portable compute. Most fleets blend all four.

Model	Discount vs. on-demand	Commitment	Interruption	Best for
On-demand	— (baseline)	None	Never	Spiky, unpredictable, short-lived

Model	Discount vs. on-demand	Commitment	Interruption	Best for
Standard RI	~40% (1 yr), ~60% (3 yr)	1-3 yr, specific family/AZ	Never (capacity reserved)	Steady-state, known shape
Convertible RI	~30% / ~45%	1-3 yr, exchangeable family	Never	Steady-state with expected family drift
Compute Savings Plan	~25-30%	1-3 yr, \$/hr commit, any family/region	Never	Steady-state with flexibility
Spot	~60-90%	None	2-min warning (AWS), 30 s (GCP)	Fault-tolerant batch, CI, stateless scale-out
Graviton (ARM)	~20% price/perf	None (just rebuild)	Never	Portable, compute-heavy, non-x86-locked

Commitment strategy

Commit only what you are confident will run — typically 60–80% of steady-state baseline. Overcommitting is worse than under-committing: an unused RI still bills. Use Cost Explorer's RI/SP recommendations and track coverage and utilization:

Coverage = committed spend / total eligible spend – target 60-80% for steady-state
 Utilization = used committed hours / purchased hours – target >90%; <80% means overbought

Purchase cadence: ladder commitments (stagger start dates quarterly) so capacity can be adjusted without a cliff. Prefer Compute Savings Plans over Standard RIs unless you need the capacity reservation or a regional AZ guarantee.

Spot for fault-tolerant workloads

Spot is the largest discount available without commitment — but the instance can be reclaimed. Use it only where interruption is handled: stateless scale-out behind an ASG, batch/ETL, CI runners, and non-critical canaries.

```

# Karpenter – spot diversification across families and AZs, with on-
demand fallback
apiVersion: karpenter.sh/v1beta1
kind: NodePool
metadata: { name: spot-pool }
spec:
  template:
    spec:
      requirements:
        - key: karpenter.sh/capacity-type
          operator: In
          values: [spot, on-demand]          # prefer spot, fall back to
on-demand
        - key: kubernetes.io/arch
          operator: In
          values: [amd64, arm64]            # blend x86 + Graviton for
diversification
        - key: karpenter.k8s.aws/instance-family
          operator: In
          values: [m7i, m7g, c7i, c7g, r7i,
r7g] # multiple families – reduces interruption correlation
        - key: topology.kubernetes.io/zone
          operator: In
          values: [us-east-1a, us-east-1b, us-east-1c]
        nodeClassRef: { name: default }
        expireAfter:
720h          # rotate nodes weekly to pick up new spot
pricing
      disruption:
        consolidationPolicy: WhenUnderutilized
        consolidateAfter: 30s
      limits:
        cpu: 1000
---
# MixedInstances ASG – alternative for non-Karpenter fleets
resource "aws_autoscaling_group" "batch" {
  mixed_instances_policy {
    instances_distribution {
      on_demand_base_capacity          = 1    # at least 1 on-
demand for baseline
      on_demand_percentage_above_base_capacity = 20 # 20% on-demand,
80% spot
      spot_allocation_strategy        = "price-capacity-
optimized"
    }
    launch_template { launch_template_specification { launch_template_i
d = aws_launch_template.batch.id } }
    override { instance_type = "m7i.large" }
    override { instance_type = "m7g.large" }
    override { instance_type = "c7i.large" }

```

```
        override { instance_type = "c7g.large" }
    }
}
```

Handle interruption gracefully — every spot workload must handle SIGTERM (AWS 2-min warning via instance metadata, Karpenter consolidation):

```
# Pod handling spot interruption – terminationGracePeriod + preStop
spec:
  terminationGracePeriodSeconds: 120 # give 2 min to drain (matches
spot warning)
  containers:
    - name: worker
      lifecycle:
        preStop:
          exec: { command: ["/bin/sh", "-c", "curl -X POST localhost:80
80/drain && sleep 10"] }
```

Right-sizing compute

Right-sizing is the process of matching provisioned capacity to actual utilization. Overprovisioning is the default — teams size for peak, forget to downsize after optimization, and accumulate slack that compounds across the fleet.

Vertical right-sizing (per pod / per instance)

The signal is `container_cpu_usage` vs. `container_spec_cpu_limit` and `container_memory_working_set` vs. `container_spec_memory_limit`, plus AWS Compute Optimizer / GCP Recommender.

```

# Vertical Pod Autoscaler – recommend (and optionally apply) right-
sized requests
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata: { name: catalog-vpa, namespace: prod }
spec:
  targetRef: { apiVersion: apps/v1, kind: Deployment, name: catalog }
  updatePolicy:
    updateMode: "Auto" # "Off" for recommendation-only, "Auto" to
evict and reschedule
    resourcePolicy:
      containerPolicies:
        - containerName: catalog
          minAllowed: { cpu: 100m, memory: 128Mi }
          maxAllowed: { cpu: "4", memory: 8Gi }
          controlledResources: [cpu, memory]
  ---
# In-place Pod resize (K8s 1.27+ alpha) – no eviction for CPU/memory
adjustment
# spec.containers[].resources with restartPolicy: NotRequired
(KEP-1287)

```

Rule of thumb: target 60–75% CPU request utilization (p95) and 70–85% memory. Below 30% sustained utilization, halve the request. Above 85%, investigate before blindly increasing — the service may need horizontal scaling or optimization, not more vertical headroom.

For EC2/GCE instances, Compute Optimizer's recommendation is the starting point — but verify against application metrics (GC pressure, p95 latency) before downsizing. A smaller instance that triggers more GC pauses is not a saving.

Horizontal right-sizing (replica count and autoscaling)

```
# Horizontal Pod Autoscaler – CPU + custom metric (requests per second
per pod)
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: catalog-hpa, namespace: prod }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: catalog }
  minReplicas: 3
  maxReplicas: 50
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization,
averageUtilization: 65 } }
    - type: Pods
      pods:
        metric: { name: http_requests_per_second }
        target: { type: AverageValue, averageValue: "1000" }
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies: [{ type: Percent, value: 50, periodSeconds: 60 }]
    scaleDown:
      stabilizationWindowSeconds: 300 # slower scale-down avoids
flapping
      policies: [{ type: Pods, value: 2, periodSeconds: 60 }]
---
# KEDA – event-driven autoscaling (queue depth, Kafka lag, custom
Prometheus)
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: { name: order-processor, namespace: prod }
spec:
  scaleTargetRef: { name: order-processor }
  minReplicaCount: 2
  maxReplicaCount: 100
  triggers:
    - type: aws-sqs-queue
      metadata:
        queueURL: https://sqs.us-east-1.amazonaws.com/123456789/orders
        queueLength: "100" # one pod per 100 messages
        awsRegion: us-east-1
    - type: prometheus
      metadata:
        serverAddress: http://prometheus.monitoring.svc:9090
        metricName: http_request_queue_depth
        threshold: "50"
        query: sum(http_requests_queued{service="catalog"})
```

Karpenter (or Cluster Autoscaler) closes the loop — HPA adds pods, Karpenter adds nodes of the right size and family, and consolidation removes underutilized nodes within 30-60 s. Without node autoscaling, HPA pods sit pending; without consolidation, the fleet accumulates half-empty nodes.

Graviton / ARM migration

For portable, compute-heavy services (API servers, data pipelines, caches), Graviton 3/4 delivers ~20-40% better price-performance over comparable x86. The migration is a rebuild + multi-arch image, not a rewrite — unless the service has x86 assembly, SIMD intrinsics, or native dependencies without ARM builds.

```
# Multi-arch image – buildx for amd64 + arm64
# docker buildx build --platform linux/amd64,linux/arm64 -t api:1.42.0
--push .
FROM --platform=$BUILDPLATFORM golang:1.22 AS build
ARG TARGETARCH
RUN GOARCH=$TARGETARCH go build -o /api ./cmd/api

FROM gcr.io/distroless/base-debian12
COPY --from=build /api /api
ENTRYPOINT ["/api"]
```

```
# Karpenter – prefer Graviton, fall back to x86 (price-performance
aware)
requirements:
  - key: kubernetes.io/arch
    operator: In
    values: [arm64, amd64]
  - key: karpenter.k8s.aws/instance-cpu
    operator: Gt
    values: ["2"]
# Weight spot + Graviton highest in provisioner priority – scheduler
prefers cheaper
```

Measure before and after — p95 latency, throughput per dollar, and any JNI/native failures — on a canary subset before fleet-wide rollout.

Storage, data transfer, and managed services

Storage

Storage cost is linear in GB-months and retention — and it is easy to forget. The largest storage wastes are: unattached EBS volumes, old snapshots, verbose logs retained at high durability, and single-tier retention for data with tiered access patterns.

```

# S3 lifecycle – tier by access pattern, expire aggressively for non-
prod
resource "aws_s3_bucket_lifecycle_configuration" "app" {
  bucket = aws_s3_bucket.app.id
  rule {
    id      = "tier-and-expire"
    status  = "Enabled"
    filter { prefix = "logs/" }
    transition { days = 30, storage_class = "STANDARD_IA" }
    transition { days = 90, storage_class = "GLACIER" }
    expiration { days = 365 }
    noncurrent_version_expiration { noncurrent_days = 30 }
  }
  rule {
    id      = "abort-multipart"
    status  = "Enabled"
    filter {}
    abort_incomplete_multipart_upload { days_after_initiation = 7 }
  }
}

# EBS – detect and alert on unattached volumes (Config rule + Lambda
cleanup)
resource "aws_config_config_rule" "unattached_ebs" {
  name = "unattached-ebs-volumes"
  source { owner = "AWS", source_identifier =
"EC2_VOLUME_INUSE_CHECK" }
}

# RDS – right-size storage, enable storage autoscaling, clean old
snapshots
resource "aws_db_instance" "prod" {
  allocated_storage      = 100
  max_allocated_storage  = 500 # autoscaling ceiling – avoids manual
resize
  storage_type           =
"gp3" # gp3 is ~20% cheaper than gp2, with provisioned IOPS
  storage_encrypted      = true
  backup_retention_period = 7 # not 35 unless compliance requires it
  delete_automated_backups = false
}

```

For Kubernetes, enforce `PersistentVolumeClaim` quotas and storage-class defaults that prevent unbounded `gp2` claims.

Data transfer

Transfer is the most regressive cost — it scales with success (more traffic → more transfer) and is easy to overlook until it is 15% of the bill. The hierarchy of cost:

Transfer	Cost (AWS, approx.)	Mitigation
Within AZ (same AZ)	Free	Co-locate chatty services in the same AZ (with HA trade-off)
Cross-AZ (same region)	\$0.01/GB each way	Reduce cross-AZ chatter, co-locate cache with compute, use AZ-aware routing
Cross-region	\$0.02/GB	Replicate only what must be global, compress, batch
Internet egress	\$0.05–0.09/GB (tiered)	CloudFront/CDN, compress, cache at edge
NAT Gateway	\$0.045/GB + hourly	VPC endpoints (Gateway/Interface) for S3/DynamoDB/ECR, egress VPC design

Mitigations, in order of impact:

1. **VPC endpoints** — eliminate NAT Gateway for S3, DynamoDB, ECR, CloudWatch. For a fleet pulling images and writing logs, this alone can save 30–50% of NAT cost.
2. **AZ-aware routing** — Istio locality-aware routing, Kafka `replica.selector.class` for AZ affinity, and Redis cluster with AZ-aligned primaries reduce cross-AZ replication.
3. **CDN for egress** — CloudFront caches at edge, reducing origin egress and improving latency; set aggressive `Cache-Control` for static assets and API responses where staleness is acceptable.
4. **Compression** — gRPC + gzip/zstd, JSON → Protobuf, image optimization — bandwidth saved is egress saved.

```
# VPC Gateway Endpoints – bypass NAT for S3 and DynamoDB
resource "aws_vpc_endpoint" "s3" {
  vpc_id      = aws_vpc.main.id
  service_name = "com.amazonaws.us-east-1.s3"
  route_table_ids = [aws_route_table.private.id]
  tags = { Name = "s3-gateway-endpoint" }
}

resource "aws_vpc_endpoint" "dynamodb" {
  vpc_id      = aws_vpc.main.id
  service_name = "com.amazonaws.us-east-1.dynamodb"
  route_table_ids = [aws_route_table.private.id]
}

# Interface Endpoints – for ECR, CloudWatch, etc. (costs hourly, saves
per-GB)
resource "aws_vpc_endpoint" "ecr_dkr" {
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.us-east-1.ecr.dkr"
  vpc_endpoint_type = "Interface"
  subnet_ids       = aws_subnet.private[*].id
  security_group_ids = [aws_security_group.endpoints.id]
  private_dns_enabled = true
}
```

```
# Istio locality-aware routing – prefer same-AZ upstream
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata: { name: catalog-locality }
spec:
  host: catalog.prod.svc.cluster.local
  trafficPolicy:
    outlierDetection: { consecutive5xxErrors: 5, interval: 10s,
baseEjectionTime: 30s }
    connectionPool: { tcp: { maxConnections: 200 }, http: {
http2MaxRequests: 200 } }
  # Outlier detection + locality failover – same AZ first, then same
region, then any
  # Configured via Envoy localityLbSetting in mesh config (istio
ConfigMap)
```

Managed services

Managed services (RDS, ElastiCache, OpenSearch, MSK) carry a premium over self-hosted — but self-hosting cost is not just instance cost; it is the engineering time to operate. The cost question for managed is: *are we using the right tier and right-sizing it?*

Common managed-service wastes: overprovisioned RDS (db.r6g.4xlarge at 15% CPU), ElastiCache with no eviction policy holding cold data, OpenSearch with excessive replica shards, MSK with under-replicated partitions. Review each managed service's utilization quarterly against spend — the same right-sizing discipline as compute.

Capacity engineering — from Little's Law to commitments

The mathematics

Capacity engineering connects demand (requests/sec) to fleet size (replicas, nodes) via queueing theory. Three results matter:

Little's Law: $L = \lambda \times W$ — concurrent requests L equals arrival rate λ times mean latency W . If $\lambda = 1000$ rps and $W = 50$ ms, you need $L = 50$ concurrent slots — i.e., enough pods $\times$ concurrency to handle 50 inflight. This sizes the fleet for *throughput*.

Universal Scalability Law (USL): $X(N) = N / (1 + \alpha(N-1) + \beta N(N-1))$ — throughput X as a function of concurrency N , where α is contention (serialization) and β is coherency (cross-node coordination). USL predicts the point where adding replicas *reduces* throughput due to coordination overhead — critical for sharded stores and consensus-bound services.

```
xychart-beta
  title "USL – Throughput vs Concurrency ( $\alpha=0.05$ ,  $\beta=0.001$ )"
  x-axis [1, 4, 8, 16, 32, 64, 128, 256]
  y-axis "Throughput (normalized)" 0 --> 20
  line [1, 3.5, 6.2, 10.1, 14.8, 16.2, 14.5, 10.2]
```

Figure 8-3: USL curve — throughput rises linearly at low concurrency, peaks where contention and coherency overhead dominate, then falls. The peak is the efficient operating point; beyond it, adding capacity hurts.

Erlang C / M/M/c queueing: For latency-sensitive services, size for p95/p99 — not mean. An M/M/c model gives the probability that a request queues ($C(c, \rho)$) given c servers and utilization $\rho = \lambda / (c\mu)$. Target $\rho \approx 60\text{--}70\%$ for p95 headroom; above 80%, queueing latency dominates.

Forecasting and commitment planning

Forecast demand from historical p95 arrival rate, growth rate, and seasonality. A simple but effective model:

Peak demand next quarter = current peak × (1 + growth_rate) × seasonality_factor × headroom
Headroom factor: 1.3–1.5 for user-facing, 1.1–1.2 for batch

Compare forecast to committed capacity:

Horizon	Instrument	Flexibility
0-1 month	On-demand + autoscaling	Maximum — handle spikes
1-12 months	Savings Plans (1 yr), Convertible RI	Moderate — can exchange
12-36 months	Standard RI / SP (3 yr), hardware reservation	Minimum — commit only to confident baseline

Commit to the *floor* (minimum observed over last 3 months), not the peak. The gap between floor and peak is served by on-demand + autoscaling. Re-evaluate quarterly — growth that was 10% last quarter may be 25% after a launch.

Unit economics

Translate fleet cost to business cost — the metric leadership cares about:

Cost per request = monthly infra cost / monthly requests
Cost per active user = monthly infra cost / MAU
Cost per order = infra cost attributed to checkout / orders

Tag-based allocation (team/service/tenant) makes these computable. Track them monthly — a rising cost-per-request with flat traffic signals efficiency regression; a falling cost-per-request with growing traffic signals healthy economies of scale. Alert when unit cost rises >10% week-over-week without a corresponding traffic or feature driver.

Operating FinOps — budgets, guardrails, and cadence

Budgets and alerts

Budgets are the circuit breaker for spend — they fire before the invoice does.

```
# AWS Budgets – monthly cost budget with alerts at 80% and 100%
resource "aws_budgets_budget" "prod" {
  name           = "prod-monthly"
  budget_type    = "COST"
  limit_amount   = "50000"
  limit_unit     = "USD"
  time_unit      = "MONTHLY"

  notification {
    comparison_operator = "GREATER_THAN"
    threshold           = 80
    threshold_type      = "PERCENTAGE"
    notification_type    = "FORECASTED"
    subscriber_email_addresses = ["platform-alerts@example.com",
"finops@example.com"]
  }
  notification {
    comparison_operator = "GREATER_THAN"
    threshold           = 100
    threshold_type      = "PERCENTAGE"
    notification_type    = "ACTUAL"
    subscriber_email_addresses = ["platform-alerts@example.com",
"finops@example.com", "cto@example.com"]
  }
}

# Anomaly detection – alert on unusual spend spikes (AWS Cost Anomaly
Detection)
resource "aws_ce_anomaly_monitor" "service" {
  name           = "service-monitor"
  monitor_type    = "CUSTOM"
  monitor_dimension = "SERVICE"
  resource_tags = { team = "platform" }
}
resource "aws_ce_anomaly_subscription" "alerts" {
  name           = "anomaly-alerts"
  frequency      = "IMMEDIATE"
  monitor_arn_list = [aws_ce_anomaly_monitor.service.arn]
  subscriber { type = "EMAIL", address = "platform-
alerts@example.com" }
}
```

GCP equivalent: `google_billing_budget` with Pub/Sub notifications to a Cloud Function that posts to Slack.

Policy guardrails

Prevent waste at provision time — cheaper than cleaning it up monthly:

```
# OPA – deny expensive instance families without approval label
package main
deny[msg] {
  input.resource.type == "aws_instance"
  expensive_families := {"p4d", "p5", "x2gd", "i4i"}
  family := split(input.resource.values.instance_type, ".")[0]
  expensive_families[family]
  not input.resource.values.tags["cost_approval"]
  msg := sprintf("Instance type %v requires cost_approval tag",
[input.resource.values.instance_type])
}
deny[msg] {
  input.resource.type == "aws_instance"
  not input.resource.values.tags["team"]
  msg := "Missing required tag: team"
}
deny[msg] {
  input.resource.type == "aws_ebs_volume"
  input.resource.values.size > 500
  not input.resource.values.tags["storage_approval"]
  msg := "EBS volume >500GB requires storage_approval tag"
}
```

Run via `conftest` or `Sentinel` in the Terraform pipeline — `plan` fails if the policy denies, before `apply` creates the waste.

Organizational cadence

Cadence	Participants	Agenda
Weekly	Platform + service owners	Top movers, untagged spend, anomaly alerts, right-sizing queue
Monthly	Eng leadership + finance	Budget vs. actual, coverage/utilization, unit economics, commitment plan
Quarterly	CTO + finance	Forecast vs. commitments, Graviton/spot adoption, architectural cost bets

Cadence	Participants	Agenda
Continuous	Automation	VPA recommendations, Karpenter consolidation, Cost Anomaly Detection, OPA gates

FinOps fails when it is a quarterly finance exercise. It succeeds when it is a weekly engineering ritual with automated guardrails — the same way reliability succeeds when it is a continuous practice, not a post-incident afterthought.

```
xychart-beta
  title "Illustrative Cost Curve – Commit vs Flexibility"
  x-axis ["All On-Demand", "60% Committed", "80% Committed", "100% Committed"]
  y-axis "Monthly Cost (normalized)" 0 --> 100
  line [100, 78, 68, 62]
```

Figure 8-4: Illustrative cost curve — committing the steady-state baseline (60–80%) captures most of the discount; the last 20% of commitment saves little but sacrifices flexibility. Commit to the floor, serve the peak with on-demand.

Key takeaways

- Cost visibility is the prerequisite — enforce tagging (team, service, env, tenant) at provision time via Terraform `default_tags` and OPA/Sentinel policy; query the CUR/Billing Export (Athena/BigQuery) for allocation, orphaned resources, and transfer breakdown. Untagged spend is unmanaged spend.
- Pricing models reward different shapes — on-demand for spiky, RI/SP (60–80% coverage of steady-state floor) for predictable baseline, spot (60–90% discount) for fault-tolerant batch with diversification, Graviton/ARM (~20% price/perf) for portable compute. Blend all four; ladder commitments quarterly.
- Right-size continuously — VPA for vertical (target 60–75% CPU request utilization), HPA/KEDA + Karpenter for horizontal and node consolidation, Compute Optimizer as the starting point but verify against p95 latency and GC. Target utilization below 30% sustained is immediate downsizing territory.

- Attack the next three drivers after compute — storage (lifecycle tiering, orphaned EBS/snapshots, gp3 vs gp2), data transfer (VPC endpoints to bypass NAT, AZ-aware routing, CDN for egress, compression), and managed services (quarterly utilization vs. spend review). A single NAT Gateway or cross-AZ Kafka can be thousands per month unnoticed.
- Capacity engineering grounds spend in queueing theory — Little's Law sizes for throughput, USL finds the concurrency peak before coordination overhead dominates, Erlang C sizes for p95 headroom (target 60-70% utilization). Forecast peak as $\text{current} \times (1 + \text{growth}) \times \text{seasonality} \times \text{headroom}$, commit to the floor, serve the peak with on-demand.
- Operate FinOps as a weekly engineering ritual — budgets with forecasted and actual alerts, anomaly detection, OPA guardrails at plan time, and unit economics (cost per request/user/order) tracked monthly. Automation (VPA, Karpenter consolidation, anomaly alerts) makes it continuous rather than quarterly.

Further reading

- FinOps Foundation — *FinOps Framework* — <https://www.finops.org/framework/> — the canonical inform/optimize/operate lifecycle, capabilities, and maturity model.
- AWS — *Cost and Usage Report (CUR) Query Library* — <https://wellarchitectedlabs.com/cost/> — Athena queries for allocation, orphaned resources, and transfer analysis.
- AWS — *Savings Plans and Reserved Instances* — <https://docs.aws.amazon.com/savingsplans/latest/userguide/what-is-savings-plan.html>
- Karpenter — <https://karpenter.sh/docs/> — consolidation, spot diversification, and price-performance-aware scheduling.
- Google Cloud — *Billing Export to BigQuery* — <https://cloud.google.com/billing/docs/how-to/export-data-bigquery>
- Gunther, N. — *Guerrilla Capacity Planning and Universal Scalability Law* — <https://www.perfdynamics.com/Manifesto/USLscalability.html> — USL derivation and application to fleet sizing.

- Tran, K. — *AWS Data Transfer Costs — The Complete Guide* — <https://www.lastweekinaws.com/blog/understanding-aws-data-transfer-costs/> — practical transfer cost breakdown and mitigations.
- OPA — <https://www.openpolicyagent.org/docs/latest/> — policy-as-code for cost guardrails in the IaC pipeline.

Chapter 9 — Capacity Planning and Performance at Cloud Scale

What this chapter covers. The cloud promises infinite elasticity, but every service still has a finite capacity envelope — and every envelope has a cost. Capacity planning is the discipline of matching that envelope to demand: enough headroom to absorb bursts and failures, not so much that you burn money on idle silicon. This chapter builds the quantitative models (Little's Law, the Universal Scalability Law, queueing theory) and the practical tooling (workload characterization, right-sizing, autoscaling with HPA/KEDA/Karpenter/ASGs, forecasting, and load testing) that turn capacity from guesswork into engineering. You will learn to characterize a workload in requests-per-second-per-core, model its concurrency and latency, choose an instance family and scaling strategy, and build the feedback loops that keep a fleet right-sized as traffic evolves.

Learning goals — after this chapter you should be able to:

- Apply Little's Law, the USL (Universal Scalability Law), and basic queueing models ($M/M/c$, $M/M/1$) to estimate concurrency, throughput ceilings, and latency-vs-utilization curves for a service tier.
- Characterize a real workload — RPS, concurrency, CPU and memory per request, p50/p99 latency, and working-set size — from production metrics and load-test data, and translate it into a core-count and instance-count target.
- Design horizontal and vertical scaling strategies: Kubernetes HPA/VPA/KEDA, Cluster Autoscaler/Karpenter, and EC2 Auto Scaling Groups with mixed instances and lifecycle hooks.
- Choose cloud instance families (general-purpose, compute-optimized, memory-optimized, Graviton/ARM, burstable, accelerated) and

placement strategies, and quantify the noisy-neighbor and NUMA effects at scale.

- Build a forecasting and capacity-buffer model that accounts for seasonality, release-driven growth, and failure-domain headroom (N+1, N+2), and wire it to an autoscaling and budgeting loop.
- Detect and remediate the performance anti-patterns that waste capacity: over-provisioned requests/limits, GC pressure, connection-pool saturation, and retry storms under autoscale events.

Boundary note. Volume 7, Chapter 2 introduced back-of-the-envelope estimation and capacity math at the system-design level; Volume 11, Chapter 7 covered load testing and chaos as verification practices. This chapter is the *operational* treatment — how to measure a running fleet, model its limits, and continuously right-size it. Volume 2 (OS/Linux) and Volume 13 (Runtimes) explain the per-node resource mechanics (scheduling, memory, GC) that underpin per-instance capacity; Volume 3 (Networking) covers the L4/L7 load balancing that spreads load across whatever capacity you provision.

Why capacity planning still matters in the cloud

The elastic-cloud narrative — "the cloud scales infinitely, so you don't need to plan" — is true only in the aggregate. For any single service, three facts make planning unavoidable:

1. **Scaling is not instant.** An EC2 Auto Scaling Group takes 60–180 seconds to launch and warm an instance. A Kubernetes HPA reacts on a 15–30 second metric window and then must schedule, pull images, and pass readiness probes. A cold Lambda in a VPC can take 2–5 seconds. Bursts that exceed headroom within that window queue, time out, or shed.
2. **Scaling is not free.** Every instance-hour, every GB-month of memory, every provisioned IOPS has a price that compounds across hundreds of services. An untuned fleet running at 12% CPU is not elastic — it is wasteful. At \$0.10/vCPU-hour, a 1,000-core fleet at 12% vs. 45% utilization is a \$250k/year difference for the same work.
3. **Scaling has ceilings.** AZ capacity, EBS throughput, NAT Gateway bandwidth (100 Gbps per GW), ALB target-group limits, and service

quotas all impose hard caps. A flash sale or a retry storm can hit a quota before it hits CPU.

Capacity planning in the cloud is therefore not about buying tin ahead of time — it is about **continuously matching a measured demand curve to a right-sized, autoscaled supply curve with explicit headroom for bursts and failures.**

```
flowchart LR
    Demand["Demand curve  
RPS over time  
seasonal + spikes + growth"] --> Model["Capacity model  
Little's Law + USL  
cores = f(RPS, latency, headroom)"]
    Model --> Supply["Supply curve  
instance count × size  
autoscaling policy"]
    Supply --> Headroom["Headroom check  
N+1 / burst buffer  
quota + AZ limits?"]
    Headroom -->|Pass| Fleet["Fleet at target  
utilization 40-65%"]
    Headroom -->|Fail| Model
    Fleet --> Metrics["Production metrics  
CPU / latency / queue depth"]
    Metrics --> Demand

    style Demand fill:#e3f2fd
    style Fleet fill:#e8f5e9
    style Headroom fill:#fff3e0
```

Figure 9-1: The capacity planning loop — demand is modeled into a supply target with explicit headroom, deployed as an autoscaled fleet, and continuously corrected by production metrics.

Quantitative foundations

Little's Law: concurrency from throughput and latency

For any stable system:

$$L = \lambda \times W$$

- **L** — average number of requests concurrently *in the system* (in-flight).

- λ — arrival rate (requests per second).
- W — average time a request spends in the system (latency in seconds).

If your service handles 5,000 RPS at an average latency of 40 ms, the average concurrency is $5,000 \times 0.04 = \mathbf{200 \text{ concurrent requests}}$. That number directly sizes thread pools, connection pools, and — when multiplied by CPU per request — core counts.

Example. A request burns 15 ms of CPU on a single core (measured via `perf` or runtime CPU profiles). At 5,000 RPS, total CPU demand is $5,000 \times 0.015 = 75$ core-seconds per second = **75 cores** of saturated CPU. At a target utilization of 50% (headroom for GC, bursts, and noisy neighbors), you need **150 cores**. On `m7g.large` (2 vCPU Graviton), that is 75 instances; on `m7g.xlarge` (4 vCPU), 38 instances. The arithmetic is simple — the hard part is measuring the 15 ms accurately and choosing the headroom factor honestly.

Universal Scalability Law (USL)

The USL models how throughput scales with concurrency under contention (α) and coherency (β) costs:

$$X(N) = N / (1 + \alpha \cdot (N-1) + \beta \cdot N \cdot (N-1))$$

- N — concurrency (threads, cores, nodes).
- α — contention (serialized access to a shared resource — a lock, a DB row, a single queue).
- β — coherency delay (cost of keeping replicas coherent — cache-line bouncing, cross-node coordination).

Fitting α and β from load-test data tells you whether your service is contention-bound (flattening curve — fix the lock), coherency-bound (retrograde — throughput *falls* past a peak — usually a cache-coherence or consensus cost), or embarrassingly parallel (near-linear — add nodes).


```

xychart-beta
  title USL: Throughput vs Concurrency for different contention/
  coherency
  x-axis "Concurrency N" [1, 4, 8, 16, 32, 64, 128]
  y-axis "Relative throughput X(N)" 0 --> 50
  line [1, 3.8, 6.5, 9.2, 11.0, 11.2, 9.5]
  line [1, 3.9, 7.2, 12.8, 20.1, 30.4, 42.0]
  line [1, 3.5, 5.5, 7.0, 7.5, 7.2, 6.0]

```

Figure 9-2 (conceptual): Three USL curves. Near-linear (middle, small α/β) scales to 64+ cores. Contention-bound (top) plateaus early. Coherency-bound (bottom) peaks and regresses — adding concurrency past the peak hurts. Fit α/β from measured $X(N)$ to diagnose which regime you are in.

In practice, collect $X(N)$ by running the same workload at 1, 2, 4, 8, ... threads/nodes and measuring throughput. Fit with any USL solver (the `usl` Python package or a simple least-squares). A retrograde curve is a signal to shard, partition, or eliminate the coordination — not to add more nodes.

Queueing theory for the impatient

Single-queue models predict how latency explodes as utilization approaches 1:

- **M/M/1** (one server, Poisson arrivals, exponential service): mean response time $R = S / (1 - \rho)$, where S is mean service time and $\rho = \lambda \cdot S$ is utilization (0-1). At 80% utilization, mean latency is 5× service time; at 90%, 10×.
- **M/M/c** (c servers, e.g., c cores or c replicas): same shape, but the knee moves right as c grows. A 32-core tier tolerates higher utilization than a 2-core tier before queueing dominates.

The takeaway is quantitative: **running a tier above ~65% steady-state CPU guarantees long-tail latency inflation**, even if mean latency looks fine. Autoscaling targets should sit left of the knee, not at 90%.

```

flowchart TB
    subgraph Models["Capacity models – when to use which"]
        L["Little's Law  
L = λW  
concurrency sizing  
always – first estimate"]
        Q["M/M/c queueing  
R = S/(1-ρ) family  
latency vs utilization  
SLA headroom"]
        U["USL  
X(N) = N / (1+α(N-1)+βN(N-1))  
scaling ceiling  
contention diagnosis"]
        F["Forecasting  
Holt-Winters / Prophet  
seasonal + growth  
weeks ahead"]
    end
    L --> Q
    Q --> U
    U --> F
    F -. "recalibrate" .-> L

    style L fill:#e3f2fd
    style Q fill:#fff3e0
    style U fill:#fce4ec
    style F fill:#e8f5e9

```

Figure 9-3: The modeling stack — Little's Law sizes concurrency, queueing sets the utilization target, USL finds the scaling ceiling, forecasting projects demand weeks out.

Workload characterization

You cannot plan capacity without measuring the workload. The four numbers that matter:

Signal	How to measure	Why it drives capacity
Arrival rate λ	ALB/NLB request count, Envoy downstream_rq_total, API gateway metrics	Directly sets core and replica count via Little's Law

Signal	How to measure	Why it drives capacity
Service time W	p50/p90/p99 latency from traces (OpenTelemetry), histogram <code>http_server_duration_seconds</code>	Determines concurrency (L) and queueing headroom
CPU per request	<code>container_cpu_usage_seconds_total</code> / RPS, or <code>perf</code> / continuous profiling (Pyroscope/Parca)	Converts RPS to cores; catches GC and serialization overhead
Memory working set	<code>container_memory_working_set_bytes</code> , heap profiles, RSS after steady state	Sizes instance memory; determines vertical vs. horizontal scaling

From metrics to a capacity target

```
#!/usr/bin/env python3
"""
capacity_model.py – Turn production metrics into an instance-count
target.

Inputs: Prometheus range query results or load-test logs.
Model: Little's Law + queueing headroom + failure-domain buffer.
"""

import math

# --- Measured inputs (replace with real numbers from Prometheus / load
# tests) ---
rps_peak = 8_000 # peak RPS (p95 of 5-min window, not
instantaneous spike)
latency_p50_s = 0.035 # p50 latency in seconds
latency_p99_s = 0.180 # p99 – used for SLO headroom check
cpu_per_req_s = 0.012 # CPU seconds per request (from
profiling)
mem_per_req_mb = 1.8 # working-set contribution per
concurrent request
mem_base_mb = 350 # per-pod baseline (runtime, caches,
sidecars)
target_cpu_util = 0.55 # queueing knee guardrail (0.50-0.65
for latency-sensitive)
failure_buffer = 1.33 # N+1 over 4 AZs ~ 1.33x (lose one AZ,
still serve peak)
burst_buffer = 1.25 # 25% burst headroom (flash traffic,
retry storms)

# --- Derived ---
concurrency_avg = rps_peak * latency_p50_s
concurrency_p99 = rps_peak * latency_p99_s
cores_needed = rps_peak * cpu_per_req_s # saturated cores
cores_with_headroom = cores_needed / target_cpu_util
cores_with_buffers = cores_with_headroom * failure_buffer * burst_buff
er

# Instance sizing – compare two families
for name, vcpu, mem_gb, price_hr in [
    ("m7g.large", 2, 8, 0.086),
    ("m7g.xlarge", 4, 16, 0.172),
    ("c7g.xlarge", 4, 8, 0.145),
    ("c7g.2xlarge", 8, 16, 0.290),
]:
    instances = math.ceil(cores_with_buffers / vcpu)
    mem_needed_per_pod = mem_base_mb + (concurrency_avg / instances) *
mem_per_req_mb
    mem_ok = "ok" if mem_needed_per_pod < (mem_gb * 1024 * 0.75) else "
MEM PRESSURE"
```

```

    cost_month = instances * price_hr * 730
    print(f"{name:14s} instances={instances:3d} cores={cores_with_buf
fers:.0f}"
          f" mem/pod~{mem_needed_per_pod:.0f}MB {mem_ok:14s}  ${cost_m
onth:.,.0f}/mo")

print(f"\nConcurrency avg={concurrency_avg:.0f}
p99={concurrency_p99:.0f}")
print(f"Cores saturated={cores_needed:.0f} with headroom={cores_with_
headroom:.0f}"
      f" with buffers={cores_with_buffers:.0f}")
print(f"Headroom model: target_cpu={target_cpu_util} failure={failure_
buffer}x"
      f" burst={burst_buffer}x combined={target_cpu_util**-1 * failur
e_buffer * burst_buffer:.2f}x over saturated")

```

Sample output:

m7g.large	instances= 99	cores=164	mem/pod~353MB ok
\$6,212/mo			
m7g.xlarge	instances= 50	cores=164	mem/pod~356MB ok
\$6,278/mo			
c7g.xlarge	instances= 50	cores=164	mem/pod~356MB ok
\$5,293/mo			
c7g.2xlarge	instances= 25	cores=164	mem/pod~371MB ok
\$5,293/mo			

The model makes trade-offs explicit: `c7g` (compute-optimized Graviton) wins on cost for CPU-bound workloads; `m7g` wins when memory per pod is the binding constraint. Re-run with your measured `cpu_per_req` and `mem_per_req` — the cheapest family for one service is the most expensive for another.

Right-sizing in Kubernetes: requests, limits, and actual usage

In Kubernetes, `requests` drive scheduling and `limits` drive throttling/OOM. Over-provisioned requests waste bin-packing; under-provisioned limits cause CPU throttling and tail-latency spikes.

Best practice:

- **requests = p90 steady-state usage** (from VPA recommender or Prometheus `quantile_over_time`).
- **limits = 1.5-2× requests for CPU** (burstable), **requests == limits for memory** (avoid OOM kills; use `Burstable` QoS only when you understand eviction).

- Reconcile weekly with VPA recommendations or a controller like Goldilocks/KRR.

```

# deployment-rightsized.yaml – rightsized requests/limits from VPA
recommender
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: prod
spec:
  replicas: 24
  selector:
    matchLabels: { app: api }
  template:
    metadata:
      labels: { app: api }
    spec:
      containers:
        - name: api
          image: registry.example.com/api:1.42.0
          ports: [{ containerPort: 8080, name: http }]
          resources:
            requests:
              cpu: "900m"
# p90 measured: ~820m; 900m leaves small buffer
            memory: "768Mi" # working set ~620Mi + 150Mi headroom
            limits:
              cpu: "1800m" # 2x requests – burst without throttle
              memory: "768Mi" # memory limit == request –避免 OOM vs
throttle tradeoff is explicit
            readinessProbe:
              httpGet: { path: /readyz, port: 8080 }
              periodSeconds: 5

# HPA drives replica count; VPA (in recommender mode) advises requests
---
apiVersion: autoscaling/v1
kind: VerticalPodAutoscaler
metadata:
  name: api-vpa
  namespace: prod
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  updatePolicy:
    updateMode: "Off" # recommender only; apply via CI after review
  resourcePolicy:
    containerPolicies:
      - containerName: api

```



```
minAllowed: { cpu: "500m", memory: "512Mi" }  
maxAllowed: { cpu: "4000m", memory: "2Gi" }
```

Horizontal and vertical autoscaling

The scaling hierarchy

```

flowchart TB
    subgraph Workload["Workload demand (RPS)"]
        RPS["RPS spike"]
    end
    end
    subgraph K8s["Kubernetes autoscaling"]
        HPA["HPA"]
        replicas = f(metric)
        15-30s loop["15-30s loop"]
        KEDA["KEDA"]
        event-driven
        queue length, Kafka lag, cron["queue length, Kafka lag, cron"]
        VPA["VPA"]
        requests/limits
        evicts & reschedules["evicts & reschedules"]
    end
    end
    subgraph Nodes["Node autoscaling"]
        CAS["Cluster Autoscaler"]
        pending pods → new nodes
        60-120s["60-120s"]
        Karp["Karpenter"]
        just-in-time, workload-aware
        30-60s, consolidation["30-60s, consolidation"]
    end
    end
    subgraph Cloud["Cloud autoscaling"]
        ASG["EC2 Auto Scaling Group"]
        mixed instances, lifecycle hooks
        60-180s["60-180s"]
        Lambda["Lambda / Fargate"]
        per-request scale
        ms to seconds["ms to seconds"]
    end
    end

    RPS --> HPA & KEDA
    HPA --> CAS
    KEDA --> CAS
    VPA --> HPA
    CAS --> ASG
    Karp --> ASG

    style HPA fill:#e3f2fd
    style KEDA fill:#e3f2fd
    style Karp fill:#e8f5e9
    style ASG fill:#fff3e0

```

Figure 9-4: The autoscaling hierarchy — workload metrics drive pod scaling (HPA/KEDA/VPA), pending pods drive node scaling (CAS/Karpenter), and node groups are ultimately EC2 ASGs with launch templates.

HPA: the workhorse

```

# hpa.yaml – HPA on custom + resource metrics, with behavior tuning
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
  namespace: prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 12          # N+1 floor: even at low traffic, survive
one AZ loss
  maxReplicas: 120
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 55  # left of queueing knee
    - type: Pods
      pods:
        metric:
          name: http_requests_per_second  # from Prometheus Adapter
          target:
            type: AverageValue
            averageValue: "800"          # 800 RPS per pod – from
capacity model
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 75
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30
    policies:
      - type: Percent
        value: 50          # at most +50% replicas per 30s –
dampen flapping
        periodSeconds: 30
      - type: Pods
        value: 8
        periodSeconds: 30
    selectPolicy: Max
  scaleDown:
    stabilizationWindowSeconds: 300  # 5 min – avoid churn on
transient dips

```

```
policies:
- type: Percent
  value: 10
  periodSeconds: 60
selectPolicy: Min
```

Key details:

- **Multiple metrics:** HPA takes the *maximum* replica count across all metrics — CPU, RPS-per-pod, and memory each independently trigger scale-up.
- **Behavior:** `scaleUp` is aggressive but capped; `scaleDown` is deliberately slow. Fast scale-down causes flapping that is worse than brief over-provisioning.
- **Stabilization windows:** prevent oscillation when metrics jitter.

KEDA: event-driven scaling

HPA polls metrics; KEDA subscribes to event sources (Kafka lag, SQS depth, Redis stream length, cron) and scales to zero when idle — essential for consumers and batch workers.

```

# keda-scaledobject.yaml – scale a Kafka consumer on lag, plus a cron
schedule for batching
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: order-consumer
  namespace: prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-consumer
  minReplicaCount: 0          # scale to zero when no lag
  maxReplicaCount: 40
  cooldownPeriod: 120        # seconds after last trigger before scaling
to zero
  pollingInterval: 15
  triggers:
    - type: kafka
      metadata:
        bootstrapServers: "kafka.prod.svc:9092"
        consumerGroup: "order-consumer"
        topic: "orders"
        lagThreshold: "500"          # one pod per 500 lag messages
        activationLagThreshold: "10" # wake from zero at lag > 10
      authenticationRef:
        name: kafka-auth
    - type: cron
      metadata:
        timezone: "UTC"
        start: "0 2 * * *"          # ensure at least 2 pods during nightly
batch window
        end: "0 5 * * *"
        desiredReplicas: "2"
  ---
apiVersion: keda.sh/v1alpha1
kind: TriggerAuthentication
metadata:
  name: kafka-auth
  namespace: prod
spec:
  secretTargetRef:
    - parameter: tls
      name: kafka-tls
      key: ca.crt

```

Karpenter: just-in-time nodes

Cluster Autoscaler works with pre-defined ASGs; Karpenter provisions nodes just-in-time by calling EC2 directly, choosing the cheapest instance type that fits pending pods, and consolidating under-utilized nodes.

```

# karpenter-nodepool.yaml – workload-aware node provisioning (Karpenter
v1 / NodePool API)
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: api-pool
spec:
  template:
    spec:
      requirements:
        - key: kubernetes.io/arch
          operator: In
          values: ["arm64"]          # Graviton – 20% cheaper
per core for this workload
        - key: kubernetes.io/os
          operator: In
          values: ["linux"]
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["spot", "on-demand"] # prefer spot, fall back
to on-demand
        - key: node.kubernetes.io/instance-type
          operator: In
          values: ["m7g.large", "m7g.xlarge", "c7g.large",
"c7g.xlarge"]
        - key: topology.kubernetes.io/zone
          operator: In
          values: ["us-east-1a", "us-east-1b", "us-east-1c"]
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: api-class
      expireAfter: 720h # recycle nodes weekly – patching via
replacement
      limits:
        cpu: 400          # fleet-wide cap – prevents runaway scale
        memory: 800Gi
      disruption:
        consolidationPolicy: WhenEmptyOrUnderutilized
        consolidateAfter: 60s
      budgets:
        - nodes: "20%"    # at most 20% of nodes disrupted at once
---
apiVersion: karpenter.k8s.aws/v1
kind: EC2NodeClass
metadata:
  name: api-class
spec:
  amiFamily: AL2023
  role: "KarpenterNodeRole-api-pool"

```



```
subnetSelectorTerms:
  - tags: { "karpenter.sh/discovery": "prod" }
securityGroupSelectorTerms:
  - tags: { "karpenter.sh/discovery": "prod" }
blockDeviceMappings:
  - deviceName: /dev/xvda
    ebs: { volumeSize: 40Gi, volumeType: gp3, encrypted: true }
tags:
  Environment: prod
  ManagedBy: karpenter
```

EC2 Auto Scaling Groups with mixed instances

When not on Kubernetes, ASGs with mixed instances and attribute-based selection achieve the same bin-packing without pinning to a single type:

```

# asg.tf – mixed-instances ASG with spot + on-demand, lifecycle hook,
and warm pool
resource "aws_autoscaling_group" "api" {
  name = "api-${var.environment}"
  vpc_zone_identifier = module.network.private_subnet_ids
  min_size = 12
  max_size = 120
  desired_capacity = 24
  health_check_type = "ELB"
  health_check_grace_period = 180

  mixed_instances_policy {
    instances_distribution {
      on_demand_base_capacity = 6 # always 6 on-
demand for baseline
      on_demand_percentage_above_base_capacity = 30 # 30% of
additional as on-demand
      spot_allocation_strategy = "price-capacity-
optimized"
    }
    launch_template {
      launch_template_specification {
        launch_template_id = aws_launch_template.api.id
        version = "$Latest"
      }
      override {
        instance_requirements {
          vcpu_count { min = 2, max = 8 }
          memory_mib { min = 8192, max = 32768 }
          cpu_manufacturers = ["aws"] # Graviton preference
          burstable_performance = "excluded"
          instance_generations = ["current"]
        }
      }
    }
  }
}

instance_refresh {
  strategy = "Rolling"
  preferences {
    checkpoint_delay = 300
    checkpoint_percentages = [33, 66, 100]
    min_healthy_percentage = 90
  }
}

tag {
  key = "Name"
  value = "api-${var.environment}"
  propagate_at_launch = true
}

```

```

    }
  }

  resource "aws_launch_template" "api" {
    name_prefix    = "api-"
    image_id       = data.aws_ami.al2023.id
    instance_type  = "m7g.large" # fallback; overridden by mixed policy
    above
    user_data      = base64encode(templatefile("${path.module}/user-
data.sh", {
      environment = var.environment
    }))
    iam_instance_profile { name = aws_iam_instance_profile.api.name }
    vpc_security_group_ids = [aws_security_group.api.id]
    block_device_mappings {
      device_name = "/dev/xvda"
      ebs { volume_size = 40, volume_type = "gp3", encrypted = true }
    }
    metadata_options {
      http_tokens = "required" # IMDSv2 only
      http_endpoint = "enabled"
    }
  }
}

resource "aws_autoscaling_lifecycle_hook" "draining" {
  name                = "draining"
  autoscaling_group_name = aws_autoscaling_group.api.name
  lifecycle_transition = "autoscaling:EC2_INSTANCE_TERMINATING"
  heartbeat_timeout   = 120
  default_result       = "CONTINUE"
  notification_target_arn = aws_sns_topic.lifecycle.arn
  role_arn             = aws_iam_role.lifecycle_hook.arn
}

```

Instance selection and performance at scale

Families and trade-offs

Family	vCPU:RAM	Best for	Watch out
m7g/m7i (general)	1:4	Balanced services, most APIs	Not cheapest per core
c7g/c7i (compute)	1:2	CPU-bound, encoding, serialization	Memory pressure if working set is large

Family	vCPU:RAM	Best for	Watch out
r7g/r7i (memory)	1:8	Caches, in-memory indexes, large heaps	Wasted if CPU-bound
Graviton (g)	same as above, 20% cheaper	Everything that runs on ARM (most Go/Java/Python)	Validate native deps (glibc, SIMD)
Burstable (t4g)	baseline + burst credits	Dev, spiky low-traffic	Credit exhaustion under sustained load
Storage (i4i, im4gn)	NVMe local	Kafka, local shuffle, low-latency stores	Ephemeral — data lost on stop

Graviton migration is the single highest-ROI capacity move for many fleets: 20% price reduction per core with comparable or better performance for most backend workloads (especially Go, Java, and Python). Validate with a canary: run the same load test on `m7g` vs. `m7i` and compare p99 latency and cost per 1k RPS.

Noisy neighbors, placement, and NUMA

At scale, the abstraction of a "vCPU" leaks:

- **Noisy neighbors** — on shared-tenancy instances, a neighbor's burst can steal LLC or memory bandwidth. Mitigate with dedicated tenancy for latency-critical tiers, or use `c7g` / `m7g` with ENA and Nitro isolation.
- **Placement groups** — `cluster` placement minimizes inter-node latency (HPC, cache clusters); `spread` and `partition` maximize failure isolation. For most APIs, AZ spread matters more than rack locality.
- **NUMA** — a `m7g.8xlarge` (32 vCPU) spans two NUMA nodes. A single-threaded process pinned to the wrong node pays cross-NUMA memory latency. For large instances, set `topologyManager` to `single-numa-node` in Kubelet or pin with `numactl / taskset`.

Forecasting and buffers

```

flowchart LR
    Hist["Historical RPS  
Prometheus / CloudWatch  
per hour, per AZ"] --> Decompose["Decompose  
trend + season + residual  
Holt-Winters / Prophet"]
    Decompose --> Trend["Trend  
MoM growth %"]
    Decompose --> Season["Seasonality  
daily / weekly / annual"]
    Decompose --> Spike["Spike model  
p95 burst multiplier"]
    Trend --> Forecast["Forecast  
peak RPS at horizon  
e.g., 90 days"]
    Season --> Forecast
    Spike --> Forecast
    Forecast --> Buffer["Add buffers  
N+1 + burst + deploy  
→ target cores"]
    Buffer --> Budget["Budget  
cost at forecast  
vs. commitment discount"]

    style Forecast fill:#e3f2fd
    style Buffer fill:#fff3e0
    style Budget fill:#e8f5e9

```

Figure 9-5: Forecasting pipeline — historical RPS is decomposed into trend, seasonality, and spike components; the forecast plus explicit buffers (N+1, burst, deploy surge) yields the capacity target and budget.

Practical forecasting:

- **Short horizon (hours-days):** HPA/KEDA handle it; no forecast needed beyond minReplicas.
- **Medium horizon (weeks):** linear extrapolation of p95 weekly peak plus seasonal multiplier (e.g., Black Friday 3×). Validate against last year's curve.
- **Long horizon (quarters):** capacity review with product — feature launches, migration waves, new regions. Model as step functions, not smooth growth.

Buffer policy (example):

Buffer	Value	Rationale
N+1 AZ	1.33× over 3 AZs, 1.5× over 2 AZs	Lose one AZ, still serve peak
Burst	1.25×	Absorb 25% spike within HPA reaction window
Deploy surge	1.20× (maxSurge: 20%)	Rolling deploy temporarily runs old + new
Combined	~2.0× saturated cores at p50	Check: is utilization at p50 still 35-50%? If >65%, add nodes.

Performance anti-patterns that waste capacity

Anti-pattern	Symptom	Fix
Over-provisioned requests	Fleet at 12% CPU, high cost	VPA recommender → lower requests → better bin-packing
CPU limits == requests, throttled	p99 spikes correlated with CPU throttle metric	Raise limits to 1.5-2× requests; use <code>cpuCFSQuota</code> tuning
GC pressure	Sawtooth heap, long STW pauses under load	Reduce allocation rate, tune heap (Vol 13), right-size memory
Undersized connection pools	Queueing at pool, not at CPU	Size pool ≈ concurrency (Little's Law) + headroom
Retry storms on scale-up	New pods cold, retries amplify load	Backoff + jitter (Vol 3 Ch 11), warm-up probes, staggered rollout
Single-AZ hotspot	One AZ at 80%, others at 30%	Check Service topology, AZ-aware routing, Karpenter zone spread

Distributed-systems lens

Capacity planning is a coordination problem across teams. Each team optimizes locally (their service's HPA), but the fleet shares global constraints (AZ capacity, NAT bandwidth, quota, budget). Three practices keep local and global aligned:

1. **Publish a capacity contract per tier.** Every service documents its measured `cpu_per_req`, `RPS per pod`, `target utilization`, and `max RPS per AZ`. The platform team aggregates these into AZ and region capacity models.
2. **Quota and budget as code.** Service quotas (EC2, EBS, ALB) and cost budgets are declared in Terraform alongside the workload — not discovered during an incident. Weekly drift checks flag services approaching 80% of quota.
3. **Load testing as a gate, not an afterthought.** Every HPA/Karpenter/ASG change is validated with a load test that ramps to `burst_buffer × peak RPS` and holds for the HPA stabilization window. If p99 exceeds SLO during the test, the capacity model is wrong — fix it before it ships.

Key takeaways

- Little's Law ($L = \lambda W$) sizes concurrency; queueing theory sets the utilization target (stay left of the knee, ~50-65%); USL diagnoses whether adding concurrency helps or hurts.
- Characterize workloads as RPS, latency distribution, CPU-per-request, and memory working set — then compute cores, instances, and cost per family before choosing hardware.
- In Kubernetes, `requests` drive scheduling and bin-packing, `limits` drive throttling — set requests to p90 usage, limits to 1.5-2× for CPU, and reconcile weekly via VPA recommender.
- HPA scales pods on multiple metrics (CPU + RPS-per-pod + memory) with slow scale-down; KEDA adds event-driven scale-to-zero; Karpenter provisions right-sized nodes just-in-time and consolidates waste.

- For EC2 fleets, mixed-instances ASGs with attribute-based selection and spot + on-demand blending cut cost without pinning to a single type.
- Graviton (ARM) is typically 20% cheaper per core — validate with a canary load test before migrating a tier.
- Forecast by decomposing historical RPS into trend + seasonality + spike, then add explicit buffers for AZ loss, bursts, and deploy surge — the combined multiplier is usually $\sim 2\times$ saturated cores.
- Treat capacity as a continuous loop — model, deploy autoscaling, measure, reforecast — not a one-time purchase.

Further reading

- Gunther, *Guerrilla Capacity Planning* (Springer) and *Analyzing Computer System Performance with Perl::PDQ* — USL and queueing models with worked examples.
- Gregg, *Systems Performance: Enterprise and the Cloud* (2nd ed.) — USE method, profiling, and per-resource capacity analysis.
- Kubernetes docs: Horizontal Pod Autoscaling, Vertical Pod Autoscaler, Karpenter — <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/> , <https://karpenter.sh/>
- KEDA documentation — <https://keda.sh/docs/>
- AWS: EC2 Auto Scaling mixed instances policy, Karpenter best practices, Graviton migration guide — <https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-mixed-instances-groups.html>
- Beyer, Jones, Petoff, *Site Reliability Engineering* (O'Reilly), Ch. 22 — Handling Overload; Ch. 23 — Managing Critical State.
- Prophet forecasting library — <https://facebook.github.io/prophet/> and Holt-Winters in `statsmodels` .

Chapter 10 — Cloud Networking, IAM, and Security Foundations

What this chapter covers. Every cloud workload runs inside two fabrics: the network that moves its packets and the identity system that

decides which packets — and which API calls — are allowed. Get either fabric wrong and the blast radius of a compromise is the entire account; get them right and a leaked credential or a compromised pod is contained to a narrow scope. This chapter builds both fabrics from first principles: VPC design (subnets, route tables, NAT, gateways, PrivateLink), network segmentation (security groups, NACLs, Network Firewall, service mesh mTLS), and the full IAM stack (principals, policies, permission boundaries, SCPs, trust policies, workload identity via IRSA/SPIFFE, and cross-account access) — with production-grade Terraform, IAM policies, and Kubernetes manifests you can apply to a real AWS/GCP account.

Learning goals — after this chapter you should be able to:

- Design a multi-AZ VPC with public, private, and isolated subnets, route tables, NAT Gateways, VPC endpoints, and PrivateLink, and explain the packet path for north-south and east-west traffic.
- Apply defense-in-depth with security groups (stateful, instance/ENI-scoped), NACLs (stateless, subnet-scoped), Network Firewall / Cloud Armor, and service-mesh mTLS — and articulate when each layer is the right tool.
- Author least-privilege IAM policies — identity-based, resource-based, permission boundaries, and SCPs — and evaluate effective permissions with the policy evaluation logic (explicit deny wins).
- Implement workload identity: IRSA / Workload Identity Federation (GCP) / Azure Workload Identity, SPIFFE/SPIRE, and cross-account roles with `sts:AssumeRole` and `ExternalId`.
- Enforce network and IAM guardrails as code — VPC flow logs, GuardDuty, IAM Access Analyzer, OPA/Kyverno network policies, and automated credential rotation.
- Diagnose and remediate the common failure modes: overly broad `*:*` policies, confused-deputy via missing `aws:SourceAccount` / `aws:SourceArn`, security-group sprawl, and NAT Gateway as a single point of failure or cost blowout.

Boundary note. Volume 3 — Networking — covered packet mechanics (IP, TCP, TLS, HTTP, DNS, load balancing) on the wire; this chapter is the *cloud control-plane* view — how those primitives are declared, segmented, and authorized via VPC and IAM APIs. Volume 9 — Security — covered applied cryptography, authentication, and authorization models (RBAC/ABAC/ReBAC) at the application layer; this chapter is the

platform enforcement — how identity and network policy are expressed in the provider's own policy language. Volume 5 — Databases — and Volume 6 — Distributed Systems — provide the consistency and replication theory behind highly available network and identity services.

Cloud networking: the VPC as a virtual data center

A VPC (Virtual Private Cloud) is a logically isolated L3 network inside the provider's fabric. Every resource — EC2, RDS, ALBs, Lambda ENIs, EKS nodes — lives in a subnet inside a VPC, and every packet is governed by the VPC's route tables and firewall layers.

VPC anatomy

```

flowchart TB
    Internet["Internet  
0.0.0.0/0"] --- IGW["Internet Gateway  
(IGW)"]
    IGW --- PubRT["Public route table  
0.0.0.0/0 → IGW"]
    PubRT --- PubA["Public subnet A  
10.0.1.0/24  
ALB + NAT GW"]
    PubRT --- PubB["Public subnet B  
10.0.2.0/24  
ALB + NAT GW"]
    PubRT --- PubC["Public subnet C  
10.0.3.0/24  
ALB + NAT GW"]

    PubA --- NATA["NAT Gateway A"]
    PubB --- NATB["NAT Gateway B"]
    PubC --- NATC["NAT Gateway C"]

    NATA --- PrivRT_A["Private route table A  
0.0.0.0/0 → NAT A"]
    NATB --- PrivRT_B["Private route table B  
0.0.0.0/0 → NAT B"]
    NATC --- PrivRT_C["Private route table C  
0.0.0.0/0 → NAT C"]

    PrivRT_A --- PrivA["Private subnet A  
10.0.10.0/24  
EKS nodes, RDS"]
    PrivRT_B --- PrivB["Private subnet B  
10.0.11.0/24  
EKS nodes, RDS"]
    PrivRT_C --- PrivC["Private subnet C  
10.0.12.0/24  
EKS nodes, RDS"]

    PrivA --- IsoRT["Isolated route table  
no 0.0.0.0/0  
only VPC local + endpoints"]
    PrivB --- IsoRT
    PrivC --- IsoRT
    IsoRT --- IsoA["Isolated subnet A  
10.0.20.0/24  
ElastiCache, isolated DBs"]
    IsoA --- VPCE["VPC Endpoints  
Gateway (S3/Dynamo)  
Interface (ECR, Secrets, CloudWatch)"]

    style PubA fill:#e3f2fd

```

```

style PubB fill:#e3f2fd
style PubC fill:#e3f2fd
style PrivA fill:#fff3e0
style PrivB fill:#fff3e0
style PrivC fill:#fff3e0
style IsoA fill:#fce4ec
style VPCE fill:#e8f5e9

```

Figure 10-1: Three-tier VPC across three AZs — public subnets host the ALB and NAT Gateways; private subnets host compute and databases with per-AZ NAT for egress; isolated subnets have no internet route and reach AWS services only via VPC endpoints.

Subnet tiers and routing

Tier	Route to 0.0.0.0/0	Reachability	Hosts
Public	IGW	Inbound from internet (if SG allows), outbound via IGW	ALBs, NLBs, bastion (if any)
Private	NAT Gateway (per AZ)	Outbound to internet via NAT; inbound only via ALB	EKS/EC2, RDS, ElastiCache
Isolated	None (only local + prefix lists for endpoints)	No internet; AWS services via VPC endpoints	Sensitive data stores, compliance workloads

One NAT Gateway per AZ is the minimum for AZ isolation — a single shared NAT is a cross-AZ single point of failure and a bandwidth bottleneck (100 Gbps per GW, shared). Cost is ~\$32/mo per GW plus data-processing charges; for high-egress fleets, consider NAT-less egress via VPC endpoints and egress VPC patterns.

Production VPC with Terraform

```
# vpc.tf – three-tier VPC with per-AZ NAT, flow logs, and endpoints

locals {
  azs = ["us-east-1a", "us-east-1b", "us-east-1c"]
  vpc_cidr = "10.0.0.0/16"
}

resource "aws_vpc" "main" {
  cidr_block      = local.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true
  tags = { Name = "${var.environment}-main" }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags = { Name = "${var.environment}-igw" }
}

# --- Subnets (one per AZ per tier) ---
resource "aws_subnet" "public" {
  for_each      = toset(local.azs)
  vpc_id        = aws_vpc.main.id
  cidr_block    = cidrsubnet(local.vpc_cidr, 8, index(local.azs, each.key))
  availability_zone = each.key
  map_public_ip_on_launch = false # ALBs get public IPs via
  allocation; instances do not
  tags = {
    Name = "${var.environment}-public-${each.key}"
    "kubernetes.io/role/elb" = "1"
    "kubernetes.io/cluster/${var.cluster_name}" = "shared"
  }
}

resource "aws_subnet" "private" {
  for_each      = toset(local.azs)
  vpc_id        = aws_vpc.main.id
  cidr_block    = cidrsubnet(local.vpc_cidr, 8, 10 + index(local.azs, each.key))
  availability_zone = each.key
  tags = {
    Name = "${var.environment}-private-${each.key}"
    "kubernetes.io/role/internal-elb" = "1"
    "kubernetes.io/cluster/${var.cluster_name}" = "shared"
  }
}

```

```

resource "aws_subnet" "isolated" {
  for_each      = toset(local.azs)
  vpc_id        = aws_vpc.main.id
  cidr_block    = cidrsubnet(local.vpc_cidr, 8, 20 + index(local.azs, each.key))
  availability_zone = each.key
  tags = { Name = "${var.environment}-isolated-${each.key}" }
}

# --- NAT per AZ (no cross-AZ NAT – AZ isolation) ---
resource "aws_eip" "nat" {
  for_each = toset(local.azs)
  domain   = "vpc"
  tags     = { Name = "${var.environment}-nat-${each.key}" }
}

resource "aws_nat_gateway" "main" {
  for_each      = toset(local.azs)
  allocation_id = aws_eip.nat[each.key].id
  subnet_id     = aws_subnet.public[each.key].id
  tags          = { Name = "${var.environment}-nat-${each.key}" }
  depends_on    = [aws_internet_gateway.main]
}

# --- Route tables ---
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }
  tags = { Name = "${var.environment}-public" }
}

resource "aws_route_table" "private" {
  for_each = toset(local.azs)
  vpc_id   = aws_vpc.main.id
  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main[each.key].id
  }
  tags = { Name = "${var.environment}-private-${each.key}" }
}

resource "aws_route_table" "isolated" {
  vpc_id = aws_vpc.main.id
  # No 0.0.0.0/0 – only local route (implicit) + gateway endpoints via
  # prefix lists
  tags = { Name = "${var.environment}-isolated" }
}

```

```

resource "aws_route_table_association" "public" {
  for_each      = toset(local.azs)
  subnet_id     = aws_subnet.public[each.key].id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "private" {
  for_each      = toset(local.azs)
  subnet_id     = aws_subnet.private[each.key].id
  route_table_id = aws_route_table.private[each.key].id
}

resource "aws_route_table_association" "isolated" {
  for_each      = toset(local.azs)
  subnet_id     = aws_subnet.isolated[each.key].id
  route_table_id = aws_route_table.isolated.id
}

# --- VPC endpoints (no NAT needed for these) ---
resource "aws_vpc_endpoint" "s3" {
  vpc_id           = aws_vpc.main.id
  service_name     = "com.amazonaws.${var.region}.s3"
  vpc_endpoint_type = "Gateway"
  route_table_ids  = concat(
    [for rt in aws_route_table.private : rt.id],
    [aws_route_table.isolated.id],
  )
  tags = { Name = "${var.environment}-s3" }
}

resource "aws_vpc_endpoint" "ecr_api" {
  vpc_id           = aws_vpc.main.id
  service_name     = "com.amazonaws.${var.region}.ecr.api"
  vpc_endpoint_type = "Interface"
  private_dns_enabled = true
  subnet_ids       = [for s in aws_subnet.private : s.id]
  security_group_ids = [aws_security_group.vpc_endpoints.id]
}

resource "aws_vpc_endpoint" "ecr_dkr" {
  vpc_id           = aws_vpc.main.id
  service_name     = "com.amazonaws.${var.region}.ecr.dkr"
  vpc_endpoint_type = "Interface"
  private_dns_enabled = true
  subnet_ids       = [for s in aws_subnet.private : s.id]
  security_group_ids = [aws_security_group.vpc_endpoints.id]
}

resource "aws_vpc_endpoint" "secretsmanager" {
  vpc_id           = aws_vpc.main.id
  service_name     = "com.amazonaws.${var.region}.secretsmanager"

```



```

    vpc_endpoint_type    = "Interface"
    private_dns_enabled  = true
    subnet_ids           = [for s in aws_subnet.private : s.id]
    security_group_ids   = [aws_security_group.vpc_endpoints.id]
}

resource "aws_security_group" "vpc_endpoints" {
    name_prefix = "${var.environment}-vpce-"
    vpc_id      = aws_vpc.main.id
    ingress {
        from_port    = 443
        to_port      = 443
        protocol     = "tcp"
        cidr_blocks  = [local.vpc_cidr] # only from within the VPC
    }
    tags = { Name = "${var.environment}-vpce" }
}

# --- Flow logs (VPC-level observability – non-negotiable) ---
resource "aws_flow_log" "main" {
    vpc_id            = aws_vpc.main.id
    traffic_type      = "ALL" # ACCEPT and REJECT – REJECT reveals
    blocked_traffic
    iam_role_arn      = aws_iam_role.flow_logs.arn
    log_destination_type = "cloud-watch-logs"
    log_destination   = aws_cloudwatch_log_group.flow_logs.arn
    max_aggregation_interval = 60 # 60s for near-real-time; 600s cheaper
}

resource "aws_cloudwatch_log_group" "flow_logs" {
    name            = "/vpc/flow-logs/${var.environment}"
    retention_in_days = 90
    kms_key_id      = aws_kms_key.logs.arn
}

```

PrivateLink and Transit Gateway

- **PrivateLink (Interface Endpoints / Endpoint Services):** expose a service (your own or a SaaS provider's NLB) as an ENI inside the consumer's VPC. Traffic never traverses the public internet. Use for cross-VPC and SaaS ingress where peering would be too broad.
- **Transit Gateway (TGW):** hub-and-spoke routing between many VPCs and on-prem. Each VPC attaches to the TGW; route tables on the TGW control which VPCs can reach which. Prefer TGW over full-mesh peering beyond ~5 VPCs — peering is $O(n^2)$ attachments, TGW is $O(n)$.

Network segmentation: defense in depth

```
flowchart TB
    Internet["Internet"] --> WAF["WAF / CloudFront / Cloud Armor  
L7 filter: OWASP, bot, rate limit"]
    WAF --> ALB["ALB / Gateway  
TLS termination  
path/host routing"]
    ALB --> SG_ALB["SG: ALB  
allow 443 from 0.0.0.0/0  
deny all else"]

    SG_ALB --> SG_App["SG: App pods/nodes  
allow 8080 from SG:ALB only  
allow 9090 from SG:monitoring"]

    SG_App --> NACL["NACL (subnet)  
stateless: allow 443/8080  
deny known-bad CIDRs  
ephemeral return ports"]

    NACL --> NFW["Network Firewall  
stateful L4/L7  
IDS/IPS, FQDN filter  
egress allow-list"]

    NFW --> Pod["Pod / ENI  
NetworkPolicy  
allow from app namespace  
deny default"]

    Pod --> mTLS["mTLS (mesh)  
SPIFFE identity  
L7 authZ  
encrypt east-west"]

    style WAF fill:#fce4ec
    style SG_App fill:#e3f2fd
    style NACL fill:#fff3e0
    style NFW fill:#fff3e0
    style mTLS fill:#e8f5e9
```

Figure 10-2: Defense in depth — each layer has a distinct scope and failure mode. WAF/ALB handle north-south L7; security groups enforce ENI-level stateful filtering; NACLs provide subnet-level stateless guardrails; Network Firewall adds stateful IDS/IPS and egress control; Kubernetes NetworkPolicy and mesh mTLS secure east-west.

Security groups vs. NACLs

Property	Security Group	NACL
Scope	ENI / instance	Subnet
State	Stateful (return traffic auto-allowed)	Stateless (explicit ingress + egress rules)
Rule model	Allow only (implicit deny)	Allow and deny, evaluated in order
Typical use	Primary firewall — every ENI gets one	Guardrail — block known-bad CIDRs, enforce subnet isolation
Common mistake	0.0.0.0/0 on app ports	Forgetting ephemeral return ports (1024-65535)

```
# security-groups.tf – least-privilege SGs with no 0.0.0.0/0 on app
ports

resource "aws_security_group" "alb" {
  name_prefix = "${var.environment}-alb-"
  vpc_id      = aws_vpc.main.id
  description = "ALB – only 443 from internet"

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "HTTPS from internet"
  }
  egress {
    from_port = 8080
    to_port   = 8080
    protocol  = "tcp"
    security_groups = [aws_security_group.app.id]
    description    = "to app pods"
  }
  tags = { Name = "${var.environment}-alb" }
}

resource "aws_security_group" "app" {
  name_prefix = "${var.environment}-app-"
  vpc_id      = aws_vpc.main.id
  description = "App – only from ALB SG, not from CIDR"

  ingress {
    from_port = 8080
    to_port   = 8080
    protocol  = "tcp"
    security_groups = [aws_security_group.alb.id] # SG-to-SG – no CIDR
    description    = "from ALB only"
  }
  ingress {
    from_port = 9090
    to_port   = 9090
    protocol  = "tcp"
    security_groups = [aws_security_group.monitoring.id]
    description    = "scraping from monitoring"
  }
  egress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # tighten with prefix lists or egress
    firewall as maturity grows
  }
}
```

```

    description = "egress to AWS APIs / internet"
  }
  tags = { Name = "${var.environment}-app" }
}

# NACL as a guardrail – block a known-bad CIDR at the subnet boundary
resource "aws_network_acl" "private" {
  vpc_id      = aws_vpc.main.id
  subnet_ids = [for s in aws_subnet.private : s.id]
  tags        = { Name = "${var.environment}-private-nacl" }
}

resource "aws_network_acl_rule" "deny_bad_cidr" {
  network_acl_id = aws_network_acl.private.id
  rule_number    = 90 # evaluated before allow rules at 100+
  egress         = false
  protocol       = "-1"
  rule_action    = "deny"
  cidr_block     = "203.0.113.0/24" # example: known scanner net
}

resource "aws_network_acl_rule" "allow_app_ingress" {
  network_acl_id = aws_network_acl.private.id
  rule_number    = 100
  egress         = false
  protocol       = "tcp"
  rule_action    = "allow"
  cidr_block     = "0.0.0.0/0"
  from_port     = 8080
  to_port       = 8080
}

resource "aws_network_acl_rule" "allow_ephemeral_egress" {
  network_acl_id = aws_network_acl.private.id
  rule_number    = 100
  egress         = true
  protocol       = "tcp"
  rule_action    = "allow"
  cidr_block     = "0.0.0.0/0"
  from_port     = 1024
  to_port       = 65535
}

```

Kubernetes NetworkPolicy (east-west)

```

# networkpolicy.yaml – default deny + explicit allow (Kubernetes-native
segmentation)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: prod
spec:
  podSelector: {}          # applies to all pods in namespace
  policyTypes: [Ingress, Egress]
  # No ingress/egress rules = deny all – every allowed flow must be
explicit

---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-allow
  namespace: prod
spec:
  podSelector:
    matchLabels: { app: api }
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
      - namespaceSelector:
          matchLabels: { name: ingress } # from ingress namespace
      - podSelector:
          matchLabels: { app: frontend } # or from frontend in same
namespace
      ports:
        - port: 8080
          protocol: TCP
    - from:
      - namespaceSelector:
          matchLabels: { name: monitoring }
      ports:
        - port: 9090 # metrics
  egress:
    - to:
      - namespaceSelector:
          matchLabels: { name: prod }
      podSelector:
          matchLabels: { app: postgres }
      ports:
        - port: 5432
    - to:
      - namespaceSelector:
          matchLabels: { name: kube-system }
      podSelector:

```

```

        matchLabels: { k8s-app: kube-dns }
    ports:
      - port: 53
        protocol: UDP
      - port: 53
        protocol: TCP
  - ports:                # allow 443 to VPC endpoints / AWS APIs
    - port: 443
      protocol: TCP

```

IAM: the other fabric

If the network decides *which packets can reach a resource*, IAM decides *which principals can call which APIs on which resources*. In AWS, every API call is authorized against IAM — there is no backdoor.

Policy evaluation logic

```

flowchart TB
    Req["API call  
principal + action + resource + context"] --> Eval["Evaluate all  
applicable policies"]
    Eval --> ExplicitDeny["Explicit Deny  
in any policy?"]
    ExplicitDeny -->|Yes| Deny["DENY  
wins over everything"]
    ExplicitDeny -->|No| AllowCheck["Allow in any  
identity or resource  
policy?"]
    AllowCheck -->|No| ImplicitDeny["Implicit DENY  
(default)"]
    AllowCheck -->|Yes| Boundary["Within  
permission boundary  
and SCP?"]
    Boundary -->|No| Deny
    Boundary -->|Yes| Allow["ALLOW"]

    style Deny fill:#ffcdd2
    style ImplicitDeny fill:#ffcdd2
    style Allow fill:#c8e6c9

```

Figure 10-3: IAM policy evaluation — an explicit deny in any applicable policy wins unconditionally; otherwise an explicit allow is required from

an identity or resource policy, and the result must also sit within the permission boundary and SCP.

Policy types and their scope:

Policy type	Attached to	Effect
Identity-based	User / group / role	Grants permissions to the principal
Resource-based	Resource (S3 bucket, SQS queue, KMS key, ECR repo)	Grants permissions to principals; required for cross-account
Permission boundary	User / role	Upper bound — caps what identity policies can grant
SCP (Service Control Policy)	OU / account (Organizations)	Upper bound across the entire account — even admin cannot exceed
Session policy	Assumed-role session	Further narrows the session
Trust policy	Role	Who can assume the role and under what conditions

For a call to succeed: **(identity allows OR resource allows) AND no explicit deny anywhere AND within boundary AND within SCP.**

Least-privilege IAM in practice

```
// iam-policies/api-role.json – application role: minimal S3 + SQS +
Secrets Manager
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadConfigFromS3",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::app-config-prod/*",
      "Condition": {
        "StringEquals": { "aws:RequestedRegion": "us-east-1" }
      }
    },
    {
      "Sid": "ConsumeOrdersQueue",
      "Effect": "Allow",
      "Action": ["sqs:ReceiveMessage", "sqs:DeleteMessage", "sqs:GetQueueAttributes"],
      "Resource": "arn:aws:sqs:us-east-1:123456789012:orders-prod"
    },
    {
      "Sid": "ReadSecretsForThisServiceOnly",
      "Effect": "Allow",
      "Action": ["secretsmanager:GetSecretValue"],
      "Resource": "arn:aws:secretsmanager:us-east-1:123456789012:secret:prod/api/*",
      "Condition": {
        "StringEquals": { "aws:ResourceTag/Service": "api" }
      }
    },
    {
      "Sid": "WriteLogsAndMetrics",
      "Effect": "Allow",
      "Action": ["logs:CreateLogStream", "logs:PutLogEvents", "cloudwatch:PutMetricData"],
      "Resource": "*",
      "Condition": {
        "StringEquals": { "aws:RequestedRegion": "us-east-1" },
        "ForAllValues:StringEquals": { "cloudwatch:namespace": "prod/api" }
      }
    },
    {
      "Sid": "DenyOutsideVPC",
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "StringNotEquals": { "aws:SourceVpc": "vpc-0a1b2c3d4e5f" }
      }
    }
  ]
}
```

```
}  
  ]  
}
```

```

# iam.tf – wire the policy to a role with a trust policy for EKS IRSA

resource "aws_iam_role" "api" {
  name           = "api-prod"
  assume_role_policy = data.aws_iam_policy_document.api_trust.json
  managed_policy_arns = [aws_iam_policy.api.arn]
  permissions_boundary = aws_iam_policy.boundary.arn # cap even if
policy is later broadened
  max_session_duration = 3600
  tags = { Service = "api", Environment = "prod" }
}

data "aws_iam_policy_document" "api_trust" {
  statement {
    effect = "Allow"
    actions = ["sts:AssumeRoleWithWebIdentity"]
    principals {
      type       = "Federated"
      identifiers = [aws_iam_openid_connect_provider.eks.arn]
    }
    condition {
      test      = "StringEquals"
      variable = "${replace(aws_iam_openid_connect_provider.eks.url,
"https://", "")}:sub"
      values    = ["system:serviceaccount:prod:api"]
    }
    condition {
      test      = "StringEquals"
      variable = "${replace(aws_iam_openid_connect_provider.eks.url,
"https://", "")}:aud"
      values    = ["sts.amazonaws.com"]
    }
  }
}

resource "aws_iam_policy" "api" {
  name     = "api-prod"
  policy = file("${path.module}/iam-policies/api-role.json")
}

# Permission boundary – even if someone attaches AdministratorAccess,
this caps it
resource "aws_iam_policy" "boundary" {
  name = "boundary-prod"
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "BoundaryAllowlist"
        Effect   = "Allow"

```

```

        Action    = ["s3:*", "sqs:*", "secretsmanager:GetSecretValue",
                    "logs:*", "cloudwatch:PutMetricData", "xray:*"]
        Resource  = "*"
        Condition = {
            StringEquals = { "aws:RequestedRegion" : "us-east-1" }
        }
    },
    {
        Sid       = "DenyIAMAndOrgs"
        Effect    = "Deny"
        Action    = ["iam:*", "organizations:*", "account:*"]
        Resource  = "*"
    }
}
}))
}

```

SCP at the OU level – no account in prod OU can launch outside approved regions or disable CloudTrail

```

resource "aws_organizations_policy" "prod_guardrails" {
    name     = "prod-guardrails"
    type     = "SERVICE_CONTROL_POLICY"
    content = jsonencode({
        Version = "2012-10-17"
        Statement = [
            {
                Sid       = "DenyOutsideApprovedRegions"
                Effect    = "Deny"
                Action    = "*"
                Resource  = "*"
                Condition = {
                    StringNotEquals = { "aws:RequestedRegion" : ["us-east-1",
"us-east-1a", "us-east-1b", "us-east-1c"] }
                }
            },
            {
                Sid       = "DenyDisableCloudTrail"
                Effect    = "Deny"
                Action    =
["cloudtrail:StopLogging", "cloudtrail:DeleteTrail",
"cloudtrail:UpdateTrail"]
                Resource  = "*"
            },
            {
                Sid       = "RequireIMDSv2"
                Effect    = "Deny"
                Action    = "ec2:RunInstances"
                Resource  = "arn:aws:ec2:*:*:instance/*"
                Condition = {
                    StringNotEquals = { "ec2:MetadataHttpTokens" : "required" }
                }
            }
        ]
    })
}

```

```

    }
  }
}

```

Workload identity: IRSA, SPIFFE, and cross-account

IRSA (IAM Roles for Service Accounts) binds a Kubernetes ServiceAccount to an IAM role via OIDC federation. No static credentials — the pod gets a projected service-account token, exchanges it for STS credentials, and assumes the role.

```

# k8s/serviceaccount.yaml – IRSA binding (EKS)
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api
  namespace: prod
  annotations:
    eks.amazonaws.com/role-arn: "arn:aws:iam::123456789012:role/api-
prod"
automountServiceAccountToken: true

---
# The Deployment references the ServiceAccount – every pod gets the
role
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: prod
spec:
  template:
    spec:
      serviceAccountName: api
      containers:
        - name: api
          image: registry.example.com/api:1.42.0
          env:
            - name: AWS_REGION
              value: us-east-1
            # No AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY –
credentials come from STS via the projected token

```

Cross-account access uses `sts:AssumeRole` with a trust policy that includes `ExternalId` (confused-deputy protection) and `aws:SourceAccount` / `aws:SourceArn` conditions:

```

# cross-account role – prod account allows CI account to deploy, but
only via a specific role and with ExternalId
data "aws_iam_policy_document" "cross_account_trust" {
  statement {
    effect = "Allow"
    actions = ["sts:AssumeRole"]
    principals {
      type        = "AWS"
      identifiers = ["arn:aws:iam::999999999999:role/ci-deployer"]
    }
    condition {
      test      = "StringEquals"
      variable = "sts:ExternalId"
      values    = [var.deploy_external_id] # random UUID, stored in
Secrets Manager
    }
    condition {
      test      = "StringEquals"
      variable = "aws:PrincipalOrgID"
      values    = [var.org_id] # only principals in our Organization
    }
  }
}

```

GCP equivalent — Workload Identity Federation:

```

# gcp-workload-identity.tf – GKE workload identity (no service-account
keys)
resource "google_service_account" "api" {
  account_id = "api-prod"
  display_name = "api prod"
}

resource "google_service_account_iam_member" "workload_identity" {
  service_account_id = google_service_account.api.name
  role                = "roles/iam.workloadIdentityUser"
  member              = "serviceAccount:$
{var.project_id}.svc.id.goog[prod/api]"
}

resource "google_project_iam_member" "api_storage" {
  project = var.project_id
  role    = "roles/storage.objectViewer"
  member  = "serviceAccount:${google_service_account.api.email}"
}

```



```
# GKE ServiceAccount annotated for Workload Identity
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api
  namespace: prod
  annotations:
    iam.gke.io/gcp-service-account: "api-prod@my-
project.iam.gserviceaccount.com"
```

The confused-deputy problem

When a service (e.g., S3, CloudTrail, or your own cross-account role) is tricked into using its authority on behalf of an attacker, it is a confused deputy. The fix is to include the caller's identity in the resource policy condition:

```
{
  "Sid": "AllowOnlyFromMyAccount",
  "Effect": "Allow",
  "Principal": { "Service": "cloudtrail.amazonaws.com" },
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3::my-cloudtrail-bucket/AWSLogs/123456789012/
*",
  "Condition": {
    "StringEquals": { "aws:SourceAccount": "123456789012" },
    "ArnLike": { "aws:SourceArn": "arn:aws:cloudtrail:us-
east-1:123456789012:trail/*" }
  }
}
```

Without `aws:SourceAccount` / `aws:SourceArn`, any account's CloudTrail could write to your bucket — or an attacker could create a trail that exfiltrates via your resource policy.

Security foundations: encryption, detection, and guardrails

Encryption at rest and in transit

- **At rest:** every VPC resource that stores data (EBS, RDS, S3, EFS, Secrets Manager, CloudWatch Logs) is encrypted with KMS. Use

customer-managed keys (CMKs) for services where you need rotation control and CloudTrail visibility into `Decrypt` calls.

- **In transit:** TLS everywhere — ALB with ACM certificates, RDS with `rds-ca-rsa2048-g1`, EFS with `stunnel`, and mesh mTLS for east-west (see below). Enforce with SCP (`Deny unencrypted S3 uploads via s3:x-amz-server-side-encryption` condition) and with Kyverno/OPA policies.

Detection: flow logs, GuardDuty, Access Analyzer

```
# detection.tf – GuardDuty + Access Analyzer + CloudTrail trail
resource "aws_guardduty_detector" "main" {
  enable = true
  datasources {
    s3_logs { enable = true }
    kubernetes { audit_logs { enable = true } }
  }
  tags = { Environment = var.environment }
}

resource "aws_accessanalyzer_analyzer" "main" {
  analyzer_name = "${var.environment}-analyzer"
  type          = "ACCOUNT"
  tags          = { Environment = var.environment }
}

resource "aws_cloudtrail" "main" {
  name = "${var.environment}-trail"
  s3_bucket_name = aws_s3_bucket.trail.id
  include_global_service_events = true
  is_multi_region_trail        = true
  enable_log_file_validation   = true
  kms_key_id                   = aws_kms_key.trail.arn
  cloud_watch_logs_group_arn   = "${aws_cloudwatch_log_group.trail.arn}:*"
  cloud_watch_logs_role_arn    = aws_iam_role.cloudtrail.arn
  insight_selector { insight_type = "ApiCallRateInsight" }
  tags = { Environment = var.environment }
}
```

Policy-as-code guardrails

```
# policy/network.rego – OPA/Conftest: deny overly broad SGs and IAM
wildcards
package main

deny[msg] {
    sg := input.resource_changes[_]
    sg.type == "aws_security_group"
    rule := sg.change.after.ingress[_]
    rule.cidr_blocks[_] == "0.0.0.0/0"
    rule.from_port != 443
    rule.from_port != 80
    msg := sprintf("SG %q allows 0.0.0.0/0 on port %v – restrict to ALB
SG or known CIDR", [sg.address, rule.from_port])
}

deny[msg] {
    pol := input.resource_changes[_]
    contains(pol.type, "aws_iam_policy")
    stmt := json.unmarshal(pol.change.after.policy).Statement[_]
    stmt.Effect == "Allow"
    stmt.Action[_] == "*:*"
    msg := sprintf("IAM policy %q contains Action '*:*' – use least
privilege", [pol.address])
}

deny[msg] {
    pol := input.resource_changes[_]
    contains(pol.type, "aws_iam_policy")
    stmt := json.unmarshal(pol.change.after.policy).Statement[_]
    stmt.Effect == "Allow"
    stmt.Resource == "*"
    not stmt.Condition # wildcard resource without condition is overly
broad
    msg :=
sprintf("IAM policy %q allows Resource '*' without Condition – scope to
specific ARNs", [pol.address])
}
```

Distributed-systems lens

Network and IAM are the two global coordination planes that every service depends on — and that every team shares. Three practices keep them coherent at scale:

1. **VPC and IAM as platform products.** No team creates its own VPC or IAM role from scratch. The platform team publishes a Terraform module (`modules/network` , `modules/workload-identity`) that encodes the approved topology, flow logs, endpoints, and permission boundary. Teams instantiate the module with narrow inputs (service name, required AWS actions) — the platform owns the invariants.
 2. **Blast-radius budgeting.** Every workload runs in a dedicated account (or at minimum a dedicated VPC + IAM boundary) so that a compromise in one service cannot call APIs or reach ENIs in another. SCPs and permission boundaries enforce the ceiling; VPC isolation enforces the floor.
 3. **Continuous verification, not periodic audit.** IAM Access Analyzer findings, GuardDuty alerts, and VPC flow-log anomalies feed the same SIEM/SOAR pipeline as application logs. A new `0.0.0.0/0` security-group rule or a new `iam:PassRole` grant triggers an alert within minutes — not at the next quarterly review.
-

Key takeaways

- A production VPC has three subnet tiers — public (ALB + NAT GW), private (compute, per-AZ NAT), and isolated (no internet, VPC endpoints only) — with one NAT Gateway per AZ for isolation and bandwidth.
- VPC endpoints (Gateway for S3/Dynamo, Interface/PrivateLink for ECR/Secrets/CloudWatch) remove NAT dependency and keep traffic on the provider backbone.
- Security groups (stateful, ENI-scoped, allow-only) are the primary firewall; NACLs (stateless, subnet-scoped, allow+deny) are a guardrail; Network Firewall adds IDS/IPS and egress allow-listing; NetworkPolicy + mesh mTLS secure east-west.

- IAM evaluation is deny-wins: an explicit deny anywhere overrides any allow, and the effective permission must sit within the permission boundary and SCP.
- Least privilege means scoping `Action` to the minimal API set, `Resource` to specific ARNs, and adding `Condition` (region, VPC, tag, `SourceAccount` / `SourceArn` for confused-deputy protection).
- Workload identity (IRSA / GKE Workload Identity / SPIFFE) eliminates static credentials — pods get short-lived STS tokens via OIDC federation, bound to a single `ServiceAccount`.
- Cross-account trust policies must include `ExternalId` and `aws:PrincipalOrgID` / `aws:SourceAccount` to prevent confused-deputy abuse.
- Encryption at rest (KMS CMKs), in transit (TLS + mesh mTLS), and detection (VPC flow logs, GuardDuty, Access Analyzer, CloudTrail) are baseline — not optional — and belong in the platform module every team inherits.

Further reading

- AWS VPC documentation — <https://docs.aws.amazon.com/vpc/latest/userguide/>
- AWS IAM policy evaluation logic — https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html
- AWS PrivateLink and Transit Gateway — <https://docs.aws.amazon.com/vpc/latest/privatelink/> , <https://docs.aws.amazon.com/vpc/latest/tgw/>
- Kubernetes NetworkPolicy — <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- SPIFFE/SPIRE — <https://spiffe.io/docs/latest/> , <https://spiffe.io/docs/latest/spire-about/>
- NIST SP 800-210 — General Access Control Guidance for Cloud Systems; NIST SP 800-53 (AC, SC families).

- Shillaker, *The Cloud Native Security Handbook* (O'Reilly) — VPC, IAM, and workload identity patterns.

Chapter 11 — Platform Engineering: Paved Roads and Internal Developer Platforms

What this chapter covers. Every engineering organization eventually hits the same bottleneck: each team re-solves the same problems — how to scaffold a service, wire CI/CD, provision infrastructure, handle secrets, observe it in production — with slightly different, slightly broken answers. Platform engineering replaces that sprawl with *paved roads*: opinionated, supported paths that make the right way the easy way, and an Internal Developer Platform (IDP) that exposes those paths as self-service. This chapter builds the mental model, architecture, and operational practice for platform engineering at cloud scale — from the paved-road philosophy and platform-as-product discipline, through IDP components (service catalog, templates, orchestration, and developer portal), to a production-grade Backstage implementation with software templates, TechDocs, Kubernetes and Terraform integration, scorecards, and the operating model that keeps a platform adopted rather than mandated.

Learning goals — after this chapter you should be able to:

- Articulate the platform engineering value proposition — cognitive load, lead time, reliability, and compliance — and distinguish platforms from traditional shared infrastructure or DevOps tooling.
- Design paved roads and golden paths: opinionated service templates, CI/CD pipelines, infrastructure modules, and observability defaults that encode organizational standards while preserving escape hatches.
- Architect an IDP — service catalog, software templates/scaffolder, orchestration (workflow engine), developer portal (Backstage or equivalent), and integrations (CI, Git, Kubernetes, cloud, observability, secrets) — and explain how data flows between them.

- Operate Backstage in production: `app-config.yaml`, `catalog-info.yaml`, software templates (`template.yaml`) with cookiecutter/templating, TechDocs, Kubernetes plugin, Terraform integration, and authentication (GitHub/Google OIDC, RBAC).
- Measure platform adoption and health with DORA metrics, service maturity scorecards, template usage, and time-to-first-deploy — and run the product discipline (discovery, roadmap, deprecation) that keeps a platform relevant.
- Avoid the failure modes that kill platforms: building a framework instead of a product, mandating without listening, over-abstraction, under-documentation, and treating the portal as the platform.

Boundary note. Chapters 1-3 built the container and Kubernetes substrate; Chapter 4 built the IaC delivery pipeline; Chapters 5-6 added cloud primitives and managed services; Chapters 9-10 sized and secured the fleet. This chapter is the *human interface* atop all of it — how developers consume the platform without needing to master every layer underneath. Volume 11 — Reliability & SRE — covers the SRE practices that the paved road encodes by default (SLOs, incident response, chaos); Volume 15 — Software Engineering Practice — covers the team and process practices that platform engineering amplifies.

Why platform engineering

The problem: undifferentiated toil at scale

At 10 services, every team can own its own pipeline, Terraform, and dashboards. At 200 services, the same work is duplicated 200 times with 200 variants of wrong:

- A new service takes 3-6 weeks to reach production — most of it scaffolding, pipeline wiring, and security review.
- No one can answer "which services use log4j 2.14?" or "which pipelines lack image signing?" because there is no catalog.
- Security and compliance are enforced by review tickets — slow, inconsistent, and resented.
- On-call is uneven — some services have SLOs and runbooks, others have a dashboard someone built two years ago.

The bottleneck is not talent — it is **cognitive load**. A product engineer who wants to ship a feature should not need to be an expert in Kubernetes, Terraform, IAM, and Prometheus to do it. Platform engineering pushes that expertise into the platform so product teams can spend their cognitive budget on the product.

Paved roads, not walled gardens

The central metaphor — from Netflix, Spotify, and later the CNCF — is the **paved road**:

- **Paved road (golden path):** the supported, opinionated path for a common need. "To build a stateless Go API, use this template — it gives you a repo, CI, IaC, Kubernetes manifests, dashboards, and alerts that pass security review automatically." The road is paved because someone maintains it, documents it, and keeps it compliant.
- **Unpaved (wilderness):** off-road is allowed — you can bring your own stack — but you own the maintenance. No one stops you; no one paves it for you.
- **Guardrails, not gates:** the platform makes the secure, compliant choice the default (signed images, least-privilege IAM, mTLS, flow logs) rather than a gate that requires a ticket.

A platform that mandates a single framework for every workload fails. A platform that offers no opinion drowns teams in choice. The art is in **opinionated defaults with explicit escape hatches**.


```

flowchart LR
    Dev["Developer  
new service / feature"] --> Q1{Question{"Common need?  
stateless API, worker,  
cron, ML batch?"}}
    Q1 -->|Yes| Paved["Paved road  
template → repo + CI + IaC  
+ K8s + observability  
compliant by default"]
    Q1 -->|No / special| Wilderness["Wilderness  
bring your own  
you own the maintenance"]
    Paved --> Catalog["Service catalog  
registered, discoverable  
scorecard + ownership"]
    Wilderness --> Catalog
    Catalog --> Prod["Production  
SLOs, runbooks, alerts  
– uniform on paved,  
– self-managed off-road"]

    Paved -.->|fast: hours| Prod
    Wilderness -.->|slow: weeks| Prod

    style Paved fill:#c8e6c9
    style Wilderness fill:#fff3e0
    style Catalog fill:#e3f2fd

```

Figure 11-1: Paved roads vs. wilderness — common needs take the paved road (hours to production, compliant by default); uncommon needs can go off-road but own the maintenance. Both end up in the catalog.

Platform as product

A platform is a **product** whose users are internal engineers. That framing has concrete consequences:

Product discipline	Platform equivalent
User research	Interviews with product teams; map their delivery pain
Roadmap	Prioritized by lead time, toil survey, and incident data — not by what's fun to build
Adoption metrics	Template usage, time-to-first-deploy, catalog coverage, DORA lead time

Product discipline	Platform equivalent
Documentation	The paved road is undocumented if the template's README and TechDocs are stale
Deprecation	Old templates and pipeline versions are sunset with migration guides, not left to rot
Support	On-call for the platform itself; SLAs for template and pipeline availability

A platform team that builds what it thinks is cool without talking to users builds a framework no one adopts. The most successful platforms (Spotify's Backstage origin story is the canonical example) started by **extracting** the paved road from what the best teams already did, not by inventing it in isolation.

IDP architecture

An Internal Developer Platform is the runtime that exposes paved roads as self-service. The CNCF Platform Engineering Working Group defines it as the layer that integrates the underlying capabilities (compute, network, IaC, CI/CD, observability, security) into a coherent developer experience. It is not a single tool — it is a composition.

```

flowchart TB
    subgraph Experience["Experience layer"]
        Portal["Developer portal"]
    end
    Backstage / custom
    catalog, docs, scorecards["CLI["CLI / API"]
    scaffold, deploy, kubeconfig
    headless access["end
    subgraph Orchestration["Orchestration layer"]
        Scaffolder["Scaffolder / workflow engine"]
    end
    cookiecutter + actions
    Git + CI + IaC + K8s calls["Workflows["Workflows"]
    Argo Workflows / Temporal
    long-running provision["end
    subgraph Catalog["Catalog and governance"]
        CatalogDB["Service catalog"]
        entities: Component, API,
        Resource, Group, User["Scorecard["Scorecards"]
        maturity, compliance
        DORA, security posture["Docs["TechDocs"]
        docs-as-code
        rendered in portal["end
    subgraph Capabilities["Capability layer (paved roads)"]
        Templates["Service templates"]
        Go API, Python worker,
        ML batch, static site["Modules["IaC modules"]
        network, workload-identity
        RDS, cache["Pipelines["CI/CD pipelines"]
        GitHub Actions / Argo CD
        signed, scanned, gated["Obs["Observability defaults"]
        dashboards, alerts, SLOs
        trace + log + metric["end
    subgraph Substrate["Substrate (Ch 1-10)"]
        K8s["Kubernetes + Cloud"]
        VPC, IAM, managed services["end

Portal --- Scaffolder
CLI --- Scaffolder
Scaffolder --- Templates & Modules & Pipelines

```

```
Scaffolder --- CatalogDB
Portal --- CatalogDB & Scorecard & Docs
CatalogDB --- K8s
Scorecard --- Pipelines & Obs
Templates --- K8s
Modules --- K8s

style Portal fill:#e3f2fd
style Scaffolder fill:#fff3e0
style CatalogDB fill:#fce4ec
style Templates fill:#c8e6c9
style K8s fill:#f3e5f5
```

Figure 11-2: IDP reference architecture — the developer interacts via portal or CLI; the scaffolder orchestrates templates, IaC modules, pipelines, and observability defaults against the Kubernetes/cloud substrate; the catalog and scorecards provide discovery and governance.

Components and their contracts:

- **Service catalog** — the single source of truth for "what exists." Every service, API, resource, team, and dependency is an entity with ownership, lifecycle, and links. Without a catalog, you cannot answer "what do we have?" — and without that, you cannot govern it.
- **Software templates (scaffolder)** — cookiecutter-style generators that produce a new repo from a golden path, wire CI, create IaC, register the catalog entry, and open the first PR. The template is the paved road made executable.
- **Orchestration / workflow engine** — long-running provisioning (VPC, database, DNS) needs a durable workflow, not a synchronous HTTP call. Argo Workflows or Temporal behind the scaffolder handle retries, approvals, and human-in-the-loop steps.
- **Developer portal** — the UI that surfaces the catalog, docs, templates, scorecards, and plugin integrations (CI status, Kubernetes, costs, incidents). Backstage is the de facto open standard; alternatives include Port, Cortex, and custom portals.
- **Capability modules** — the actual paved roads: Terraform modules, pipeline definitions, Helm charts, and observability mixins that templates compose.

Backstage in production

Backstage (Spotify, CNCF Incubating) is the most widely adopted IDP portal. It is a React frontend + Node.js backend with a plugin architecture — the platform team assembles the portal from core and community plugins rather than building from scratch.

App configuration

```

# app-config.yaml – Backstage production configuration (secrets via env
vars)
app:
  title: Acme Developer Portal
  baseUrl: https://backstage.acme.example.com

organization:
  name: Acme

backend:
  baseUrl: https://backstage.acme.example.com
  listen:
    port: 7007
  csp:
    connect-src: ["'self'", "https://api.github.com", "https://
gitlab.acme.example.com"]
  cors:
    origin: https://backstage.acme.example.com
    credentials: true
  database:
    client: pg
    connection:
      host: ${POSTGRES_HOST}
      port: ${POSTGRES_PORT}
      user: ${POSTGRES_USER}
      password: ${POSTGRES_PASSWORD}
      ssl: { rejectUnauthorized: false }
  cache:
    store: redis
    connection: ${REDIS_URL}

integrations:
  github:
    - host: github.com
      token: ${GITHUB_TOKEN} # GitHub App token – not a PAT

auth:
  environment: production
  providers:
    github:
      development:
        clientId: ${AUTH_GITHUB_CLIENT_ID}
        clientSecret: ${AUTH_GITHUB_CLIENT_SECRET}
      signIn:
        resolvers:
          - resolver: usernameMatchingUserEntityName
    oidc:
      google:
        metadataUrl: https://accounts.google.com/.well-known/openid-
configuration

```

```

    clientId: ${AUTH_OIDC_CLIENT_ID}
    clientSecret: ${AUTH_OIDC_CLIENT_SECRET}

catalog:
  rules:
    - allow: [Component, System, API, Resource, Group, User, Location,
Template]
  locations:
    - type: url
      target: https://github.com/acme/backstage-catalog/blob/main/
all.yaml
    - type: url
      target: https://github.com/acme/service-catalog/blob/main/
catalog-info.yaml
  providers:
    githubOrg:
      id: acme-org
      githubUrl: https://github.com
      orgs: [acme]
      schedule:
        frequency: { minutes: 30 }
        timeout: { minutes: 5 }

permission:
  enabled: true # RBAC – see below

scaffolder:
  defaultAuthor:
    name: Acme Scaffolder
    email: scaffolder@acme.example.com

techdocs:
  builder: local
  generator:
    runIn: docker
  publisher:
    type: awsS3
    awsS3:
      bucketName: acme-techdocs-${ENVIRONMENT}
      region: us-east-1
      s3ForcePathStyle: false
      credentials:
        accessKeyId: ${TECHDOCS_AWS_ACCESS_KEY_ID}
        secretAccessKey: ${TECHDOCS_AWS_SECRET_ACCESS_KEY}

kubernetes:
  serviceLocatorMethod:
    type: multiTenant
  clusterLocatorMethods:
    - type: config
      clusters:

```



```
- name: prod-us-east-1
  url: https://k8s-prod.acme.example.com
  authProvider: oidc
  oidcTokenProvider: google
  dashboardUrl: https://k8s-prod.acme.example.com
  dashboardApp: gke

costInsights:
  engineerCost: 200000 # for cost-vs-build decisions in Cost Insights
  plugin
```

Service catalog: catalog-info.yaml

Every repo contains a `catalog-info.yaml` that registers it with the portal. Backstage discovers these via the `catalog.locations` or GitHub discovery provider.

```

# catalog-info.yaml – lives at the repo root; registered automatically
via GitHub discovery
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: orders-api
  title: Orders API
  description: Order lifecycle service – creates, validates, and
fulfills customer orders
  tags: [go, api, orders, team-checkout]
  annotations:
    github.com/project-slug: acme/orders-api
    backstage.io/techdocs-ref: dir:.
    backstage.io/kubernetes-id: orders-api
    aws.amazon.com/role-arn: arn:aws:iam::123456789012:role/orders-api-
prod
    pagerduty.com/service-id: PDSVC123
    cost-insights.io/product: orders
spec:
  type: service
  lifecycle: production
  owner: group:checkout      # must be a Group entity – ownership is
mandatory
  system: commerce          # System groups related components
  dependsOn:
    - component:payments-api
    - resource:orders-db
    - resource:orders-queue
  providesApis:
    - orders-rest
  consumesApis:
    - payments-rest
    - inventory-grpc
---
apiVersion: backstage.io/v1alpha1
kind: API
metadata:
  name: orders-rest
  title: Orders REST API
  description: REST API for order management
spec:
  type: openapi
  lifecycle: production
  owner: group:checkout
  system: commerce
  definition: |
    openapi: 3.0.3
    info: { title: Orders API, version: 1.2.0 }
    paths:
      /orders:

```

```

      post:
        operationId: createOrder
        summary: Create a new order
    ---
  apiVersion: backstage.io/v1alpha1
  kind: Resource
  metadata:
    name: orders-db
    title: Orders Postgres (RDS)
    description: RDS Postgres 16 – primary store for orders
    tags: [postgres, rds, managed]
    annotations:
      aws.amazon.com/arn: arn:aws:rds:us-east-1:123456789012:db:orders-
prod
spec:
  type: database
  owner: group:checkout
  system: commerce
  dependsOn:
    - resource:prod-vpc
  ---
  apiVersion: backstage.io/v1alpha1
  kind: Group
  metadata:
    name: checkout
    title: Checkout Team
    description: Owns orders, payments, and cart
spec:
  type: team
  parent: commerce-org
  children: []
  profile:
    displayName: Checkout
    email: checkout@acme.example.com
    members: [alice, bob, carol]
  ---
  apiVersion: backstage.io/v1alpha1
  kind: System
  metadata:
    name: commerce
    title: Commerce System
    description: E-commerce core – orders, payments, inventory,
fulfillment
spec:
  owner: group:commerce-org
  domain: commerce

```

Software templates: the paved road made executable

A Backstage software template is a `template.yaml` that declares parameters (inputs), steps (actions), and output (the generated repo). When a developer clicks "Create Component," the scaffolder runs the steps.

```

# templates/go-api/template.yaml – golden path for a Go stateless API
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: go-api
  title: Go API Service
  description: Opinionated Go API – Go 1.22, net/http + chi, Postgres,
  CI, IaC, K8s, observability
  tags: [go, api, paved-road, recommended]
spec:
  owner: group:platform
  type: service

  parameters:
    - title: Service metadata
      required: [name, owner, system]
      properties:
        name:
          title: Service name
          type: string
          pattern: ^[a-z][a-z0-9-]{2,30}$
          description: Lowercase, hyphenated – becomes repo and K8s
name
  description:
    title: Description
    type: string
    maxLength: 200
  owner:
    title: Owner
    type: string
    ui:field: OwnerPicker
    ui:options: { catalogFilter: { kind: Group } }
  system:
    title: System
    type: string
    ui:field: EntityPicker
    ui:options: { catalogFilter: { kind: System } }
  - title: Deployment
    properties:
      environment:
        title: Initial environment
        type: string
        enum: [prod, staging]
        default: staging
      region:
        title: AWS region
        type: string
        enum: [us-east-1, eu-west-1]
        default: us-east-1
      enableDb:

```

```

        title: Provision Postgres (RDS)?
        type: boolean
        default: false

steps:
  # 1. Generate from cookiecutter skeleton
  - id: fetchSkeleton
    name: Fetch skeleton
    action: fetch:template
    input:
      url: ./skeleton
      values:
        name: ${ parameters.name }
        description: ${ parameters.description }
        owner: ${ parameters.owner }
        system: ${ parameters.system }
        goModule: github.com/acme/${ parameters.name }

  # 2. Publish to GitHub – creates the repo
  - id: publish
    name: Publish to GitHub
    action: publish:github
    input:
      repoUrl: github.com?owner=acme&repo=${ parameters.name }
      defaultBranch: main
      protectDefaultBranch: true
      repoVisibility: private
      requiredStatusChecks: [ci / test, ci / lint, ci / vuln-scan]

  # 3. Register in Backstage catalog
  - id: register
    name: Register in catalog
    action: catalog:register
    input:
      repoContentsUrl: ${ steps.publish.output.repoContentsUrl }
      catalogInfoPath: /catalog-info.yaml

  # 4. Trigger IaC – create ECR repo, K8s namespace, and (optionally) RDS
  via Terraform Cloud
  - id: provisionInfra
    name: Provision infrastructure
    action: acme:terraform-cloud:run
    input:
      workspace: ${ parameters.name }-${ parameters.environment }
      variables:
        service_name: ${ parameters.name }
        environment: ${ parameters.environment }
        region: ${ parameters.region }
        enable_db: ${ parameters.enableDb }

```

```

# 5. Wire CI – trigger initial pipeline run
- id: triggerCi
  name: Trigger initial CI
  action: github:actions:dispatch
  input:
    repoUrl: github.com?owner=acme&repo={{ parameters.name }}
    workflowId: ci.yaml
    branchOrTagName: main

output:
  links:
    - title: Repository
      url: {{ steps.publish.output.remoteUrl }}
    - title: Open in catalog
      entityRef: component:default/{{ parameters.name }}
    - title: Pipeline run
      url: {{ steps.publish.output.remoteUrl }}/actions

```

The skeleton (templates/go-api/skeleton/) contains the actual files with `{{ values.name }}` substitution:

```

skeleton/
├─ catalog-info.yaml           # templated from parameters
├─ Dockerfile                 # distroless, non-root, slim
├─ .github/workflows/ci.yaml  # test, lint, vuln-scan, build,
sign, push, deploy
├─ deploy/
│   └─ deployment.yaml        # K8s Deployment with probes,
resources, IRSA
│   └─ service.yaml
│   └─ hpa.yaml              # HPA from Ch 9 defaults
│   └─ networkpolicy.yaml    # default-deny from Ch 10
├─ terraform/
│   └─ main.tf
# ECR, IAM role (IRSA), optional RDS – thin wrapper over modules
├─ variables.tf
├─ docs/
│   └─ index.md              # TechDocs entry point
│   └─ runbook.md           # incident runbook skeleton
├─ Makefile
└─ README.md

```

CI pipeline produced by the template (abridged):

```

# .github/workflows/ci.yaml – generated per service from the paved-road
pipeline template
name: ci
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }

permissions:
  contents: read
  id-token: write          # OIDC – no static secrets
  packages: write
  security-events: write

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with: { go-version: "1.22" }
      - run: go test -race -count=1 ./...
      - run: go vet ./...

  vuln-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: govulncheck-action@v1
      - uses: aquasecurity/trivy-action@0.24.0
        with: { image-ref: app, format: sarif, output: trivy.sarif }
      - uses: github/codeql-action/upload-sarif@v3
        with: { sarif_file: trivy.sarif }

  build-and-push:
    needs: [test, vuln-scan]
    runs-on: ubuntu-latest
    outputs:
      digest: ${ steps.build.outputs.digest }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/build-push-action@v6
        id: build
        with:
          context: .
          push: true
          tags: ghcr.io/acme/${ github.event.repository.name }:$
            { github.sha }
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - uses: sigstore/cosign-installer@v3.5.0

```



```

- run: cosign sign --yes ghcr.io/acme/$
{{ github.event.repository.name }}@${{ steps.build.outputs.digest }}

deploy-staging:
  if: github.ref == 'refs/heads/main'
  needs: [build-and-push]
  runs-on: ubuntu-latest
  environment: staging
  steps:
    - uses: actions/checkout@v4
    - uses: aws-actions/configure-aws-credentials@v4
      with: { role-to-assume: arn:aws:iam::123456789012:role/gha-
deployer-staging, aws-region: us-east-1 }
    - run: |
        aws eks update-kubeconfig --name staging --region us-east-1
        kubectl set image deployment/$
{{ github.event.repository.name }} \
        app=ghcr.io/acme/${{ github.event.repository.name }}@$
{{ needs.build-and-push.outputs.digest }} \
        -n ${{ github.event.repository.name }}
        kubectl rollout status deployment/$
{{ github.event.repository.name }} -n $
{{ github.event.repository.name }} --timeout=120s

```

TechDocs

TechDocs renders Markdown from the repo (docs/) inside Backstage via MkDocs — docs live next to the code and are versioned with it.

```

# mkdocs.yml – at the repo root, consumed by TechDocs
site_name: Orders API
site_description: Order lifecycle service
repo_url: https://github.com/acme/orders-api

nav:
  - Home: index.md
  - Runbook: runbook.md
  - API: api.md
  - ADRs: adr/index.md

plugins:
  - techdocs-core

theme:
  name: material

```

```

<!-- docs/index.md – rendered inside Backstage -->
# Orders API

Owner: @checkout · Lifecycle: production · System: commerce

## Quick start

\\`\\`\\`bash
make run          # local dev with docker-compose (Postgres + Redis)
make test         # unit + integration
gh workflow run ci.yaml # trigger CI
\\`\\`\\`

## Architecture

See [ADR-003: Postgres vs DynamoDB](adr/003-postgres-vs-dynamodb.md).

## On-call

- PagerDuty: [orders-api service](https://acme.pagerduty.com/services/PDSVC123)
- Runbook: [runbook.md](runbook.md)
- SLOs: 99.9% availability, p99 < 250ms (see [SLOs](slo.md))

```

RBAC and Kubernetes plugin

```

# rbac-policy.csv – Backstage permission framework (Casbin-style)
# p, role, rule, resource, action, effect
p, role:default/guests, catalog.entity.read, catalog-entity, read, allow
p, role:default/developers, scaffolder.template.execute, scaffolder-template, execute, allow
p, role:default/developers, scaffolder.action.execute, scaffolder-action, execute, allow
p, role:default/platform-admins, catalog.entity.delete, catalog-entity, delete, allow
p, role:default/platform-admins, scaffolder.template.execute, scaffolder-template, execute, allow

g, group:default/checkout, role:default/developers
g, group:default/platform, role:default/platform-admins
g, user:default/alice, role:default/developers

```

Backstage deployment on Kubernetes (abridged):

```

# deploy/backstage.yaml – Backstage itself on Kubernetes
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backstage
  namespace: platform
spec:
  replicas: 2
  selector:
    matchLabels: { app: backstage }
  template:
    metadata:
      labels: { app: backstage }
    spec:
      serviceAccountName: backstage
      containers:
        - name: backstage
          image: ghcr.io/acme/backstage:1.28.0
          ports: [{ containerPort: 7007 }]
          env:
            - name: POSTGRES_HOST
              valueFrom: { secretKeyRef: { name: backstage-secrets,
key: postgres-host } }
            - name: GITHUB_TOKEN
              valueFrom: { secretKeyRef: { name: backstage-secrets,
key: github-token } }
          resources:
            requests: { cpu: "1000m", memory: "1Gi" }
            limits: { cpu: "2000m", memory: "2Gi" }
          readinessProbe:
            httpGet: { path: /.backstage/health/v1/readiness, port: 700
7 }
            periodSeconds: 10
          livenessProbe:
            httpGet: { path: /.backstage/health/v1/liveness, port:
7007 }
            periodSeconds: 30
          ---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: backstage
  namespace: platform
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/
backstage-prod

```

Golden paths in depth

A paved road is not just a repo template — it is a **vertical slice** that encodes standards at every layer. A "Go API" golden path, for example, is a contract:

```
flowchart TB
    Template["Template: go-api  
Backstage scaffolder"] --> Repo["Repo"]
    Repo --> CI["CI (GHA)"]
    CI --> IaC["IaC (Terraform)"]
    IaC --> K8s["Kubernetes"]
    K8s --> Obs["Observability"]
    Obs --> Docs["Docs + ownership"]
    Docs --> Repo

    style Template fill:#e3f2fd
    style CI fill:#fff3e0
    style IaC fill:#fce4ec
    style Obs fill:#c8e6c9
    style Docs fill:#f3e5f5
    style Custom fill:#fff9c4
```

Go 1.22 + chi
Dockerfile (distroless)
catalog-info.yaml"]

test + vet + vuln-scan
build + cosign + push"]

ECR + IRSA role
optional RDS
via modules/network"]

Deployment + HPA
NetworkPolicy
ServiceMonitor"]

Grafana dashboard
Prometheus alerts
SLO (99.9 / p99 250ms)"]

TechDocs + runbook
PagerDuty + catalog"]

replace any layer"| Custom["Custom stack
you own the delta"]

Figure 11-3: A golden path as a vertical slice — the template composes repo, CI, IaC, Kubernetes, observability, and docs into one coherent, compliant default. Any layer can be replaced via an escape hatch, but the default is fully wired.

What makes a golden path "golden" is not the tool choice — it is the **guarantees**:

Layer	Default	Guarantee
Language/runtime	Go 1.22, distroless image, non-root	No CVEs in base image; reproducible builds
CI	GHA with OIDC, cosign, Trivy, govulncheck	Every image is signed and scanned before it can be deployed
IaC	Terraform modules with <code>required_version</code> , policy checks	Every resource is tagged, encrypted, and in the right VPC/subnet
Kubernetes	Deployment + HPA + NetworkPolicy + ServiceMonitor	Every service is autoscaled, network-isolated, and monitored
Observability	Dashboard + alerts + SLO from mixins	Every service has an SLO and an on-call rotation on day one
Security	IRSA / Workload Identity, secrets via Secrets Manager, mTLS via mesh	No static credentials; no <code>0.0.0.0/0</code> SGs; TLS everywhere

The platform team owns the **mixins** — a reusable dashboard, alert, or policy fragment — and the template composes them. When the logging standard changes, the platform updates one mixin and the next scaffold (and optionally a bulk PR via Renovate/Dependabot) propagates it.

Operating the platform

Measuring success

A platform without metrics is a cost center. The signals that matter:

Metric	What it tells you	How to measure
Time to first deploy	Is the paved road actually fast?	Scaffolder timestamp → first <code>kubectrl</code> rollout success

Metric	What it tells you	How to measure
Template adoption	Are teams using the road or going off-road?	Catalog: % of new services from templates
DORA lead time	Is the platform speeding up delivery?	CI: commit → production deploy duration
Catalog coverage	Can you answer "what do we have?"	% of repos with catalog-info.yaml , orphan detection
Scorecard pass rate	Are services meeting standards?	TechInsights / custom scorecard: prod readiness checks
Platform NPS / toil survey	Do engineers like the platform?	Quarterly survey: "how much toil did the platform remove?"

Service maturity scorecards (Backstage TechInsights or a custom plugin) encode the standards as checks:

```
# scorecards/prod-readiness.yaml – TechInsights-style checks
apiVersion: backstage.io/v1alpha1
kind: Scorecard
metadata:
  name: prod-readiness
  title: Production Readiness
spec:
  checks:
    - id: has-catalog-info
      name: Registered in catalog
      fact: catalog-info-exists
    - id: has-runbook
      name: Runbook present (docs/runbook.md)
      fact: runbook-exists
    - id: ci-signed-images
      name: Images are signed (cosign)
      fact: cosign-verified
    - id: has-slo
      name: SLO defined and monitored
      fact: slo-exists
    - id: no-static-secrets
      name: No static AWS keys in repo
      fact: no-long-lived-credentials
    - id: hpa-configured
      name: HPA configured
      fact: k8s-hpa-exists
    - id: networkpolicy-present
      name: NetworkPolicy present
      fact: k8s-networkpolicy-exists
    - id: pagerduty-wired
      name: PagerDuty service linked
      fact: pagerduty-service-exists
```

The catalog page for each service renders its scorecard — green checks and red gaps are visible to the team and to leadership without a spreadsheet.

The platform operating model

```

flowchart LR
    Discovery["Discovery  
interviews, toil survey  
incident analysis"] --> Roadmap["Roadmap  
prioritized by  
lead time + toil + risk"]
    Roadmap --> Build["Build  
template / module / mixin  
with docs + tests"]
    Build --> Release["Release  
versioned, changelog  
migration guide"]
    Release --> Adopt["Adopt  
comms, office hours  
migration PRs"]
    Adopt --> Measure["Measure  
adoption, lead time  
NPS, scorecards"]
    Measure --> Discovery

    style Roadmap fill:#e3f2fd
    style Build fill:#fff3e0
    style Adopt fill:#c8e6c9
    style Measure fill:#fce4ec

```

Figure 11-4: The platform product loop — discovery → roadmap → build → release → adopt → measure → repeat. The loop is continuous; the platform is never "done."

Practical operating choices:

- **Versioning golden paths.** Templates and modules are versioned (e.g., `go-api@v2.4`). Breaking changes get a new major version and a migration guide. A bot (Renovate) opens PRs to update consumers — adoption is nudged, not forced.
- **Office hours and champions.** The platform team holds weekly office hours and seeds champions in product teams who advocate for the paved road and feed back pain.
- **Deprecation with a deadline.** Old pipeline versions or Terraform modules are deprecated with a 90-day window and automated codemods where possible. After the window, the platform stops supporting the old path — but does not break it overnight.
- **Incident-driven improvement.** Every production incident that traces to a missing paved-road feature (no HPA, no NetworkPolicy, no

SLO) becomes a platform backlog item. The next team never hits the same gap.

Anti-patterns

Anti-pattern	Symptom	Fix
Framework, not product	"We built a great abstraction — no one uses it"	Start from user pain, not from a cool tool; extract from what best teams already do
Mandate without adoption	Forced migration, resentment, shadow tooling	Make the road better than the wilderness; measure adoption, not compliance
Portal is the platform	Beautiful Backstage, no paved roads underneath	The portal is the storefront — the platform is the templates, modules, and pipelines behind it
Over-abstraction	One template with 40 flags tries to serve every workload	Fewer, opinionated templates per workload type; escape hatches for the rest
Under-documentation	Template exists, no one knows how to use it	TechDocs, README, and a 5-minute video per golden path — docs are part of the template
No deprecation	Five pipeline versions coexist forever	Version, changelog, migration guide, deadline — treat the platform as a product with a lifecycle

Distributed-systems lens

Platform engineering is the organizational answer to the coordination cost of distributed systems. Each service is a distributed system; the fleet is a distributed system of distributed systems. The platform reduces coordination cost in three ways:

1. **Consistency by default.** Paved roads encode the consistency choices (retry policy, timeout, idempotency, SLO) that individual

teams would otherwise choose inconsistently. The fleet converges on a coherent set of defaults without a central review bottleneck.

2. **Discovery and dependency reasoning.** The catalog is the fleet's service registry for humans — "what depends on what, who owns it, where are its docs and runbooks?" Without it, incident response starts with "which Slack channel owns this service?" With it, the on-call path is one click from the failing dashboard.
 3. **Safe evolution at scale.** Versioned templates + automated migration PRs let the platform evolve the fleet's 200 services without 200 manual tickets. A new security standard (e.g., "all images must be signed") is rolled out as a template update + bulk PR + scorecard check — not as a 6-month program.
-

Key takeaways

- Platform engineering reduces cognitive load by encoding expertise into paved roads — opinionated, supported paths where the right way is the easy way — with explicit escape hatches for off-road needs.
- Treat the platform as a product: user research, prioritized roadmap, adoption metrics, documentation, and deprecation discipline — not as a side project or a mandate.
- An IDP composes a service catalog, software templates/scaffolder, workflow orchestration, developer portal, and capability modules (IaC, CI/CD, observability) atop the Kubernetes/cloud substrate.
- Backstage is the de facto portal standard: `catalog-info.yaml` registers every component with ownership and dependencies; `template.yaml` scaffolds new services as executable golden paths; TechDocs keeps docs next to code; RBAC and the Kubernetes plugin surface runtime state.
- A golden path is a vertical slice — repo, CI (signed/scanned), IaC (tagged/encrypted), Kubernetes (autoscaled/isolated), observability (SLO/alerts), and docs — composed from reusable mixins that the platform team maintains.
- Measure the platform with time-to-first-deploy, template adoption, DORA lead time, catalog coverage, scorecard pass rate, and engineer NPS — not just portal page views.

- The platform loop is continuous: discovery → roadmap → build → release → adopt → measure → repeat. The most dangerous moment for a platform is when the team declares it "done."
- The portal is the storefront, not the platform. A beautiful portal with no paved roads underneath is a catalog of toil, not a reduction of it.

Further reading

- Backstage documentation — <https://backstage.io/docs/>
- CNCF Platform Engineering Working Group — white papers and maturity model — <https://tag-app-delivery.cncf.io/whitepapers/platform-eng-maturity-model/>
- Pauley, *Platform Engineering* (O'Reilly, 2024) — paved roads, IDPs, and the product discipline.
- Skelton & Pais, *Team Topologies* (IT Revolution, 2019) — platform team as an enabling team, cognitive load, and team interaction modes.
- Spotify engineering blog — Backstage origin and adoption — <https://backstage.spotify.com/learn/>
- Thoughtworks Technology Radar — Backstage, platform engineering patterns — <https://www.thoughtworks.com/radar>
- *Accelerate* (Forsgren, Humble, Kim) — DORA metrics and the link between platform capabilities and delivery performance.